OmniCode: Chapter 4: Track 1 - Core Types & Process Tape Ribbon

TapeHead 5-Ws Ribbon, Cell, Word, Reels & Operators

CHAPTER IDENTIFIER
CH 04

NOTEBOOKLM WORD COUNT
42,278 words

CATALOGED CHAPTER FILES
44 files (11,219 lines)

GENERATION DATE
2026-10-16 20:35:50

EPISTEMIC COVENANT BEDROCK & IMMUTABLE LANDMARKS

"And he gave unto Moses... two tables of testimony, tables of stone, written with the finger of God." — Exodus 31:18

"Prepare thy work without, and make it fit for thyself in the field; and afterwards build thine house." — Proverbs 24:27

"Remove not the ancient landmark, which thy fathers have set." — Proverbs 22:28

GOOGLE NOTEBOOKLM GROUNDING FOCUS & LEARNING PROMPT

- **Chapter Focus:** *"How does the TapeHead coordinate the 5 Ws and 1 H in time?"*
- **Architectural Rule:** All definitions conform to 4-block structure & spatial ruler.

CHAPTER MANIFEST & INDEX (44 FILES)

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
1	**<TRACK 1> : {Host Engine & Virtual Machine}/TRACK.md**	MD	14	85	0.7
2	**<TRACK 1> : {Host Engine & Virtual Machine}/src/lib.rs**	RS	87	332	3.7
3	**<TRACK 1> : {Host Engine & Virtual Machine}/src/main.rs**	RS	1,355	5,052	60.2
4	**<TRACK 1> : {Host Engine & Virtual Machine}/src/core/boundaries.rs**	RS	167	778	6.9
5	**<TRACK 1> : {Host Engine & Virtual Machine}/src/core/cell.rs**	RS	345	1,236	11.7
6	**<TRACK 1> : {Host Engine & Virtual Machine}/src/core/filetype.rs**	RS	420	1,554	16.3
7	**<TRACK 1> : {Host Engine & Virtual Machine}/src/core/linguistics.rs**	RS	1,237	4,208	54.2
8	**<TRACK 1> : {Host Engine & Virtual Machine}/src/core/mmio.rs**	RS	300	1,055	9.7
9	**<TRACK 1> : {Host Engine & Virtual Machine}/src/core/mod.rs**	RS	54	205	2.1
10	**<TRACK 1> : {Host Engine & Virtual Machine}/src/core/operators.rs**	RS	380	1,303	14.6
11	**<TRACK 1> : {Host Engine & Virtual Machine}/src/core/reel.rs**	RS	286	1,090	11.5
12	**<TRACK 1> : {Host Engine & Virtual Machine}/src/core/scene.rs**	RS	130	449	4.3
13	**<TRACK 1> : {Host Engine & Virtual Machine}/src/core/tape.rs**	RS	497	1,828	19.1
14	**<TRACK 1> : {Host Engine & Virtual Machine}/src/core/ternary.rs**	RS	386	1,528	14.5
15	**<TRACK 1> : {Host Engine & Virtual Machine}/src/core/vessel.rs**	RS	320	1,088	11.2
16	**<TRACK 1> : {Host Engine & Virtual Machine}/src/core/word.rs**	RS	676	2,608	22.6
17	**<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/lexicon/domain.rs**	RS	38	166	1.5
18	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/mod.rs**	RS	37	122	1.5
19	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/debugger.rs**	RS	471	1,835	20.2
20	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/mod.rs**	RS	54	264	2.6
21	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/restorator.rs**	RS	667	2,387	27.8
22	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/README.md**	MD	78	588	4.5
23	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/mod.rs**	RS	356	1,031	12.2
24	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/inscriber/log_writer.rs**	RS	91	264	3.4
25	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/inscriber/mod.rs**	RS	16	53	0.6
26	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/inscriber/naming.rs**	RS	69	231	2.1
27	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/inscriber/rotation.rs**	RS	86	300	3.0
28	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/inscriber/scroll_writer.rs**	RS	133	460	5.3
29	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/telemetry/frame.rs**	RS	82	263	2.7
30	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/telemetry/mod.rs**	RS	14	45	0.4
31	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/telemetry/note.rs**	RS	79	305	3.0
32	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/telemetry/ring_buffer.rs**	RS	95	313	3.0
33	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/failpoint/mod.rs**	RS	12	38	0.4
34	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/failpoint/remediation.rs**	RS	45	235	2.0
35	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/failpoint/types.rs**	RS	61	276	2.7
36	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/query/index.rs**	RS	32	139	1.2

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
37	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/query/mod.rs**	RS	12	36	0.4
38	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/query/search.rs**	RS	273	798	9.0
39	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/universal/bus.rs**	RS	96	305	3.2
40	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/universal/clock.rs**	RS	56	176	1.7
41	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/universal/mod.rs**	RS	12	40	0.4
42	**<TRACK 1> : {Host Engine & Virtual Machine}/src/cogs/mod.rs**	RS	32	132	1.4
43	**<TRACK 1> : {Host Engine & Virtual Machine}/src/cogs/platform.rs**	RS	940	4,296	42.8
44	**<TRACK 1> : {Host Engine & Virtual Machine}/src/cogs/tripwire.rs**	RS	628	2,781	28.2

<TRACK 1> : {Host Engine & Virtual Machine}/TRACK.md MD 14 LINES 85 WORDS 0.7 KB

```
# <TRACK 1> : {Host Engine & Virtual Machine}

## Scope & Invariant

The host compiler, balanced ternary virtual machine, process scheduler, and test harness.

- `src/core/`: Boundaries, Cell, 32-Trit Word, 4-Directional Reel, MMIO, Linguistics.
- `src/runtime/`: Assembler, Transactional Environment, Evaluator.
- `src/terminal/`: Multi-Track OmniScheduler, Living TerminalSession, Studio Renderer.
- `src/ai/`: Cognitive loop, AI Cortex, AILearner, AIMemory, Biblical Lexicon.
- `src/parser/` & `src/ast/`: Directional Lexer, recursive-descent Parser, AST.
- `src/diagnostics/`: Restorator, Debugger, Linter, Logger, Hover Engine.
- `tests/`: 3-tier automated test harness (unit, integrated, functional).
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/lib.rs RS 87 LINES 332 WORDS 3.7 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Library Root Identity, Universal Subsystem Modules & Core Re-exports )
// ============================================================================
//! OmniCode Language Engine
//! High-performance language engine based on formal ontology,
//! set theory, and balanced ternary quad-state execution,
//! engineered to be symmetrically human and machine readable.

pub mod core;
pub mod diagnostics;
pub mod ast;
pub mod parser;
pub mod runtime;
pub mod cli;
pub mod ai;
pub mod terminal;
pub mod cogs;


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Public Re-export Declarations & High-Level API Surface )
// ============================================================================
pub use cogs::platform::{HostPlatform, HostArch};
pub use cogs::tripwire::{Tripwire, TripwireError};
pub use core::ternary::{QuadState, TransportAction, ROLLBACK, AWAIT, PROCEED, HALT};
pub use core::boundaries::BoundaryType;
pub use core::operators::{BeingRelation, DirectionalVector, Connective, Polarity, AffixPosition};
pub use core::word::{Word32Trit, Word64};
pub use core::scene::Scene;
pub use core::reel::MemoryReel;
pub use core::filetype::OmniFileType;
pub use core::tape::{Tape, Roll};
pub use core::mmio::{PhysicalMemoryMap, VgaBuffer, VgaColor, VGA_BUFFER_BASE, COM1_SERIAL_PORT};
pub use terminal::{OmniScheduler, OmniTask, Track, TaskPriority};

pub use diagnostics::logger::{DiagnosticLogger, FailpointType, FixKind, FixSuggestion};
pub use diagnostics::debugger::OmniDebugger;
pub use diagnostics::linter::{
    BlockKind, BodyScore, ClosingScore, FileAuditScore, LintReport, MetadataScore, OmniLinter,
    OpeningScore, QuadStateGate,
};
pub use diagnostics::restorator::{Restorator, RestorationReport, AppliedRepair};
pub use diagnostics::hover::{HoverEngine, HoverCard, OmniDefinition, TapeRollContext};
pub use ai::{
    AITransducer, TransductionSession, AICortex, ScrollLogger, ExecutionScroll, AILearner,
    EpisodicSummary, EpisodicExperience, CognitiveResult, BiblicalLexicon, LexiconDomain,
    LexiconEntry, MoralEvaluation,
};
pub use parser::lexer::Lexer;
pub use parser::parser::Parser;
pub use runtime::environment::Environment;
pub use runtime::evaluator::Evaluator;
pub use terminal::OmniStudio;
pub use cli::{
    run_repl, run_gui_server, run_boot, run_boot_with_config, BootArch, BootConfig, BootContext,
    run_assemble, run_assemble_with_config, AssembleConfig, AssembleContext, AssembleFormat,
};



// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Core Convenience Execution Pipeline )
// ============================================================================
/// Convenience execution helper that tokenizes, parses, and evaluates an OmniCode string.
pub fn execute(source: &str, print_logs: bool) -> Result<runtime::RuntimeValue, String> {
    let logger = DiagnosticLogger::new(print_logs);
    let mut lexer = Lexer::new(source, logger.clone());
    let tokens = lexer.tokenize()?;

    let debugger = OmniDebugger::new(logger, 10);
    let mut parser = Parser::new(tokens, debugger);
```

```
    let program = parser.parse_program()?;

    let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
    evaluator.eval(&ast::nodes::ASTNode::Program(program))
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Engine Status & Library Invariants )
// ============================================================================
// OmniCode library surface verified and sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/main.rs RS 1355 LINES 5052 WORDS 60.2 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( CLI Executable Identity, Subsystem Routing & Core Imports )
// ============================================================================
//! OmniCode CLI Entrypoint
//!
//! Provides the primary terminal interface for the OmniCode language engine:
//!     - Interactive REPL Terminal (default)
//!     - Single-file Script Runner (`run <file.omni>`)
//!     - 4-Block Structural Linter (`lint <file.omni>`)
//!     - Canonical Template Scaffolder (`scaffold <Name>`)
//!     - Ontological Verification Demo (`demo`)

use std::env;
use std::fs;
use std::path::PathBuf;
use std::process;
use std::thread;

use omnicode::{
    run_repl, run_gui_server, DiagnosticLogger, OmniDebugger, OmniLinter, Restorator,
    AICortex, AILearner, OmniFileType, Lexer, Parser, Evaluator, Environment,
    BiblicalLexicon, HoverEngine, Tape, OmniStudio,
};
use omnicode::cli::commands::{
    BootConfig, run_boot_with_config,
    AssembleConfig, run_assemble_with_config,
    RunConfig, run_with_config,
};


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( CLI Help Display, Version Banner, and Command Route Setup )
// ============================================================================
fn print_cli_help() {
    println!("\x1b[1;36m================================================================================\x1b[0m");
    println!("\x1b[1;36m  OmniCode: Dual Human & Machine Readable Language Engine (v0.1.0)\x1b[0m");
    println!("\x1b[1;36m  Core: Balanced Ternary & Boundary Physics (Rust Engine)\x1b[0m");
    println!("\x1b[1;36m================================================================================\x1b[0m\n");
    println!("\x1b[1;33mUSAGE:\x1b[0m");
    println!("  omnicode                    Start the interactive terminal shell (REPL)");
    println!("  omnicode repl               Start the interactive terminal shell (REPL)");
    println!("  omnicode run <file>         Execute an OmniCode source (.omni) or scroll (.scroll)
  omnicode echo <char|msg>    Echo character or message through native balanced ternary I/O transducer");
    println!("  omnicode assemble <file>    Assemble an OmniCode process tape into concrete memory layout (.tape)");
    println!("  omnicode studio <file.tape> Project process tape in the cinematic HUD terminal projector");
    println!("  omnicode project <file.tape> Project process tape in the cinematic HUD terminal projector");
    println!("  omnicode boot [--uefi|--multiboot|--bios] [--gui] Boot bare-metal OS in QEMU");
    println!("  omnicode lint <file>        Validate 4-Block architecture compliance (.omni / .scroll)");
    println!("  omnicode hover <file> <l> <c> Inspect cross-scale 3-tier hovercard for symbol at position");
    println!("  omnicode doc <symbol>       Query architectural docstring, invariants, and machine intent");
    println!("  omnicode restore <file>     Auto-heal syntax, boundary, and block anomalies");
    println!("  omnicode ai \"<prompt>\"      Transduce prompt with AI Cortex, self-heal & inscribe");
    println!("  omnicode interact           Launch live CPI-SI cognitive discourse & learning shell");
    println!("  omnicode train              Train balanced-ternary brain on ideal reels and logs");
    println!("  omnicode noise [stress]     Simulate noise degradation breakdown ladder");
    println!("  omnicode query <session>    Query an episodic memory scroll in O(1) time");
    println!("  omnicode diagnose <session> Diagnose failpoints and view immune repairs");
    println!("  omnicode rotate [--cap N]   Enforce storehouse capacity bounds (Occupancy <= Capacity)");
    println!("  omnicode scrolls            Inspect episodic memory bank and learning statistics");
    println!("  omnicode manifest           Display the canonical Covenant Manifest and moral axioms");
    println!("  omnicode lexicon [term]     Inspect the 100% offline Biblical world lexicon");
    println!("  omnicode gui [--port 8080]  Launch the local web GUI Studio & Terminal interface");
    println!("  omnicode desktop            Launch the native desktop GUI Studio & Terminal shell");
    println!("  omnicode scaffold <Name> [--type omni|scroll] Generate canonical 4-Block template");
    println!("  omnicode filetypes          Display all formal OmniCode file types and MIME definitions");
    println!("  omnicode demo               Run the built-in ontological verification suite");
    println!("  omnicode --help, -h         Show this help information");
    println!("  omnicode --version, -v      Display engine version\n");
    println!("\x1b[1;33mREPL FEATURES:\x1b[0m");
    println!("  • Real-time evaluation of balanced ternary expressions (-1, 0, +1, NULL)");
    println!("  • Boundary delimiter matching with multiline input buffering");
    println!("  • Interactive failpoint alerts and transaction snapshot rollback");
```

```
    println!("  • Live symbol inspection via `:env` and AST inspection via `:ast <expr>`");
    println!("  • Persistent command line history\n");
}

fn print_version() {
    println!("omnicode 0.1.0 (Rust Core: Balanced Ternary & Boundary Physics)");
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Command Handlers: Script Runner, Linter, Scaffolder, Ontological Demo )
// ============================================================================
fn run_studio(path: &str) {
    let logger = DiagnosticLogger::new(false);
    let tape = match Tape::from_file(path, logger) {
        Ok(t) => t,
        Err(e) => {
            eprintln!("\x1b[1;31mError loading tape for studio:\x1b[0m {e}");
            process::exit(1);
        }
    };
    let mut studio = OmniStudio::new(tape.head.name.clone(), tape.head.required_partition_size);
    studio.project_tape(&tape);
}

fn run_lint_file(path: &str) {
    let ft = OmniFileType::from_path(path);
    let title = ft.map(|f| f.display_name()).unwrap_or("OmniCode file");
    println!("\x1b[1;33m--- Linting Architecture ({title}): {} ---\x1b[0m", path);
    let content = match fs::read_to_string(path) {
        Ok(c) => c,
        Err(e) => {
            eprintln!("\x1b[1;31m✗ File read error:\x1b[0m Could not open '{}': {}", path, e);
            process::exit(1);
        }
    };

    let logger = DiagnosticLogger::new(false);
    let linter = OmniLinter::new(logger);
    let report = linter.lint_source(path, &content);

    if report.is_valid {
        println!("\x1b[1;32m✓ {title} Validation PASSED\x1b[0m");
        println!("  Blocks verified in sequence:");
        for b in &report.blocks_found {
            println!("    - Line {}: {}", b.line_number, b.kind.name());
        }

        if let Some(audit) = &report.audit_score {
            let is_tape = matches!(ft, Some(OmniFileType::Tape));

            // Block 1: Metadata / Tape Head
            let b1 = &audit.block1_metadata;
            let b1_label = if is_tape { "Tape Head Manifest" } else { "Metadata Block" };
            println!("\n  \x1b[1;34m[ BLOCK 1 : {b1_label} ]\x1b[0m");
            let score_color = if b1.total >= 90 { "\x1b[1;32m" } else if b1.total >= 70 { "\x1b[1;32m" } else if b1.total
>= 50 { "\x1b[1;33m" } else { "\x1b[1;31m" };
            println!("  {score_color}★ Block 1 Invariant Score: {}/100 [{}]\x1b[0m", b1.total, b1.rating);
            println!("    Subsystem Breakdown:");
            if is_tape {
                println!("      • Formal Tape Head:        {}/20", b1.identity_score);
                println!("      • Partition & Domain:      {}/25", b1.provenance_score);
                println!("      • Invariant Scripture:     {}/20", b1.witness_score);
                println!("      • Cardinal Coordinates:    {}/20", b1.ontology_score);
                println!("      • Spatial & Naming Hygiene:{}/15", b1.spatial_hygiene_score);
            } else {
                println!("      • Identity & Declaration:  {}/20", b1.identity_score);
                println!("      • Provenance & Context:    {}/25", b1.provenance_score);
                println!("      • Covenant Witness Anchor: {}/20", b1.witness_score);
                println!("      • Ontology & Coordinates:  {}/20", b1.ontology_score);
                println!("      • Spatial & Naming Hygiene:{}/15", b1.spatial_hygiene_score);
            }
            for item in &b1.breakdown {
                println!("      {}", item);
            }

            // Block 2: Opening / Tape Setup
```

```
            let b2 = &audit.block2_opening;
            let b2_label = if is_tape { "Roll 1 Initial Setup & Preconditions" } else { "Opening Block & Precondition
Equilibrium" };
            println!("\n  \x1b[1;34m[ BLOCK 2 : {b2_label} ]\x1b[0m");
            let score_color = if b2.total >= 90 { "\x1b[1;32m" } else if b2.total >= 70 { "\x1b[1;32m" } else if b2.total
>= 50 { "\x1b[1;33m" } else { "\x1b[1;31m" };
            println!("  {score_color}★ Block 2 Invariant Score: {}/100 [{}]\x1b[0m", b2.total, b2.rating);
            println!("    Subsystem Breakdown:");
            if is_tape {
                println!("      • Roll 1 Entity Setup:     {}/25", b2.entity_allocation_score);
                println!("      • Memory Cell Target:      {}/20", b2.cell_container_score);
                println!("      • Inscription Vector:      {}/20", b2.substrate_bridge_score);
                println!("      • Container Parity Guard:  {}/20", b2.preconditions_target_score);
                println!("      • Spatial Ruler & Asides:  {}/15", b2.spatial_asides_score);
            } else {
                println!("      • Typed Entity Allocation: {}/25", b2.entity_allocation_score);
                println!("      • Cell Container Sizing:   {}/20", b2.cell_container_score);
                println!("      • Substrate Bridge/Trits:  {}/20", b2.substrate_bridge_score);
                println!("      • Preconditions & Targets: {}/20", b2.preconditions_target_score);
                println!("      • Spatial Ruler & Asides:  {}/15", b2.spatial_asides_score);
            }
            for item in &b2.breakdown {
                println!("      {}", item);
            }

            // Block 3: Body / Tape Rolls
            let b3 = &audit.block3_body;
            let b3_label = if is_tape { "Operational Tape Rolls Section" } else { "Body Block & Causal Pipelines" };
            println!("\n  \x1b[1;34m[ BLOCK 3 : {b3_label} ]\x1b[0m");
            let score_color = if b3.total >= 90 { "\x1b[1;32m" } else if b3.total >= 70 { "\x1b[1;32m" } else if b3.total
>= 50 { "\x1b[1;33m" } else { "\x1b[1;31m" };
            println!("  {score_color}★ Block 3 Invariant Score: {}/100 [{}]\x1b[0m", b3.total, b3.rating);
            println!("    Subsystem Breakdown:");
            println!("      • Operational SVO Verbs:   {}/25", b3.svo_verbs_score);
            println!("      • Directional Vectors:     {}/20", b3.directional_vectors_score);
            println!("      • State-Prime Progression: {}/20", b3.state_prime_score);
            println!("      • Adversative Guard Caveat:{}/20", b3.guard_caveats_score);
            println!("      • Spatial Ruler & Asides:  {}/15", b3.spatial_asides_score);
            for item in &b3.breakdown {
                println!("      {}", item);
            }

            // Block 4: Closing / Commit Status
            let b4 = &audit.block4_closing;
            let b4_label = if is_tape { "Terminal Commit Status Roll" } else { "Closing Block & Sealing Gate" };
            println!("\n  \x1b[1;34m[ BLOCK 4 : {b4_label} ]\x1b[0m");
            let score_color = if b4.total >= 90 { "\x1b[1;32m" } else if b4.total >= 70 { "\x1b[1;32m" } else if b4.total
>= 50 { "\x1b[1;33m" } else { "\x1b[1;31m" };
            println!("  {score_color}★ Block 4 Invariant Score: {}/100 [{}]\x1b[0m", b4.total, b4.rating);
            println!("    Subsystem Breakdown:");
            if is_tape {
                println!("      • Commit Roll Identity:    {}/20", b4.verification_proofs_score);
                println!("      • Preceding Roll Condition:{}/20", b4.invariant_validation_score);
                println!("      • Zero-Tolerance Tripwire: {}/20", b4.tripwire_breaker_score);
                println!("      • Fourfold T4 Machine:     {}/25", b4.fourfold_state_machine_score);
                println!("      • Spatial Ruler & Asides:  {}/15", b4.telemetry_asides_score);
            } else {
                println!("      • Operational Proofs:      {}/20", b4.verification_proofs_score);
                println!("      • Invariant Bounds Parity: {}/20", b4.invariant_validation_score);
                println!("      • Zero-Tolerance Tripwire: {}/20", b4.tripwire_breaker_score);
                println!("      • Fourfold T4 Machine:     {}/25", b4.fourfold_state_machine_score);
                println!("      • Telemetry & Spatial:     {}/15", b4.telemetry_asides_score);
            }
            for item in &b4.breakdown {
                println!("      {}", item);
            }

            // Composite 4-Block Summary & QuadState Gate
            println!("\n
\x1b[1;36m================================================================================\x1b[0m");
            let comp_color = if audit.composite_score >= 70 {
                "\x1b[1;32m"
            } else if audit.composite_score >= 50 {
                "\x1b[1;33m"
            } else {
                "\x1b[1;31m"
            };
            println!("  {comp_color}COMPOSITE 4-BLOCK AUDIT SCORE: {:+}/100\x1b[0m", audit.composite_score);
```

```
            println!("  COMPILATION GATE: \x1b[1m{}\x1b[0m", audit.gate.name());
            if audit.eligible_to_compile {
                println!("  \x1b[1;32m✓ ELIGIBLE TO COMPILE: Source unit certified for AST lowering & assembly\x1b[0m");
            } else {
                println!("  \x1b[1;31m✗ COMPILATION DISQUALIFIED: Fails threshold for assembly lowering\x1b[0m");
            }
            println!("
\x1b[1;36m================================================================================\x1b[0m\n");
        }

        if !report.warnings.is_empty() {
            println!("\x1b[1;33m⚠ Lint Warnings / Recommendations ({}):\x1b[0m", report.warnings.len());
            for warn in &report.warnings {
                println!("    • {}", warn);
            }
        }
    } else {
        println!("\x1b[1;31m✗ {title} Validation FAILED\x1b[0m");
        for err in &report.errors {
            eprintln!("    • {}", err);
        }
        process::exit(1);
    }
}

fn run_scaffold(name: &str, file_type: Option<&str>) {
    let ft = file_type
        .and_then(OmniFileType::from_extension)
        .unwrap_or(OmniFileType::SourceReel);
    let scaffolded = match ft {
        OmniFileType::SourceReel => OmniLinter::auto_scaffold_omni(name, "User-defined module"),
        OmniFileType::EpisodicScroll => OmniLinter::auto_scaffold_scroll(name, "User cognitive discourse intent"),
        OmniFileType::Tape => ft.scaffold_template(name, "User process execution tape"),
    };
    println!("{}", scaffolded);
}

fn run_filetypes() {
    println!("\x1b[1;36m================================================================================\x1b[0m");
    println!("\x1b[1;36m  OmniCode File Type System & Taxonomy Manifest                                  \x1b[0m");
    println!("\x1b[1;36m================================================================================\x1b[0m\n");

    for (i, ft) in OmniFileType::all().iter().enumerate() {
        println!("  \x1b[1;33m[{}]\x1b[0m \x1b[1;32m.{}\x1b[0m  —  \x1b[1m{}\x1b[0m", i + 1, ft.extension(),
ft.display_name());
        println!("      MIME:        \x1b[36m{}\x1b[0m", ft.mime_type());
        println!("      Language ID: \x1b[35m{}\x1b[0m", ft.language_id());
        println!("      Purpose:     {}\n", ft.description());
    }
}

fn run_demo() {
    println!("\x1b[1;36m======================================================\x1b[0m");
    println!("\x1b[1;36m  OmniCode: Ontological Conceptual Testbed           \x1b[0m");
    println!("\x1b[1;36m======================================================\x1b[0m\n");

    let demo_code = fs::read_to_string("examples/canonical_notation.omni")
        .unwrap_or_else(|_| OmniLinter::auto_scaffold_omni("DemoModule", "Ontological verification module"));


    println!("\x1b[1;33m--- Running 4-Block Structural Linter ---\x1b[0m");
    let logger = DiagnosticLogger::new(true);
    let linter = OmniLinter::new(logger.clone());
    let lint_report = linter.lint_source("demo_code", &demo_code);

    if lint_report.is_valid {
        println!("\x1b[1;32m✓ 4-Block Structural Check Passed:\x1b[0m Found {} blocks in order.",
lint_report.blocks_found.len());
    } else {
        println!("\x1b[1;31m✗ Structural Linter Errors:\x1b[0m {:?}", lint_report.errors);
    }

    println!("\n\x1b[1;33m--- Running Code Evaluation Pipeline ---\x1b[0m");
    let mut lexer = Lexer::new(&demo_code, logger.clone());

    match lexer.tokenize() {
        Ok(tokens) => {
            println!("\x1b[1;32m✓ Lexer Tokenization Succeeded:\x1b[0m ({} tokens)", tokens.len());
```

```
            let debugger = OmniDebugger::new(logger.clone(), 5);
            let mut parser = Parser::new(tokens, debugger);

            match parser.parse_program() {
                Ok(prog) => {
                    println!("\x1b[1;32m✓ Parser Succeeded:\x1b[0m Generated AST with {} statements",
prog.statements.len());
                    let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
                    match evaluator.eval(&omnicode::ast::nodes::ASTNode::Program(prog)) {
                        Ok(res) => {
                            println!("\n\x1b[1;32m✓ Execution Final Result:\x1b[0m => {}", res.format_colored());
                        }
                        Err(e) => eprintln!("\x1b[1;31m✗ Evaluator Error:\x1b[0m {}", e),
                    }
                }
                Err(e) => eprintln!("\x1b[1;31m✗ Parser Error:\x1b[0m {}", e),
            }
        }
        Err(e) => eprintln!("\x1b[1;31m✗ Lexer Error:\x1b[0m {}", e),
    }

    if logger.has_failpoints() {
        println!("\n\x1b[1;33m--- Diagnostic Failpoints Caught During Run ---\x1b[0m");
        for fp in logger.failpoints() {
            println!("  * [{}] {}", fp.channel, fp.message);
        }
    } else {
        println!("\n\x1b[1;32m✓ No unexpected failpoints encountered during demo run.\x1b[0m");
    }
}

fn run_restore(path: &str, in_place: bool) {
    println!("\x1b[1;36mAutonomous Self-Healing Restoration:\x1b[0m {}", path);
    let logger = DiagnosticLogger::new(false);
    let restorator = Restorator::new(logger);

    match restorator.restore_file(path, in_place) {
        Ok(report) => {
            if !report.was_modified {
                println!("\x1b[1;32m✓ Source is already healthy:\x1b[0m No repairs needed.");
            } else {
                println!("\x1b[1;33m⚠ Repairs Applied ({}):\x1b[0m", report.repairs_applied.len());
                for repair in &report.repairs_applied {
                    println!("    • L{}: [{:?}] {}", repair.line, repair.kind, repair.description);
                }
                if in_place {
                    println!("\x1b[1;32m✓ Healed file written in-place to {}\x1b[0m", path);
                } else {
                    println!("\n\x1b[1;36m--- Restored Content Preview ---\x1b[0m\n{}", report.restored_content);
                }
            }

            if report.verified {
                println!("\x1b[1;32m✓ Post-Restoration Verification PASSED:\x1b[0m Code executes to => +1:PROCEED");
            } else if !report.errors.is_empty() {
                println!("\x1b[1;33m⚠ Verification Diagnostics:\x1b[0m {:?}", report.errors);
            }
        }
        Err(e) => {
            eprintln!("\x1b[1;31m✗ Restoration Failed:\x1b[0m {}", e);
            process::exit(1);
        }
    }
}

fn print_deliberation_trace(
    res: &omnicode::ai::CognitiveResult,
    prompt: &str,
) {
    println!(
        "\x1b[1;36m==================================================\
         ============================\x1b[0m"
    );
    println!(
        "\x1b[1;36m  OmniCode AI: CPI-SI Cognitive Thought \
         Deliberation Trace                    \x1b[0m"
    );
    println!(
```

```
        "\x1b[1;36m================================================\
         ============================\x1b[0m\n"
    );
    println!("\x1b[1;33m[Discourse Prompt]:\x1b[0m\n  \"{}\"\n", prompt);

    println!(
        "\x1b[1;34m[1. Sensory Inscription & Neural Forward Pass]:\x1b[0m"
    );
    let stim_trits: Vec<i8> = res
        .stimulus
        .iter()
        .map(|s| s.as_int().unwrap_or(0))
        .collect();
    println!(
        "  • Encoded Balanced-Ternary Stimulus (8 Cells): {:?}",
        stim_trits
    );
    let brain_acts: Vec<String> = res
        .brain_activation
        .iter()
        .map(|s| {
            format!("{:+}:{}", s.as_int().unwrap_or(0), s.macro_op())
        })
        .collect();
    println!(
        "  • Multi-Layer Neural Activation (4 Neurons):   {:?}",
        brain_acts
    );
    println!(
        "  • Dominant Resonance:                          => +1:{}\n",
        res.execution_state.macro_op()
    );

    println!(
        "\x1b[1;34m[2. OmniCode Syntax Breakdown & 5Ws + H Anchoring]:\
         \x1b[0m"
    );
    println!("  • WHO:   {}", res.thought.who);
    println!("  • WHAT:  {}", res.thought.what);
    println!("  • WHEN:  {}", res.thought.when_clause);
    println!("  • WHERE: {}", res.thought.where_clause);
    println!("  • WHY:   {}", res.thought.why);
    println!("  • HOW:   {}", res.thought.how);
    if let Some(ref anchor) = res.thought.biblical_anchor {
        println!("  • Covenant Invariant: {}\n", anchor);
    } else {
        println!();
    }

    if !res.repairs_applied.is_empty() {
        println!(
            "\x1b[1;33m⚠ Restorator Immune Repairs ({}):\x1b[0m",
            res.repairs_applied.len()
        );
        for r in &res.repairs_applied {
            println!(
                "    • L{}: [{:?}] {}",
                r.line, r.kind, r.description
            );
        }
        println!();
    }

    println!("\x1b[1;32m✓ Synthesized 4-Block OmniCode Turing Reel:\x1b[0m");
    println!(
        "-------------------------------------------------\
         ----------------------------"
    );
    println!("{}", res.synthesized_omnicode.trim());
    println!(
        "-------------------------------------------------\
         ----------------------------"
    );
    println!(
        "\x1b[1;32m✓ Machine Evaluator Verification Gate:\x1b[0m => +1:{}",
        res.execution_state.macro_op()
    );

```

```
    println!(
        "\n\x1b[1;34m[3. Cognitive Conclusion Derivation \
         (Grounded vs Keyword)]:\x1b[0m"
    );
    println!(
        "  • Pathway:      Neuro-Symbolic Formulation \
         (Not Keyword Trigger)"
    );
    println!("  • Anchor:       {}", res.thought.premise);
    println!("  • Causal Law:   {}", res.thought.why);
    println!(
        "  • Verification: AST Evaluator verified state => +1:{}",
        res.execution_state.macro_op()
    );
    println!("\n\x1b[1;35m[Machine Layer Response]:\x1b[0m");
    println!("  \"{}\"\n", res.natural_response);

    let (b_e, b_n, b_i) = res.synaptic_counts_before;
    let (a_e, a_n, a_i) = res.synaptic_counts_after;
    println!(
        "\x1b[1;34m[Episodic Storage & Lifelong Neuroplasticity]:\x1b[0m"
    );
    println!("  • Prompt Scroll:   {:?}", res.scroll_path);
    println!("  • Telemetry Log:   {:?}", res.log_path);
    println!(
        "  • Synaptic Delta:  +1: {b_e}->{a_e} | 0: {b_n}->{a_n} \
         | -1: {b_i}->{a_i}"
    );
    println!("  • Session ID:      {}\n", res.session_id);
}

fn run_ai(prompt: &str, verbose: bool) {
    let mut cortex = AICortex::new();

    match cortex.process_prompt(prompt) {
        Ok(res) => {
            if verbose {
                print_deliberation_trace(&res, prompt);
            } else {
                println!("[CPI-SI Base]: \"{}\"", res.natural_response);
            }
        }
        Err(e) => {
            eprintln!("\x1b[1;31m✗ AI Cortex Error:\x1b[0m {}", e);
            process::exit(1);
        }
    }
}

fn run_scrolls() {
    println!("\x1b[1;36m================================================================================\x1b[0m");
    println!("\x1b[1;36m  OmniCode AI: Episodic Memory Bank & Experiential Learning Archive           \x1b[0m");
    println!("\x1b[1;36m================================================================================\x1b[0m\n");
    let scroll_dir = omnicode::core::filetype::resolve_scroll_storage_dir();
    let learner = AILearner::new(&scroll_dir);
    match learner.digest_experiences() {
        Ok(summary) => {
            println!("\x1b[1;33mEPISODIC LEARNING SUMMARY:\x1b[0m");
            println!("  • Total Inscribed Episodes:     {}", summary.total_episodes);
            println!("  • State Distribution:           Proceed (+1): {} | Halt (NULL): {} | Rollback (-1): {} | Await
(0): {}",
                summary.proceed_count, summary.halt_count, summary.rollback_count, summary.await_count
            );
            println!("  • Restorator Repairs Learned:   {}\n", summary.total_repairs_learned);

            if summary.experiences.is_empty() {
                println!("  (No scrolls recorded yet in `scrolls/`. Run `omnicode ai \"...\"` to generate
experiences.)\n");
            } else {
                println!("\x1b[1;33mRECENT EPISODES:\x1b[0m");
                for (i, exp) in summary.experiences.iter().enumerate() {
                    let log_status = if exp.log_path.is_some() { "yes" } else { "no" };
                    println!(
                        "  [{:>2}] Session: {:<12} | State: +1:{:<8} | Repairs: {:>2} | Failpoints: {:>2} | .log: {}",
                        i + 1, exp.session_id, exp.final_state.macro_op(), exp.repair_count, exp.failpoint_count,
log_status
                    );
                    println!("       Prompt: \"{}\"", exp.prompt);
```

```
                    println!("       Scroll: {:?}\n", exp.scroll_path);
                }
            }
        }
        Err(e) => {
            eprintln!("\x1b[1;31m✗ Failed to read scrolls:\x1b[0m {}", e);
            process::exit(1);
        }
    }
}

fn run_query(session_id: &str) {
    println!("\x1b[1;36m================================================================================\x1b[0m");
    println!("\x1b[1;36m  OmniCode AI: O(1) Episodic Memory & Telemetry Query Engine                  \x1b[0m");
    println!("\x1b[1;36m================================================================================\x1b[0m\n");

    let cortex = AICortex::new();
    match cortex.query_episode(session_id) {
        Some(session) => {
            let state_str = match session.final_state.as_int() {
                Some(i) => format!("{i:+}:{}", session.final_state.macro_op()),
                _ => format!("*:{}", session.final_state.macro_op()),
            };
            println!("\x1b[1;32m✓ Session Located in O(1) Time:\x1b[0m");
            println!("  • Session ID:        {}", session.session_id);
            println!("  • Domain Category:   {}", session.category);
            println!("  • Commit Quad-State: {}", state_str);
            println!("  • Inscribed Prompt:  \"{}\"", session.prompt);
            println!("  • Trapped Failpoints: {}", session.failpoint_count);
            println!("  • Restorator Repairs: {}", session.repair_count);
            println!("  • Inscribed Scroll:  {:?}", session.scroll_path);
            if let Some(ref lp) = session.log_path {
                println!("  • Paired Telemetry:  {:?}", lp);
            } else {
                println!("  • Paired Telemetry:  (none)");
            }
            println!();
        }
        None => {
            eprintln!(
                "\x1b[1;31m✗ Episode Not Found:\x1b[0m Session '{}' does not exist in storehouse.",
                session_id
            );
            process::exit(1);
        }
    }
}

fn run_diagnose(session_id: &str) {
    println!("\x1b[1;36m================================================================================\x1b[0m");
    println!("\x1b[1;36m  OmniCode AI: Failpoint & Anomaly Diagnostic Oracle                          \x1b[0m");
    println!("\x1b[1;36m================================================================================\x1b[0m\n");

    let cortex = AICortex::new();
    match cortex.diagnose_failpoint(session_id) {
        Some(report) => {
            println!("{report}");
        }
        None => {
            eprintln!(
                "\x1b[1;31m✗ Episode Not Found:\x1b[0m Session '{}' does not exist in storehouse.",
                session_id
            );
            process::exit(1);
        }
    }
}

fn run_rotate(max_files: usize) {
    println!("\x1b[1;36m================================================================================\x1b[0m");
    println!("\x1b[1;36m  OmniCode Storehouse: Vessel Capacity Rotation Engine                         \x1b[0m");
    println!("\x1b[1;36m  Invariant Law: Occupancy <= Capacity (2 Kings 4:4)                          \x1b[0m");
    println!("\x1b[1;36m================================================================================\x1b[0m\n");

    let scroll_dir = omnicode::core::filetype::resolve_scroll_storage_dir();
    let policy = omnicode::diagnostics::logger::RotationPolicy::new(
        max_files, true,
    );
```

```
    let logger = omnicode::ai::ScrollLogger::with_rotation_policy(
        &scroll_dir, policy,
    );

    match logger.rotate_storehouse() {
        Ok(rotated) => {
            println!("\x1b[1;32m✓ Storehouse Rotation Enforced:\x1b[0m");
            println!("  • Target Partition Capacity:  {} files", max_files);
            println!(
                "  • Excess Files Archived:     {} files moved to 'archive/'\n",
                rotated
            );
        }
        Err(e) => {
            eprintln!("\x1b[1;31m✗ Storehouse Rotation Failed:\x1b[0m {}", e);
            process::exit(1);
        }
    }
}

fn run_train() {
    println!("\x1b[1;36m================================================================================\x1b[0m");
    println!("\x1b[1;36m  OmniCode AI: CPI-SI Base Neural Trainer & Weight Freezing                    \x1b[0m");
    println!("\x1b[1;36m  Ingesting Archetypes & Runtime Logs into Balanced-Ternary Weights           \x1b[0m");
    println!("\x1b[1;36m================================================================================\x1b[0m\n");

    let scroll_dir = omnicode::core::filetype::resolve_scroll_storage_dir();
    let learner = omnicode::ai::AILearner::new(&scroll_dir);

    match learner.digest_experiences() {
        Ok(summary) => {
            let logger = DiagnosticLogger::new(false);
            let mut brain = omnicode::ai::NeuralBrain::new(
                &[8, 16, 4], 1, logger,
            );
            let trainer = omnicode::ai::NeuralTrainer::new();

            // 1. Stage 1 Foundational Archetype Training
            let archetypes = [
                "Idea { Type } : Character (Atomic rune, 1 Cell, 5 trits)",
                "Exemplar 'A': 65 <=> [ -1, +1, +1, -1, +1 ]",
                "Idea { Type } : Number (Radix-3 balanced scalar value)",
                "Exemplar '0': 0 <=> [ 0, 0, 0, 0, 0 ]",
                "Idea { Type } : Container (Primordial boundary perimeter)",
                "Boundary Invariant: Strict capacity and leak defense",
                "Covenant Identity: CPI-SI Base model conceived 2 years ago",
                "Neural Plasticity: BCM homeostatic threshold adaptation",
            ];
            let archetypes_adapted =
                trainer.train_archetypes(&mut brain, &archetypes);

            // 2. Episodic Experience Training
            let report = trainer.train(&mut brain, &summary.experiences);

            // 3. Persist Frozen Baseline Weights
            let weights_path =
                PathBuf::from("storage/weights/cpisi_base.weights");
            let save_result = brain.save_weights(&weights_path);

            let (exc, neu, inh) = brain.synaptic_counts();

            println!("\x1b[1;32m✓ CPI-SI Base Training Completed:\x1b[0m");
            println!("  • Archetype Inscriptions:    {}", archetypes_adapted);
            println!("  • Episodes Digested:         {}", report.total_episodes);
            println!("  • Total Synapses Adapted:    {}", report.synapses_adapted);
            println!("  • Clean (+1:Proceed) Paths:  {}", report.clean_proceed_count);
            println!("  • Rollback (-1) Protections: {}", report.degraded_rollback_count);
            println!(
                "  • Synapse Breakdown:         +1: {} | 0: {} | -1: {}",
                exc, neu, inh
            );
            println!(
                "  • Resilience Before:         {:.1}%",
                report.noise_resilience_before * 100.0
            );
            println!(
                "  • Resilience After:          {:.1}%",
                report.noise_resilience_after * 100.0
```

```
            );
            match save_result {
                Ok(_) => println!("  • Saved Frozen Weights:      {:?}\n", weights_path),
                Err(e) => println!("  • Weight Save Warning:       {}\n", e),
            }
        }
        Err(e) => {
            eprintln!("\x1b[1;31m✗ Training Failed:\x1b[0m {}", e);
            process::exit(1);
        }
    }
}

fn run_interact() {
    use std::io::Write;
    println!(
        "\x1b[1;36m==================================================\
         ============================\x1b[0m"
    );
    println!(
        "\x1b[1;36m  OmniCode AI: CPI-SI Base Interactive Cognitive \
         Discourse Shell              \x1b[0m"
    );
    println!(
        "\x1b[1;36m  Covenant Partnered Intelligence (CPI-SI) - \
         Structured Intelligence          \x1b[0m"
    );
    println!(
        "\x1b[1;36m==================================================\
         ============================\x1b[0m\n"
    );
    println!(
        "\x1b[1;33mStage 3 Adult Neuroplasticity Active.\x1b[0m \
         Enter prompt or inquiry below."
    );
    println!(
        "Commands: \x1b[1;32m:brain\x1b[0m (status), \
         \x1b[1;32m:verbose\x1b[0m (toggle trace), \
         \x1b[1;32m:save\x1b[0m (persist weights), \
         \x1b[1;31mexit\x1b[0m (quit).\n"
    );

    let mut cortex = AICortex::new();
    let mut verbose = false;
    let stdin = std::io::stdin();

    loop {
        print!("\x1b[1;35mCPI-SI> \x1b[0m");
        std::io::stdout().flush().ok();

        let mut line = String::new();
        if stdin.read_line(&mut line).is_err() || line.trim().is_empty() {
            continue;
        }

        let trimmed = line.trim();
        if trimmed == "exit" || trimmed == "quit" {
            println!(
                "\x1b[1;33mClosing CPI-SI interactive session \
                 with shalom.\x1b[0m\n"
            );
            break;
        }

        if trimmed == ":brain" {
            let (exc, neu, inh) = cortex.brain.synaptic_counts();
            println!("\n\x1b[1;32m--- CPI-SI Neural Brain Status ---\x1b[0m");
            println!(
                "  • Architecture:       \
                 4-Block Multi-Layer Balanced-Ternary"
            );
            println!("  • Layers:             {}", cortex.brain.layers.len());
            println!("  • Excitatory (+1):    {}", exc);
            println!("  • Neutral (0):        {}", neu);
            println!("  • Inhibitory (-1):    {}", inh);
            println!("  • Total Synapses:     {}\n", exc + neu + inh);
            continue;
        }
```

```
        if trimmed == ":verbose" || trimmed == ":thought" {
            verbose = !verbose;
            if verbose {
                println!(
                    "\n\x1b[1;32m✓ Verbose deliberation trace enabled.\
                     \x1b[0m Showing full thought process.\n"
                );
            } else {
                println!(
                    "\n\x1b[1;33m✓ Verbose deliberation trace disabled.\
                     \x1b[0m Returning clean machine responses.\n"
                );
            }
            continue;
        }

        if trimmed == ":save" {
            let weights_path =
                PathBuf::from("storage/weights/cpisi_base.weights");
            match cortex.brain.save_weights(&weights_path) {
                Ok(_) => {
                    println!(
                        "\x1b[1;32m✓ Weights saved to {:?}\x1b[0m\n",
                        weights_path
                    );
                }
                Err(e) => println!("\x1b[1;31m✗ Save failed:\x1b[0m {}\n", e),
            }
            continue;
        }

        match cortex.process_prompt(trimmed) {
            Ok(res) => {
                if verbose {
                    print_deliberation_trace(&res, trimmed);
                } else {
                    println!(
                        "\x1b[1;35m[CPI-SI Base]:\x1b[0m \"{}\"\n",
                        res.natural_response
                    );
                }
            }
            Err(e) => {
                println!("\x1b[1;31m✗ Deliberation Error:\x1b[0m {}\n", e);
            }
        }
    }
}

fn run_noise(stress: Option<f64>) {
    println!("\x1b[1;36m================================================================================\x1b[0m");
    println!("\x1b[1;36m  OmniCode AI: Structured Non-Random Degradation & Noise Engine              \x1b[0m");
    println!("\x1b[1;36m  Simulating Systemic Entropy & Physical Mind Breakdown Dynamics             \x1b[0m");
    println!("\x1b[1;36m================================================================================\x1b[0m\n");

    let engine = omnicode::ai::NoiseEngine::default();
    let ideal_stimulus = &[
        omnicode::core::ternary::QuadState::Proceed,
        omnicode::core::ternary::QuadState::Proceed,
        omnicode::core::ternary::QuadState::Proceed,
        omnicode::core::ternary::QuadState::Proceed,
        omnicode::core::ternary::QuadState::Proceed,
        omnicode::core::ternary::QuadState::Proceed,
        omnicode::core::ternary::QuadState::Proceed,
        omnicode::core::ternary::QuadState::Proceed,
    ];

    let logger = DiagnosticLogger::new(false);
    let brain = omnicode::ai::NeuralBrain::new(&[8, 16, 4], 1, logger);

    if let Some(s) = stress {
        let (perturbed, out) =
            brain.think_with_noise(ideal_stimulus, &engine, s);
        println!("\x1b[1;33mSTRESS LEVEL: {:.2}\x1b[0m", s);
        println!("  • Input Vector:     {:?}", perturbed);
        println!("  • Cortical Output:  {:?}\n", out);
    } else {
```

```
        println!("\x1b[1;33mDEGRADATION LADDER (Ideal -> Breakdown):\x1b[0m");
        let ladder = engine.simulate_breakdown_ladder(ideal_stimulus);
        for step in &ladder {
            let out = brain.think(&step.perturbed_vector);
            let mode_str = match step.dominant_mode {
                omnicode::ai::DegradationMode::Ideal => "Pristine (Ideal)",
                omnicode::ai::DegradationMode::TritDecay => {
                    "Trit Attenuation (Decay)"
                }
                omnicode::ai::DegradationMode::BoundaryLeak => {
                    "Boundary Cross-Talk (Leak)"
                }
                omnicode::ai::DegradationMode::ThermalInversion => {
                    "Thermal Perturbation (Inversion)"
                }
                omnicode::ai::DegradationMode::FailpointTrap => {
                    "Circuit Halt (Catastrophic)"
                }
            };
            println!(
                "  [Stress {:>4.0}%] Mode: {:<32} | Fidelity: {:>5.1}%",
                step.stress_level * 100.0,
                mode_str,
                step.signal_fidelity * 100.0
            );
            println!(
                "    Sensory: {:?} -> Motor: {:?}",
                step.perturbed_vector, out
            );
        }
        let resilience =
            brain.evaluate_noise_resilience(ideal_stimulus, &engine);
        println!(
            "\n  • Overall Neural Noise Resilience Score: {:.1}%\n",
            resilience * 100.0
        );
    }
}

fn run_manifest() {
    println!("\x1b[1;36m================================================================================\x1b[0m");
    println!("\x1b[1;36m  OmniCode AI: Invariant Covenant Manifest (manifest.omni)                     \x1b[0m");
    println!("\x1b[1;36m  Anchored in Public Domain Scriptures (King James Version & World English Bible)\x1b[0m");
    println!("\x1b[1;36m================================================================================\x1b[0m\n");

    match fs::read_to_string("manifest.omni") {
        Ok(content) => println!("{}", content),
        Err(e) => {
            eprintln!("\x1b[1;31m✗ Could not open manifest.omni:\x1b[0m {}", e);
            process::exit(1);
        }
    }
}

fn run_lexicon(query: Option<&str>) {
    println!("\x1b[1;36m================================================================================\x1b[0m");
    println!("\x1b[1;36m  OmniCode AI: 100% Offline Biblical World Lexicon & Moral Ontology           \x1b[0m");
    println!("\x1b[1;36m  Public Domain Primary Witness: King James Version (1611) & World English Bible\x1b[0m");
    println!("\x1b[1;36m================================================================================\x1b[0m\n");

    let lex = BiblicalLexicon::new();
    if let Some(q) = query {
        let results = lex.search(q);
        if results.is_empty() {
            println!("No concepts found matching '{}'. Total concepts available: {}\n", q, lex.all_entries().len());
        } else {
            println!("\x1b[1;33mSEARCH RESULTS FOR '{}' ({} found):\x1b[0m\n", q, results.len());
            for e in results {
                println!("\x1b[1;32m[{}]\x1b[0m \x1b[1m{}\x1b[0m (State Resonance: +1:{})", e.domain.as_str(),
e.term.to_uppercase(), e.quad_state_resonance.macro_op());
                println!("  • Definition: {}", e.definition);
                println!("  • KJV:        {}", e.kjv_passage);
                println!("  • WEB:        {}", e.web_passage);
                println!("  • Guidance:   {}\n", e.moral_guidance);
            }
        }
    } else {
        println!("\x1b[1;33mCANONICAL VOCABULARY OVERVIEW ({} concepts available offline):\x1b[0m\n",
```

```
lex.all_entries().len());
            for e in lex.all_entries() {
                println!("  • \x1b[1;32m{:<20}\x1b[0m | Domain: {:<30} | Resonance: +1:{}", e.term, e.domain.as_str(),
e.quad_state_resonance.macro_op());
            }
            println!("\nTip: Query a specific concept with `omnicode lexicon <term>` (e.g. `omnicode lexicon truth`)\n");
        }
    }

    fn run_gui(port: u16, auto_open: bool, path: &str) {
        if auto_open {
            let url = format!("http://localhost:{port}{path}");
            thread::spawn(move || {
                thread::sleep(std::time::Duration::from_millis(300));
                omnicode::cogs::platform::HostPlatform::current().open_browser(&url);
            });
        }
        if let Err(e) = run_gui_server(port) {
            eprintln!("\x1b[1;31m✗ GUI Server Fatal Error:\x1b[0m {}", e);
            process::exit(1);
        }
    }



    fn run_hover(file: &str, line: usize, col: usize, as_json: bool) {
        let logger = DiagnosticLogger::new(false);
        let mut engine = HoverEngine::new(logger);
        engine.index_workspace(".");

        match engine.hover_at(file, line, col) {
            Some(card) => {
                if as_json {
                    println!("{}", card.to_json());
                } else {
                    println!("{}", card.render_terminal());
                }
            }
            None => {
                if as_json {
                    println!(r#"{{"symbol":"","found":false,"error":"No symbol found at coordinate"}}"#);
                } else {
                    eprintln!("\x1b[1;33m⚠ No symbol located at {}:{}:{}\x1b[0m", file, line, col);
                }
                process::exit(1);
            }
        }
    }

    fn run_doc(symbol: &str, file_hint: Option<&str>, as_json: bool) {
        let logger = DiagnosticLogger::new(false);
        let mut engine = HoverEngine::new(logger);
        engine.index_workspace(".");

        match engine.document_symbol(symbol, file_hint) {
            Some(card) => {
                if as_json {
                    println!("{}", card.to_json());
                } else {
                    println!("{}", card.render_terminal());
                }
            }
            None => {
                if as_json {
                    println!(r#"{{"symbol":"{}","found":false,"error":"Symbol definition not found"}}"#, symbol);
                } else {
                    eprintln!("\x1b[1;33m⚠ Symbol '{}' not found in workspace or target file\x1b[0m", symbol);
                }
                process::exit(1);
            }
        }
    }


    // ================================================================================
    // BLOCK 4: CLOSING BLOCK
    // ( CLI Main Entrypoint Dispatcher & Process Termination )
    // ================================================================================
```

```
fn main() {
    let args: Vec<String> = env::args().collect();

    if args.len() <= 1 {
        // Default behavior: launch interactive terminal environment (REPL)
        if let Err(e) = run_repl() {
            eprintln!("\x1b[1;31mFatal REPL Error:\x1b[0m {}", e);
            process::exit(1);
        }
        return;
    }

    match args[1].as_str() {
        "repl" => {
            if let Err(e) = run_repl() {
                eprintln!("\x1b[1;31mFatal REPL Error:\x1b[0m {}", e);
                process::exit(1);
            }
        }
        "terminal" | "term" | "shell" => {
            let port = if args.len() >= 4 && (args[2] == "--port" || args[2] == "-p") {
                args[3].parse::<u16>().unwrap_or(8080)
            } else if args.len() >= 3 && args[2].starts_with("--port=") {
                args[2]["--port=".len()..].parse::<u16>().unwrap_or(8080)
            } else if args.len() >= 3 && args[2] != "terminal" && args[2] != "term" {
                args[2].parse::<u16>().unwrap_or(8080)
            } else {
                8080
            };
            println!("\x1b[1;36mLaunching OmniCode Native Interactive Terminal (CreativeWorkzStudio LLC)...\x1b[0m");
            run_gui(port, true, "/terminal");
        }
        "project" => {
            if args.len() < 3 {
                eprintln!("\x1b[1;31mError:\x1b[0m Missing file path. Usage: omnicode project <file.tape>");
                process::exit(1);
            }
            run_studio(&args[2]);
        }
        "studio" => {
            if args.len() >= 3 && (args[2].ends_with(".tape") || args[2].ends_with(".omni") || !args[2].starts_with("-"))
&& args[2].parse::<u16>().is_err() {
                run_studio(&args[2]);
            } else {
                let port = if args.len() >= 4 && (args[2] == "--port" || args[2] == "-p") {
                    args[3].parse::<u16>().unwrap_or(8080)
                } else if args.len() >= 3 && args[2].starts_with("--port=") {
                    args[2]["--port=".len()..].parse::<u16>().unwrap_or(8080)
                } else if args.len() >= 3 && args[2] != "studio" {
                    args[2].parse::<u16>().unwrap_or(8080)
                } else {
                    8080
                };
                println!("\x1b[1;36mLaunching OmniCode Studio & Platform (CreativeWorkzStudio LLC)...\x1b[0m");
                run_gui(port, true, "/studio");
            }
        }
        "gui" | "serve" | "desktop" | "app" => {
            let port = if args.len() >= 4 && (args[2] == "--port" || args[2] == "-p") {
                args[3].parse::<u16>().unwrap_or(8080)
            } else if args.len() >= 3 && args[2].starts_with("--port=") {
                args[2]["--port=".len()..].parse::<u16>().unwrap_or(8080)
            } else if args.len() >= 3 && args[2] != "gui" && args[2] != "serve" && args[2] != "desktop" && args[2] !=
"app" {
                args[2].parse::<u16>().unwrap_or(8080)
            } else {
                8080
            };
            println!("\x1b[1;36mLaunching OmniCode Studio & Platform (CreativeWorkzStudio LLC)...\x1b[0m");
            run_gui(port, true, "/studio");
        }
        "boot" | "qemu" => {
            let config = BootConfig::from_args(&args);
            if let Err(e) = run_boot_with_config(config) {
                eprintln!("\x1b[1;31mMachine Inception Halted:\x1b[0m {e}");
                process::exit(1);
            }
        }
```

```
        "assemble" | "asm" => {
            let sub_args: Vec<String> = args.iter().skip(2).cloned().collect();
            let config = match AssembleConfig::from_args(&sub_args) {
                Ok(c) => c,
                Err(e) => {
                    eprintln!("\x1b[1;31mError:\x1b[0m {e}");
                    process::exit(1);
                }
            };
            if let Err(e) = run_assemble_with_config(config) {
                eprintln!("\x1b[1;31m✗ Assemble Tripwire Error:\x1b[0m {e}");
                process::exit(1);
            }
        }
        "run" => {
            if args.len() < 3 {
                eprintln!("\x1b[1;31mError:\x1b[0m Missing file path. Usage: omnicode run <file> [--format
omni|tape|c|lua] [--transpile c|lua]");
                process::exit(1);
            }
            let sub_args: Vec<String> = args.iter().skip(2).cloned().collect();
            let config = match RunConfig::from_args(&sub_args) {
                Ok(c) => c,
                Err(e) => {
                    eprintln!("\x1b[1;31mError:\x1b[0m {e}");
                    process::exit(1);
                }
            };
            match run_with_config(config) {
                Ok(res) => {
                    println!("{}", res.output);
                }
                Err(e) => {
                    eprintln!("\x1b[1;31m✗ Run Error:\x1b[0m {e}");
                    process::exit(1);
                }
            }
        }
        "echo" => {
            let verbose = args.iter().any(|a| a == "--verbose" || a == "-v" || a == "--log");
            let filtered_args: Vec<String> = args.iter().skip(2).filter(|a| !a.starts_with("-")).cloned().collect();
            let payload = filtered_args.join(" ");
            let unquoted = if (payload.starts_with('\'') && payload.ends_with('\'') && payload.len() >= 2)
                || (payload.starts_with('"') && payload.ends_with('"') && payload.len() >= 2)
            {
                &payload[1..payload.len() - 1]
            } else {
                &payload
            };
            match omnicode::terminal::universal::call_response_engine::EchoEngine::echo_stream(unquoted) {
                Ok(echoed_resp) => {
                    let echoed = &echoed_resp.echoed;
                    let mut log_details = String::new();
                    for (i, c) in unquoted.chars().enumerate() {
                        if let Ok(res) = omnicode::terminal::universal::call_response_engine::EchoEngine::echo_char(c) {
                            log_details.push_str(&format!(
                                "  [{:>2}] Glyph: '{}' | Code: {:>3} (0x{:02X}) | 5-Trits: {:?} | Footprint: {} Cell |
State: +1:{}\n",
                                i + 1, res.glyph, res.glyph as u32, res.glyph as u32, res.trits, res.cell_footprint,
res.state.macro_op()
                            ));
                        }
                    }

                    // Inscribe persistent dual log (scroll and telemetry log)
                    let session_id = format!("echo_{}", std::process::id());
                    let timestamp_str = omnicode::ai::model::scroll::generate_timestamp_id();
                    let scroll = omnicode::ai::model::scroll::ExecutionScroll {
                        session_id,
                        timestamp_str,
                        prompt: format!("Echo call: \"{unquoted}\""),
                        synthesized_omnicode: format!("# Character Transduction Echo Reel\necho [ \"{unquoted}\" ] => [
\"{echoed}\" ]\n"),
                        execution_state: omnicode::core::ternary::QuadState::Proceed,
                        natural_response: format!("Echoed: \"{echoed}\""),
                        failpoints: Vec::new(),
                        repairs_applied: Vec::new(),
                        remediations_suggested: Vec::new(),
```

```
                        elapsed_micros: 100,
                    };
                    let saved_paths = omnicode::ai::model::scroll::ScrollLogger::default().save_dual_logs(&scroll).ok();

                    if verbose {
                        println!
("\x1b[1;36m================================================================================\x1b[0m");
                        println!("\x1b[1;36m  OMNICODE PRIMORDIAL ECHO: CHARACTER TRANSDUCTION TELEMETRY LOG
\x1b[0m");
                        println!
("\x1b[1;36m================================================================================\x1b[0m");
                        println!("  • Call Input (FROM Port 0x03F8):        \"{}\"", unquoted);
                        println!("  • Response Output (INTO Port 0x000B8000): \"{}\"", echoed);
                        println!("  • Stream Length:                        {} glyph(s)", echoed.len());
                        if let Some((ref sp, ref lp)) = saved_paths {
                            println!("  • Inscribed Episodic Scroll:            {:?}", sp);
                            println!("  • Inscribed Telemetry Log:              {:?}", lp);
                        }
                        println!("--------------------------------------------------------------------------------");
                        print!("{log_details}");
                        println!("--------------------------------------------------------------------------------");
                        println!("\x1b[1;32m✓ Parity Verified:\x1b[0m Call-Response dialogue sealed in equilibrium
(State: +1:PROCEED)\n");
                    } else {
                        println!("{echoed}");
                    }
                }
                Err(e) => {
                    eprintln!("\x1b[1;31m✗ Echo Anomaly:\x1b[0m {e}");
                    process::exit(1);
                }
            }
        }
        "lint" => {
            if args.len() < 3 {
                eprintln!("\x1b[1;31mError:\x1b[0m Missing file path. Usage: omnicode lint <file.omni>");
                process::exit(1);
            }
            run_lint_file(&args[2]);
        }
        "hover" => {
            if args.len() < 5 {
                eprintln!("\x1b[1;31mError:\x1b[0m Usage: omnicode hover <file> <line> <col> [--json]");
                process::exit(1);
            }
            let file = &args[2];
            let line = args[3].parse::<usize>().unwrap_or(1);
            let col = args[4].parse::<usize>().unwrap_or(1);
            let as_json = args.iter().any(|a| a == "--json" || a == "-j");
            run_hover(file, line, col, as_json);
        }
        "doc" | "explain" => {
            if args.len() < 3 {
                eprintln!("\x1b[1;31mError:\x1b[0m Usage: omnicode doc <symbol> [--file <path>] [--json]");
                process::exit(1);
            }
            let symbol = &args[2];
            let as_json = args.iter().any(|a| a == "--json" || a == "-j");
            let file_hint = args.iter().position(|a| a == "--file" || a == "-f").and_then(|idx| args.get(idx + 1).map(|s|
s.as_str()));
            run_doc(symbol, file_hint, as_json);
        }
        "restore" => {
            if args.len() < 3 {
                eprintln!("\x1b[1;31mError:\x1b[0m Missing file path. Usage: omnicode restore <file.omni> [--write]");
                process::exit(1);
            }
            let in_place = args.iter().any(|a| a == "--write" || a == "-w");
            run_restore(&args[2], in_place);
        }
        "query" => {
            if args.len() < 3 {
                eprintln!(
                    "\x1b[1;31mError:\x1b[0m Missing session ID. \
                     Usage: omnicode query <session_id>"
                );
                process::exit(1);
            }
```

```
            run_query(&args[2]);
        }
        "diagnose" => {
            if args.len() < 3 {
                eprintln!(
                    "\x1b[1;31mError:\x1b[0m Missing session ID. \
                     Usage: omnicode diagnose <session_id>"
                );
                process::exit(1);
            }
            run_diagnose(&args[2]);
        }
        "rotate" => {
            let cap = if args.len() >= 3 {
                if args[2].starts_with("--cap=") {
                    args[2]["--cap=".len()..]
                        .parse::<usize>()
                        .unwrap_or(1000)
                } else if (args[2] == "--cap" || args[2] == "-c")
                    && args.len() >= 4
                {
                    args[3].parse::<usize>().unwrap_or(1000)
                } else {
                    args[2].parse::<usize>().unwrap_or(1000)
                }
            } else {
                1000
            };
            run_rotate(cap);
        }
        "interact" | "chat" | "cpisi" => {
            run_interact();
        }
        "train" => {
            run_train();
        }
        "noise" => {
            let stress = args.get(2).and_then(|s| s.parse::<f64>().ok());
            run_noise(stress);
        }
        "ai" => {
            if args.len() < 3 {
                eprintln!(
                    "\x1b[1;31mError:\x1b[0m Missing prompt. \
                     Usage: omnicode ai \"<prompt>\" | omnicode ai query <id>"
                );
                process::exit(1);
            }
            if args[2] == "query" {
                if args.len() < 4 {
                    eprintln!(
                        "\x1b[1;31mError:\x1b[0m Usage: omnicode ai query \
                         <session_id>"
                    );
                    process::exit(1);
                }
                run_query(&args[3]);
            } else if args[2] == "diagnose" {
                if args.len() < 4 {
                    eprintln!(
                        "\x1b[1;31mError:\x1b[0m Usage: omnicode ai \
                         diagnose <session_id>"
                    );
                    process::exit(1);
                }
                run_diagnose(&args[3]);
            } else if args[2] == "interact" || args[2] == "chat" {
                run_interact();
            } else if args[2] == "train" {
                run_train();
            } else if args[2] == "noise" {
                let stress = args.get(3).and_then(|s| s.parse::<f64>().ok());
                run_noise(stress);
            } else {
                let mut verbose = false;
                let mut prompt_parts = Vec::new();
                for a in &args[2..] {
                    if a == "--verbose" || a == "-v" {
```

```
                        verbose = true;
                    } else {
                        prompt_parts.push(a.as_str());
                    }
                }
                if prompt_parts.is_empty() {
                    eprintln!(
                        "\x1b[1;31mError:\x1b[0m Missing prompt. \
                         Usage: omnicode ai [-v|--verbose] \"<prompt>\""
                    );
                    process::exit(1);
                }
                let prompt = prompt_parts.join(" ");
                run_ai(&prompt, verbose);
            }
        }
        "scrolls" | "history" | "learn" => {
            run_scrolls();
        }
        "manifest" => {
            run_manifest();
        }
        "lexicon" | "bible" | "vocab" => {
            let query = args.get(2).map(|s| s.as_str());
            run_lexicon(query);
        }
        "filetypes" | "types" => {
            run_filetypes();
        }
        "scaffold" => {
            let (name, kind) = if args.len() >= 4 && (args[2] == "--type" || args[2] == "-t") {
                (args.get(4).map(|s| s.as_str()).unwrap_or("MyModule"), Some(args[3].as_str()))
            } else if args.len() >= 4 && (args[3] == "--type" || args[3] == "-t") {
                (args[2].as_str(), args.get(4).map(|s| s.as_str()))
            } else if args.len() >= 4 && (args[2] == "scroll" || args[2] == "omni") {
                (args[3].as_str(), Some(args[2].as_str()))
            } else if args.len() >= 3 {
                (args[2].as_str(), None)
            } else {
                ("MyModule", None)
            };
            run_scaffold(name, kind);
        }
        "demo" => {
            run_demo();
        }
        "--help" | "-h" | "help" => {
            print_cli_help();
        }
        "--version" | "-v" | "version" => {
            print_version();
        }
        other => {
            eprintln!("\x1b[1;31mUnknown argument:\x1b[0m '{}'", other);
            print_cli_help();
            process::exit(1);
        }
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/core/boundaries.rs RS 167 LINES 778 WORDS 6.9 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Module Identity, Morphological Boundary Invariants & Delimiter Tables )
// ================================================================================
//! OmniCode Core: Boundary Mechanics & Morphological Delimiters
//!
//! Delimiters are active morphological operators that govern the boundary physics
//! between an entity and its surrounding universe.
//!
//! The 6 Canonical Boundaries:
//!     1. Denotation  : [ ... ]  (Mutable Definition)
//!     2. Invariant   : < ... >  (Immutable Axiom / Constant)
//!     3. Connotation : { ... }  (In Context Of / Environment)
//!     4. TopDown     : { ... ]  (Context Defines Entity)
//!     5. BottomUp    : [ ... }  (Entity Defined by Context)
//!     6. Aside       : ( ... )  (Aside To / First-Class Machine Intent)

use std::fmt;


// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Context Declarations, BoundaryType Enum, and Delimiter Mapping )
// ================================================================================
/// Morphological boundary types ranked by ontological hierarchy:
///   Rank 1: < ... > (Invariant / Axiom / Law)
///   Rank 2: [ ... ] (Denotation / Essence / Mutable State)
///   Rank 3: { ... } (Connotation / Ambient Context / Environment)
///   Rank 4: [ ... } (Bottom-Up / Inside-Out / Projection into Context)
///   Rank 5: { ... ] (Top-Down / Outside-In / Imposition by Context)
///   Rank 6: ( ... ) (Aside / First-Class Intent / Observer Rationale)
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub enum BoundaryType {
    /// Rank 1: < ... > Immutable Axiom / Law
    Invariant,
    /// Rank 2: [ ... ] Mutable Definition / Essence / State
    Denotation,
    /// Rank 3: { ... } Ambient Context / Environment
    Connotation,
    /// Rank 4: [ ... } Defined by context (Inside-Out / Emergent)
    BottomUp,
    /// Rank 5: { ... ] Context defines (Outside-In / Imposition)
    TopDown,
    /// Rank 6: ( ... ) Aside to (Intent / Observer Rationale)
    Aside,
}


// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Core Resolution Logic: Boundary Properties, Directionality, Pair Matching )
// ================================================================================
impl BoundaryType {
    /// Returns the ontological rank of the boundary (1 = Highest authority / ground truth).
    pub fn rank(&self) -> u8 {
        match self {
            BoundaryType::Invariant => 1,
            BoundaryType::Denotation => 2,
            BoundaryType::Connotation => 3,
            BoundaryType::BottomUp => 4,
            BoundaryType::TopDown => 5,
            BoundaryType::Aside => 6,
        }
    }

    /// Descriptive name of the ontological rank tier.
    pub fn rank_name(&self) -> &'static str {
        match self {
            BoundaryType::Invariant => "Rank 1: Axiomatic Invariant",
            BoundaryType::Denotation => "Rank 2: Denotative Essence",
            BoundaryType::Connotation => "Rank 3: Connotative Environment",
            BoundaryType::BottomUp => "Rank 4: Inside-Out Emergence",
            BoundaryType::TopDown => "Rank 5: Outside-In Imposition",
            BoundaryType::Aside => "Rank 6: Intentional Aside",
```

```
        }
    }

    /// Natural language grammatical clause template for this boundary.
    pub fn linguistic_clause(&self) -> &'static str {
        match self {
            BoundaryType::Invariant => "It is an invariant law that",
            BoundaryType::Denotation => "The entity defines and executes",
            BoundaryType::Connotation => "In the context of the ambient universe",
            BoundaryType::BottomUp => "Projecting outward from entity into context",
            BoundaryType::TopDown => "As conditioned and imposed by the context",
            BoundaryType::Aside => "With the intent and rationale that",
        }
    }

    /// Returns the (open, close) character pair for this boundary type.
    pub fn delimiters(&self) -> (char, char) {
        match self {
            BoundaryType::Invariant => ('<', '>'),
            BoundaryType::Denotation => ('[', ']'),
            BoundaryType::Connotation => ('{', '}'),
            BoundaryType::BottomUp => ('[', '}'),
            BoundaryType::TopDown => ('{', ']'),
            BoundaryType::Aside => ('(', ')'),
        }
    }

    /// True if opening and closing delimiters come from different bracket families.
    pub fn is_asymmetric(&self) -> bool {
        matches!(self, BoundaryType::TopDown | BoundaryType::BottomUp)
    }

    /// True if boundary enforces unalterable immutability.
    pub fn is_immutable(&self) -> bool {
        matches!(self, BoundaryType::Invariant)
    }

    /// Resolves opening and closing characters into a valid BoundaryType.
    ///
    /// # Failpoint
    /// Returns `Err` if the pair forms an illegal boundary (e.g. `(`, `]` or `{`, `>`).
    pub fn from_delimiters(open: char, close: char) -> Result<Self, String> {
        match (open, close) {
            ('[', ']') => Ok(BoundaryType::Denotation),
            ('<', '>') => Ok(BoundaryType::Invariant),
            ('{', '}') => Ok(BoundaryType::Connotation),
            ('{', ']') => Ok(BoundaryType::TopDown),
            ('[', '}') => Ok(BoundaryType::BottomUp),
            ('(', ')') => Ok(BoundaryType::Aside),
            (o, c) => Err(format!(
                "FAILPOINT [IllegalBoundaryPair]: Delimiter pair '{o}...{c}' does not form a legal boundary. \
                 Legal pairs: [...] (Denotation), <...> (Invariant), {{...}} (Connotation), \
                 {{...] (Top-Down), [...}} (Bottom-Up), (...) (Aside)"
            )),
        }
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Display Trait Verification and String Formatter )
// ============================================================================
impl fmt::Display for BoundaryType {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        let (o, c) = self.delimiters();
        write!(f, "{:?}({o}...{c})", self)
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_boundary_ranking_hierarchy() {
        assert_eq!(BoundaryType::Invariant.rank(), 1);
        assert_eq!(BoundaryType::Denotation.rank(), 2);
```

```
        assert_eq!(BoundaryType::Connotation.rank(), 3);
        assert_eq!(BoundaryType::BottomUp.rank(), 4);
        assert_eq!(BoundaryType::TopDown.rank(), 5);
        assert_eq!(BoundaryType::Aside.rank(), 6);

        // Verify ordinal comparison preserves exact hierarchy
        assert!(BoundaryType::Invariant < BoundaryType::Denotation);
        assert!(BoundaryType::Denotation < BoundaryType::Connotation);
        assert!(BoundaryType::Connotation < BoundaryType::BottomUp);
        assert!(BoundaryType::BottomUp < BoundaryType::TopDown);
        assert!(BoundaryType::TopDown < BoundaryType::Aside);
    }
}

```

<TRACK 1> : {Host Engine & Virtual Machine}/src/core/cell.rs RS 345 LINES 1236 WORDS 11.7 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Cell Engine Identity, 8-bit to 5-Trit Codec, & Balanced Radix-3 Mapping )
// ============================================================================
//! OmniCode Core: Cell Architecture & Byte-Trit Hardware Transducer
//!
//! Provides the physical hardware bridge between 8-bit binary silicon
//! and OmniCode's Balanced Ternary Quad-State physics:
//!     - 8 Bits (256 states) <--> 5 Trits (243 symmetric states) + 13 Control Traps
//!     - Dense packing: 1 Cell = exactly 1 physical byte (zero struct overhead)
//!     - Invariant Out-of-band Quad-States:
//!         * 243: NULL / HALT (Void / Unmapped memory)
//!         * 244: AWAIT (Superposition / Pending / Lock)
//!         * 245: ROLLBACK (Veto / Tainted / Unwound cell)
//!         * 246: PROCEED (Affirmative sealed landmark)
//!         * 247..255: Hardware Traps & System Landmarks

use std::fmt;
use crate::core::linguistics::{GrammaticalClassification, GrammaticalEntity, PartOfSpeech};
use crate::core::ternary::QuadState;


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Out-of-Band Constants, Trit Definition, and Cell Structures )
// ============================================================================

pub const BYTE_HALT: u8 = 243;
pub const BYTE_AWAIT: u8 = 244;
pub const BYTE_ROLLBACK: u8 = 245;
pub const BYTE_PROCEED: u8 = 246;
pub const BYTE_TRAP_BASE: u8 = 247;
pub const MAX_NATURAL_CELL_VALUE: u8 = 242;
pub const MIN_SIGNED_CELL_VALUE: i8 = -121;
pub const MAX_SIGNED_CELL_VALUE: i8 = 121;

/// The atomic ternary digit (Radix-3 balanced unit).
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum Trit {
    /// -1: Negative polarity / Contra (denoted 'T')
    Neg,
    ///  0: Neutral balance / Equilibrium (denoted '0')
    Zero,
    /// +1: Positive polarity / Affirmation (denoted '1')
    Pos,
}

impl Trit {
    pub fn as_i8(&self) -> i8 {
        match self {
            Trit::Neg => -1,
            Trit::Zero => 0,
            Trit::Pos => 1,
        }
    }

    pub fn from_i8(val: i8) -> Result<Self, String> {
        match val {
            -1 => Ok(Trit::Neg),
            0 => Ok(Trit::Zero),
            1 => Ok(Trit::Pos),
            other => Err(format!("FAILPOINT [InvalidTrit]: '{other}' is not in {{-1, 0, +1}}")),
        }
    }

    pub fn char_symbol(&self) -> char {
        match self {
            Trit::Neg => 'T',
            Trit::Zero => '0',
            Trit::Pos => '1',
        }
    }

    /// Algebraic negation of a trit: NOT(x) = -x.
```

```rust
    pub fn negate(&self) -> Self {
        match self {
            Trit::Neg => Trit::Pos,
            Trit::Zero => Trit::Zero,
            Trit::Pos => Trit::Neg,
        }
    }
}

impl fmt::Display for Trit {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{}", self.char_symbol())
    }
}

/// The atomic storage unit of memory in OmniCode.
///
/// Holds 5 balanced trits (spanning 243 values) alongside its Quad-State resonance.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub struct Cell {
    pub trits: [Trit; 5],
    pub state: QuadState,
    pub is_marker: bool,
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Mathematical Conversion, Packing, Unpacking, and Word Operations )
// ============================================================================

impl Cell {
    /// Constructs a Cell with explicit trits and Quad-State.
    pub fn new(trits: [Trit; 5], state: QuadState) -> Self {
        Self { trits, state, is_marker: false }
    }

    /// Creates an empty cell at equilibrium (0 in value, AWAIT in state).
    pub fn equilibrium() -> Self {
        Self {
            trits: [Trit::Zero; 5],
            state: QuadState::Await,
            is_marker: true,
        }
    }

    /// Creates a cell initialized with a pure out-of-band Quad-State marker.
    pub fn from_state(state: QuadState) -> Self {
        Self {
            trits: [Trit::Zero; 5],
            state,
            is_marker: true,
        }
    }

    /// Encodes a natural number in [0 ..= 242] into a 5-trit Cell with state PROCEED.
    pub fn from_natural(value: u8) -> Result<Self, String> {
        if value > MAX_NATURAL_CELL_VALUE {
            return Err(format!(
                "FAILPOINT [CellOverflow]: Value {value} exceeds single-cell capacity ({MAX_NATURAL_CELL_VALUE}). \
                 Use multi-cell Word chaining."
            ));
        }
        let signed_val = (value as i16) + (MIN_SIGNED_CELL_VALUE as i16);
        Self::from_signed(signed_val as i8)
    }

    /// Encodes a signed balanced integer in [-121 ..= +121] into 5 balanced trits.
    pub fn from_signed(value: i8) -> Result<Self, String> {
        if !(MIN_SIGNED_CELL_VALUE..=MAX_SIGNED_CELL_VALUE).contains(&value) {
            return Err(format!(
                "FAILPOINT [CellRangeError]: Signed value {value} out of range [{MIN_SIGNED_CELL_VALUE} ..=
{MAX_SIGNED_CELL_VALUE}]."
            ));
        }

        let mut x = value;
        let mut trits = [Trit::Zero; 5];
```

```
        for i in 0..5 {
            let rem = ((x % 3) + 3) % 3;
            match rem {
                0 => {
                    trits[i] = Trit::Zero;
                    x /= 3;
                }
                1 => {
                    trits[i] = Trit::Pos;
                    x = (x - 1) / 3;
                }
                2 => {
                    trits[i] = Trit::Neg;
                    x = (x + 1) / 3;
                }
                _ => unreachable!(),
            }
        }

        Ok(Self {
            trits,
            state: QuadState::Proceed,
            is_marker: false,
        })
    }

    /// Decodes the 5 trits back to a signed integer in [-121 ..= +121].
    pub fn to_signed(&self) -> Option<i8> {
        if self.is_marker || self.state == QuadState::Halt {
            return None;
        }

        let mut total: i32 = 0;
        let mut multiplier: i32 = 1;

        for trit in &self.trits {
            total += (trit.as_i8() as i32) * multiplier;
            multiplier *= 3;
        }

        Some(total as i8)
    }

    /// Decodes the 5 trits back to a natural number in [0 ..= 242].
    pub fn to_natural(&self) -> Option<u8> {
        self.to_signed().map(|s| (s as i16 - (MIN_SIGNED_CELL_VALUE as i16)) as u8)
    }

    /// Packs this Cell into a single physical 8-bit byte.
    pub fn pack(&self) -> u8 {
        if self.is_marker {
            match self.state {
                QuadState::Halt => BYTE_HALT,
                QuadState::Await => BYTE_AWAIT,
                QuadState::Rollback => BYTE_ROLLBACK,
                QuadState::Proceed => BYTE_PROCEED,
            }
        } else {
            self.to_natural().unwrap_or(BYTE_HALT)
        }
    }

    /// Unpacks a single physical 8-bit byte into a Cell with full 5-trit fidelity and Quad-State.
    pub fn unpack(byte: u8) -> Self {
        match byte {
            BYTE_HALT => Self::from_state(QuadState::Halt),
            BYTE_AWAIT => Self::from_state(QuadState::Await),
            BYTE_ROLLBACK => Self::from_state(QuadState::Rollback),
            BYTE_PROCEED => Self::from_state(QuadState::Proceed),
            trap if trap >= BYTE_TRAP_BASE => {
                // Reserved system trap / hardware landmark
                Self::from_state(QuadState::Halt)
            }
            natural => {
                Self::from_natural(natural).unwrap_or_else(|_| Self::from_state(QuadState::Halt))
            }
        }
```

```
    }
}

impl GrammaticalEntity for Cell {
    fn classify(&self) -> GrammaticalClassification {
        GrammaticalClassification::new(
            PartOfSpeech::Noun,
            Some(PartOfSpeech::Preposition),
            None,
            "Atomic memory vessel; stores 5 balanced trits and Quad-State resonance",
            "Single 8-bit memory byte carrying payload and state",
        )
    }
}

impl GrammaticalEntity for Trit {
    fn classify(&self) -> GrammaticalClassification {
        GrammaticalClassification::new(
            PartOfSpeech::Noun,
            Some(PartOfSpeech::Adjective),
            None,
            "Atomic ternary digit (-1, 0, +1)",
            "Single Radix-3 unit of computational polarity",
        )
    }
}

impl fmt::Display for Cell {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        if self.state == QuadState::Halt {
            write!(f, "Cell[NULL]")
        } else if let Some(nat) = self.to_natural() {
            write!(
                f,
                "Cell[Val:{nat} | {}{}{}{}{} | {:?}]",
                self.trits[4], self.trits[3], self.trits[2], self.trits[1], self.trits[0], self.state
            )
        } else {
            write!(f, "Cell[{:?}]", self.state)
        }
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Bijective Round-Trip Verification & Mathematical Unit Tests )
// ============================================================================
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_signed_round_trip_all_243_values() {
        for s in MIN_SIGNED_CELL_VALUE..=MAX_SIGNED_CELL_VALUE {
            let cell = Cell::from_signed(s).expect("Signed conversion must succeed");
            let recovered = cell.to_signed().expect("To signed must succeed");
            assert_eq!(s, recovered, "Failed round-trip for signed value {s}");
        }
    }

    #[test]
    fn test_natural_round_trip_all_243_values() {
        for n in 0..=MAX_NATURAL_CELL_VALUE {
            let cell = Cell::from_natural(n).expect("Natural conversion must succeed");
            let recovered = cell.to_natural().expect("To natural must succeed");
            assert_eq!(n, recovered, "Failed round-trip for natural number {n}");

            // Verify physical 8-bit byte pack and unpack
            let packed_byte = cell.pack();
            assert_eq!(packed_byte, n);
            let unpacked_cell = Cell::unpack(packed_byte);
            assert_eq!(unpacked_cell.to_natural(), Some(n));
            assert_eq!(unpacked_cell.state, QuadState::Proceed);
        }
    }

    #[test]
```

```
    fn test_out_of_band_quad_state_markers() {
        // Test HALT (NULL)
        let halt_cell = Cell::from_state(QuadState::Halt);
        let packed_halt = halt_cell.pack();
        assert_eq!(packed_halt, BYTE_HALT);
        let unpacked_halt = Cell::unpack(packed_halt);
        assert_eq!(unpacked_halt.state, QuadState::Halt);

        // Test AWAIT
        let await_cell = Cell::from_state(QuadState::Await);
        let packed_await = await_cell.pack();
        assert_eq!(packed_await, BYTE_AWAIT);
        let unpacked_await = Cell::unpack(packed_await);
        assert_eq!(unpacked_await.state, QuadState::Await);

        // Test ROLLBACK
        let rollback_cell = Cell::from_state(QuadState::Rollback);
        let packed_rollback = rollback_cell.pack();
        assert_eq!(packed_rollback, BYTE_ROLLBACK);
        let unpacked_rollback = Cell::unpack(packed_rollback);
        assert_eq!(unpacked_rollback.state, QuadState::Rollback);

        // Test PROCEED marker
        let proceed_cell = Cell::from_state(QuadState::Proceed);
        assert_eq!(proceed_cell.pack(), BYTE_PROCEED);
        let unpacked_proceed = Cell::unpack(BYTE_PROCEED);
        assert_eq!(unpacked_proceed.state, QuadState::Proceed);
    }

    #[test]
    fn test_zero_and_equilibrium_properties() {
        let zero_cell = Cell::from_signed(0).unwrap();
        assert_eq!(zero_cell.trits, [Trit::Zero; 5]);
        assert_eq!(zero_cell.to_signed(), Some(0));
        assert_eq!(zero_cell.to_natural(), Some(121)); // 121 is the balanced center of [0 .. 242]
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/core/filetype.rs RS 420 LINES 1554 WORDS 16.3 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( OmniCode File Type Subsystem Identity, Taxonomy & Invariants )
// ============================================================================
//! OmniCode File Type Taxonomy & Formal Extension System
//!
//! Classifies and handles all recognized OmniCode file artifacts:
//!   - `.omni`: Canonical OmniCode Source Reel (Primary executable program logic,
//!     modules, libraries, and ontological domain models).
//!   - `.scroll`: Cognitive Episodic Scroll (Homoiconic execution trace reel,
//!     recording prompts, synthesized programs, Restorator repairs, failpoints,
//!     and natural responses for AI continual learning).

use std::fmt;
use std::path::Path;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( File Type Enumeration, Mime Types, and Constructor Declarations )
// ============================================================================
/// Formal file classifications in the OmniCode ecosystem.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum OmniFileType {
    /// `.omni` - Canonical executable source reel.
    SourceReel,
    /// `.scroll` - Homoiconic episodic cognitive trace and AI learning reel.
    EpisodicScroll,
    /// `.tape` - Sequential execution process tape led by a Tape Head.
    Tape,
}

pub const ALL_FILE_TYPES: [OmniFileType; 3] = [
    OmniFileType::SourceReel,
    OmniFileType::EpisodicScroll,
    OmniFileType::Tape,
];

impl OmniFileType {
    pub const EXT_OMNI: &'static str = "omni";
    pub const EXT_SCROLL: &'static str = "scroll";
    pub const EXT_TAPE: &'static str = "tape";

    pub const ALL: &'static [OmniFileType] = &ALL_FILE_TYPES;

    /// Returns all supported OmniCode file types.
    pub fn all() -> &'static [OmniFileType] {
        Self::ALL
    }

    /// Detects file type from extension string (with or without leading dot).
    pub fn from_extension(ext: &str) -> Option<Self> {
        let clean = ext.trim().trim_start_matches('.').to_lowercase();
        match clean.as_str() {
            "omni" => Some(OmniFileType::SourceReel),
            "scroll" => Some(OmniFileType::EpisodicScroll),
            "tape" => Some(OmniFileType::Tape),
            _ => None,
        }
    }

    /// Detects file type from a file path.
    pub fn from_path(path: impl AsRef<Path>) -> Option<Self> {
        let p = path.as_ref();
        p.extension()
            .and_then(|ext| ext.to_str())
            .and_then(Self::from_extension)
    }

    /// Checks if a file path is a known OmniCode artifact.
    pub fn is_omnicode_file(path: impl AsRef<Path>) -> bool {
        Self::from_path(path).is_some()
    }
}

```

```
// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Metadata Queries, Scaffolding Templates & Formatting Properties )
// ================================================================================
impl OmniFileType {
    /// Canonical file extension without leading dot.
    pub fn extension(&self) -> &'static str {
        match self {
            OmniFileType::SourceReel => Self::EXT_OMNI,
            OmniFileType::EpisodicScroll => Self::EXT_SCROLL,
            OmniFileType::Tape => Self::EXT_TAPE,
        }
    }

    /// File extension with leading dot.
    pub fn dotted_extension(&self) -> &'static str {
        match self {
            OmniFileType::SourceReel => ".omni",
            OmniFileType::EpisodicScroll => ".scroll",
            OmniFileType::Tape => ".tape",
        }
    }

    /// Human-readable display title.
    pub fn display_name(&self) -> &'static str {
        match self {
            OmniFileType::SourceReel => "OmniCode Source Reel",
            OmniFileType::EpisodicScroll => "OmniCode Episodic Scroll",
            OmniFileType::Tape => "OmniCode Process Tape",
        }
    }

    /// VS Code / IDE language identifier.
    pub fn language_id(&self) -> &'static str {
        "omnicode"
    }

    /// Formal MIME type.
    pub fn mime_type(&self) -> &'static str {
        match self {
            OmniFileType::SourceReel => "text/x-omnicode",
            OmniFileType::EpisodicScroll => "text/x-omnicode-scroll",
            OmniFileType::Tape => "text/x-omnicode-tape",
        }
    }

    /// Short technical summary.
    pub fn description(&self) -> &'static str {
        match self {
            OmniFileType::SourceReel => {
                "Canonical 4-Block executable program module conforming to balanced ternary physics."
            }
            OmniFileType::EpisodicScroll => {
                "Homoiconic episodic execution reel recording cognitive discourse, repairs, and telemetry."
            }
            OmniFileType::Tape => {
                "Sequential process tape consisting of a leading Tape Head and subordinate rolls."
            }
        }
    }

    /// Generates canonical 4-Block scaffolding template for this file type.
    pub fn scaffold_template(&self, identifier: &str, intent_or_prompt: &str) -> String {
        match self {
            OmniFileType::SourceReel => {
                format!(
r#"# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( File Identity, Version, Author, Intent Aside, and Invariants )
# ================================================================================
Module < {name} > ( {intent} ) {{
    version: < "0.1.0" >,
    system: < OmniCode.Core >,
    safety: < MemorySafe :: Deterministic >
}}

# Invariant Axioms & Types
Type_{name} < +1 > ( Invariant definition )
```

```
153
154 # ================================================================================
155 # BLOCK 2: OPENING BLOCK
156 # ( Environment Setup, Preconditions, Scoped Context, and Declarations )
157 # ================================================================================
158 Opening {{
159     import: [ Core.Arithmetic :: Core.Logic :: Core.Vectors ],
160     preconditions: [ +1 ]
161 }}
162
163 # Initial state instance
164 state [ +1 ] ( Initialized state instance )
165
166 # ================================================================================
167 # BLOCK 3: BODY BLOCK
168 # ( Core Transformations, Causality Vectors, Logic, and Caveat Pipelines )
169 # ================================================================================
170 Body [
171     # Forward action: state acts upon transformation
172     state --> next_state [ +1 ]
173     next_state ! [ state = +1 ] ( Caveat verification )
174 ]
175
176 # ================================================================================
177 # BLOCK 4: CLOSING BLOCK
178 # ( Postconditions, Invariant Proofs, Teardown, and Commit Status )
179 # ================================================================================
180 Closing {{
181     verify: [ next_state = +1 ],
182     commit_status [ +1 ] ( Commit verified state )
183 }}
184 "#,
185                     name = identifier,
186                     intent = intent_or_prompt
187                 )
188             }
189             OmniFileType::EpisodicScroll => {
190                 format!(
191 r#"# ================================================================================
192 # BLOCK 1: METADATA BLOCK (THE COGNITIVE SCROLL)
193 # ( Execution Trace ID, Prompt Intent, Inferred Types, and Neural Snapshot )
194 # ================================================================================
195 Scroll < EpistemicTrace.{session} > ( {prompt} ) {{
196     version: < "1.0.0" >,
197     system: < OmniCode.AI.Model >,
198     timestamp: < "{timestamp}" >,
199     session_id: < "{session}" >,
200     cognitive_mode: < NeuralSymbolicTransduction >
201 }}
202
203 # ================================================================================
204 # BLOCK 2: OPENING BLOCK (EXPERIENCE INITIALIZATION)
205 # ( Hardware Invariants, Preconditions, and Initial Context )
206 # ================================================================================
207 Opening {{
208     import: [ Core.Arithmetic :: Core.Logic :: Core.Vectors ],
209     preconditions: [ +1 ]
210 }}
211
212 # Original Cognitive Discourse Prompt:
213 user_prompt < "{prompt}" >
214
215 # ================================================================================
216 # BLOCK 3: BODY BLOCK (CHRONICLE OF EXECUTION & RESTORATION)
217 # ( Replay Tape: Synthesized Program, Sensory Telemetry & Immune Interventions )
218 # ================================================================================
219 Body [
220     # Immune System Interventions (Restorator):
221     repair_entry [ L1: CleanExecution --> "Zero repairs required." ]
222
223     # Executed Turing Reel:
224     synthesized_reel [
225         state [ +1 ]
226     ]
227 ]
228
229 # ================================================================================
230 # BLOCK 4: CLOSING BLOCK (EPISODIC COMMIT & LEARNING PROOF)
```

```
# ( Verification Proof, Final Quad-State, and Conversational Synthesis )
# ================================================================================
Closing {{
    commit_status [ <PROCEED> ],
    machine_utterance ( "Execution verified to +1:PROCEED." )
}}
"#,
                    session = identifier,
                    prompt = intent_or_prompt,
                    timestamp = "0"
                )
            }
            OmniFileType::Tape => {
                format!(
r#"# ================================================================================
# TAPE HEAD (COVENANT DECLARATION & PROCESS MANIFEST)
# ================================================================================
[Tape.{name}] : WHO {{Process}} --> DISPATCH {{{name}}}
    WHERE {{partition}} = [ 256 ]
    WHEN {{Mount}}
    WHY ( {intent} )
    HOW [ +1 : PROCEED ]
    :: < covenant > ( verified )

# ================================================================================
# SUBORDINATE ROLLS (SEQUENTIAL EXECUTION STREAM)
# ================================================================================
roll_1 [ 10 --> Buf @ 0 ] ( Inscribe initial state into cell 0 )
roll_2 [ Buf[0] = 10 ! [ +1 ] ] ( Verify invariant bounds )
roll_3 [ commit_status [ PROCEED ] ] ( Tape execution completed successfully )
"#,
                    name = identifier,
                    intent = intent_or_prompt
                )
            }
        }
    }
}

/// Resolves a requested file path against the canonical 6-Track filesystem hierarchy.
///
/// If `path` exists directly as specified, it is returned immediately.
/// Otherwise, searches through the designated Track directories:
/// - `<TRACK 0> : {Silicon & Bare-Metal Kernel}`
/// - `<TRACK 1> : {Host Engine & Virtual Machine}`
/// - `<TRACK 2> : {OmniCode Language & Standard Library}`
/// - `<TRACK 3> : {Applications & OmniStudio Interfaces}`
/// - `<TRACK 4> : {Variable Storage & Episodic Memory}`
/// - `<TRACK 5> : {Infrastructure, Operations & Archives}`
pub fn resolve_track_path(path: impl AsRef<Path>) -> std::path::PathBuf {
    let p = path.as_ref();
    if p.exists() {
        return p.to_path_buf();
    }

    const TRACK_PREFIXES: [&str; 6] = [
        "<TRACK 2> : {OmniCode Language & Standard Library}",
        "<TRACK 3> : {Applications & OmniStudio Interfaces}",
        "<TRACK 0> : {Silicon & Bare-Metal Kernel}",
        "<TRACK 1> : {Host Engine & Virtual Machine}",
        "<TRACK 4> : {Variable Storage & Episodic Memory}",
        "<TRACK 5> : {Infrastructure, Operations & Archives}",
    ];

    for prefix in &TRACK_PREFIXES {
        let candidate = std::path::PathBuf::from(prefix).join(p);
        if candidate.exists() {
            return candidate;
        }
        let sibling = std::path::PathBuf::from("..").join(prefix).join(p);
        if sibling.exists() {
            return sibling;
        }
    }

    // Modularized type resolution: check compiled/ and interpreted/ folders
    let path_str = p.to_string_lossy();
    if path_str.contains("ideas/types/") {
```

```
        if let Some(file_name) = p.file_name().and_then(|f| f.to_str()) {
            let mod_p = if file_name.ends_with(".omni") {
                let stem = file_name.trim_end_matches(".omni");
                let old_s = format!("{file_name}");
                let new_s = format!("{stem}/compiled/{file_name}");
                Some(std::path::PathBuf::from(path_str.replace(&old_s, &new_s)))
            } else if file_name.ends_with(".tape") {
                let stem = file_name.trim_end_matches(".tape");
                let old_s = format!("{file_name}");
                let new_s = format!("{stem}/interpreted/{file_name}");
                Some(std::path::PathBuf::from(path_str.replace(&old_s, &new_s)))
            } else {
                None
            };

            if let Some(mp) = mod_p {
                for prefix in &TRACK_PREFIXES {
                    let candidate = std::path::PathBuf::from(prefix).join(&mp);
                    if candidate.exists() {
                        return candidate;
                    }
                    let sibling =
                        std::path::PathBuf::from("..").join(prefix).join(&mp);
                    if sibling.exists() {
                        return sibling;
                    }
                }
            }
        }
    }

    p.to_path_buf()
}

/// Resolves the canonical episodic memory scroll storage directory across the 6-Track filesystem.
pub fn resolve_scroll_storage_dir() -> std::path::PathBuf {
    let candidates = [
        std::path::PathBuf::from("../<TRACK 4> : {Variable Storage & Episodic Memory}/storage/scrolls"),
        std::path::PathBuf::from("<TRACK 4> : {Variable Storage & Episodic Memory}/storage/scrolls"),
        std::path::PathBuf::from("storage/scrolls"),
        std::path::PathBuf::from("runtime/scrolls"),
        std::path::PathBuf::from("scrolls"),
    ];

    for candidate in &candidates {
        if candidate.exists() {
            return candidate.clone();
        }
    }

    let track4_sibling = std::path::Path::new("../<TRACK 4> : {Variable Storage & Episodic Memory}/storage/scrolls");
    if let Some(parent) = track4_sibling.parent() {
        if parent.exists() {
            let _ = std::fs::create_dir_all(track4_sibling);
            return track4_sibling.to_path_buf();
        }
    }

    std::path::PathBuf::from("storage/scrolls")
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Display Trait Implementation, Sealing & Unit Tests )
// ============================================================================
impl fmt::Display for OmniFileType {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{}(.{})", self.display_name(), self.extension())
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_file_type_detection() {
        assert_eq!(OmniFileType::from_extension("omni"), Some(OmniFileType::SourceReel));
```

```
        assert_eq!(OmniFileType::from_extension(".omni"), Some(OmniFileType::SourceReel));
        assert_eq!(OmniFileType::from_extension("scroll"), Some(OmniFileType::EpisodicScroll));
        assert_eq!(OmniFileType::from_extension(".scroll"), Some(OmniFileType::EpisodicScroll));
        assert_eq!(OmniFileType::from_extension("tape"), Some(OmniFileType::Tape));
        assert_eq!(OmniFileType::from_extension(".tape"), Some(OmniFileType::Tape));
        assert_eq!(OmniFileType::from_extension("txt"), None);

        assert_eq!(OmniFileType::from_path("src/core.omni"), Some(OmniFileType::SourceReel));
        assert_eq!(OmniFileType::from_path("scrolls/test.scroll"), Some(OmniFileType::EpisodicScroll));
        assert_eq!(OmniFileType::from_path("lib/core/cell.tape"), Some(OmniFileType::Tape));
        assert!(OmniFileType::is_omnicode_file("lib/vector.omni"));
        assert!(OmniFileType::is_omnicode_file("history/session.scroll"));
        assert!(OmniFileType::is_omnicode_file("lib/core/cell.tape"));
        assert!(!OmniFileType::is_omnicode_file("Cargo.toml"));
    }

    #[test]
    fn test_scaffold_generation() {
        let omni_code = OmniFileType::SourceReel.scaffold_template("MathModule", "Basic math");
        assert!(omni_code.contains("Module < MathModule >"));
        assert!(omni_code.contains("BLOCK 1: METADATA BLOCK"));
        assert!(omni_code.contains("BLOCK 4: CLOSING BLOCK"));

        let scroll_code = OmniFileType::EpisodicScroll.scaffold_template("sess_1234", "Add 2 and 3");
        assert!(scroll_code.contains("Scroll < EpistemicTrace.sess_1234 >"));
        assert!(scroll_code.contains("commit_status [ <PROCEED> ]"));

        let tape_code = OmniFileType::Tape.scaffold_template("BootProcess", "Kernel Genesis");
        assert!(tape_code.contains("[Tape.BootProcess]"));
        assert!(tape_code.contains("TAPE HEAD"));
        assert!(tape_code.contains("SUBORDINATE ROLLS"));
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/core/linguistics.rs RS 1237 LINES 4208 WORDS 54.2 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Linguistic Engine Identity, 8 Parts of Speech, & Grammatical Classification )
// ================================================================================
//! OmniCode Core: Universal Linguistics & Grammatical Classification
//!
//! Maps every token, delimiter, vector, and quad-state directly into the
//! 8 canonical parts of speech in human natural language:
//!     1. Noun          (Entities, types, structures, literals, values)
//!     2. Pronoun       (Pointers, self-references, context handles)
//!     3. Verb          (Vectors, state transitions, operations, relations of being)
//!     4. Adjective     (Invariants, type qualifiers, contracts, quad-states)
//!     5. Adverb        (Execution manner, scheduling priority, negation)
//!     6. Preposition   (Indexing, spatial offsets, projections, scoping)
//!     7. Conjunction   (Connectives, boolean logic, pipeline caveats)
//!     8. Interjection   (Interrupts, traps, halts, parenthetical Asides)

use crate::core::boundaries::BoundaryType;
use crate::core::operators::{BeingRelation, Connective, DirectionalVector, Polarity};
use crate::core::ternary::QuadState;
use crate::parser::lexer::TokenType;


// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Grammatical Role Definitions, Classifications & Mapping Structures )
// ================================================================================

/// The canonical 8 parts of speech in natural human grammar.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum PartOfSpeech {
    Noun,
    Pronoun,
    Verb,
    Adjective,
    Adverb,
    Preposition,
    Conjunction,
    Interjection,
}

impl PartOfSpeech {
    pub fn as_str(&self) -> &'static str {
        match self {
            Self::Noun => "Noun",
            Self::Pronoun => "Pronoun",
            Self::Verb => "Verb",
            Self::Adjective => "Adjective",
            Self::Adverb => "Adverb",
            Self::Preposition => "Preposition",
            Self::Conjunction => "Conjunction",
            Self::Interjection => "Interjection",
        }
    }
}

/// A comprehensive grammatical classification supporting primary, secondary,
/// and tertiary functional roles across natural language and computation.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct GrammaticalClassification {
    /// The primary syntactic and semantic role in the sentence.
    pub primary: PartOfSpeech,
    /// The secondary role when functioning within different syntactic contexts.
    pub secondary: Option<PartOfSpeech>,
    /// The tertiary role for edge cases or multi-tier semantics.
    pub tertiary: Option<PartOfSpeech>,
    /// Natural language explanation of its linguistic function.
    pub linguistic_role: &'static str,
    /// Corresponding computer science / systems counterpart.
    pub computing_counterpart: &'static str,
}

impl GrammaticalClassification {
    pub fn new(
```

```
        primary: PartOfSpeech,
        secondary: Option<PartOfSpeech>,
        tertiary: Option<PartOfSpeech>,
        linguistic_role: &'static str,
        computing_counterpart: &'static str,
    ) -> Self {
        Self {
            primary,
            secondary,
            tertiary,
            linguistic_role,
            computing_counterpart,
        }
    }

    pub fn is(&self, pos: PartOfSpeech) -> bool {
        self.primary == pos || self.secondary == Some(pos) || self.tertiary == Some(pos)
    }
}

/// Grammatical case of a pronoun in sentence structure.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum PronounCase {
    /// Subjective / Nominative: Initiates action / holds execution frame (e.g., `I`, `we`, `you`, `he`, `she`, `it`, `they`).
    Subjective,
    /// Objective / Accusative: Recipient / target of action / inbound buffer (e.g., `me`, `us`, `him`, `her`, `them`).
    Objective,
    /// Reflexive / Introspective: Self-referential inspection / recursion (e.g., `myself`, `itself`, `themselves`).
    Reflexive,
    /// Possessive Determiner / Capability Token: Ownership / access rights (e.g., `my`, `your`, `his`, `her`, `its`, `our`, `their`).
    Possessive,
}

/// Person perspective of a pronoun.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum PronounPerson {
    /// First person: Active self, kernel engine, or cooperative covenant (`I`, `me`, `we`, `us`, `myself`).
    First,
    /// Second person: Interlocutor, caller, user, or client socket (`you`, `yourself`, `your`).
    Second,
    /// Third person: External entity, data vessel, antecedent, or peer thread (`it`, `he`, `she`, `they`).
    Third,
}

/// Grammatical number of a pronoun.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum PronounNumber {
    Singular,
    Plural,
}

/// The canonical spectrum of grammatical pronouns in OmniCode.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum PronounKind {
    // --- First Person Singular (Kernel / Self) ---
    I,
    Me,
    Myself,
    My,
    Mine,

    // --- First Person Plural (Covenant / Composite) ---
    We,
    Us,
    Ourselves,
    Our,
    Ours,

    // --- Second Person (Caller / User / Interlocutor) ---
    You,
    Yourself,
    Yourselves,
    Your,
    Yours,

    // --- Third Person Singular Animate (Autonomous Worker / Peer) ---
```

```
    He,
    Him,
    Himself,
    His,
    She,
    Her,
    Herself,
    Hers,

    // --- Third Person Singular Inanimate (Data Vessel / Antecedent / Accumulator) ---
    It,
    Itself,
    Its,

    // --- Third Person Plural (Worker Pool / Swarm / Quorum) ---
    They,
    Them,
    Themselves,
    Their,
    Theirs,

    // --- System Context Handles & Wildcards ---
    SelfHandle,
    CallerHandle,
    OwnerHandle,
    Wildcard,
}

impl PronounKind {
    pub fn as_str(&self) -> &'static str {
        match self {
            Self::I => "I",
            Self::Me => "me",
            Self::Myself => "myself",
            Self::My => "my",
            Self::Mine => "mine",
            Self::We => "we",
            Self::Us => "us",
            Self::Ourselves => "ourselves",
            Self::Our => "our",
            Self::Ours => "ours",
            Self::You => "you",
            Self::Yourself => "yourself",
            Self::Yourselves => "yourselves",
            Self::Your => "your",
            Self::Yours => "yours",
            Self::He => "he",
            Self::Him => "him",
            Self::Himself => "himself",
            Self::His => "his",
            Self::She => "she",
            Self::Her => "her",
            Self::Herself => "herself",
            Self::Hers => "hers",
            Self::It => "it",
            Self::Itself => "itself",
            Self::Its => "its",
            Self::They => "they",
            Self::Them => "them",
            Self::Themselves => "themselves",
            Self::Their => "their",
            Self::Theirs => "theirs",
            Self::SelfHandle => "self",
            Self::CallerHandle => "caller",
            Self::OwnerHandle => "owner",
            Self::Wildcard => "_",
        }
    }

    pub fn case(&self) -> PronounCase {
        match self {
            Self::I | Self::We | Self::You | Self::He | Self::She | Self::It | Self::They | Self::SelfHandle |
Self::CallerHandle | Self::OwnerHandle | Self::Wildcard => PronounCase::Subjective,
            Self::Me | Self::Us | Self::Him | Self::Her | Self::Them => PronounCase::Objective,
            Self::Myself | Self::Ourselves | Self::Yourself | Self::Yourselves | Self::Himself | Self::Herself |
Self::Itself | Self::Themselves => PronounCase::Reflexive,
            Self::My | Self::Mine | Self::Our | Self::Ours | Self::Your | Self::Yours | Self::His | Self::Hers |
Self::Its | Self::Their | Self::Theirs => PronounCase::Possessive,
```

```
        }
    }

    pub fn person(&self) -> PronounPerson {
        match self {
            Self::I | Self::Me | Self::Myself | Self::My | Self::Mine |
            Self::We | Self::Us | Self::Ourselves | Self::Our | Self::Ours |
            Self::SelfHandle => PronounPerson::First,

            Self::You | Self::Yourself | Self::Yourselves | Self::Your | Self::Yours |
            Self::CallerHandle => PronounPerson::Second,

            Self::He | Self::Him | Self::Himself | Self::His |
            Self::She | Self::Her | Self::Herself | Self::Hers |
            Self::It | Self::Itself | Self::Its |
            Self::They | Self::Them | Self::Themselves | Self::Their | Self::Theirs |
            Self::OwnerHandle | Self::Wildcard => PronounPerson::Third,
        }
    }

    pub fn number(&self) -> PronounNumber {
        match self {
            Self::We | Self::Us | Self::Ourselves | Self::Our | Self::Ours |
            Self::Yourselves |
            Self::They | Self::Them | Self::Themselves | Self::Their | Self::Theirs => PronounNumber::Plural,
            _ => PronounNumber::Singular,
        }
    }

    pub fn from_keyword(s: &str) -> Option<Self> {
        match s.to_lowercase().as_str() {
            "i" => Some(Self::I),
            "me" => Some(Self::Me),
            "myself" => Some(Self::Myself),
            "my" => Some(Self::My),
            "mine" => Some(Self::Mine),
            "we" => Some(Self::We),
            "us" => Some(Self::Us),
            "ourselves" => Some(Self::Ourselves),
            "our" => Some(Self::Our),
            "ours" => Some(Self::Ours),
            "you" => Some(Self::You),
            "yourself" => Some(Self::Yourself),
            "yourselves" => Some(Self::Yourselves),
            "your" => Some(Self::Your),
            "yours" => Some(Self::Yours),
            "he" => Some(Self::He),
            "him" => Some(Self::Him),
            "himself" => Some(Self::Himself),
            "his" => Some(Self::His),
            "she" => Some(Self::She),
            "her" => Some(Self::Her),
            "herself" => Some(Self::Herself),
            "hers" => Some(Self::Hers),
            "it" => Some(Self::It),
            "itself" => Some(Self::Itself),
            "its" => Some(Self::Its),
            "they" => Some(Self::They),
            "them" => Some(Self::Them),
            "themselves" => Some(Self::Themselves),
            "their" => Some(Self::Their),
            "theirs" => Some(Self::Theirs),
            "self" => Some(Self::SelfHandle),
            "caller" => Some(Self::CallerHandle),
            "owner" => Some(Self::OwnerHandle),
            "_" => Some(Self::Wildcard),
            _ => None,
        }
    }

    pub fn is_subjective(&self) -> bool {
        self.case() == PronounCase::Subjective
    }

    pub fn is_objective(&self) -> bool {
        self.case() == PronounCase::Objective
    }

```

```
    pub fn is_reflexive(&self) -> bool {
        self.case() == PronounCase::Reflexive
    }

    pub fn is_possessive(&self) -> bool {
        self.case() == PronounCase::Possessive
    }

    pub fn linguistic_role(&self) -> &'static str {
        match self {
            Self::I => "First-person singular subjective; represents the active executing identity / kernel",
            Self::Me => "First-person singular objective; represents self as the receiver of inbound action",
            Self::Myself => "First-person reflexive; drives introspective self-audit and invariant verification",
            Self::My | Self::Mine => "First-person possessive; denotes exclusive ownership and write capability",
            Self::We => "First-person plural subjective; denotes cooperative covenant (Kernel + User or Multi-Agent)",
            Self::Us => "First-person plural objective; denotes composite entity as target",
            Self::Ourselves => "First-person plural reflexive; drives collective group audit",
            Self::Our | Self::Ours => "First-person plural possessive; denotes shared covenant resource access",
            Self::You => "Second-person pronoun; identifies external interlocutor, user, or caller socket",
            Self::Yourself | Self::Yourselves => "Second-person reflexive; triggers remote caller verification",
            Self::Your | Self::Yours => "Second-person possessive; denotes borrowed caller capability token",
            Self::He | Self::She => "Third-person singular animate; designates specific peer worker or thread",
            Self::Him | Self::Her => "Third-person singular objective; identifies designated worker as target",
            Self::Himself | Self::Herself => "Third-person singular reflexive; isolates worker-internal state",
            Self::His | Self::Hers => "Third-person singular possessive; denotes worker-delegated capability",
            Self::It => "Third-person singular inanimate; represents data vessel, antecedent, or accumulator",
            Self::Itself => "Third-person reflexive; drives homoiconic tape rewriting or fixed-point recursion",
            Self::Its => "Third-person possessive; denotes vessel-internal property encapsulation",
            Self::They => "Third-person plural subjective; identifies worker pool, quorum, or parallel swarm",
            Self::Them => "Third-person plural objective; identifies worker pool as destination targets",
            Self::Themselves => "Third-person plural reflexive; verifies Byzantine swarm agreement or consensus",
            Self::Their | Self::Theirs => "Third-person plural possessive; denotes process-group capability mask",
            Self::SelfHandle => "Architectural local frame handle; resolves active stack/partition base",
            Self::CallerHandle => "Architectural caller handle; resolves invoking principal and frame return",
            Self::OwnerHandle => "Architectural owner handle; resolves root capability authority",
            Self::Wildcard => "Anonymous discard handle; suppresses destination binding or ignores noise",
        }
    }

    pub fn computing_counterpart(&self) -> &'static str {
        match self {
            Self::I => "Active Task PID / Execution Context Register / `&mut self`",
            Self::Me => "Destination Buffer in Self / Callee Register Inflow",
            Self::Myself => "Self-Audit Telemetry / Local Frame Invariant Check / Introspection",
            Self::My | Self::Mine => "Exclusive Memory Partition Lease / Write Capability Token",
            Self::We => "Composite Covenant Task / Dual-Principal Session / Multi-Agent Swarm",
            Self::Us => "Composite Destination Queue / Shared Inbound Buffer",
            Self::Ourselves => "Group Invariant Consensus Check / Quorum Self-Verification",
            Self::Our | Self::Ours => "Shared Memory Partition Capability / Covenant Lock",
            Self::You => "Caller Context Handle / `STDIN` Channel / Remote IPC Socket",
            Self::Yourself | Self::Yourselves => "Caller Self-Test RPC / Remote Health Probe",
            Self::Your | Self::Yours => "Borrowed Caller Capability Token / Foreign Slice Reference",
            Self::He | Self::She => "Designated Worker Thread PID / Specific Actor Descriptor",
            Self::Him | Self::Her => "Worker Inflow Channel / Task Mailbox",
            Self::Himself | Self::Herself => "Worker Local State Isolation / Actor Invariant Seal",
            Self::His | Self::Hers => "Worker Capability Delegation / Sub-Partition Lease",
            Self::It => "Pipeline Antecedent / Accumulator Register (`ACC`) / Top of Stack",
            Self::Itself => "Homoiconic Tape Rewriter / Circular Reel Wrap / Fixed-Point Recursion",
            Self::Its => "Vessel Field Encapsulation / Internal Property Offset",
            Self::They => "Process Group Descriptor / Parallel Thread Pool / Consensus Quorum",
            Self::Them => "Broadcast Channel / Multi-Partition Destination Slices",
            Self::Themselves => "Swarm Peer-to-Peer Consensus Alignment / Byzantine Quorum Proof",
            Self::Their | Self::Theirs => "Process Group Access Control List / Collective Capability Mask",
            Self::SelfHandle => "Frame Pointer Register (`FP`) / Local Execution Fiber",
            Self::CallerHandle => "Return Address / Caller Task Descriptor (`CALLER`)",
            Self::OwnerHandle => "Capability Authority Root / Security Principal Token",
            Self::Wildcard => "Bit Bucket (`/dev/null`) / Black Hole Register (`_`)",
        }
    }
}

impl GrammaticalEntity for PronounKind {
    fn classify(&self) -> GrammaticalClassification {
        let sec = match self.case() {
            PronounCase::Possessive => Some(PartOfSpeech::Adjective),
            PronounCase::Reflexive => Some(PartOfSpeech::Verb),
            PronounCase::Objective => Some(PartOfSpeech::Noun),
            PronounCase::Subjective => Some(PartOfSpeech::Noun),
```

```
        };
        GrammaticalClassification::new(
            PartOfSpeech::Pronoun,
            sec,
            None,
            self.linguistic_role(),
            self.computing_counterpart(),
        )
    }
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Trait Implementations & Grammatical Classification Mappings )
// ============================================================================

pub trait GrammaticalEntity {
    fn classify(&self) -> GrammaticalClassification;
}

impl GrammaticalEntity for TokenType {
    fn classify(&self) -> GrammaticalClassification {
        match self {
            // Delimiters
            TokenType::LBracket | TokenType::RBracket => GrammaticalClassification::new(
                PartOfSpeech::Noun,
                Some(PartOfSpeech::Preposition),
                None,
                "Denotation boundary container; spatially indexes inner state",
                "Array / Struct container; memory offset indexing `buf[idx]`",
            ),
            TokenType::LBrace | TokenType::RBrace => GrammaticalClassification::new(
                PartOfSpeech::Adjective,
                Some(PartOfSpeech::Conjunction),
                None,
                "Invariant contract; specifies necessary ambient conditions",
                "Type invariant / Precondition constraint / Scope qualification",
            ),
            TokenType::LAngle | TokenType::RAngle => GrammaticalClassification::new(
                PartOfSpeech::Adjective,
                Some(PartOfSpeech::Noun),
                None,
                "Immutable axiom boundary; names high-order categorical domains",
                "Type parameter / Scale invariant / Module domain constant",
            ),
            TokenType::Aside(_) => GrammaticalClassification::new(
                PartOfSpeech::Interjection,
                Some(PartOfSpeech::Adverb),
                None,
                "First-class teleological Aside; states machine intent and rationale",
                "Non-executable intent annotation / Compiler reasoning trace",
            ),

            // Grammatical Pronouns (Perspective, Identity, Ownership & Capability)
            TokenType::Pronoun(p) => p.classify(),

            // Polarity & Boundary Cast Operators
            TokenType::AffixedIdent { polarity, .. } => match polarity {
                Polarity::Abstract => GrammaticalClassification::new(
                    PartOfSpeech::Adjective,
                    Some(PartOfSpeech::Noun),
                    None,
                    "Abstract archetype polarity modifier (+Word); denotes universal concept or intension",
                    "Uninstantiated archetype / Type specification / Zero-cost blueprint",
                ),
                Polarity::Concrete => GrammaticalClassification::new(
                    PartOfSpeech::Noun,
                    Some(PartOfSpeech::Adjective),
                    None,
                    "Concrete substance polarity modifier (-Word / Word-); denotes physical instance",
                    "Physical memory allocation / Concrete hardware register footprint",
                ),
            },
            TokenType::BoundaryCast { .. } => GrammaticalClassification::new(
                PartOfSpeech::Preposition,
                Some(PartOfSpeech::Verb),
                None,
```

```
                "Boundary casting operator; modulates entity essence across boundary rank",
                "Type cast / Transduction between abstract and concrete representations",
            ),

            // Directional Vectors (Verbs of Action & Causation)
            TokenType::ActUpon => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Forward causation; acts upon target entity",
                "Store / Mutation / Pipeline emission / MOV",
            ),
            TokenType::ActAgain => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Retroactive reflection; draws from or reflects back upon source",
                "Load / Feedback / State rollback inspection",
            ),
            TokenType::BuildsTo => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Ascending action; synthesizes lower primitive into higher concept",
                "Type promotion / Emergence / Layering up to abstraction",
            ),
            TokenType::BreaksDownFrom => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Descending action; decomposes abstract concept into primitives",
                "Destructuring / Register unpack / Compilation lowering",
            ),
            TokenType::Major => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Adjective),
                None,
                "Major scale vector; ascending forward diagonal leap",
                "Major scale promotion / Leap to higher abstraction tier (`/-^`)",
            ),
            TokenType::Minor => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Adjective),
                None,
                "Minor scale vector; descending forward diagonal step",
                "Minor scale reduction / Settling to lower execution tier (`\\-v`)",
            ),
            TokenType::Great => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Adjective),
                None,
                "Great scale vector; ascending backward diagonal reflection",
                "Great scale elevation / Architectural reflection (`\\-^`)",
            ),
            TokenType::Small => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Adjective),
                None,
                "Small scale vector; descending backward diagonal micro-adjustment",
                "Small scale reduction / Micro-destructuring (`/-v`)",
            ),
            TokenType::UpperRight => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Quadrant I compass vector; forward ascending northeast movement",
                "2D Quadrant I vector: +X, +Y (`/^`)",
            ),
            TokenType::LowerRight => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Quadrant IV compass vector; forward descending southeast movement",
                "2D Quadrant IV vector: +X, -Y (`\\v`)",
            ),
            TokenType::UpperLeft => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
```

```
                None,
                "Quadrant II compass vector; backward ascending northwest movement",
                "2D Quadrant II vector: -X, +Y (`^\\`)",
            ),
            TokenType::LowerLeft => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Quadrant III compass vector; backward descending southwest movement",
                "2D Quadrant III vector: -X, -Y (`v/`)",
            ),

            // Relations of Being (Verbs of Identity & State)
            TokenType::Is => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Copula of being; asserts exact equivalence and identity",
                "Equality assertion (`==`) / Identity binding",
            ),
            TokenType::IsNot => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Negative copula; asserts distinctness and exclusion",
                "Inequality assertion (`!=`)",
            ),
            TokenType::ToBeNot => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "State copula; asserts negative state predicate (=!)",
                "State negation predicate (`=!`)",
            ),
            TokenType::ToBeLike => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Adjective),
                None,
                "Analogy copula; asserts structural likeness or homomorphism",
                "Structural compatibility / Duck typing / Approximate equivalence",
            ),
            TokenType::InvariantEquivalence => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Invariant copula; immutable balance, identity, and eternal equilibrium (<=>)",
                "Invariant to be / Axiomatic Equivalence (`<=>`)",
            ),
            TokenType::LessThan => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Relational ordering; strictly less than within context bounds (<)",
                "Contextual less than comparison (`[ { A } < { B } ]`)",
            ),
            TokenType::GreaterThan => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Relational ordering; strictly greater than within context bounds (>)",
                "Contextual greater than comparison (`[ { A } > { B } ]`)",
            ),
            TokenType::LessEqual => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Relational ordering; less than or equal within context bounds (<=)",
                "Contextual less than or equal comparison (`[ { A } <= { B } ]`)",
            ),
            TokenType::GreaterEqual => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Relational ordering; greater than or equal within context bounds (>=)",
                "Contextual greater than or equal comparison (`[ { A } >= { B } ]`)",
            ),

            // Connectives
```

```
            TokenType::But => GrammaticalClassification::new(
                PartOfSpeech::Conjunction,
                Some(PartOfSpeech::Interjection),
                None,
                "Adversative conjunction; introduces strict caveat condition",
                "Caveat guard; triggers `-1: ROLLBACK` if caveat fails",
            ),
            TokenType::And => GrammaticalClassification::new(
                PartOfSpeech::Conjunction,
                Some(PartOfSpeech::Preposition),
                None,
                "Cumulative conjunction; asserts simultaneous co-presence",
                "Conjunction (`&&`) / Simultaneous consensus",
            ),
            TokenType::HardwareLocator => GrammaticalClassification::new(
                PartOfSpeech::Preposition,
                Some(PartOfSpeech::Conjunction),
                None,
                "Dedicated hardware locator (::@); binds entity co-presence directly to MMIO address or port",
                "MMIO port locator / Hardware address binding (`Glyph & Cell ::@ Port`)",
            ),
            TokenType::InRelationTo => GrammaticalClassification::new(
                PartOfSpeech::Preposition,
                Some(PartOfSpeech::Verb),
                None,
                "Relational projection; positions entity in relation to a category",
                "Type annotation (`x : Type`) / Contextual role assignment",
            ),
            TokenType::Or => GrammaticalClassification::new(
                PartOfSpeech::Conjunction,
                Some(PartOfSpeech::Adverb),
                None,
                "Disjunctive conjunction; introduces alternatives or choices",
                "Disjunction (`||`) / Non-deterministic branch",
            ),
            TokenType::Not => GrammaticalClassification::new(
                PartOfSpeech::Adverb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Negating adverb; inverts the state or predicate across balance line",
                "Logical / Algebraic negation",
            ),
            TokenType::BoundPair => GrammaticalClassification::new(
                PartOfSpeech::Conjunction,
                Some(PartOfSpeech::Noun),
                None,
                "Hand-in-hand inseparable dual binding; binds entities so neither can be referenced without the other",
                "Inseparable atomic pair / Dual-rail entangled binding (`A&B`)",
            ),

            // Prepositions & Spatial Locators
            TokenType::At => GrammaticalClassification::new(
                PartOfSpeech::Preposition,
                Some(PartOfSpeech::Noun),
                None,
                "Spatial preposition specifying absolute or relative location, address, or memory site",
                "Memory address / MMIO locator / Absolute site (`Storehouse @ Offset`)",
            ),

            // Measures & Proportions
            TokenType::Percent => GrammaticalClassification::new(
                PartOfSpeech::Adjective,
                Some(PartOfSpeech::Noun),
                None,
                "Proportional measure specifying capacity fraction, ratio, or utilization percentage",
                "Capacity fullness ratio / Percentage scale (`Capacity % 80`)",
            ),

            // Ranges
            TokenType::DotDot => GrammaticalClassification::new(
                PartOfSpeech::Preposition,
                Some(PartOfSpeech::Noun),
                None,
                "Range span preposition; defines closed interval bounds between coordinates",
                "Coordinate interval / Memory span `[0 .. 256]`",
            ),

            // Cardinal Classification Clauses (The 5 W's and 1 H)
```

```
            TokenType::Who => GrammaticalClassification::new(
                PartOfSpeech::Pronoun,
                Some(PartOfSpeech::Noun),
                None,
                "Agent pronoun clause; identifies actor, authority, caller, or process identity",
                "Process owner / Security principal / Caller identity (`WHO`)",
            ),
            TokenType::What => GrammaticalClassification::new(
                PartOfSpeech::Noun,
                Some(PartOfSpeech::Pronoun),
                None,
                "Substance noun clause; identifies entity, payload, data vessel, or type",
                "Data vessel / Entity payload / Operand specification (`WHAT`)",
            ),
            TokenType::When => GrammaticalClassification::new(
                PartOfSpeech::Adverb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Temporal adverb clause; identifies execution cadence, lifecycle block, or trigger",
                "Execution phase / Scheduler quantum / Event trigger (`WHEN`)",
            ),
            TokenType::Where => GrammaticalClassification::new(
                PartOfSpeech::Preposition,
                Some(PartOfSpeech::Conjunction),
                None,
                "Spatial preposition clause; identifies memory location, address, scope, or invariant bounds",
                "Spatial address / MMIO site / Invariant scope (`WHERE`)",
            ),
            TokenType::Why => GrammaticalClassification::new(
                PartOfSpeech::Interjection,
                Some(PartOfSpeech::Adjective),
                None,
                "Teleological inquiry clause; specifies rationale, contractual reason, or axiom",
                "Teleological rationale / Contractual reason / Axiom link (`WHY`)",
            ),
            TokenType::How => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Adverb),
                None,
                "Causal mechanism clause; specifies vector method, transducer, or execution policy",
                "Causal mechanism / Vector method / Transducer (`HOW`)",
            ),

            // Natural Arithmetic Keywords
            TokenType::Plus => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Action of addition; unites quantities",
                "Arithmetic addition opcode",
            ),
            TokenType::Minus => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Action of subtraction; separates quantities",
                "Arithmetic subtraction opcode",
            ),
            TokenType::Times => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Action of multiplication; scales quantities",
                "Arithmetic multiplication opcode",
            ),
            TokenType::Div => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Action of division; partitions quantities",
                "Arithmetic division opcode",
            ),

            // Infer
            TokenType::Infer => GrammaticalClassification::new(
                PartOfSpeech::Interjection,
                Some(PartOfSpeech::Verb),
                None,
```

```
                "Interrogative inquiry; seeks resolution or truth state",
                "Pattern match query / Ontological inference operator",
            ),

            // Literals
            TokenType::Identifier(_) => GrammaticalClassification::new(
                PartOfSpeech::Noun,
                Some(PartOfSpeech::Pronoun),
                None,
                "Named subject, object, or entity in discourse",
                "Variable name / Symbol / Type identifier",
            ),
            TokenType::Number(_) => GrammaticalClassification::new(
                PartOfSpeech::Noun,
                Some(PartOfSpeech::Adjective),
                None,
                "Numeric scalar value; quantifies entities",
                "Immediate scalar numeric literal",
            ),
            TokenType::StringLit(_) => GrammaticalClassification::new(
                PartOfSpeech::Noun,
                None,
                None,
                "Textual noun; semantic content literal",
                "String literal / Byte payload",
            ),
            TokenType::CharLit(_) => GrammaticalClassification::new(
                PartOfSpeech::Noun,
                None,
                None,
                "Atomic character glyph rune literal ('A'); discrete single-cell textual scalar",
                "Character literal / 5-trit scalar byte (`'A' <=> 65`)",
            ),
            TokenType::Ternary(state) => state.classify(),

            TokenType::Eof => GrammaticalClassification::new(
                PartOfSpeech::Interjection,
                None,
                None,
                "Terminal punctuation sealing the discourse",
                "End of file / Stream termination",
            ),
        }
    }
}

impl GrammaticalEntity for QuadState {
    fn classify(&self) -> GrammaticalClassification {
        match self {
            QuadState::Proceed => GrammaticalClassification::new(
                PartOfSpeech::Adjective,
                Some(PartOfSpeech::Noun),
                Some(PartOfSpeech::Interjection),
                "Affirmative state of truth, covenant alignment, and forward momentum",
                "Logic +1 / Monotonic commit / Success",
            ),
            QuadState::Await => GrammaticalClassification::new(
                PartOfSpeech::Adjective,
                Some(PartOfSpeech::Verb),
                None,
                "Superposition state; yielding in wait for consensus",
                "Logic 0 / Cooperative schedule yield / Non-blocking pending",
            ),
            QuadState::Rollback => GrammaticalClassification::new(
                PartOfSpeech::Interjection,
                Some(PartOfSpeech::Verb),
                Some(PartOfSpeech::Adjective),
                "Exclamation of veto; immediate transactional rewind",
                "Logic -1 / Transaction abort and rollback",
            ),
            QuadState::Halt => GrammaticalClassification::new(
                PartOfSpeech::Interjection,
                Some(PartOfSpeech::Noun),
                None,
                "Terminal trap of void and non-existence; complete cessation",
                "Logic NULL / Fatal CPU trap / Irreducible collapse",
            ),
        }
```

```
    }
}

impl GrammaticalEntity for BoundaryType {
    fn classify(&self) -> GrammaticalClassification {
        match self {
            BoundaryType::Denotation => GrammaticalClassification::new(
                PartOfSpeech::Noun,
                Some(PartOfSpeech::Preposition),
                None,
                "Concrete essence and mutable data literal",
                "Data structure container / Mutable memory slice",
            ),
            BoundaryType::Invariant => GrammaticalClassification::new(
                PartOfSpeech::Adjective,
                Some(PartOfSpeech::Noun),
                None,
                "Immutable axiom and universal physical constant",
                "Constant literal / Read-only memory / Axiomatic constraint",
            ),
            BoundaryType::Connotation => GrammaticalClassification::new(
                PartOfSpeech::Adjective,
                Some(PartOfSpeech::Conjunction),
                None,
                "Ambient context, required capabilities, and environmental predicates",
                "Precondition contract / Ambient capabilities / Context scope",
            ),
            BoundaryType::TopDown => GrammaticalClassification::new(
                PartOfSpeech::Preposition,
                Some(PartOfSpeech::Adjective),
                None,
                "Outside-in conditioning: the environment dictates entity essence",
                "Hardware MMIO window / Environmental dependency injection",
            ),
            BoundaryType::BottomUp => GrammaticalClassification::new(
                PartOfSpeech::Preposition,
                Some(PartOfSpeech::Verb),
                None,
                "Inside-out emergence: entity essence projects into environment",
                "Polymorphic emission / Register export to ABI",
            ),
            BoundaryType::Aside => GrammaticalClassification::new(
                PartOfSpeech::Interjection,
                Some(PartOfSpeech::Adverb),
                None,
                "Aside to; teleological intent container preserving reasoning trace",
                "First-class intent container / AI reasoning trace",
            ),
        }
    }
}

impl GrammaticalEntity for DirectionalVector {
    fn classify(&self) -> GrammaticalClassification {
        match self {
            DirectionalVector::ActUpon => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Forward causation; acts upon target",
                "Forward pipeline dispatch / Store operation",
            ),
            DirectionalVector::ActAgain => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Retroactive reflection; draws from or re-evaluates source",
                "Feedback loop / Rollback inspection / Load operation",
            ),
            DirectionalVector::BuildsTo => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Ascending synthesis; primitives build up to abstraction",
                "Type promotion / Abstraction synthesis",
            ),
            DirectionalVector::BreaksDownFrom => GrammaticalClassification::new(
                PartOfSpeech::Verb,
```

```
                Some(PartOfSpeech::Preposition),
                None,
                "Descending decomposition; concept lowers into hardware primitives",
                "Destructuring / Register lowering",
            ),
            DirectionalVector::Major => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Adjective),
                None,
                "Major scale vector; ascending forward diagonal leap",
                "Major scale promotion (`/-^`)",
            ),
            DirectionalVector::Minor => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Adjective),
                None,
                "Minor scale vector; descending forward diagonal step",
                "Minor scale reduction (`\\-v`)",
            ),
            DirectionalVector::Great => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Adjective),
                None,
                "Great scale vector; ascending backward diagonal reflection",
                "Great scale elevation (`\\-^`)",
            ),
            DirectionalVector::Small => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Adjective),
                None,
                "Small scale vector; descending backward diagonal micro-adjustment",
                "Small scale reduction (`/-v`)",
            ),
            DirectionalVector::UpperRight => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Quadrant I compass vector; forward ascending movement",
                "2D Quadrant I vector: +X, +Y (`/^`)",
            ),
            DirectionalVector::LowerRight => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Quadrant IV compass vector; forward descending movement",
                "2D Quadrant IV vector: +X, -Y (`\\v`)",
            ),
            DirectionalVector::UpperLeft => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Quadrant II compass vector; backward ascending movement",
                "2D Quadrant II vector: -X, +Y (`^\\`)",
            ),
            DirectionalVector::LowerLeft => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Preposition),
                None,
                "Quadrant III compass vector; backward descending movement",
                "2D Quadrant III vector: -X, -Y (`v/`)",
            ),
        }
    }
}

impl GrammaticalEntity for BeingRelation {
    fn classify(&self) -> GrammaticalClassification {
        match self {
            BeingRelation::Is => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Identity copula: exact equivalence of essence",
                "Equality assertion / Equational binding",
            ),
            BeingRelation::IsNot => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
```

```
                None,
                "Negative copula: distinctness and mutual exclusion",
                "Inequality assertion",
            ),
            BeingRelation::ToBeNot => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "State copula: asserts negative state predicate (=!)",
                "State negation predicate",
            ),
            BeingRelation::ToBeLike => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Adjective),
                None,
                "Analogy copula: structural or behavioral compatibility",
                "Homomorphism / Approximate equality",
            ),
            BeingRelation::Equivalence => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Invariant copula: immutable balance, identity, and eternal equilibrium (<=>)",
                "Invariant to be / Axiomatic Equivalence",
            ),
            BeingRelation::LessThan => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Relational ordering: strictly less than within context bounds (<)",
                "Less than relational query",
            ),
            BeingRelation::GreaterThan => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Relational ordering: strictly greater than within context bounds (>)",
                "Greater than relational query",
            ),
            BeingRelation::LessEqual => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Relational ordering: less than or equal within context bounds (<=)",
                "Less than or equal relational query",
            ),
            BeingRelation::GreaterEqual => GrammaticalClassification::new(
                PartOfSpeech::Verb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Relational ordering: greater than or equal within context bounds (>=)",
                "Greater than or equal relational query",
            ),
        }
    }
}

impl GrammaticalEntity for Connective {
    fn classify(&self) -> GrammaticalClassification {
        match self {
            Connective::But => GrammaticalClassification::new(
                PartOfSpeech::Conjunction,
                Some(PartOfSpeech::Interjection),
                None,
                "Adversative conjunction: caveat requirement that vetoes on failure",
                "Guard condition with automatic rollback",
            ),
            Connective::And => GrammaticalClassification::new(
                PartOfSpeech::Conjunction,
                Some(PartOfSpeech::Preposition),
                None,
                "Cumulative conjunction: co-present simultaneous execution",
                "Conjunction (`&&`) / Joint consensus",
            ),
            Connective::InRelationTo => GrammaticalClassification::new(
                PartOfSpeech::Preposition,
                Some(PartOfSpeech::Verb),
                None,
```

```
                "Relational projection: positions an entity within a role or type",
                "Type annotation (`:`) / Role assignment",
            ),
            Connective::Or => GrammaticalClassification::new(
                PartOfSpeech::Conjunction,
                Some(PartOfSpeech::Adverb),
                None,
                "Disjunctive conjunction: alternative path or branching",
                "Disjunction (`||`)",
            ),
            Connective::Not => GrammaticalClassification::new(
                PartOfSpeech::Adverb,
                Some(PartOfSpeech::Conjunction),
                None,
                "Negating adverb: inverts the evaluation balance",
                "Algebraic negation (`--`)",
            ),
        }
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Linguistic Unit Tests & Invariant Verification )
// ============================================================================
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_eight_parts_of_speech_coverage() {
        let act_upon = TokenType::ActUpon.classify();
        assert_eq!(act_upon.primary, PartOfSpeech::Verb);
        assert_eq!(act_upon.secondary, Some(PartOfSpeech::Preposition));

        let but = TokenType::But.classify();
        assert_eq!(but.primary, PartOfSpeech::Conjunction);
        assert_eq!(but.secondary, Some(PartOfSpeech::Interjection));

        let bracket = TokenType::LBracket.classify();
        assert_eq!(bracket.primary, PartOfSpeech::Noun);
        assert_eq!(bracket.secondary, Some(PartOfSpeech::Preposition));

        let brace = TokenType::LBrace.classify();
        assert_eq!(brace.primary, PartOfSpeech::Adjective);
        assert_eq!(brace.secondary, Some(PartOfSpeech::Conjunction));

        let aside = TokenType::Aside("intent".to_string()).classify();
        assert_eq!(aside.primary, PartOfSpeech::Interjection);
        assert_eq!(aside.secondary, Some(PartOfSpeech::Adverb));

        let not = TokenType::Not.classify();
        assert_eq!(not.primary, PartOfSpeech::Adverb);
        assert_eq!(not.secondary, Some(PartOfSpeech::Conjunction));

        let relation = TokenType::InRelationTo.classify();
        assert_eq!(relation.primary, PartOfSpeech::Preposition);
        assert_eq!(relation.secondary, Some(PartOfSpeech::Verb));

        let bound = TokenType::BoundPair.classify();
        assert_eq!(bound.primary, PartOfSpeech::Conjunction);
        assert_eq!(bound.secondary, Some(PartOfSpeech::Noun));

        let at = TokenType::At.classify();
        assert_eq!(at.primary, PartOfSpeech::Preposition);
        assert_eq!(at.secondary, Some(PartOfSpeech::Noun));

        let pct = TokenType::Percent.classify();
        assert_eq!(pct.primary, PartOfSpeech::Adjective);
        assert_eq!(pct.secondary, Some(PartOfSpeech::Noun));
    }

    #[test]
    fn test_quad_state_linguistic_classifications() {
        let proceed = QuadState::Proceed.classify();
        assert_eq!(proceed.primary, PartOfSpeech::Adjective);
        assert!(proceed.is(PartOfSpeech::Interjection));
```

```
        let rollback = QuadState::Rollback.classify();
        assert_eq!(rollback.primary, PartOfSpeech::Interjection);
        assert!(rollback.is(PartOfSpeech::Verb));
    }

    #[test]
    fn test_pronoun_matrix_and_capability_classifications() {
        // 1. Subjective (Agency / Execution context)
        let p_i = PronounKind::I;
        assert_eq!(p_i.case(), PronounCase::Subjective);
        assert_eq!(p_i.person(), PronounPerson::First);
        assert_eq!(p_i.number(), PronounNumber::Singular);
        assert!(p_i.is_subjective());
        let class_i = p_i.classify();
        assert_eq!(class_i.primary, PartOfSpeech::Pronoun);
        assert_eq!(class_i.secondary, Some(PartOfSpeech::Noun));
        assert!(class_i.computing_counterpart.contains("&mut self"));

        // 2. Objective (Inbound target)
        let p_me = PronounKind::Me;
        assert_eq!(p_me.case(), PronounCase::Objective);
        assert_eq!(p_me.person(), PronounPerson::First);
        assert!(p_me.is_objective());

        // 3. Reflexive (Introspection & self-audit)
        let p_myself = PronounKind::Myself;
        assert_eq!(p_myself.case(), PronounCase::Reflexive);
        assert!(p_myself.is_reflexive());
        let class_myself = p_myself.classify();
        assert_eq!(class_myself.secondary, Some(PartOfSpeech::Verb));
        assert!(class_myself.computing_counterpart.contains("Self-Audit"));

        let p_itself = PronounKind::Itself;
        assert_eq!(p_itself.case(), PronounCase::Reflexive);
        assert_eq!(p_itself.person(), PronounPerson::Third);
        let class_itself = p_itself.classify();
        assert!(class_itself.computing_counterpart.contains("Homoiconic"));

        let p_themselves = PronounKind::Themselves;
        assert_eq!(p_themselves.case(), PronounCase::Reflexive);
        assert_eq!(p_themselves.number(), PronounNumber::Plural);
        assert!(p_themselves.classify().computing_counterpart.contains("Consensus"));

        // 4. Possessive (Ownership & Capability tokens)
        let p_my = PronounKind::My;
        assert_eq!(p_my.case(), PronounCase::Possessive);
        assert!(p_my.is_possessive());
        let class_my = p_my.classify();
        assert_eq!(class_my.secondary, Some(PartOfSpeech::Adjective));
        assert!(class_my.computing_counterpart.contains("Capability Token"));

        let p_their = PronounKind::Their;
        assert_eq!(p_their.case(), PronounCase::Possessive);
        assert_eq!(p_their.number(), PronounNumber::Plural);

        // 5. Keyword parsing round-trip
        assert_eq!(PronounKind::from_keyword("we"), Some(PronounKind::We));
        assert_eq!(PronounKind::from_keyword("you"), Some(PronounKind::You));
        assert_eq!(PronounKind::from_keyword("it"), Some(PronounKind::It));
        assert_eq!(PronounKind::from_keyword("she"), Some(PronounKind::She));
        assert_eq!(PronounKind::from_keyword("he"), Some(PronounKind::He));
        assert_eq!(PronounKind::from_keyword("herself"), Some(PronounKind::Herself));
        assert_eq!(PronounKind::from_keyword("himself"), Some(PronounKind::Himself));
        assert_eq!(PronounKind::from_keyword("theirs"), Some(PronounKind::Theirs));
        assert_eq!(PronounKind::from_keyword("self"), Some(PronounKind::SelfHandle));
        assert_eq!(PronounKind::from_keyword("caller"), Some(PronounKind::CallerHandle));
        assert_eq!(PronounKind::from_keyword("owner"), Some(PronounKind::OwnerHandle));
        assert_eq!(PronounKind::from_keyword("_"), Some(PronounKind::Wildcard));

        // 6. TokenType delegation
        let tok = TokenType::Pronoun(PronounKind::Myself);
        assert_eq!(tok.classify().primary, PartOfSpeech::Pronoun);
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/core/mmio.rs RS 300 LINES 1055 WORDS 9.7 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Physical Memory-Mapped I/O: Bare-Metal VGA Text Buffer & Serial Console )
// ============================================================================
//! OmniCode Physical MMIO Subsystem
//!
//! Provides the bare-metal hardware interaction layer for the bootable machine:
//!
//! # Physical Memory Architecture
//! - **0x000B8000 .. 0x000B8FA0**: Standard VGA text-mode video buffer (80 cols × 25 rows = 2000 cells).
//!   Each cell consists of an ASCII character code byte + color attribute byte.
//! - **0x03F8 .. 0x03FF**: COM1 UART 16550 Serial Port for low-level kernel telemetry.
//! - **0x00100000 .. 0x00FFFFFF**: Extended 16MB RAM partition for kernel and reels.
//!
//! When instructions on `Track 0 [KERNEL]` inscribe physical words to memory,
//! addresses in the MMIO range drive hardware video output and serial telemetry.

use crate::core::word::Word64;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( VGA Data Structures, Colors, and Serial Port Models )
// ============================================================================

pub const VGA_BUFFER_BASE: u64 = 0x000B8000;
pub const VGA_COLS: usize = 80;
pub const VGA_ROWS: usize = 25;
pub const VGA_CELL_COUNT: usize = VGA_COLS * VGA_ROWS;

pub const COM1_SERIAL_PORT: u64 = 0x03F8;

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
#[repr(u8)]
pub enum VgaColor {
    Black = 0,
    Blue = 1,
    Green = 2,
    Cyan = 3,
    Red = 4,
    Magenta = 5,
    Brown = 6,
    LightGray = 7,
    DarkGray = 8,
    LightBlue = 9,
    LightGreen = 10,
    LightCyan = 11,
    LightRed = 12,
    Pink = 13,
    Yellow = 14,
    White = 15,
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct VgaCell {
    pub ascii: u8,
    pub attribute: u8,
}

impl Default for VgaCell {
    fn default() -> Self {
        Self {
            ascii: b' ',
            attribute: (VgaColor::Black as u8) << 4 | (VgaColor::LightGray as u8),
        }
    }
}

pub struct VgaBuffer {
    pub cells: [VgaCell; VGA_CELL_COUNT],
    pub cursor_col: usize,
    pub cursor_row: usize,
}

impl Default for VgaBuffer {
```

```
    fn default() -> Self {
        Self::new()
    }
}

pub struct SerialPort {
    pub buffer: Vec<u8>,
}

impl Default for SerialPort {
    fn default() -> Self {
        Self::new()
    }
}

pub struct PhysicalMemoryMap {
    pub vga: VgaBuffer,
    pub serial: SerialPort,
    pub ram_cells: Vec<Word64>,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( MMIO Driver Methods, Video Screen Rendering, and Address Dispatch )
// ============================================================================

impl VgaBuffer {
    /// Creates an empty VGA text buffer cleared with light gray on black.
    pub fn new() -> Self {
        Self {
            cells: [VgaCell::default(); VGA_CELL_COUNT],
            cursor_col: 0,
            cursor_row: 0,
        }
    }

    /// Clears the video text buffer.
    pub fn clear(&mut self) {
        self.cells = [VgaCell::default(); VGA_CELL_COUNT];
        self.cursor_col = 0;
        self.cursor_row = 0;
    }

    /// Inscribes a single character byte at the current cursor location.
    pub fn write_char(&mut self, ch: u8, fg: VgaColor, bg: VgaColor) {
        if ch == b'\n' {
            self.new_line();
            return;
        }

        let attr = ((bg as u8) << 4) | (fg as u8);
        let idx = self.cursor_row * VGA_COLS + self.cursor_col;
        if idx < VGA_CELL_COUNT {
            self.cells[idx] = VgaCell { ascii: ch, attribute: attr };
        }

        self.cursor_col += 1;
        if self.cursor_col >= VGA_COLS {
            self.new_line();
        }
    }

    /// Writes an ASCII string to the video buffer.
    pub fn write_str(&mut self, s: &str, fg: VgaColor, bg: VgaColor) {
        for b in s.bytes() {
            self.write_char(b, fg, bg);
        }
    }

    /// Advances the cursor to the beginning of the next row, scrolling if needed.
    pub fn new_line(&mut self) {
        self.cursor_col = 0;
        if self.cursor_row + 1 < VGA_ROWS {
            self.cursor_row += 1;
        } else {
            // Scroll buffer up by 1 line
            for row in 1..VGA_ROWS {
                for col in 0..VGA_COLS {
```

```
                    let from_idx = row * VGA_COLS + col;
                    let to_idx = (row - 1) * VGA_COLS + col;
                    self.cells[to_idx] = self.cells[from_idx];
                }
            }
            // Clear last row
            let blank = VgaCell::default();
            for col in 0..VGA_COLS {
                let last_idx = (VGA_ROWS - 1) * VGA_COLS + col;
                self.cells[last_idx] = blank;
            }
        }
    }

    /// Renders the 80x25 video buffer as a framed ASCII string for terminal display.
    pub fn render_screen(&self, max_rows: usize) -> String {
        let mut out = String::new();
        out.push_str("\x1b[1;34m┌──────────────────────────────── [BARE-METAL VGA CONSOLE @ 0x000B8000] ────────────────────────────────┐\x1b[0m\n");
        let rows_to_show = max_rows.min(VGA_ROWS);
        for row in 0..rows_to_show {
            out.push_str("\x1b[1;34m│\x1b[0m ");
            for col in 0..VGA_COLS {
                let cell = self.cells[row * VGA_COLS + col];
                let ch = if (32..=126).contains(&cell.ascii) {
                    cell.ascii as char
                } else {
                    ' '
                };
                out.push(ch);
            }
            out.push_str(" \x1b[1;34m│\x1b[0m\n");
        }

        out.push_str("\x1b[1;34m└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘\x1b[0m\n");
        out
    }
}

impl SerialPort {
    pub fn new() -> Self {
        Self { buffer: Vec::new() }
    }

    pub fn write_byte(&mut self, b: u8) {
        self.buffer.push(b);
    }

    pub fn write_str(&mut self, s: &str) {
        self.buffer.extend_from_slice(s.as_bytes());
    }

    pub fn read_output(&self) -> String {
        String::from_utf8_lossy(&self.buffer).to_string()
    }
}

impl PhysicalMemoryMap {
    pub fn new(ram_capacity: usize) -> Self {
        Self {
            vga: VgaBuffer::new(),
            serial: SerialPort::new(),
            ram_cells: vec![Word64::zero(); ram_capacity],
        }
    }

    /// Dispatches a 64-bit word write to RAM or hardware MMIO peripherals.
    pub fn write(&mut self, address: u64, word: Word64) -> Result<(), String> {
        if (VGA_BUFFER_BASE..VGA_BUFFER_BASE + (VGA_CELL_COUNT as u64)).contains(&address) {
            let offset = (address - VGA_BUFFER_BASE) as usize;
            let ch = (word.payload().to_i64() as u8).max(32);
            let attr = ((word.raw >> 8) & 0xFF) as u8;
            self.vga.cells[offset] = VgaCell {
                ascii: ch,
                attribute: if attr == 0 { 0x07 } else { attr },
            };
            return Ok(());
```

```
        }

        if address == COM1_SERIAL_PORT {
            let b = (word.payload().to_i64() & 0xFF) as u8;
            self.serial.write_byte(b);
            return Ok(());
        }

        let idx = address as usize;
        if idx < self.ram_cells.len() {
            self.ram_cells[idx] = word;
            Ok(())
        } else {
            Err(format!("FAILPOINT [PhysicalBusFault]: Address 0x{:08X} unmapped on physical bus", address))
        }
    }

    /// Reads a 64-bit word from RAM or MMIO.
    pub fn read(&self, address: u64) -> Option<Word64> {
        if (VGA_BUFFER_BASE..VGA_BUFFER_BASE + (VGA_CELL_COUNT as u64)).contains(&address) {
            let offset = (address - VGA_BUFFER_BASE) as usize;
            let cell = self.vga.cells[offset];
            return Some(Word64::from_signed_int(cell.ascii as i64));
        }

        let idx = address as usize;
        self.ram_cells.get(idx).copied()
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests )
// ============================================================================
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_vga_write_and_scroll() {
        let mut vga = VgaBuffer::new();
        vga.write_str("OMNICODE BOOT", VgaColor::White, VgaColor::Black);
        assert_eq!(vga.cells[0].ascii, b'O');
        assert_eq!(vga.cells[1].ascii, b'M');
        assert_eq!(vga.cells[12].ascii, b'T');
        assert_eq!(vga.cursor_col, 13);
        assert_eq!(vga.cursor_row, 0);

        vga.new_line();
        assert_eq!(vga.cursor_col, 0);
        assert_eq!(vga.cursor_row, 1);
    }

    #[test]
    fn test_physical_memory_map_mmio_dispatch() {
        let mut mem = PhysicalMemoryMap::new(1024);

        // Write to VGA MMIO
        let word_char = Word64::from_signed_int(b'A' as i64);
        mem.write(VGA_BUFFER_BASE, word_char).expect("VGA write");
        assert_eq!(mem.vga.cells[0].ascii, b'A');

        // Write to Serial COM1
        let word_serial = Word64::from_signed_int(b'!' as i64);
        mem.write(COM1_SERIAL_PORT, word_serial).expect("Serial write");
        assert_eq!(mem.serial.read_output(), "!");

        // Write to RAM
        mem.write(42, Word64::from_signed_int(999)).expect("RAM write");
        assert_eq!(mem.read(42).unwrap().payload().to_i64(), 999);
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/core/mod.rs RS 54 LINES 205 WORDS 2.1 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Core Subsystem Identity, Module Architecture, and Invariant Re-exports )
// ============================================================================
//! OmniCode Core Subsystem
//! Encapsulates the algebraic foundation: Ternary Quad-State, Boundaries, and Operators.

pub mod ternary;
pub mod boundaries;
pub mod operators;
pub mod filetype;
pub mod linguistics;
pub mod cell;
pub mod vessel;
pub mod tape;
pub mod word;
pub mod scene;
pub mod reel;
pub mod mmio;


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Re-export Declarations for Direct Core Access )
// ============================================================================
pub use ternary::{QuadState, TransportAction, ROLLBACK, AWAIT, PROCEED, HALT};
pub use boundaries::BoundaryType;
pub use operators::{BeingRelation, DirectionalVector, Connective, Polarity, AffixPosition};
pub use filetype::OmniFileType;
pub use linguistics::{PartOfSpeech, GrammaticalClassification, GrammaticalEntity, PronounKind, PronounCase, PronounPerson, PronounNumber};
pub use cell::{Cell, Trit, BYTE_HALT, BYTE_AWAIT, BYTE_ROLLBACK, BYTE_PROCEED};
pub use vessel::{Vessel, Offset, Capacity, Count};
pub use tape::{Roll, Tape, TapeHead};
pub use word::{Word32Trit, Word64};
pub use scene::Scene;
pub use reel::MemoryReel;
pub use mmio::{PhysicalMemoryMap, VgaBuffer, VgaColor, VgaCell, SerialPort, VGA_BUFFER_BASE, COM1_SERIAL_PORT};




// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Core Subsystem Invariant Definitions )
// ============================================================================
// Invariants and operators are actively materialized via re-exported submodules.


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Verification & Subsystem Status )
// ============================================================================
// Core module namespace verified and closed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/core/operators.rs RS 380 LINES 1303 WORDS 14.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Module Identity, 3D Coordinate Operators, Connectives & Vector Maps )
// ============================================================================
//! OmniCode Core: Relational, Directional, and Connective Operators
//!
//! Governs the 3-Dimensional Coordinate System:
//!     - Y-Axis (Being/State)      : = (IS), != (IS NOT), ~= (IS LIKE)
//!     - X-Axis (Causality/Action) : --> (Act Upon), <-- (Act Again)
//!     - Z-Axis (Scale/Layering)   : ^-- (Builds To), v-- (Breaks Down From)
//!     - Connectives               : ! (But), :: (And), : (In Relation To), | (Or), -- (Not)

use std::fmt;


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Type Declarations: BeingRelation, DirectionalVector, and Connective Enums )
// ============================================================================
/// Relations of Being along the Ontological Y-Axis.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum BeingRelation {
    /// Exact identity / equivalence (=)
    Is,
    /// Distinction / exclusion (!=) (Not to be / inequality)
    IsNot,
    /// State negation / copular denial (=!) (Is not / To be not)
    ToBeNot,
    /// Homomorphism / structural analogy / duck typing (~=)
    ToBeLike,
    /// Invariant Balance / Equivalence (<=>)
    Equivalence,
    /// Relational strictly less than (<)
    LessThan,
    /// Relational strictly greater than (>)
    GreaterThan,
    /// Relational less than or equal (<=)
    LessEqual,
    /// Relational greater than or equal (>=)
    GreaterEqual,
}

/// Vectors of Action, Scale, and Spatial Quadrants across X, Y, and Z axes.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum DirectionalVector {
    // X-Axis (Temporal / Dynamics)
    /// Forward causation, execution, emission (-->)
    ActUpon,
    /// Retroactive feedback, retry, rollback (<--)
    ActAgain,

    // Z-Axis (Scale / Hierarchy)
    /// Layering Up: synthesis, abstraction, emergence (^-- / --^)
    BuildsTo,
    /// Layering Down: decomposition, destructuring, compilation to IR (v-- / --v)
    BreaksDownFrom,

    // Diagonal Modulated Scales
    /// Major ascending forward diagonal (/-^)
    Major,
    /// Minor descending forward diagonal (\-v)
    Minor,
    /// Great ascending backward diagonal (\-^)
    Great,
    /// Small descending backward diagonal (/-v)
    Small,

    // 2D Quadrant / Compass Vectors
    /// Upper Right (Quadrant I: +X, +Y) (/^)
    UpperRight,
    /// Lower Right (Quadrant IV: +X, -Y) (\v)
    LowerRight,
    /// Upper Left (Quadrant II: -X, +Y) (^\)
    UpperLeft,
```

```
    /// Lower Left (Quadrant III: -X, -Y) (v/)
    LowerLeft,
}

/// Logical and Structural Connectives.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum Connective {
    /// Adversative caveat: A ! B (A executes, BUT must satisfy caveat B)
    But,
    /// Cumulative conjunction: A :: B (A AND B simultaneous)
    And,
    /// Relational role / typing: A : B (A in relation to B)
    InRelationTo,
    /// Disjunction / alternative: A | B (A OR B)
    Or,
    /// Ontological inversion: -- A (NOT A)
    Not,
}

/// Polarity of an entity, keyword, or value.
/// '+' represents Abstract (archetype, intension, zero-cost blueprint).
/// '-' represents Concrete (extension, physical instance, allocated substance).
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum Polarity {
    Abstract,
    Concrete,
}

/// Affix position of a polarity operator attached to an identifier or keyword.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum AffixPosition {
    Prefix,  // +Dog, -Dog, +Cell
    Postfix, // Dog+, Dog-, PROCEED-
}

impl Polarity {
    pub fn from_symbol(sym: &str) -> Result<Self, String> {
        match sym.trim() {
            "+" => Ok(Polarity::Abstract),
            "-" => Ok(Polarity::Concrete),
            other => Err(format!("FAILPOINT [InvalidPolarity]: Unknown polarity symbol '{other}'")),
        }
    }

    pub fn symbol(&self) -> &'static str {
        match self {
            Polarity::Abstract => "+",
            Polarity::Concrete => "-",
        }
    }
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Core Symbol Parsing & Semantic Transformations )
// ============================================================================
impl BeingRelation {
    pub fn from_symbol(sym: &str) -> Result<Self, String> {
        match sym.trim() {
            "=" => Ok(BeingRelation::Is),
            "!=" | "<>" => Ok(BeingRelation::IsNot),
            "=!" => Ok(BeingRelation::ToBeNot),
            "~=" | "=~" | "≈" => Ok(BeingRelation::ToBeLike),
            "<=>" => Ok(BeingRelation::Equivalence),
            "<" => Ok(BeingRelation::LessThan),
            ">" => Ok(BeingRelation::GreaterThan),
            "<=" => Ok(BeingRelation::LessEqual),
            ">=" => Ok(BeingRelation::GreaterEqual),
            other => Err(format!("FAILPOINT [InvalidBeingRelation]: Unknown symbol '{other}'")),
        }
    }

    pub fn symbol(&self) -> &'static str {
        match self {
            BeingRelation::Is => "=",
            BeingRelation::IsNot => "!=",
            BeingRelation::ToBeNot => "=!",
```

```
            BeingRelation::ToBeLike => "~=",
            BeingRelation::Equivalence => "<=>",
            BeingRelation::LessThan => "<",
            BeingRelation::GreaterThan => ">",
            BeingRelation::LessEqual => "<=",
            BeingRelation::GreaterEqual => ">=",
        }
    }
}

impl DirectionalVector {
    pub fn from_symbol(sym: &str) -> Result<Self, String> {
        match sym.trim() {
            "-->" | "->" => Ok(DirectionalVector::ActUpon),
            "<--" | "<-" => Ok(DirectionalVector::ActAgain),
            "--^" | "-^" | "^--" | "^-" | "↑" => Ok(DirectionalVector::BuildsTo),
            "--v" | "-v" | "v--" | "v-" | "↓" => Ok(DirectionalVector::BreaksDownFrom),
            "/-^" => Ok(DirectionalVector::Major),
            "\\-v" => Ok(DirectionalVector::Minor),
            "\\-^" => Ok(DirectionalVector::Great),
            "/-v" => Ok(DirectionalVector::Small),
            "/^" => Ok(DirectionalVector::UpperRight),
            "\\v" => Ok(DirectionalVector::LowerRight),
            "^\\" => Ok(DirectionalVector::UpperLeft),
            "v/" => Ok(DirectionalVector::LowerLeft),
            other => Err(format!("FAILPOINT [InvalidDirectionalVector]: Unknown vector symbol '{other}'")),
        }
    }

    /// Canonical ASCII symbol for the vector.
    pub fn symbol(&self) -> &'static str {
        match self {
            DirectionalVector::ActUpon => "-->",
            DirectionalVector::ActAgain => "<--",
            DirectionalVector::BuildsTo => "--^",
            DirectionalVector::BreaksDownFrom => "--v",
            DirectionalVector::Major => "/-^",
            DirectionalVector::Minor => "\\-v",
            DirectionalVector::Great => "\\-^",
            DirectionalVector::Small => "/-v",
            DirectionalVector::UpperRight => "/^",
            DirectionalVector::LowerRight => "\\v",
            DirectionalVector::UpperLeft => "^\\",
            DirectionalVector::LowerLeft => "v/",
        }
    }

    /// Fast 2-character ASCII shorthand symbol for the vector.
    pub fn shorthand(&self) -> &'static str {
        match self {
            DirectionalVector::ActUpon => "->",
            DirectionalVector::ActAgain => "<-",
            DirectionalVector::BuildsTo => "-^",
            DirectionalVector::BreaksDownFrom => "-v",
            DirectionalVector::Major => "/-^",
            DirectionalVector::Minor => "\\-v",
            DirectionalVector::Great => "\\-^",
            DirectionalVector::Small => "/-v",
            DirectionalVector::UpperRight => "/^",
            DirectionalVector::LowerRight => "\\v",
            DirectionalVector::UpperLeft => "^\\",
            DirectionalVector::LowerLeft => "v/",
        }
    }

    /// Cardinal direction on the computational coordinate plane.
    pub fn cardinal_direction(&self) -> &'static str {
        match self {
            DirectionalVector::ActUpon => "Right (Forward Causation / Advance)",
            DirectionalVector::ActAgain => "Left (Retroaction / Feedback / Rollback)",
            DirectionalVector::BuildsTo => "Up (Synthesis / Abstraction / Ascend)",
            DirectionalVector::BreaksDownFrom => "Down (Decomposition / Analysis / Descend)",
            DirectionalVector::Major => "Major (Ascending Forward Diagonal / Major Scale)",
            DirectionalVector::Minor => "Minor (Descending Forward Diagonal / Minor Scale)",
            DirectionalVector::Great => "Great (Ascending Backward Diagonal / Great Scale)",
            DirectionalVector::Small => "Small (Descending Backward Diagonal / Small Scale)",
            DirectionalVector::UpperRight => "Upper Right (Quadrant I: +X, +Y / Northeast)",
            DirectionalVector::LowerRight => "Lower Right (Quadrant IV: +X, -Y / Southeast)",
```

```
            DirectionalVector::UpperLeft => "Upper Left (Quadrant II: -X, +Y / Northwest)",
            DirectionalVector::LowerLeft => "Lower Left (Quadrant III: -X, -Y / Southwest)",
        }
    }
}

impl Connective {
    pub fn from_symbol(sym: &str) -> Result<Self, String> {
        match sym.trim() {
            "!" => Ok(Connective::But),
            "::" | "&&" => Ok(Connective::And),
            ":" => Ok(Connective::InRelationTo),
            "|" | "||" => Ok(Connective::Or),
            "--" | "not" | "~" => Ok(Connective::Not),
            other => Err(format!("FAILPOINT [InvalidConnective]: Unknown connective symbol '{other}'")),
        }
    }

    pub fn symbol(&self) -> &'static str {
        match self {
            Connective::But => "!",
            Connective::And => "::",
            Connective::InRelationTo => ":",
            Connective::Or => "|",
            Connective::Not => "--",
        }
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Display Trait Implementations and String Formatting )
// ============================================================================
impl fmt::Display for BeingRelation {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{}", self.symbol())
    }
}

impl fmt::Display for DirectionalVector {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{}", self.symbol())
    }
}

impl fmt::Display for Connective {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{}", self.symbol())
    }
}

impl fmt::Display for Polarity {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{}", self.symbol())
    }
}

impl fmt::Display for AffixPosition {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            AffixPosition::Prefix => write!(f, "Prefix"),
            AffixPosition::Postfix => write!(f, "Postfix"),
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_polarity_and_affix_operators() {
        assert_eq!(Polarity::from_symbol("+").unwrap(), Polarity::Abstract);
        assert_eq!(Polarity::from_symbol("-").unwrap(), Polarity::Concrete);
        assert_eq!(Polarity::Abstract.symbol(), "+");
        assert_eq!(Polarity::Concrete.symbol(), "-");
        assert_eq!(format!("{}", Polarity::Abstract), "+");
```

```
        assert_eq!(format!("{}", Polarity::Concrete), "-");
    }

    #[test]
    fn test_directional_vectors_canonical_and_shorthand() {
        // Canonical 3-character symbols
        assert_eq!(DirectionalVector::from_symbol("-->").unwrap(), DirectionalVector::ActUpon);
        assert_eq!(DirectionalVector::from_symbol("<--").unwrap(), DirectionalVector::ActAgain);
        assert_eq!(DirectionalVector::from_symbol("--^").unwrap(), DirectionalVector::BuildsTo);
        assert_eq!(DirectionalVector::from_symbol("--v").unwrap(), DirectionalVector::BreaksDownFrom);

        // Fast 2-character shorthands
        assert_eq!(DirectionalVector::from_symbol("->").unwrap(), DirectionalVector::ActUpon);
        assert_eq!(DirectionalVector::from_symbol("<-").unwrap(), DirectionalVector::ActAgain);
        assert_eq!(DirectionalVector::from_symbol("-^").unwrap(), DirectionalVector::BuildsTo);
        assert_eq!(DirectionalVector::from_symbol("-v").unwrap(), DirectionalVector::BreaksDownFrom);

        // Legacy symbols
        assert_eq!(DirectionalVector::from_symbol("^--").unwrap(), DirectionalVector::BuildsTo);
        assert_eq!(DirectionalVector::from_symbol("v--").unwrap(), DirectionalVector::BreaksDownFrom);

        // Symbols emit canonical forms
        assert_eq!(DirectionalVector::BuildsTo.symbol(), "--^");
        assert_eq!(DirectionalVector::BreaksDownFrom.symbol(), "--v");

        // Scale Modulations
        assert_eq!(DirectionalVector::from_symbol("/-^").unwrap(), DirectionalVector::Major);
        assert_eq!(DirectionalVector::from_symbol("\\-v").unwrap(), DirectionalVector::Minor);
        assert_eq!(DirectionalVector::from_symbol("\\-^").unwrap(), DirectionalVector::Great);
        assert_eq!(DirectionalVector::from_symbol("/-v").unwrap(), DirectionalVector::Small);

        // 2D Quadrants
        assert_eq!(DirectionalVector::from_symbol("/^").unwrap(), DirectionalVector::UpperRight);
        assert_eq!(DirectionalVector::from_symbol("\\v").unwrap(), DirectionalVector::LowerRight);
        assert_eq!(DirectionalVector::from_symbol("^\\").unwrap(), DirectionalVector::UpperLeft);
        assert_eq!(DirectionalVector::from_symbol("v/").unwrap(), DirectionalVector::LowerLeft);
    }

    #[test]
    fn test_being_relations_canonical() {
        assert_eq!(BeingRelation::from_symbol("=").unwrap(), BeingRelation::Is);
        assert_eq!(
            BeingRelation::from_symbol("!=").unwrap(),
            BeingRelation::IsNot
        );
        assert_eq!(
            BeingRelation::from_symbol("=!").unwrap(),
            BeingRelation::ToBeNot
        );
        assert_eq!(
            BeingRelation::from_symbol("~=").unwrap(),
            BeingRelation::ToBeLike
        );
        assert_eq!(
            BeingRelation::from_symbol("=~").unwrap(),
            BeingRelation::ToBeLike
        );
        assert_eq!(
            BeingRelation::from_symbol("<=>").unwrap(),
            BeingRelation::Equivalence
        );

        assert_eq!(BeingRelation::Is.symbol(), "=");
        assert_eq!(BeingRelation::IsNot.symbol(), "!=");
        assert_eq!(BeingRelation::ToBeNot.symbol(), "=!");
        assert_eq!(BeingRelation::ToBeLike.symbol(), "~=");
        assert_eq!(BeingRelation::Equivalence.symbol(), "<=>");
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/core/reel.rs RS 286 LINES 1090 WORDS 11.5 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( 4-Directional Toroidal Memory Reel: Transport Kinematics & Scene Spool )
// ============================================================================
//! OmniCode Core: 4-Directional Memory Reel
//!
//! Spools memory across two orthogonal mechanical axes:
//! 1. **Horizontal Ribbon Navigation:**
//!    - `push_forward(word)` (`-->`): Advances write head along the track.
//!    - `pull_back()` (`<--`): Recedes backward along the track to inspect/undo.
//! 2. **Vertical Spool Hierarchy Navigation:**
//!    - `sink_in(name, cap)` (`v--`): Cuts into a child scene (pushes stack frame).
//!    - `pop_out()` (`^--`): Cuts back to parent scene (pops stack frame).
//!
//! Governed by the Quad-State Transport Kinematics:
//! `PLAY` (+1), `PAUSE` (0), `REWIND` (-1), `STOP` (NULL).

use std::fmt;
use crate::core::scene::Scene;
use crate::core::ternary::TransportAction;
use crate::core::word::Word64;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( MemoryReel Struct Definition & Constructor )
// ============================================================================

/// A 4-directional spooled memory stack and multi-track film canister.
#[derive(Debug, Clone, PartialEq)]
pub struct MemoryReel {
    pub name: String,
    pub transport: TransportAction,
    pub scenes: Vec<Scene>,
    pub next_scene_id: usize,
}

impl MemoryReel {
    /// Creates a new MemoryReel initialized with a root master scene (depth 0).
    pub fn new(name: impl Into<String>, root_capacity: usize) -> Self {
        let root_scene = Scene::new(0, "MasterSet_0", root_capacity, 0, Some("genesis".to_string()));
        Self {
            name: name.into(),
            transport: TransportAction::Pause,
            scenes: vec![root_scene],
            next_scene_id: 1,
        }
    }

    /// Returns a reference to the currently active camera scene.
    pub fn active_scene(&self) -> &Scene {
        self.scenes.last().expect("Reel must always contain at least one scene")
    }

    /// Returns a mutable reference to the currently active camera scene.
    pub fn active_scene_mut(&mut self) -> &mut Scene {
        self.scenes.last_mut().expect("Reel must always contain at least one scene")
    }

    /// Returns the current spindle depth level (0 = Master set).
    pub fn depth(&self) -> usize {
        self.active_scene().depth
    }

    /// Returns the total number of scenes currently on the reel spindle.
    pub fn scene_count(&self) -> usize {
        self.scenes.len()
    }
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( The 4 Mechanical Motions: Push Forward, Pull Back, Sink In, Pop Out )
// ============================================================================
```

```

impl MemoryReel {
    // ---------------------------------------------------------------------------
    // Axis 1: Horizontal Track Motions (--> and <--)
    // ---------------------------------------------------------------------------

    /// **Push Forward (`-->`):**
    /// Advances the track head, inscribes the physical word at the new position,
    /// and sets transport to `PLAY`.
    pub fn push_forward(&mut self, word: Word64) -> Result<usize, String> {
        let scene = self.active_scene_mut();
        let target_pos = scene.head_pos;

        if target_pos >= scene.capacity {
            return Err(format!(
                "FAILPOINT [ReelCapacityReached]: Track head at @0x{:04X} exceeded scene capacity {}",
                target_pos, scene.capacity
            ));
        }

        scene.write(target_pos, word)?;
        scene.head_pos += 1;
        self.transport = TransportAction::Play;
        Ok(target_pos)
    }

    /// **Pull Back (`<--`):**
    /// Recedes the track head backward along the ribbon to inspect antecedent props
    /// or perform transactional retreat without destroying future frame cells.
    pub fn pull_back(&mut self) -> Option<Word64> {
        let scene = self.active_scene_mut();
        if scene.head_pos > 0 {
            scene.head_pos -= 1;
            scene.read(scene.head_pos)
        } else {
            scene.read(0)
        }
    }

    // ---------------------------------------------------------------------------
    // Axis 2: Vertical Spool Hierarchy Motions (v-- and ^--)
    // ---------------------------------------------------------------------------

    /// **Sink In (`v--`):**
    /// Cuts into a subordinate scene (pushes a new stack frame onto the spindle).
    /// The parent scene remains frozen and untouched above it.
    pub fn sink_in(
        &mut self,
        scene_name: impl Into<String>,
        capacity: usize,
        landmark: Option<String>,
    ) -> usize {
        let parent_depth = self.depth();
        let scene_id = self.next_scene_id;
        self.next_scene_id += 1;

        let child_scene = Scene::new(scene_id, scene_name, capacity, parent_depth + 1, landmark);
        self.scenes.push(child_scene);
        self.depth()
    }

    /// **Pop Out (`^--`):**
    /// Completes the subordinate scene, pops the stack frame off the spindle,
    /// and restores the camera framing to the supervisory parent set.
    pub fn pop_out(&mut self) -> Result<Scene, String> {
        if self.scenes.len() <= 1 {
            return Err("FAILPOINT [RootScenePop]: Cannot pop out of the master root scene (Depth 0)".to_string());
        }
        let completed = self.scenes.pop().expect("Scene pop must succeed");
        Ok(completed)
    }

    // ---------------------------------------------------------------------------
    // Transport Kinematics Controls: Play, Pause, Rewind, Stop
    // ---------------------------------------------------------------------------

    /// Sets transport to `PLAY` (+1 / PROCEED).
    pub fn play(&mut self) -> TransportAction {
```

```
        self.transport = TransportAction::Play;
        self.transport
    }

    /// Sets transport to `PAUSE` (0 / AWAIT).
    pub fn pause(&mut self) -> TransportAction {
        self.transport = TransportAction::Pause;
        self.transport
    }

    /// **Rewinds (`<--` / -1 / ROLLBACK):**
    /// Spools the track head backward to a designated scene landmark cue.
    pub fn rewind_to_landmark(&mut self, target_cue: &str) -> Result<usize, String> {
        let scene = self.active_scene_mut();
        if let Some(ref landmark) = scene.landmark {
            if landmark == target_cue {
                scene.head_pos = 0;
                self.transport = TransportAction::Rewind;
                return Ok(0);
            }
        }
        Err(format!(
            "FAILPOINT [LandmarkNotFound]: Active scene '{}' does not match landmark cue '<{target_cue}>'",
            scene.name
        ))
    }

    /// Sets transport to `STOP` (NULL / HALT).
    pub fn stop(&mut self) -> TransportAction {
        self.transport = TransportAction::Stop;
        self.transport
    }

    /// Generates a cinematic ASCII status dashboard of the memory reel.
    pub fn inspect(&self) -> String {
        let mut out = String::new();
        out.push_str(&format!("═══════════════════════════════════════════════════════════════\n"));
        out.push_str(&format!("REEL: [{}] | TRANSPORT: [{:?}]\n", self.name, self.transport));
        out.push_str(&format!("SPINDLE DEPTH: {} | TOTAL SCENES: {}\n", self.depth(), self.scene_count()));
        out.push_str(&format!("---------------------------------------------------------------\n"));
        for (i, sc) in self.scenes.iter().enumerate() {
            let active_marker = if i == self.scenes.len() - 1 { "▶ [ACTIVE]" } else { "  [FROZEN]" };
            out.push_str(&format!("  {} Level {}: {}\n", active_marker, sc.depth, sc));
        }
        out.push_str(&format!("═══════════════════════════════════════════════════════════════\n"));
        out
    }
}

impl fmt::Display for MemoryReel {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(
            f,
            "Reel [{} | Transport: {:?} | Depth: {} | Active: {}]",
            self.name, self.transport, self.depth(), self.active_scene().name
        )
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests & Verification Proof )
// ============================================================================
#[cfg(test)]
mod tests {
    use super::*;
    use crate::core::word::Word32Trit;
    use crate::core::ternary::QuadState;
    use crate::core::operators::Polarity;
    use crate::core::boundaries::BoundaryType;

    #[test]
    fn test_reel_4_directional_motions() {
        let mut reel = MemoryReel::new("CinemaReel", 32);
        assert_eq!(reel.depth(), 0);
        assert_eq!(reel.scene_count(), 1);

```

```
        // 1. Push Forward (-->) on master set
        let w1 = Word64::new(Word32Trit::from_i64(10), QuadState::Proceed, Polarity::Concrete, BoundaryType::Denotation,
0);
        let pos1 = reel.push_forward(w1).expect("Push forward 1");
        assert_eq!(pos1, 0);
        assert_eq!(reel.transport, TransportAction::Play);

        let w2 = Word64::new(Word32Trit::from_i64(20), QuadState::Proceed, Polarity::Concrete, BoundaryType::Denotation,
0);
        let pos2 = reel.push_forward(w2).expect("Push forward 2");
        assert_eq!(pos2, 1);

        // 2. Pull Back (<--) along master track
        let back_word = reel.pull_back().expect("Pull back");
        assert_eq!(back_word.payload().to_i64(), 20);

        // 3. Sink In (v--) into nested sub-scene
        let depth = reel.sink_in("CloseUp_Scene", 16, Some("closeup_take".to_string()));
        assert_eq!(depth, 1);
        assert_eq!(reel.depth(), 1);
        assert_eq!(reel.scene_count(), 2);
        assert_eq!(reel.active_scene().name, "CloseUp_Scene");

        // Write in nested sub-scene
        let w_sub = Word64::new(Word32Trit::from_i64(999), QuadState::Proceed, Polarity::Concrete,
BoundaryType::Denotation, 0);
        reel.push_forward(w_sub).expect("Push in sub-scene");
        assert_eq!(reel.active_scene().read(0).unwrap().payload().to_i64(), 999);

        // 4. Pop Out (^--) back to master wide shot
        let popped = reel.pop_out().expect("Pop out to master");
        assert_eq!(popped.name, "CloseUp_Scene");
        assert_eq!(reel.depth(), 0);
        assert_eq!(reel.active_scene().name, "MasterSet_0");
        assert_eq!(reel.scene_count(), 1);

        // Verify master set props were unaffected by sub-scene
        assert_eq!(reel.active_scene().read(0).unwrap().payload().to_i64(), 10);
        assert_eq!(reel.active_scene().read(1).unwrap().payload().to_i64(), 20);
    }

    #[test]
    fn test_reel_rewind_to_landmark() {
        let mut reel = MemoryReel::new("TakeReel", 32);
        reel.sink_in("TakeScene", 16, Some("scene_cue".to_string()));

        let w = Word64::new(Word32Trit::from_i64(42), QuadState::Proceed, Polarity::Concrete, BoundaryType::Denotation,
0);
        reel.push_forward(w).unwrap();
        reel.push_forward(w).unwrap();
        assert_eq!(reel.active_scene().head_pos, 2);

        // Rewind to landmark
        let reset_pos = reel.rewind_to_landmark("scene_cue").expect("Rewind to cue");
        assert_eq!(reset_pos, 0);
        assert_eq!(reel.active_scene().head_pos, 0);
        assert_eq!(reel.transport, TransportAction::Rewind);
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/core/scene.rs RS 130 LINES 449 WORDS 4.3 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Scene Memory Frame: Set, Props & Landmark Clapper )
// ============================================================================
//! OmniCode Core: Scene Memory Frame
//!
//! Represents a unified memory stack frame / cinematic set where instructions
//! (rolls) operate upon local props (memory cells):
//! - **Set:** Bounded memory partition slice.
//! - **Props:** Physical 64-bit / 32-trit memory cells.
//! - **Clapper Landmark:** Scene cue point for retakes and rewind operations.

use std::fmt;
use crate::core::word::Word64;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Scene Struct Definition & Constructor )
// ============================================================================

/// A memory stack frame and camera set in the OmniCode cinema architecture.
#[derive(Debug, Clone, PartialEq)]
pub struct Scene {
    pub id: usize,
    pub name: String,
    pub depth: usize,
    pub landmark: Option<String>,
    pub capacity: usize,
    pub head_pos: usize,
    pub cells: Vec<Word64>,
}

impl Scene {
    /// Creates a new Scene set with bounded cell capacity.
    pub fn new(
        id: usize,
        name: impl Into<String>,
        capacity: usize,
        depth: usize,
        landmark: Option<String>,
    ) -> Self {
        let cap = capacity.max(8);
        Self {
            id,
            name: name.into(),
            depth,
            landmark,
            capacity: cap,
            head_pos: 0,
            cells: vec![Word64::default(); cap],
        }
    }

    /// Returns the number of allocated cells in this scene.
    pub fn len(&self) -> usize {
        self.cells.len()
    }

    /// Returns true if this scene has zero capacity.
    pub fn is_empty(&self) -> bool {
        self.cells.is_empty()
    }

    /// Reads a memory cell prop at an explicit offset.
    pub fn read(&self, offset: usize) -> Option<Word64> {
        self.cells.get(offset).copied()
    }

    /// Inscribes a memory cell prop at an explicit offset with bounds enforcement.
    pub fn write(&mut self, offset: usize, word: Word64) -> Result<(), String> {
        if offset >= self.capacity {
            return Err(format!(
                "FAILPOINT [SceneBoundsOverflow]: Scene '{}' capacity is {}, but attempted write at offset {}",
                self.name, self.capacity, offset
```

```
            ));
        }
        self.cells[offset] = word;
        Ok(())
    }

    /// Reads the prop directly under the active scene head.
    pub fn current(&self) -> Option<Word64> {
        self.read(self.head_pos)
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Scene Display Formatting )
// ============================================================================

impl fmt::Display for Scene {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        let landmark_str = self.landmark.as_deref().unwrap_or("none");
        write!(
            f,
            "Scene [{} | ID: {} | Depth: {} | Landmark: <{}> | Head: @0x{:04X} | Cells: {}/{}]",
            self.name, self.id, self.depth, landmark_str, self.head_pos, self.cells.len(), self.capacity
        )
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests )
// ============================================================================
#[cfg(test)]
mod tests {
    use super::*;
    use crate::core::word::Word32Trit;
    use crate::core::ternary::QuadState;
    use crate::core::operators::Polarity;
    use crate::core::boundaries::BoundaryType;

    #[test]
    fn test_scene_creation_and_bounded_write() {
        let mut scene = Scene::new(0, "TestSet", 16, 0, Some("take_1".to_string()));
        assert_eq!(scene.name, "TestSet");
        assert_eq!(scene.depth, 0);
        assert_eq!(scene.capacity, 16);

        let word = Word64::new(Word32Trit::from_i64(77), QuadState::Proceed, Polarity::Concrete,
BoundaryType::Denotation, 1);
        scene.write(3, word).expect("Write to offset 3");
        assert_eq!(scene.read(3).unwrap().payload().to_i64(), 77);

        // Bounds overflow rejection
        assert!(scene.write(16, word).is_err());
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/core/tape.rs RS 497 LINES 1828 WORDS 19.1 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Tape Subsystem Identity, Ribbon Physical Topology & Invariants )
// ================================================================================
//! OmniCode Core Subsystem: Physical Tape & Roll Execution Topology
//!
//! # Physical Computing Hierarchy
//! In the OmniCode Operating System, execution does not use raw unstructured threads.
//! It follows a strict 3-tier physical hierarchy:
//!
//! $$\text{Roll} \longrightarrow \text{Tape} \longrightarrow \text{Track}$$
//!
//! 1. **Roll (Instruction Line):** An atomic line of code executing an SVO transformation.
//! 2. **Tape (Process Ribbon):** A sequential chain of rolls led by a **Tape Head**
//!    (the covenant manifest declaring Who, What, When, Where, Why, and How).
//! 3. **Track (Subsystem Channel):** A concurrent execution lane on the scheduler
//!    (Track 0: Kernel, Track 1: Terminal, Track 2: Apps).
//!
//! # Tape Head Anatomy
//! The Tape Head is the first roll on the ribbon. It establishes:
//! - Identity & Name (`[Tape.<Name>]`)
//! - Principal & Caller (`WHO {Caller}`)
//! - Target Entity (`WHAT {Target}`)
//! - Lifecycle Trigger (`WHEN {Phase}`)
//! - Memory Partition Ceiling (`WHERE {partition} = [ <N> ]`)
//! - Teleological Purpose Aside (`WHY ( ... )`)
//! - Operational Vector (`HOW [ ... ]`)

use std::fs;
use std::path::Path;
use crate::core::ternary::QuadState;
use crate::diagnostics::logger::DiagnosticLogger;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Tape Data Structures, Roll Containers and Context )
// ================================================================================

/// An atomic line of code on a Tape ribbon.
#[derive(Debug, Clone, PartialEq)]
pub struct Roll {
    /// 0-indexed position within the tape's rolls.
    pub index: usize,
    /// 1-indexed source line number.
    pub line_number: usize,
    /// Raw text content of the roll.
    pub text: String,
    /// Current execution state of this roll (+1: PROCEED, 0: AWAIT, -1: ROLLBACK, NULL: HALT).
    pub status: QuadState,
    /// Whether this roll is the leading Tape Head covenant.
    pub is_head: bool,
}

/// The leading covenant roll establishing the process manifest and memory bounds.
#[derive(Debug, Clone, PartialEq)]
pub struct TapeHead {
    /// Identifier or process name.
    pub name: String,
    /// Cardinal Clause: WHO (Caller / Principal).
    pub who: String,
    /// Cardinal Clause: WHAT (Process substance).
    pub what: String,
    /// Cardinal Clause: WHEN (Trigger precondition).
    pub when_clause: String,
    /// Cardinal Clause: WHERE (Target partition or spatial bounds).
    pub where_clause: String,
    /// Cardinal Clause: WHY (Teleological intent aside).
    pub why: String,
    /// Cardinal Clause: HOW (Directional vector and methodology).
    pub how: String,
    /// Required memory partition size in cells (carved from Storehouse via `v--`).
    pub required_partition_size: usize,
    /// Complete raw text of the Tape Head roll.
    pub raw_text: String,
```

```rust
}

impl Default for TapeHead {
    fn default() -> Self {
        Self {
            name: "Anonymous_Tape".to_string(),
            who: "System".to_string(),
            what: "Process".to_string(),
            when_clause: "Immediate".to_string(),
            where_clause: "Memory".to_string(),
            why: "Execution".to_string(),
            how: "Forward".to_string(),
            required_partition_size: 256,
            raw_text: String::new(),
        }
    }
}

/// A complete sequential process ribbon executed roll-by-roll.
#[derive(Debug, Clone)]
pub struct Tape {
    /// The leading covenant manifest.
    pub head: TapeHead,
    /// The sequence of subordinate execution rolls.
    pub rolls: Vec<Roll>,
    /// Active cursor pointing to the roll currently being executed.
    pub cursor: usize,
    /// Name of the isolated memory partition allocated for this tape.
    pub bound_partition: Option<String>,
    /// Overall execution quad-state.
    pub execution_state: QuadState,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Tape Parsing, Stepping, and Execution Management )
// ============================================================================

impl Tape {
    /// Parses a native `.tape` source string into a structured [`Tape`].
    pub fn from_source(source: &str, logger: DiagnosticLogger) -> Result<Self, String> {
        let mut head_opt: Option<TapeHead> = None;
        let mut raw_roll_lines: Vec<(usize, String)> = Vec::new();

        let mut head_text_accumulator = String::new();
        let mut parsing_multiline_head = false;
        let mut is_block_head = false;

        for (zero_idx, raw_line) in source.lines().enumerate() {
            let line_number = zero_idx + 1;
            let trimmed = raw_line.trim();

            // Skip comment boundaries (# ...)
            if trimmed.starts_with('#') || trimmed.is_empty() {
                continue;
            }

            // Detect Tape Head
            if head_opt.is_none() {
                if !parsing_multiline_head {
                    if trimmed.starts_with('[')
                        || trimmed.starts_with("= [Tape")
                        || trimmed.starts_with("Tape [")
                        || trimmed.starts_with("Tape <")
                    {
                        parsing_multiline_head = true;
                        is_block_head = trimmed.ends_with('{');
                        head_text_accumulator.push_str(trimmed);
                        head_text_accumulator.push(' ');
                        // Check if closed on same line (only if not opening a block with '{')
                        if !is_block_head && (trimmed.contains(":: <") || (!trimmed.contains("WHERE") &&
!trimmed.contains("WHO"))) {
                            parsing_multiline_head = false;
                            let head = Self::parse_tape_head(&head_text_accumulator, line_number)?;
                            logger.log("TAPE", format!("Discovered Tape Head: [{}]", head.name));
                            head_opt = Some(head);
                            head_text_accumulator.clear();
                        }
```

```
                    } else {
                        // Plain line without leading bracket treated as fallback roll
                        raw_roll_lines.push((line_number, trimmed.to_string()));
                    }
                } else {
                    // Accumulating multiline Tape Head
                    if is_block_head {
                        if trimmed == "}" || trimmed.ends_with('}') {
                            // Closing brace of block-manifest Tape Head
                            parsing_multiline_head = false;
                            is_block_head = false;
                            let head = Self::parse_tape_head(&head_text_accumulator, line_number)?;
                            logger.log("TAPE", format!("Discovered Tape Head: [{}]", head.name));
                            head_opt = Some(head);
                            head_text_accumulator.clear();
                            continue;
                        } else {
                            head_text_accumulator.push_str(trimmed);
                            head_text_accumulator.push(' ');
                        }
                    } else if trimmed.starts_with("WHERE")
                        || trimmed.starts_with("WHEN")
                        || trimmed.starts_with("WHY")
                        || trimmed.starts_with("HOW")
                        || trimmed.starts_with("WHO")
                        || trimmed.starts_with("WHAT")
                        || trimmed.starts_with("< Scripture")
                        || trimmed.starts_with("::")
                        || trimmed.contains(":: <")
                    {
                        head_text_accumulator.push_str(trimmed);
                        head_text_accumulator.push(' ');
                    } else {
                        // Current line is not part of the head; the multiline head is complete
                        parsing_multiline_head = false;
                        let head = Self::parse_tape_head(&head_text_accumulator, line_number.saturating_sub(1))?;
                        logger.log("TAPE", format!("Discovered Tape Head: [{}]", head.name));
                        head_opt = Some(head);
                        head_text_accumulator.clear();

                        raw_roll_lines.push((line_number, trimmed.to_string()));
                    }
                }
                continue;
            }

            raw_roll_lines.push((line_number, trimmed.to_string()));
        }

        // If multiline head was accumulating at EOF
        if head_opt.is_none() && !head_text_accumulator.is_empty() {
            let head = Self::parse_tape_head(&head_text_accumulator, 1)?;
            head_opt = Some(head);
        }

        let head = head_opt.unwrap_or_else(|| TapeHead {
            name: "Default_Tape".to_string(),
            raw_text: "[Default_Tape]".to_string(),
            ..Default::default()
        });

        // Group raw_roll_lines into cohesive, multiline-aware Rolls
        let mut rolls = Vec::new();
        let mut cur_roll_text = String::new();
        let mut cur_roll_line = 1;
        let mut bracket_balance: i32 = 0;

        let is_new_roll_start = |s: &str| -> bool {
            let upper = s.to_uppercase();
            upper.starts_with("ROLL_")
                || upper.starts_with("[ ROLL")
                || upper.starts_with("[ROLL")
        };

        for (line_num, line_str) in raw_roll_lines {
            let starts_new = is_new_roll_start(&line_str);

            if starts_new && !cur_roll_text.is_empty() && bracket_balance <= 0 {
```

```
                rolls.push(Roll {
                    index: rolls.len(),
                    line_number: cur_roll_line,
                    text: cur_roll_text.trim().to_string(),
                    status: QuadState::Proceed,
                    is_head: false,
                });
                cur_roll_text.clear();
                bracket_balance = 0;
            }

            if cur_roll_text.is_empty() {
                cur_roll_line = line_num;
            }

            for ch in line_str.chars() {
                match ch {
                    '[' | '{' => bracket_balance += 1,
                    ']' | '}' => bracket_balance -= 1,
                    _ => {}
                }
            }

            cur_roll_text.push_str(&line_str);
            cur_roll_text.push(' ');

            // If not an explicit roll_... format and bracket_balance == 0, flush standalone roll
            if !starts_new && !cur_roll_text.starts_with("roll_") && !cur_roll_text.to_uppercase().starts_with("[ ROLL")
&& bracket_balance <= 0 {
                rolls.push(Roll {
                    index: rolls.len(),
                    line_number: cur_roll_line,
                    text: cur_roll_text.trim().to_string(),
                    status: QuadState::Proceed,
                    is_head: false,
                });
                cur_roll_text.clear();
                bracket_balance = 0;
            }
        }

        if !cur_roll_text.trim().is_empty() {
            rolls.push(Roll {
                index: rolls.len(),
                line_number: cur_roll_line,
                text: cur_roll_text.trim().to_string(),
                status: QuadState::Proceed,
                is_head: false,
            });
        }

        Ok(Self {
            head,
            rolls,
            cursor: 0,
            bound_partition: None,
            execution_state: QuadState::Proceed,
        })
    }

    /// Loads and parses a `.tape` file from the filesystem.
    pub fn from_file(path: impl AsRef<Path>, logger: DiagnosticLogger) -> Result<Self, String> {
        let p = path.as_ref();
        let resolved = crate::core::filetype::resolve_track_path(p);
        let content = fs::read_to_string(&resolved)
            .map_err(|e| format!("Failed to read tape file {:?}: {e}", resolved))?;
        Self::from_source(&content, logger)
    }

    /// Parses the raw text of a Tape Head roll into structured cardinal clauses.
    fn parse_tape_head(text: &str, line_number: usize) -> Result<TapeHead, String> {
        let clean = text.trim();

        // 1. Extract Name (e.g., from `[ Process { State } ]`, `[Tape.Name]` or `[Cell]`)
        let name = if let Some(proc_idx) = clean.find("Process {") {
            if let Some(end_brace) = clean[proc_idx..].find('}') {
                clean[proc_idx + 9..proc_idx + end_brace].trim().to_string()
            } else {
```

```
                "Process".to_string()
            }
        } else if let Some(start) = clean.find('[') {
            if let Some(end) = clean[start..].find(']') {
                let inner = &clean[start + 1..start + end];
                inner.trim_start_matches("Tape.").trim().to_string()
            } else {
                format!("Tape_L{}", line_number)
            }
        } else {
            format!("Tape_L{}", line_number)
        };

        // 2. Extract Cardinal Clauses
        let who = extract_clause(clean, "WHO").unwrap_or_else(|| "System".to_string());
        let what = extract_clause(clean, "WHAT").unwrap_or_else(|| name.clone());
        let when_clause = extract_clause(clean, "WHEN").unwrap_or_else(|| "Opening".to_string());
        let where_clause = extract_clause(clean, "WHERE").unwrap_or_else(|| "Memory".to_string());
        let why = extract_parenthetical(clean, "WHY").unwrap_or_else(|| "Operational task".to_string());
        let how = extract_clause(clean, "HOW").unwrap_or_else(|| "+1 : PROCEED".to_string());

        // 3. Extract required partition size from WHERE clause
        let mut required_partition_size = 256;
        if clean.contains("WHERE") {
            for token in clean.split_whitespace() {
                if let Ok(num) = token.trim_matches(|c: char| !c.is_ascii_digit()).parse::<usize>() {
                    if num > 0 {
                        required_partition_size = num;
                        break;
                    }
                }
            }
        }

        Ok(TapeHead {
            name,
            who,
            what,
            when_clause,
            where_clause,
            why,
            how,
            required_partition_size,
            raw_text: clean.to_string(),
        })
    }

    /// Returns the roll currently under the Tape Head cursor.
    pub fn active_roll(&self) -> Option<&Roll> {
        self.rolls.get(self.cursor)
    }

    /// Advances the Tape Head cursor to the next roll, returning the newly focused roll.
    pub fn advance(&mut self) -> Option<&Roll> {
        if self.cursor < self.rolls.len() {
            self.cursor += 1;
        }
        self.active_roll()
    }

    /// Returns true if all subordinate rolls have been traversed.
    pub fn is_finished(&self) -> bool {
        self.cursor >= self.rolls.len()
    }

    /// Resets the Tape Head back to the first subordinate roll.
    pub fn reset(&mut self) {
        self.cursor = 0;
        for r in &mut self.rolls {
            r.status = QuadState::Proceed;
        }
        self.execution_state = QuadState::Proceed;
    }

    /// Rewinds the Tape Head cursor back to an explicit roll index.
    pub fn rewind_to(&mut self, target_index: usize) {
        if target_index < self.rolls.len() {
            self.cursor = target_index;
```

```
        }
    }

    /// Searches for a roll containing a scene landmark cue `< landmark >` or name.
    pub fn find_landmark_index(&self, landmark: &str) -> Option<usize> {
        let trimmed = landmark.trim();
        for (idx, roll) in self.rolls.iter().enumerate() {
            if roll.text.contains(&format!("< {trimmed} >"))
                || roll.text.contains(&format!("<{trimmed}>"))
                || roll.text.contains(&format!("::{trimmed}"))
                || roll.text.contains(trimmed)
            {
                return Some(idx);
            }
        }
        None
    }

    /// Total number of subordinate rolls on the tape.
    pub fn roll_count(&self) -> usize {
        self.rolls.len()
    }
}


/// Helper function to extract clause content bounded by braces `{...}` or brackets `[...]`.
fn extract_clause(text: &str, keyword: &str) -> Option<String> {
    if let Some(pos) = text.find(keyword) {
        let rest = &text[pos + keyword.len()..];
        // Look for { ... }
        if let Some(open) = rest.find('{') {
            if let Some(close) = rest[open..].find('}') {
                return Some(rest[open + 1..open + close].trim().to_string());
            }
        }
        // Look for [ ... ]
        if let Some(open) = rest.find('[') {
            if let Some(close) = rest[open..].find(']') {
                return Some(rest[open + 1..open + close].trim().to_string());
            }
        }
    }
    None
}

/// Helper function to extract aside content bounded by parentheses `(...)`.
fn extract_parenthetical(text: &str, keyword: &str) -> Option<String> {
    if let Some(pos) = text.find(keyword) {
        let rest = &text[pos + keyword.len()..];
        if let Some(open) = rest.find('(') {
            if let Some(close) = rest[open..].find(')') {
                return Some(rest[open + 1..open + close].trim().to_string());
            }
        }
    }
    None
}

// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests, Verification & Subsystem Sealing )
// ================================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_tape_parsing_and_head_extraction() {
        let source = r#"
            # Definition Roll / Tape Head
            [Tape.TaskWorker] : WHO {User} WHAT {Worker} --> DISPATCH {Worker}
                WHEN {Opening}
                WHERE {partition} = [ 512 ]
                WHY ( Secure background task execution )
                HOW [ +1 : PROCEED ]
                :: < covenant > ( verified )

```

```
            # Subordinate Rolls
            roll_1 [ 100 --> Buf @ 0 ] ( Initial write )
            roll_2 [ Buf[0] = 100 ! [ +1 ] ] ( Integrity verify )
            roll_3 [ commit_status [ PROCEED ] ] ( Tape sealed )
        "#;

        let logger = DiagnosticLogger::new(false);
        let mut tape = Tape::from_source(source, logger).expect("Parsing tape");

        assert_eq!(tape.head.name, "TaskWorker");
        assert_eq!(tape.head.who, "User");
        assert_eq!(tape.head.what, "Worker");
        assert_eq!(tape.head.when_clause, "Opening");
        assert_eq!(tape.head.why, "Secure background task execution");
        assert_eq!(tape.head.required_partition_size, 512);

        assert_eq!(tape.roll_count(), 3);
        assert_eq!(tape.cursor, 0);

        let r1 = tape.active_roll().unwrap();
        assert!(r1.text.contains("100 --> Buf @ 0"));

        tape.advance();
        assert_eq!(tape.cursor, 1);
        let r2 = tape.active_roll().unwrap();
        assert!(r2.text.contains("Buf[0] = 100"));

        tape.advance();
        assert_eq!(tape.cursor, 2);

        tape.advance();
        assert!(tape.is_finished());
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/core/ternary.rs RS 386 LINES 1528 WORDS 14.5 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Module Identity, Balanced Ternary Invariants, and Mathematical Basis )
// ============================================================================
//! OmniCode Core: Balanced Ternary Quad-State Engine
//!
//! Grounded in Balanced Radix-3 logic with a terminal Null/Void state:
//!     T4 = { -1 (Rollback), 0 (Await), +1 (Proceed), NULL (Halt) }
//!
//! Algebraic properties:
//!     - Symmetry: NOT(x) = -x
//!     - Conjunction (:: AND): min(A, B) with NULL absorption
//!     - Disjunction (| OR): max(A, B) with NULL absorption
//!     - Adversative Caveat (! BUT): Action A constrained by Caveat B

use std::fmt;

// Invariant QuadState Constants
pub const ROLLBACK: QuadState = QuadState::Rollback;
pub const AWAIT: QuadState = QuadState::Await;
pub const PROCEED: QuadState = QuadState::Proceed;
pub const HALT: QuadState = QuadState::Halt;


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Context Declarations, State Enum, and Constructor Initialization )
// ============================================================================
/// The four fundamental ontological states of computation in OmniCode.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum QuadState {
    /// -1: NO / Rollback. Triggers retroactive vector (`<--`), unwinds mutated state.
    Rollback,
    /// 0: OK / Await. Equilibrium, standby, asynchronous yield, pass-through.
    Await,
    /// +1: YES / Proceed. Advances forward vector (`-->`), commits state.
    Proceed,
    /// NULL: VOID / Halt. Bottom type (⊥), boundary trap, terminal exit.
    Halt,
}

/// The cinematic transport kinematics of a reel along a track.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum TransportAction {
    /// Advance the track forward (matches +1, YES, PROCEED)
    Play,
    /// Hold the spindle in equilibrium (matches 0, OK, AWAIT)
    Pause,
    /// Rewind the track back to scene landmark (matches -1, NO, ROLLBACK)
    Rewind,
    /// Emergency brake detent / halt (matches NULL, VOID, HALT)
    Stop,
}

impl TransportAction {
    pub fn to_quad_state(&self) -> QuadState {
        match self {
            TransportAction::Play => QuadState::Proceed,
            TransportAction::Pause => QuadState::Await,
            TransportAction::Rewind => QuadState::Rollback,
            TransportAction::Stop => QuadState::Halt,
        }
    }

    pub fn as_str(&self) -> &'static str {
        match self {
            TransportAction::Play => "PLAY",
            TransportAction::Pause => "PAUSE",
            TransportAction::Rewind => "REWIND",
            TransportAction::Stop => "STOP",
        }
    }
}

```

```rust
impl QuadState {
    /// Parses a raw integer or string representation into a QuadState.
    ///
    /// # Failpoints
    /// Returns `Err` if the value does not belong to {-1, 0, 1, NULL}.
    pub fn from_raw(raw: &str) -> Result<Self, String> {
        match raw.trim().to_uppercase().as_str() {
            "-1" | "NO" | "ROLLBACK" | "REWIND" => Ok(QuadState::Rollback),
            "0" | "OK" | "AWAIT" | "PAUSE" => Ok(QuadState::Await),
            "1" | "+1" | "YES" | "PROCEED" | "PLAY" => Ok(QuadState::Proceed),
            "NULL" | "VOID" | "HALT" | "STOP" => Ok(QuadState::Halt),
            other => Err(format!(
                "FAILPOINT [InvalidQuadState]: '{other}' is not a valid balanced ternary state. \
                 Permitted states: -1, 0, +1, NULL (or PLAY, PAUSE, REWIND, STOP)"
            )),
        }
    }

    /// Transport kinematics level (cinematic reel/track motor):
    /// - Proceed: Play
    /// - Await: Pause
    /// - Rollback: Rewind
    /// - Halt: Stop
    pub fn transport(&self) -> TransportAction {
        match self {
            QuadState::Proceed => TransportAction::Play,
            QuadState::Await => TransportAction::Pause,
            QuadState::Rollback => TransportAction::Rewind,
            QuadState::Halt => TransportAction::Stop,
        }
    }

    /// Macro transport operation string: "PLAY", "PAUSE", "REWIND", "STOP"
    pub fn transport_str(&self) -> &'static str {
        self.transport().as_str()
    }


    /// Returns the integer value, or None for Halt (NULL).
    pub fn as_int(&self) -> Option<i8> {
        match self {
            QuadState::Rollback => Some(-1),
            QuadState::Await => Some(0),
            QuadState::Proceed => Some(1),
            QuadState::Halt => None,
        }
    }

    /// Micro level (linguistic / logical / epistemic stance):
    /// - Proceed: "yes" (1)
    /// - Rollback: "no" (-1)
    /// - Await: "ok" (0)
    /// - Halt: "void" (NULL)
    pub fn micro(&self) -> &'static str {
        match self {
            QuadState::Proceed => "yes",
            QuadState::Rollback => "no",
            QuadState::Await => "ok",
            QuadState::Halt => "void",
        }
    }

    /// Macro level (systems / OS state-machine physics):
    /// - Proceed: "PROCEED" (commit state / advance PC)
    /// - Rollback: "ROLLBACK" (unwind snapshot / retry)
    /// - Await: "AWAIT" (async yield / wait queue)
    /// - Halt: "HALT" (boundary trap / terminal exit)
    pub fn macro_op(&self) -> &'static str {
        match self {
            QuadState::Proceed => "PROCEED",
            QuadState::Rollback => "ROLLBACK",
            QuadState::Await => "AWAIT",
            QuadState::Halt => "HALT",
        }
    }

    /// Bridges the Micro and Macro ontological perspective.
    pub fn description(&self) -> &'static str {
```

```
        match self {
            QuadState::Proceed => "+1 / yes (Micro: Affirmation) | PROCEED (Macro: Forward commit)",
            QuadState::Rollback => "-1 / no (Micro: Refutation) | ROLLBACK (Macro: Retroactive rewind)",
            QuadState::Await => "0 / ok (Micro: Neutral balance) | AWAIT (Macro: Async yield / Standby)",
            QuadState::Halt => "NULL / void (Micro: Absence) | HALT (Macro: Terminal trap / Exit)",
        }
    }
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Core Algebraic Transformations: Inversion, Conjunction, Disjunction, Caveat )
// ============================================================================
impl QuadState {
    /// Algebraic Inversion (`--` NOT).
    ///
    /// Rules:
    /// - `--(+1)` == `-1` (NOT Proceed is Rollback)
    /// - `--(-1)` == `+1` (NOT Rollback is Proceed)
    /// - `--(0)`  ==  `0` (NOT Await is Await - zero is symmetric balance)
    /// - `--(NULL)` == `NULL` (NOT Halt is Halt - void remains void)
    pub fn invert(&self) -> Self {
        match self {
            QuadState::Proceed => QuadState::Rollback,
            QuadState::Rollback => QuadState::Proceed,
            QuadState::Await => QuadState::Await,
            QuadState::Halt => QuadState::Halt,
        }
    }

    /// Cumulative Conjunction (`::` AND).
    ///
    /// Evaluates to minimum with NULL acting as an absorbing trap.
    pub fn and(&self, other: Self) -> Self {
        if *self == QuadState::Halt || other == QuadState::Halt {
            return QuadState::Halt;
        }
        let a = self.as_int().unwrap();
        let b = other.as_int().unwrap();
        match a.min(b) {
            -1 => QuadState::Rollback,
            0 => QuadState::Await,
            1 => QuadState::Proceed,
            _ => unreachable!(),
        }
    }

    /// Disjunction (`|` OR).
    ///
    /// Evaluates to maximum with NULL acting as an absorbing trap.
    pub fn or(&self, other: Self) -> Self {
        if *self == QuadState::Halt || other == QuadState::Halt {
            return QuadState::Halt;
        }
        let a = self.as_int().unwrap();
        let b = other.as_int().unwrap();
        match a.max(b) {
            -1 => QuadState::Rollback,
            0 => QuadState::Await,
            1 => QuadState::Proceed,
            _ => unreachable!(),
        }
    }

    /// Adversative Caveat (`!` BUT).
    ///
    /// `action.but(caveat)`: Action proceeds ONLY if caveat does not veto.
    /// - Caveat = Rollback (-1) -> forces Rollback (-1)
    /// - Caveat = Halt (NULL)   -> forces Halt (NULL)
    /// - Caveat = Await (0)     -> degrades Proceed to Await (0)
    /// - Caveat = Proceed (+1)  -> action executes unhindered
    pub fn but(&self, caveat: Self) -> Self {
        match caveat {
            QuadState::Halt => QuadState::Halt,
            QuadState::Rollback => QuadState::Rollback,
            QuadState::Await => {
                if *self == QuadState::Proceed {
```

```
                    QuadState::Await
                } else {
                    *self
                }
            }
            QuadState::Proceed => *self,
        }
    }

    // ---------------------------------------------------------------------------
    // Dual-Rail Balanced Ternary Substrate & Rail Kinematics
    // ---------------------------------------------------------------------------

    /// Returns the 2-bit dual-rail hardware encoding of this QuadState:
    /// - `00` = 0 / AWAIT (neutral equilibrium)
    /// - `01` = +1 / PROCEED (positive rail asserted)
    /// - `10` = -1 / ROLLBACK (negative rail asserted)
    /// - `11` = NULL / HALT (collision trap / boundary fault)
    pub fn to_dual_rail_code(&self) -> u8 {
        match self {
            QuadState::Await => 0b00,
            QuadState::Proceed => 0b01,
            QuadState::Rollback => 0b10,
            QuadState::Halt => 0b11,
        }
    }

    /// Reconstructs a QuadState from a 2-bit dual-rail encoding.
    pub fn from_dual_rail_code(code: u8) -> Self {
        match code & 0b11 {
            0b00 => QuadState::Await,
            0b01 => QuadState::Proceed,
            0b10 => QuadState::Rollback,
            0b11 => QuadState::Halt,
            _ => unreachable!(),
        }
    }

    /// Returns the dual-rail bit-plane tuple `(pos_rail, neg_rail)`.
    pub fn to_rail_pair(&self) -> (u8, u8) {
        match self {
            QuadState::Await => (0, 0),
            QuadState::Proceed => (1, 0),
            QuadState::Rollback => (0, 1),
            QuadState::Halt => (1, 1),
        }
    }

    /// Reconstructs a QuadState from separate `(pos_rail, neg_rail)` bits.
    pub fn from_rail_pair(pos: u8, neg: u8) -> Self {
        match (pos & 1, neg & 1) {
            (0, 0) => QuadState::Await,
            (1, 0) => QuadState::Proceed,
            (0, 1) => QuadState::Rollback,
            (1, 1) => QuadState::Halt,
            _ => unreachable!(),
        }
    }

    /// SIMD Rail-Swap Negation: swaps the positive and negative rails,
    /// inverting +1 to -1 and -1 to +1 while leaving 0 and NULL invariant.
    pub fn simd_rail_swap(&self) -> Self {
        let (pos, neg) = self.to_rail_pair();
        Self::from_rail_pair(neg, pos)
    }

    /// Returns `true` if this state represents a dual-rail collision trap (NULL).
    pub fn is_collision(&self) -> bool {
        *self == QuadState::Halt
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Verification, Formatter Proofs, Display Formatting, and Export )
// ============================================================================
impl fmt::Display for QuadState {
```

```
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            QuadState::Rollback => write!(f, "-1:ROLLBACK"),
            QuadState::Await => write!(f, "0:AWAIT"),
            QuadState::Proceed => write!(f, "+1:PROCEED"),
            QuadState::Halt => write!(f, "NULL:HALT"),
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_quad_state_micro_and_macro() {
        // Micro labels (linguistic stance)
        assert_eq!(QuadState::Proceed.micro(), "yes");
        assert_eq!(QuadState::Rollback.micro(), "no");
        assert_eq!(QuadState::Await.micro(), "ok");
        assert_eq!(QuadState::Halt.micro(), "void");

        // Macro labels (systems execution physics)
        assert_eq!(QuadState::Proceed.macro_op(), "PROCEED");
        assert_eq!(QuadState::Rollback.macro_op(), "ROLLBACK");
        assert_eq!(QuadState::Await.macro_op(), "AWAIT");
        assert_eq!(QuadState::Halt.macro_op(), "HALT");

        // Parsing micro and macro strings
        assert_eq!(QuadState::from_raw("yes").unwrap(), QuadState::Proceed);
        assert_eq!(QuadState::from_raw("no").unwrap(), QuadState::Rollback);
        assert_eq!(QuadState::from_raw("ok").unwrap(), QuadState::Await);
        assert_eq!(QuadState::from_raw("void").unwrap(), QuadState::Halt);

        assert_eq!(QuadState::from_raw("PROCEED").unwrap(), QuadState::Proceed);
        assert_eq!(QuadState::from_raw("ROLLBACK").unwrap(), QuadState::Rollback);
        assert_eq!(QuadState::from_raw("AWAIT").unwrap(), QuadState::Await);
        assert_eq!(QuadState::from_raw("HALT").unwrap(), QuadState::Halt);
    }

    #[test]
    fn test_quad_state_dual_rail_and_rail_swap() {
        // Dual-rail encodings: 00=Await, 01=Proceed, 10=Rollback, 11=Halt
        assert_eq!(QuadState::Await.to_dual_rail_code(), 0b00);
        assert_eq!(QuadState::Proceed.to_dual_rail_code(), 0b01);
        assert_eq!(QuadState::Rollback.to_dual_rail_code(), 0b10);
        assert_eq!(QuadState::Halt.to_dual_rail_code(), 0b11);

        assert_eq!(QuadState::from_dual_rail_code(0b00), QuadState::Await);
        assert_eq!(QuadState::from_dual_rail_code(0b01), QuadState::Proceed);
        assert_eq!(QuadState::from_dual_rail_code(0b10), QuadState::Rollback);
        assert_eq!(QuadState::from_dual_rail_code(0b11), QuadState::Halt);

        // Rail pairs (pos, neg)
        assert_eq!(QuadState::Await.to_rail_pair(), (0, 0));
        assert_eq!(QuadState::Proceed.to_rail_pair(), (1, 0));
        assert_eq!(QuadState::Rollback.to_rail_pair(), (0, 1));
        assert_eq!(QuadState::Halt.to_rail_pair(), (1, 1));

        // SIMD rail swap: (pos, neg) -> (neg, pos)
        // Proceed (+1) swaps to Rollback (-1)
        assert_eq!(QuadState::Proceed.simd_rail_swap(), QuadState::Rollback);
        // Rollback (-1) swaps to Proceed (+1)
        assert_eq!(QuadState::Rollback.simd_rail_swap(), QuadState::Proceed);
        // Await (0) swaps to Await (0)
        assert_eq!(QuadState::Await.simd_rail_swap(), QuadState::Await);
        // Halt (NULL) swaps to Halt (NULL)
        assert_eq!(QuadState::Halt.simd_rail_swap(), QuadState::Halt);

        // Collision check
        assert!(!QuadState::Proceed.is_collision());
        assert!(!QuadState::Rollback.is_collision());
        assert!(!QuadState::Await.is_collision());
        assert!(QuadState::Halt.is_collision());
    }
}

```

<TRACK 1> : {Host Engine & Virtual Machine}/src/core/vessel.rs RS 320 LINES 1088 WORDS 11.2 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Vessel Subsystem Identity, Memory Containers & Boundary Invariants )
// ============================================================================
//! OmniCode Core: Memory Vessel & Storehouse Subsystem
//!
//! Codifies the foundational storage nouns of OmniCode:
//!     - Units: Trit, Cell, Word
//!     - Vessels: Vessel, Storehouse, Partition
//!     - Coordinates: Offset, Origin, Landmark
//!     - Measures: Capacity, Count
//!     - Substance: Value, State
//!
//! Governed by Invariant Boundary Physics: An access beyond `Capacity`
//! produces an immediate `-1: ROLLBACK` (Caveat Veto), preserving memory integrity.

use std::fmt;
use crate::core::cell::Cell;
use crate::core::linguistics::{GrammaticalClassification, GrammaticalEntity, PartOfSpeech};
use crate::core::ternary::QuadState;


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Vessel, Partition, and Coordinate Struct Definitions )
// ============================================================================

/// A bounded container of memory cells.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct Vessel {
    /// Canonical identifier / name of the vessel.
    pub name: String,
    /// Physical or logical origin address / base coordinate.
    pub origin: usize,
    /// Maximum number of cells this vessel is allocated to hold.
    pub capacity: usize,
    /// Number of cells currently written or active.
    pub count: usize,
    /// Dense cell array (where each cell packs to exactly 1 byte in physical RAM).
    pub cells: Vec<Cell>,
    /// Whether this vessel acts as an ambient system Storehouse.
    pub is_storehouse: bool,
    /// If this vessel is a Partition, references the parent Storehouse name.
    pub parent: Option<String>,
}

/// Coordinate offset within a vessel (`Vessel[Offset]`).
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub struct Offset(pub usize);

/// Measurement extent of a vessel's total bounds.
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub struct Capacity(pub usize);

/// Measurement count of active cells.
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub struct Count(pub usize);


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Vessel Allocation, Partitioning, Bounds Checking, and Cell Read/Write )
// ============================================================================

impl Vessel {
    /// Allocates a new bounded Vessel with the given capacity.
    /// Initialized with empty equilibrium cells (`AWAIT`).
    pub fn new(name: &str, capacity: usize) -> Self {
        Self {
            name: name.to_string(),
            origin: 0,
            capacity,
            count: 0,
            cells: vec![Cell::equilibrium(); capacity],
```

```
            is_storehouse: false,
            parent: None,
        }
    }

    /// Allocates an ambient system Storehouse.
    pub fn new_storehouse(name: &str, capacity: usize) -> Self {
        let mut v = Self::new(name, capacity);
        v.is_storehouse = true;
        v
    }

    /// Grants an entrusted, bounded Partition from this Storehouse.
    ///
    /// # Invariants
    /// - The partition capacity cannot exceed the remaining unallocated capacity.
    /// - Breaches within the partition are contained and cannot overrun the parent.
    pub fn partition(&mut self, partition_name: &str, partition_capacity: usize) -> Result<Self, String> {
        let available = self.capacity.saturating_sub(self.count);
        if partition_capacity > available {
            return Err(format!(
                "FAILPOINT [StorehouseExhaustion]: Requested partition '{partition_name}' of {partition_capacity} cells, \
                 but Storehouse '{}' only has {available} cells remaining (Capacity: {}, Count: {}).",
                self.name, self.capacity, self.count
            ));
        }

        let partition_origin = self.origin + self.count;
        self.count += partition_capacity;

        Ok(Self {
            name: partition_name.to_string(),
            origin: partition_origin,
            capacity: partition_capacity,
            count: 0,
            cells: vec![Cell::equilibrium(); partition_capacity],
            is_storehouse: false,
            parent: Some(self.name.clone()),
        })
    }

    /// Writes a Cell into this vessel at the specified Offset.
    ///
    /// # Invariant Physics
    /// If `offset >= capacity`, boundary physics triggers an immediate `-1: ROLLBACK`!
    pub fn write(&mut self, offset: usize, value: Cell) -> Result<QuadState, String> {
        if offset >= self.capacity {
            return Ok(QuadState::Rollback);
        }

        self.cells[offset] = value;
        if offset >= self.count {
            self.count = offset + 1;
        }

        Ok(QuadState::Proceed)
    }

    /// Reads a Cell from this vessel at the specified Offset.
    ///
    /// # Invariant Physics
    /// If `offset >= capacity`, returns `Cell::from_state(QuadState::Rollback)`.
    pub fn read(&self, offset: usize) -> Cell {
        if offset >= self.capacity {
            Cell::from_state(QuadState::Rollback)
        } else {
            self.cells[offset]
        }
    }

    /// Packs the entire vessel into a dense array of physical 8-bit bytes (1 byte per cell).
    pub fn pack_bytes(&self) -> Vec<u8> {
        self.cells.iter().map(|c| c.pack()).collect()
    }

    /// Reconstructs a Vessel directly from raw physical bytes (e.g. from RAM, MMIO, or disk).
    pub fn unpack_bytes(name: &str, bytes: &[u8]) -> Self {
```

```
        let cells: Vec<Cell> = bytes.iter().map(|&b| Cell::unpack(b)).collect();
        let capacity = cells.len();
        Self {
            name: name.to_string(),
            origin: 0,
            capacity,
            count: capacity,
            cells,
            is_storehouse: false,
            parent: None,
        }
    }
}

impl GrammaticalEntity for Vessel {
    fn classify(&self) -> GrammaticalClassification {
        let role = if self.is_storehouse {
            "System Storehouse; root reservoir of physical or virtual memory cells"
        } else if self.parent.is_some() {
            "Entrusted Partition; bounded memory slice derived from a storehouse"
        } else {
            "Bounded memory vessel; holds sequential cells under invariant law"
        };

        GrammaticalClassification::new(
            PartOfSpeech::Noun,
            Some(PartOfSpeech::Preposition),
            None,
            role,
            "Contiguous memory block / Buffer / Page frame",
        )
    }
}

impl GrammaticalEntity for Offset {
    fn classify(&self) -> GrammaticalClassification {
        GrammaticalClassification::new(
            PartOfSpeech::Preposition,
            Some(PartOfSpeech::Noun),
            None,
            "Spatial coordinate index within a vessel: Vessel[Offset]",
            "Array index / Memory byte offset",
        )
    }
}

impl GrammaticalEntity for Capacity {
    fn classify(&self) -> GrammaticalClassification {
        GrammaticalClassification::new(
            PartOfSpeech::Noun,
            Some(PartOfSpeech::Adjective),
            None,
            "Total allocated capacity measure of a vessel",
            "Buffer size / Maximum allocation limit",
        )
    }
}

impl GrammaticalEntity for Count {
    fn classify(&self) -> GrammaticalClassification {
        GrammaticalClassification::new(
            PartOfSpeech::Noun,
            Some(PartOfSpeech::Adjective),
            None,
            "Active or written element count within a vessel",
            "Active buffer length / Populated element count",
        )
    }
}

impl fmt::Display for Vessel {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        let kind = if self.is_storehouse {
            "Storehouse"
        } else if self.parent.is_some() {
            "Partition"
        } else {
            "Vessel"
```

```
        };
        write!(
            f,
            "{kind} < {} > [ Capacity: {}, Count: {}, Origin: 0x{:X} ]",
            self.name, self.capacity, self.count, self.origin
        )
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Vessel Invariant Tests, Bounds Safety & Partition Verification )
// ============================================================================
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_vessel_creation_and_bounds_checking() {
        let mut vessel = Vessel::new("Scratchpad", 10);
        assert_eq!(vessel.capacity, 10);
        assert_eq!(vessel.count, 0);

        // In-bounds write
        let cell42 = Cell::from_natural(42).unwrap();
        let res = vessel.write(0, cell42).unwrap();
        assert_eq!(res, QuadState::Proceed);
        assert_eq!(vessel.count, 1);

        // Read back
        let read_cell = vessel.read(0);
        assert_eq!(read_cell.to_natural(), Some(42));
        assert_eq!(read_cell.state, QuadState::Proceed);

        // Out-of-bounds write -> Automatic ROLLBACK (-1)
        let oob_res = vessel.write(15, cell42).unwrap();
        assert_eq!(oob_res, QuadState::Rollback);

        // Out-of-bounds read -> Automatic ROLLBACK cell
        let oob_read = vessel.read(15);
        assert_eq!(oob_read.state, QuadState::Rollback);
    }

    #[test]
    fn test_storehouse_partitioning() {
        let mut storehouse = Vessel::new_storehouse("Main_RAM", 1024);
        assert!(storehouse.is_storehouse);

        // Partition 256 cells
        let mut partition = storehouse.partition("Process_1_Stack", 256).unwrap();
        assert_eq!(partition.capacity, 256);
        assert_eq!(partition.origin, 0);
        assert_eq!(storehouse.count, 256);

        // Write inside partition
        let val = Cell::from_natural(99).unwrap();
        assert_eq!(partition.write(255, val).unwrap(), QuadState::Proceed);
        assert_eq!(partition.count, 256);

        // Boundary breach inside partition triggers rollback
        assert_eq!(partition.write(256, val).unwrap(), QuadState::Rollback);

        // Storehouse exhaustion
        let err = storehouse.partition("Excessive", 1000);
        assert!(err.is_err());
    }

    #[test]
    fn test_dense_byte_packing_round_trip() {
        let mut vessel = Vessel::new("PackedVessel", 4);
        vessel.write(0, Cell::from_natural(10).unwrap()).unwrap();
        vessel.write(1, Cell::from_natural(20).unwrap()).unwrap();
        vessel.write(2, Cell::from_state(QuadState::Rollback)).unwrap();
        vessel.write(3, Cell::from_state(QuadState::Halt)).unwrap();

        let bytes = vessel.pack_bytes();
        assert_eq!(bytes.len(), 4);
```

```
        assert_eq!(bytes[0], 10);
        assert_eq!(bytes[1], 20);
        assert_eq!(bytes[2], crate::core::cell::BYTE_ROLLBACK);
        assert_eq!(bytes[3], crate::core::cell::BYTE_HALT);

        let reconstructed = Vessel::unpack_bytes("Reconstructed", &bytes);
        assert_eq!(reconstructed.read(0).to_natural(), Some(10));
        assert_eq!(reconstructed.read(1).to_natural(), Some(20));
        assert_eq!(reconstructed.read(2).state, QuadState::Rollback);
        assert_eq!(reconstructed.read(3).state, QuadState::Halt);
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/core/word.rs RS 676 LINES 2608 WORDS 22.6 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Physical Machine Word Architecture: 64-Bit Silicon & 32-Trit OS Substrate )
// ================================================================================
//! OmniCode Core: Physical 64-Bit Machine Word & Logical 32-Trit Vector
//!
//! Bridges physical 64-bit binary silicon hardware with the 32-trit balanced
//! ternary operating system kernel:
//!
//! # Bitfield Layout (64 Physical Bits)
//! - **Bits 0..50 (51 bits):** 32-Trit Balanced Ternary Payload (Range: $3^{32} \approx 1.85 \times 10^{15}$)
//! - **Bits 51..52 (2 bits):** Quad-State / Transport Action (Play, Pause, Rewind, Stop)
//! - **Bit 53 (1 bit):** Polarity Affix (0 = Abstract `+`, 1 = Concrete `-`)
//! - **Bits 54..56 (3 bits):** Boundary Rank Delimiter (`()`, `[]`, `<>`, `{}`)
//! - **Bits 57..63 (7 bits):** Moral Invariant Checksum & Axiom Flags (Truth, Life, Justice, etc.)

use std::fmt;
use crate::core::boundaries::BoundaryType;
use crate::core::operators::Polarity;
use crate::core::ternary::{QuadState, TransportAction};

pub const POWERS_OF_3: [u64; 33] = {
    let mut p = [0u64; 33];
    p[0] = 1;
    let mut i = 1;
    while i <= 32 {
        p[i] = p[i - 1] * 3;
        i += 1;
    }
    p
};

/// Dual-rail bit-plane masks (32 trits x 2 bits = 64 bits native)
pub const BITPLANE_POS_MASK: u64 = 0x5555_5555_5555_5555;
pub const BITPLANE_NEG_MASK: u64 = 0xAAAA_AAAA_AAAA_AAAA;

/// 2-bit dual-rail polar encodings:
/// 00 = 0 (Equilibrium)
/// 01 = +1 (Positive rail)
/// 10 = -1 (Negative rail)
/// 11 = NULL (Collision trap)
pub const TRIT_CODE_ZERO: u8 = 0;
pub const TRIT_CODE_POS: u8 = 1;
pub const TRIT_CODE_NEG: u8 = 2;
pub const TRIT_CODE_NULL: u8 = 3;

/// Kinematic bit-packing topologies across 8-bit byte windows
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum PackMode {
    /// 2-2-2-2: Standard dense storage (4 trits)
    Dense2222,
    /// 2-1-2-1-2: Symmetric resting equilibrium with carry channels
    Kinematic21212,
    /// 1-2-1-2-2: Prefix carry/overflow channel
    Kinematic12122,
    /// 2-2-1-2-1: Postfix carry/overflow channel
    Kinematic22121,
    /// 3-3-2: Directional vector (X:3, Y:3, Z:2)
    Directional332,
    /// 3-2-3: Directional vector (X:3, Y:2, Z:3)
    Directional323,
    /// 2-3-3: Directional vector (X:2, Y:3, Z:3)
    Directional233,
}

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Word32Trit: 32-Trit Balanced Ternary ALU Vector )
// ================================================================================

/// A 32-trit balanced ternary word.
/// Each trit is in `{-1, 0, +1}`.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub struct Word32Trit {
```

```
    pub trits: [i8; 32],
}

impl Default for Word32Trit {
    fn default() -> Self {
        Self::zero()
    }
}

impl Word32Trit {
    /// Creates a zeroed 32-trit word (equilibrium: all 0).
    pub const fn zero() -> Self {
        Self { trits: [0; 32] }
    }

    /// Creates a 32-trit word from an explicit array of trits.
    pub fn new(trits: [i8; 32]) -> Result<Self, String> {
        for (idx, &t) in trits.iter().enumerate() {
            if t < -1 || t > 1 {
                return Err(format!("Invalid trit at index {idx}: {t}. Permitted: -1, 0, 1"));
            }
        }
        Ok(Self { trits })
    }

    /// Converts a signed 64-bit integer into a balanced ternary 32-trit word.
    pub fn from_i64(mut val: i64) -> Self {
        let mut trits = [0i8; 32];
        for i in 0..32 {
            if val == 0 {
                break;
            }
            let rem = ((val % 3) + 3) % 3;
            if rem == 0 {
                trits[i] = 0;
                val /= 3;
            } else if rem == 1 {
                trits[i] = 1;
                val = (val - 1) / 3;
            } else {
                // rem == 2 -> trit is -1, adjust carry (+1)
                trits[i] = -1;
                val = (val + 1) / 3;
            }
        }
        Self { trits }
    }

    /// Decodes the 32-trit word into a standard signed 64-bit integer.
    pub fn to_i64(&self) -> i64 {
        let mut total: i64 = 0;
        let mut power: i64 = 1;
        for i in 0..32 {
            total = total.saturating_add((self.trits[i] as i64).saturating_mul(power));
            if i < 31 {
                power = power.saturating_mul(3);
            }
        }
        total
    }

    /// Negates the 32-trit word in-place with zero sign-extension cost.
    pub fn negate(&self) -> Self {
        let mut trits = [0i8; 32];
        for i in 0..32 {
            trits[i] = -self.trits[i];
        }
        Self { trits }
    }

    /// Balanced ternary full-adder for two 32-trit words.
    pub fn add(&self, other: &Self) -> (Self, i8) {
        let mut trits = [0i8; 32];
        let mut carry: i8 = 0;

        for i in 0..32 {
            let sum = self.trits[i] + other.trits[i] + carry;
            match sum {
```

```
                3 => {
                    trits[i] = 0;
                    carry = 1;
                }
                2 => {
                    trits[i] = -1;
                    carry = 1;
                }
                1 => {
                    trits[i] = 1;
                    carry = 0;
                }
                0 => {
                    trits[i] = 0;
                    carry = 0;
                }
                -1 => {
                    trits[i] = -1;
                    carry = 0;
                }
                -2 => {
                    trits[i] = 1;
                    carry = -1;
                }
                -3 => {
                    trits[i] = 0;
                    carry = -1;
                }
                _ => unreachable!("Sum out of range: {sum}"),
            }
        }
        (Self { trits }, carry)
    }

    /// Packs the 32-trit word into an unsigned 51-bit binary representation.
    pub fn pack_51bits(&self) -> u64 {
        let mut packed: u64 = 0;
        for i in 0..32 {
            let digit = (self.trits[i] + 1) as u64; // Map -1,0,1 to 0,1,2
            packed += digit * POWERS_OF_3[i];
        }
        packed
    }

    /// Unpacks a 32-trit word from an unsigned 51-bit binary representation.
    pub fn unpack_51bits(mut val: u64) -> Self {
        let mut trits = [0i8; 32];
        for i in 0..32 {
            let rem = (val % 3) as i8;
            trits[i] = rem - 1; // Map 0,1,2 back to -1,0,1
            val /= 3;
        }
        Self { trits }
    }

    /// Encodes a single trit into a 2-bit polar pair:
    /// 0 -> 0b00, 1 -> 0b01, -1 -> 0b10, other -> 0b11.
    pub fn trit_to_2bit(trit: i8) -> u8 {
        match trit {
            0 => TRIT_CODE_ZERO,
            1 => TRIT_CODE_POS,
            -1 => TRIT_CODE_NEG,
            _ => TRIT_CODE_NULL,
        }
    }

    /// Decodes a 2-bit polar pair into a balanced trit:
    /// 0b00 -> 0, 0b01 -> +1, 0b10 -> -1, 0b11 -> Err(Collision).
    pub fn two_bit_to_trit(code: u8) -> Result<i8, String> {
        match code & 0x03 {
            TRIT_CODE_ZERO => Ok(0),
            TRIT_CODE_POS => Ok(1),
            TRIT_CODE_NEG => Ok(-1),
            _ => Err("Dual-rail collision trap: code 11 (both rails active)".to_string()),
        }
    }

    /// Packs the 32-trit word into a 64-bit dual-rail polar bit-plane word.
```

```
    pub fn pack_dual_rail(&self) -> u64 {
        let mut packed: u64 = 0;
        for i in 0..32 {
            let code = Self::trit_to_2bit(self.trits[i]) as u64;
            packed |= (code & 0x03) << (i * 2);
        }
        packed
    }

    /// Unpacks a 64-bit dual-rail polar bit-plane word into a 32-trit word.
    /// Trips interlock (Err) if any 2-bit slot has collision code 11.
    pub fn unpack_dual_rail(val: u64) -> Result<Self, String> {
        let mut trits = [0i8; 32];
        for i in 0..32 {
            let code = ((val >> (i * 2)) & 0x03) as u8;
            trits[i] = Self::two_bit_to_trit(code)?;
        }
        Ok(Self { trits })
    }

    /// Checks if a 64-bit dual-rail word has a NULL collision (code 11).
    pub fn has_null_collision(val: u64) -> bool {
        let pos = val & BITPLANE_POS_MASK;
        let neg = (val >> 1) & BITPLANE_POS_MASK;
        (pos & neg) != 0
    }

    /// Negates a 64-bit dual-rail packed word in O(1) via SIMD rail swap.
    pub fn simd_rail_swap_negate(val: u64) -> u64 {
        let pos = val & BITPLANE_POS_MASK;
        let neg = val & BITPLANE_NEG_MASK;
        ((pos << 1) & BITPLANE_NEG_MASK) | ((neg >> 1) & BITPLANE_POS_MASK)
    }

    /// Packs a slice of trits into a single byte using the specified kinematic topology.
    pub fn pack_byte(trits: &[i8], mode: PackMode) -> u8 {
        let get = |idx: usize| -> u8 {
            if idx < trits.len() {
                Self::trit_to_2bit(trits[idx]) & 0x03
            } else {
                0
            }
        };
        let mut b: u8 = 0;
        match mode {
            PackMode::Dense2222 => {
                for i in 0..4 {
                    b |= get(i) << (i * 2);
                }
            }
            PackMode::Kinematic21212 => {
                b |= get(0) & 0x03;
                b |= (get(1) & 0x01) << 2;
                b |= (get(1) & 0x03) << 3;
                b |= ((get(1) >> 1) & 0x01) << 5;
                b |= (get(2) & 0x03) << 6;
            }
            PackMode::Kinematic12122 => {
                b |= (get(0) & 0x01) << 0;
                b |= (get(0) & 0x03) << 1;
                b |= ((get(1) >> 1) & 0x01) << 3;
                b |= (get(1) & 0x03) << 4;
                b |= (get(2) & 0x03) << 6;
            }
            PackMode::Kinematic22121 => {
                b |= (get(0) & 0x03) << 0;
                b |= (get(1) & 0x03) << 2;
                b |= (get(2) & 0x01) << 4;
                b |= (get(2) & 0x03) << 5;
                b |= ((get(2) >> 1) & 0x01) << 7;
            }
            PackMode::Directional332 => {
                b |= get(0) << 0;
                b |= get(1) << 3;
                b |= get(2) << 6;
            }
            PackMode::Directional323 => {
                b |= get(0) << 0;
```

```
                b |= get(1) << 3;
                b |= get(2) << 5;
            }
            PackMode::Directional233 => {
                b |= get(0) << 0;
                b |= get(1) << 2;
                b |= get(2) << 5;
            }
        }
        b
    }

    /// Unpacks a single byte into up to 4 trits using the specified kinematic topology.
    pub fn unpack_byte(b: u8, mode: PackMode) -> Result<[i8; 4], String> {
        let mut out = [0i8; 4];
        match mode {
            PackMode::Dense2222 => {
                for i in 0..4 {
                    out[i] = Self::two_bit_to_trit((b >> (i * 2)) & 0x03)?;
                }
            }
            PackMode::Kinematic21212 => {
                out[0] = Self::two_bit_to_trit(b & 0x03)?;
                out[1] = Self::two_bit_to_trit((b >> 3) & 0x03)?;
                out[2] = Self::two_bit_to_trit((b >> 6) & 0x03)?;
                out[3] = 0;
            }
            PackMode::Kinematic12122 => {
                out[0] = Self::two_bit_to_trit((b >> 1) & 0x03)?;
                out[1] = Self::two_bit_to_trit((b >> 4) & 0x03)?;
                out[2] = Self::two_bit_to_trit((b >> 6) & 0x03)?;
                out[3] = 0;
            }
            PackMode::Kinematic22121 => {
                out[0] = Self::two_bit_to_trit(b & 0x03)?;
                out[1] = Self::two_bit_to_trit((b >> 2) & 0x03)?;
                out[2] = Self::two_bit_to_trit((b >> 5) & 0x03)?;
                out[3] = 0;
            }
            PackMode::Directional332 => {
                out[0] = Self::two_bit_to_trit(b & 0x03)?;
                out[1] = Self::two_bit_to_trit((b >> 3) & 0x03)?;
                out[2] = Self::two_bit_to_trit((b >> 6) & 0x03)?;
                out[3] = 0;
            }
            PackMode::Directional323 => {
                out[0] = Self::two_bit_to_trit(b & 0x03)?;
                out[1] = Self::two_bit_to_trit((b >> 3) & 0x03)?;
                out[2] = Self::two_bit_to_trit((b >> 5) & 0x03)?;
                out[3] = 0;
            }
            PackMode::Directional233 => {
                out[0] = Self::two_bit_to_trit(b & 0x03)?;
                out[1] = Self::two_bit_to_trit((b >> 2) & 0x03)?;
                out[2] = Self::two_bit_to_trit((b >> 5) & 0x03)?;
                out[3] = 0;
            }
        }
        Ok(out)
    }
}

impl fmt::Display for Word32Trit {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "0t")?;
        for i in (0..32).rev() {
            match self.trits[i] {
                1 => write!(f, "+")?,
                0 => write!(f, "0")?,
                -1 => write!(f, "-")?,
                _ => write!(f, "?")?,
            }
        }
        write!(f, " ({})", self.to_i64())
    }
}
```

```rust
// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Word64: 64-Bit Physical Silicon Word Container )
// ============================================================================

/// A 64-bit physical silicon machine word combining a 32-trit payload
/// with out-of-band operational metadata.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub struct Word64 {
    pub raw: u64,
}

impl Default for Word64 {
    fn default() -> Self {
        Self::new(Word32Trit::zero(), QuadState::Proceed, Polarity::Concrete, BoundaryType::Denotation, 0)
    }
}

impl Word64 {
    const PAYLOAD_MASK: u64   = 0x0007_FFFF_FFFF_FFFF; // Bits 0..50 (51 bits)
    const STATE_SHIFT: u32    = 51;
    const STATE_MASK: u64     = 0x3;                   // Bits 51..52 (2 bits)
    const POLARITY_SHIFT: u32 = 53;
    const POLARITY_MASK: u64  = 0x1;                   // Bit 53 (1 bit)
    const BOUNDARY_SHIFT: u32 = 54;
    const BOUNDARY_MASK: u64  = 0x7;                   // Bits 54..56 (3 bits)
    const CHECKSUM_SHIFT: u32 = 57;
    const CHECKSUM_MASK: u64  = 0x7F;                  // Bits 57..63 (7 bits)

    /// Constructs a physical 64-bit word from components.
    pub fn new(
        payload: Word32Trit,
        state: QuadState,
        polarity: Polarity,
        boundary: BoundaryType,
        checksum: u8,
    ) -> Self {
        let p_val = payload.pack_51bits() & Self::PAYLOAD_MASK;
        let s_val = match state {
            QuadState::Await => 0u64,
            QuadState::Proceed => 1u64,
            QuadState::Rollback => 2u64,
            QuadState::Halt => 3u64,
        };
        let pol_val = match polarity {
            Polarity::Abstract => 0u64,
            Polarity::Concrete => 1u64,
        };
        let b_val = match boundary {
            BoundaryType::Invariant => 0u64,
            BoundaryType::Denotation => 1u64,
            BoundaryType::Connotation => 2u64,
            BoundaryType::BottomUp => 3u64,
            BoundaryType::TopDown => 4u64,
            BoundaryType::Aside => 5u64,
        };
        let c_val = (checksum as u64) & Self::CHECKSUM_MASK;

        let raw = p_val
            | (s_val << Self::STATE_SHIFT)
            | (pol_val << Self::POLARITY_SHIFT)
            | (b_val << Self::BOUNDARY_SHIFT)
            | (c_val << Self::CHECKSUM_SHIFT);

        Self { raw }
    }

    /// Returns a zeroed physical 64-bit word with equilibrium payload and Proceed/PLAY transport.
    pub fn zero() -> Self {
        Self::from_signed_int(0)
    }

    /// Convenience constructor from a signed integer value with default Proceed/PLAY transport.
    pub fn from_signed_int(val: i64) -> Self {
        Self::new(
            Word32Trit::from_i64(val),
            QuadState::Proceed,
            Polarity::Concrete,
```

```
            BoundaryType::Denotation,
            0,
        )
    }

    /// Convenience constructor from a 32-trit word with default Proceed/PLAY transport.
    pub fn from_trit_payload(payload: Word32Trit) -> Self {
        Self::new(
            payload,
            QuadState::Proceed,
            Polarity::Concrete,
            BoundaryType::Denotation,
            0,
        )
    }

    /// Extracts the 32-trit payload.
    pub fn payload(&self) -> Word32Trit {
        let p_bits = self.raw & Self::PAYLOAD_MASK;
        Word32Trit::unpack_51bits(p_bits)
    }

    /// Extracts the QuadState / Transport Action.
    pub fn state(&self) -> QuadState {
        let s_bits = (self.raw >> Self::STATE_SHIFT) & Self::STATE_MASK;
        match s_bits {
            0 => QuadState::Await,
            1 => QuadState::Proceed,
            2 => QuadState::Rollback,
            _ => QuadState::Halt,
        }
    }

    /// Returns the cinematic transport action: Play, Pause, Rewind, Stop.
    pub fn transport(&self) -> TransportAction {
        self.state().transport()
    }

    /// Extracts the Polarity affix.
    pub fn polarity(&self) -> Polarity {
        let p_bit = (self.raw >> Self::POLARITY_SHIFT) & Self::POLARITY_MASK;
        if p_bit == 0 {
            Polarity::Abstract
        } else {
            Polarity::Concrete
        }
    }

    /// Extracts the Boundary Type.
    pub fn boundary(&self) -> BoundaryType {
        let b_bits = (self.raw >> Self::BOUNDARY_SHIFT) & Self::BOUNDARY_MASK;
        match b_bits {
            0 => BoundaryType::Invariant,
            1 => BoundaryType::Denotation,
            2 => BoundaryType::Connotation,
            3 => BoundaryType::BottomUp,
            4 => BoundaryType::TopDown,
            _ => BoundaryType::Aside,
        }
    }

    /// Extracts the moral invariant checksum byte.
    pub fn checksum(&self) -> u8 {
        ((self.raw >> Self::CHECKSUM_SHIFT) & Self::CHECKSUM_MASK) as u8
    }

    /// Formats a complete human-readable physical register inspection string.
    pub fn inspect(&self) -> String {
        format!(
            "0x{:016X} | {} | State: {:<8} (Transport: {:<6}) | Polarity: {} | Bound: {}",
            self.raw,
            self.payload(),
            self.state().macro_op(),
            self.transport().as_str(),
            self.polarity().symbol(),
            self.boundary().delimiters().0
        )
    }
```

```
}



// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests & Verification Proof )
// ============================================================================
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_word32_trit_integer_conversion_round_trip() {
        let test_cases = [0, 1, -1, 4, -4, 42, -42, 243, -243, 100_000, -100_000, 1_000_000_000, -1_000_000_000];
        for &val in &test_cases {
            let w = Word32Trit::from_i64(val);
            let back = w.to_i64();
            assert_eq!(val, back, "Round trip failed for value {val}");
        }
    }

    #[test]
    fn test_word32_trit_pack_unpack_round_trip() {
        let values = [0, 1, -1, 12345, -54321, 999_999_999, -999_999_999];
        for &v in &values {
            let w = Word32Trit::from_i64(v);
            let packed = w.pack_51bits();
            let unpacked = Word32Trit::unpack_51bits(packed);
            assert_eq!(w, unpacked, "Pack/unpack mismatch for value {v}");
            assert_eq!(v, unpacked.to_i64());
        }
    }

    #[test]
    fn test_word32_trit_balanced_addition() {
        let a = Word32Trit::from_i64(42);
        let b = Word32Trit::from_i64(58);
        let (sum, carry) = a.add(&b);
        assert_eq!(carry, 0);
        assert_eq!(sum.to_i64(), 100);

        let c = Word32Trit::from_i64(15);
        let d = Word32Trit::from_i64(-25);
        let (sum2, carry2) = c.add(&d);
        assert_eq!(carry2, 0);
        assert_eq!(sum2.to_i64(), -10);
    }

    #[test]
    fn test_word64_physical_container_round_trip() {
        let payload = Word32Trit::from_i64(1337);
        let w64 = Word64::new(payload, QuadState::Proceed, Polarity::Concrete, BoundaryType::Denotation, 42);

        assert_eq!(w64.payload().to_i64(), 1337);
        assert_eq!(w64.state(), QuadState::Proceed);
        assert_eq!(w64.transport(), TransportAction::Play);
        assert_eq!(w64.polarity(), Polarity::Concrete);
        assert_eq!(w64.boundary(), BoundaryType::Denotation);
        assert_eq!(w64.checksum(), 42);
    }

    #[test]
    fn test_word32_dual_rail_bitplanes() {
        assert_eq!(Word32Trit::trit_to_2bit(0), TRIT_CODE_ZERO);
        assert_eq!(Word32Trit::trit_to_2bit(1), TRIT_CODE_POS);
        assert_eq!(Word32Trit::trit_to_2bit(-1), TRIT_CODE_NEG);

        assert_eq!(Word32Trit::two_bit_to_trit(0).unwrap(), 0);
        assert_eq!(Word32Trit::two_bit_to_trit(1).unwrap(), 1);
        assert_eq!(Word32Trit::two_bit_to_trit(2).unwrap(), -1);
        assert!(Word32Trit::two_bit_to_trit(3).is_err());

        let values = [0, 1, -1, 4096, -4096, 65536, -65536, 123456789, -123456789];
        for &v in &values {
            let w = Word32Trit::from_i64(v);
            let packed = w.pack_dual_rail();
            assert!(!Word32Trit::has_null_collision(packed));
```

```
            let unpacked = Word32Trit::unpack_dual_rail(packed).expect("Unpack dual rail");
            assert_eq!(w, unpacked, "Dual-rail mismatch for {v}");
            assert_eq!(v, unpacked.to_i64());
        }
    }

    #[test]
    fn test_word32_simd_rail_swap_negation() {
        let w = Word32Trit::from_i64(4096);
        let packed = w.pack_dual_rail();
        let negated_packed = Word32Trit::simd_rail_swap_negate(packed);
        let unpacked_neg = Word32Trit::unpack_dual_rail(negated_packed).expect("Unpack negated");
        assert_eq!(unpacked_neg.to_i64(), -4096);

        // Involution: -(-x) == x
        let double_neg = Word32Trit::simd_rail_swap_negate(negated_packed);
        assert_eq!(double_neg, packed);
    }

    #[test]
    fn test_word32_null_collision_trap() {
        let w = Word32Trit::from_i64(1);
        let mut packed = w.pack_dual_rail();
        assert!(!Word32Trit::has_null_collision(packed));

        // Inject 11 into slot 0
        packed |= 3;
        assert!(Word32Trit::has_null_collision(packed));
        assert!(Word32Trit::unpack_dual_rail(packed).is_err());
    }

    #[test]
    fn test_word32_dynamic_packing_topologies() {
        let modes = [
            PackMode::Dense2222,
            PackMode::Kinematic21212,
            PackMode::Kinematic12122,
            PackMode::Kinematic22121,
            PackMode::Directional332,
            PackMode::Directional323,
            PackMode::Directional233,
        ];
        let trits = [1, -1, 1, -1];
        for &mode in &modes {
            let byte_val = Word32Trit::pack_byte(&trits, mode);
            let unpacked = Word32Trit::unpack_byte(byte_val, mode).expect("Unpack byte");
            assert_eq!(unpacked[0], trits[0]);
            assert_eq!(unpacked[1], trits[1]);
            assert_eq!(unpacked[2], trits[2]);
            if mode == PackMode::Dense2222 {
                assert_eq!(unpacked[3], trits[3]);
            }
        }
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/ai/lexicon/domain.rs RS 38 LINES 166 WORDS 1.5 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Categorical Domains of Reality & Biblical World Ontology )
// ============================================================================
//! # Lexicon Domains (`domain.rs`)
//!
//! Categorical domains covering the biblical ontology of reality.

/// Categorical domains covering the biblical ontology of reality.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum LexiconDomain {
    /// The physical cosmos: light, darkness, heavens, earth, seas, time.
    Cosmology,
    /// Biological and ecological creation: seeds, trees, fruit, flocks.
    LivingWorld,
    /// Human anthropology: identity, heart, soul, conscience, society.
    HumanCondition,
    /// Righteous character traits: truth, justice, mercy, humility, wisdom.
    MoralVirtue,
    /// Fallen behaviors: pride, deceit, theft, murder, wrath.
    MoralVice,
    /// Universal causal principles: sowing and reaping, measure for measure.
    CausalLaw,
}

impl LexiconDomain {
    pub fn as_str(&self) -> &'static str {
        match self {
            Self::Cosmology => "Cosmology & Creation",
            Self::LivingWorld => "Living World & Stewardship",
            Self::HumanCondition => "Human Condition & Society",
            Self::MoralVirtue => "Moral Virtues & Righteousness",
            Self::MoralVice => "Moral Vices & Warnings",
            Self::CausalLaw => "Causal Laws & Invariants",
        }
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/mod.rs RS 37 LINES 122 WORDS 1.5 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Runtime Subsystem Identity & Component Re-exports )
// ================================================================================
//! OmniCode Runtime Subsystem
//! Combines Transactional Environment with Quad-State Evaluator.

pub mod environment;
pub mod evaluator;
pub mod assembler;


// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Re-export Declarations for Runtime Access )
// ================================================================================
pub use environment::{Environment, RuntimeValue, RegisterBank, CallFrame};
pub use evaluator::Evaluator;
pub use assembler::{
    OmniAssembler, AssembledTape, PhysicalSymbol, AssembledInstruction,
    MachineOp, MachineRegister, transpile_tape_to_omni, transpile_omni_to_tape,
};


// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Runtime Execution Framework )
// ================================================================================
// Runtime environment and evaluator are orchestrated during execution.


// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Verification & Subsystem Status )
// ================================================================================
// Runtime subsystem verified and closed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/debugger.rs RS 471 LINES 1835 WORDS 20.2 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( OmniDebugger Identity, Invariant Verification Engine & Safety Constraints )
// ================================================================================
//! OmniCode Diagnostics: Debugger & Invariant Inspector
//!
//! # Purpose & Cybernetic Architecture
//! Acts as the runtime invariant enforcement organ and rollback controller:
//! - **Boundary Physics**: Manages the active delimiter stack, trapping leaks and mismatches.
//! - **Immutability Protection**: Enforces Rank 1 axiom protection `< ... >` against illegal mutation.
//! - **Retroaction Bounding**: Prevents infinite rollback loops via threshold counters.
//! - **Self-Healing Test Oracles**: Connects candidate restorations directly to test verification
//!   harnesses, certifying patches before committing state (+1:PROCEED).
//!
//! # IDE Hover & AI Agent Context
//! When hovering over [`OmniDebugger`] or its methods in an IDE:
//! - Hovering over [`OmniDebugger::execute_verification_test`] explains how verification sandboxes work.
//! - Hovering over [`OmniDebugger::execute_restoration_from_log`] details log-driven surgical patching.
//! - Hovering over [`OmniDebugger::pop_boundary`] shows delimiter matching and remediation synthesis.

use std::collections::{HashMap, HashSet};
use crate::core::boundaries::BoundaryType;
use crate::core::ternary::QuadState;
use crate::diagnostics::logger::{DiagnosticLogger, FailpointType, FixKind, FixSuggestion, LogEvent};


// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( BoundaryFrame Definition and OmniDebugger Context Setup )
// ================================================================================
/// Stack frame recording an unclosed boundary delimiter's opening context.
#[derive(Debug, Clone)]
pub struct BoundaryFrame {
    /// The opening delimiter character: `<`, `[`, `{`, or `(`.
    pub open_char: char,
    /// 1-based source line where the boundary opened.
    pub line: usize,
    /// 1-based source column where the delimiter occurred.
    pub col: usize,
}

/// Runtime invariant inspector, boundary stack tracker, and test verification oracle.
#[derive(Debug, Clone)]
pub struct OmniDebugger {
    /// Shared diagnostic logger for emitting debug events, failpoints, and cognitive traces.
    pub logger: DiagnosticLogger,
    /// Active LIFO stack of unclosed boundary frames.
    pub boundary_stack: Vec<BoundaryFrame>,
    /// Set of entity names declared inside `< ... >` sealed against mutation.
    pub immutable_registry: HashSet<String>,
    /// Execution rollback counters keyed by target label to detect infinite retroaction loops.
    pub retroaction_counters: HashMap<String, usize>,
    /// Staged queue of automated remediation suggestions awaiting application by the Restorator.
    pub remediation_queue: Vec<FixSuggestion>,
    /// Maximum permitted rollback operations per target before triggering cyclic retroaction failure.
    pub max_retroaction_depth: usize,
    /// Total operational steps and state evaluations inspected by this debugger.
    pub step_count: usize,
}

impl OmniDebugger {
    /// Constructs a new [`OmniDebugger`].
    ///
    /// # Arguments
    /// * `logger` - The shared [`DiagnosticLogger`] instance for telemetry recording.
    /// * `max_retroaction_depth` - Threshold ceiling for retroactions before trapping cyclic loops.
    pub fn new(logger: DiagnosticLogger, max_retroaction_depth: usize) -> Self {
        Self {
            logger,
            boundary_stack: Vec::new(),
            immutable_registry: HashSet::new(),
            retroaction_counters: HashMap::new(),
            remediation_queue: Vec::new(),
            max_retroaction_depth,
```

```
            step_count: 0,
        }
    }
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Core Invariant Enforcement: Boundary Push/Pop, Immutability, Retroaction )
// ============================================================================
impl OmniDebugger {
    /// Pushes an opening boundary delimiter onto the active stack.
    ///
    /// # Arguments
    /// * `open` - Delimiter character (`<`, `[`, `{`, `(`).
    /// * `line` - 1-based line number.
    /// * `col` - 1-based column number.
    pub fn push_boundary(&mut self, open: char, line: usize, col: usize) {
        self.logger.log(
            "BOUNDARY",
            format!("Pushing delimiter '{open}' at L{line}:C{col} (depth: {})", self.boundary_stack.len() + 1),
        );
        self.boundary_stack.push(BoundaryFrame { open_char: open, line, col });
    }

    /// Pops a closing boundary delimiter and validates the pair against the stack top.
    ///
    /// # Failpoints Intercepted
    /// - **Empty Stack**: Traps unmatched closing delimiter as [`FailpointType::BoundaryMismatch`].
    /// - **Mismatched Delimiter**: Traps illegal pair crossing (e.g. `< ... ]`), logs a
    ///   [`FixSuggestion`] pointing to the correct partner, and queues remediation.
    ///
    /// # Returns
    /// `Ok(BoundaryType)` on valid resolution, or `Err(String)` on failpoint mismatch.
    pub fn pop_boundary(&mut self, close: char, line: usize, col: usize) -> Result<BoundaryType, String> {
        if self.boundary_stack.is_empty() {
            let msg = format!("Unmatched closing delimiter '{close}' at L{line}:C{col} (stack is empty)");
            self.logger.failpoint(FailpointType::BoundaryMismatch, "DEBUGGER", &msg);
            return Err(format!("FAILPOINT: {msg}"));
        }

        let frame = self.boundary_stack.pop().unwrap();
        match BoundaryType::from_delimiters(frame.open_char, close) {
            Ok(btype) => {
                self.logger.log(
                    "BOUNDARY",
                    format!(
                        "Successfully resolved boundary {} '{}{}' (remaining depth: {})",
                        btype, frame.open_char, close, self.boundary_stack.len()
                    ),
                );
                Ok(btype)
            }
            Err(err) => {
                let msg = format!(
                    "Boundary mismatch: opened with '{}' (L{}:C{}) but closed with '{}' at L{}:C{}",
                    frame.open_char, frame.line, frame.col, close, line, col
                );
                let matching = match frame.open_char {
                    '<' => '>',
                    '[' => ']',
                    '{' => '}',
                    _ => ')',
                };
                let suggestion = FixSuggestion {
                    kind: FixKind::FixBoundaryMismatch,
                    line,
                    col,
                    target_token: close.to_string(),
                    suggested_patch: matching.to_string(),
                    rationale: format!(
                        "Replace mismatched closing delimiter '{close}' with matching '{matching}' to seal boundary
opened with '{}' at L{}:C{}",
                        frame.open_char, frame.line, frame.col
                    ),
                };
                self.logger.failpoint_with_remediation(
                    FailpointType::BoundaryMismatch,
```

```
                    "DEBUGGER",
                    &msg,
                    suggestion.clone(),
                );
                self.remediation_queue.push(suggestion);
                Err(err)
            }
        }
    }

    /// Asserts that no boundaries remain unclosed at EOF.
    ///
    /// If unclosed delimiters remain on the stack, logs [`FailpointType::BoundaryLeak`]
    /// with a concrete [`FixSuggestion`] indicating the required closing tokens.
    pub fn assert_all_boundaries_closed(&mut self) -> Result<(), String> {
        if let Some(frame) = self.boundary_stack.last() {
            let msg = format!(
                "Boundary leak: delimiter '{}' opened at L{}:C{} was never closed",
                frame.open_char, frame.line, frame.col
            );
            let matching = match frame.open_char {
                '<' => '>',
                '[' => ']',
                '{' => '}',
                _ => ')',
            };
            let suggestion = FixSuggestion {
                kind: FixKind::CloseBoundary,
                line: frame.line,
                col: frame.col,
                target_token: frame.open_char.to_string(),
                suggested_patch: format!("\n{matching}"),
                rationale: format!(
                    "Append missing closing delimiter '{matching}' for boundary opened with '{}' at L{}:C{}",
                    frame.open_char, frame.line, frame.col
                ),
            };
            self.logger.failpoint_with_remediation(
                FailpointType::BoundaryLeak,
                "DEBUGGER",
                &msg,
                suggestion.clone(),
            );
            self.remediation_queue.push(suggestion);
            return Err(format!("FAILPOINT: {msg}"));
        }
        Ok(())
    }

    /// Registers an entity identifier defined inside `< ... >` as globally immutable Rank 1 axiom.
    pub fn register_immutable(&mut self, name: &str) {
        self.immutable_registry.insert(name.to_string());
        self.logger.log("IMMUTABILITY", format!("Registered invariant <{name}>"));
    }

    /// Verifies that mutation is permitted for the given entity identifier.
    ///
    /// # Failpoints Intercepted
    /// If the entity was registered as immutable, traps [`FailpointType::ImmutableMutation`],
    /// and suggests converting the declaration to a mutable Rank 2 denotation `[ ... ]`.
    pub fn assert_mutation_permitted(&mut self, name: &str) -> Result<(), String> {
        if self.immutable_registry.contains(name) {
            let msg = format!(
                "Illegal mutation attempted on immutable invariant <{name}>! Entity cannot be changed."
            );
            let suggestion = FixSuggestion {
                kind: FixKind::ConvertImmutableToMutable,
                line: 0,
                col: 0,
                target_token: name.to_string(),
                suggested_patch: format!("[ {name} ]"),
                rationale: format!(
                    "Entity <{name}> was sealed as a Rank 1 immutable invariant; declare it with '[ {name} ]' if updates
are required"
                ),
            };
            self.logger.failpoint_with_remediation(
                FailpointType::ImmutableMutation,
```

```
                "DEBUGGER",
                &msg,
                suggestion.clone(),
            );
            self.remediation_queue.push(suggestion);
            return Err(format!("FAILPOINT: {msg}"));
        }
        Ok(())
    }

    /// Retrieves and empties the active remediation queue.
    pub fn drain_remediations(&mut self) -> Vec<FixSuggestion> {
        std::mem::take(&mut self.remediation_queue)
    }

    /// Monitors retroaction steps on a named target to prevent infinite rollback loops.
    ///
    /// # Failpoints Intercepted
    /// If rollback iterations exceed `max_retroaction_depth`, traps [`FailpointType::CyclicRetroaction`].
    pub fn track_retroaction(&mut self, target: &str) -> Result<usize, String> {
        let count = self.retroaction_counters.entry(target.to_string()).or_insert(0);
        *count += 1;
        let cur = *count;

        self.logger.log(
            "VECTOR",
            format!("Retroaction step #{cur} on '{target}'"),
        );

        if cur > self.max_retroaction_depth {
            let msg = format!(
                "Cyclic retroaction loop detected! Target '{target}' rolled back {cur} times, \
                 exceeding threshold ({})",
                self.max_retroaction_depth
            );
            self.logger.failpoint(FailpointType::CyclicRetroaction, "DEBUGGER", &msg);
            return Err(format!("FAILPOINT: {msg}"));
        }

        Ok(cur)
    }

    /// Inspects QuadState and raises appropriate signals or failpoints.
    ///
    /// - `QuadState::Halt`: Traps void / NULL halt trap (`FP_NULL_TRAP`).
    /// - `QuadState::Rollback`: Traps rollback trigger (`FP_ROLLBACK_TRIGGERED`).
    /// - `QuadState::Await`: Standby state logged.
    /// - `QuadState::Proceed`: Commit state logged.
    pub fn inspect_state(&mut self, state: QuadState, context: &str) {
        self.step_count += 1;
        match state {
            QuadState::Halt => {
                self.logger.failpoint(
                    FailpointType::NullTrap,
                    "DEBUGGER",
                    format!("NULL / Void halt trap encountered in context: {context}"),
                );
            }
            QuadState::Rollback => {
                self.logger.failpoint(
                    FailpointType::RollbackTriggered,
                    "DEBUGGER",
                    format!("Rollback (-1) triggered in context: {context}"),
                );
            }
            QuadState::Await => {
                self.logger.log("TERNARY", format!("Await (0) standby in context: {context}"));
            }
            QuadState::Proceed => {
                self.logger.log("TERNARY", format!("Proceed (+1) committed in context: {context}"));
            }
        }
    }

    /// Executes a verification test against an isolated child debugger context.
    ///
    /// # Why Test Oracles Matter for Self-Healing
    /// Acts as an execution oracle for hypothesis verification before state commit.
```

```rust
    /// The debugger executes the candidate logic in a sandbox:
    /// - If the test passes: records an `ORACLE_SUCCESS` observation for AI memory.
    /// - If the test fails: traps a rollback failpoint and logs `ORACLE_FAILURE`.
    ///
    /// # Arguments
    /// * `test_name` - Human and AI readable identifier for the verification assertion.
    /// * `test_runner` - Closure executing the test logic against a sandboxed child debugger.
    pub fn execute_verification_test<F>(&mut self, test_name: &str, test_runner: F) -> Result<(), String>
    where
        F: FnOnce(&mut OmniDebugger) -> Result<(), String>,
    {
        self.logger.log("TEST_ORACLE", format!("Spawning verification test: {test_name}"));
        let mut test_debugger = OmniDebugger::new(self.logger.clone(), self.max_retroaction_depth);
        match test_runner(&mut test_debugger) {
            Ok(()) => {
                self.logger.log("TEST_ORACLE", format!("Verification test '{test_name}' passed (+1:PROCEED)"));
                self.logger.record_ai_observation(
                    "ORACLE_SUCCESS",
                    &format!("Test '{test_name}' passed"),
                    "Candidate state passed invariant verification without failpoints",
                    ["test_oracle", "verified", "proceed"].as_slice(),
                    Some(test_name),
                );
                Ok(())
            }
            Err(err) => {
                self.logger.failpoint(
                    FailpointType::RollbackTriggered,
                    "TEST_ORACLE",
                    format!("Verification test '{test_name}' failed: {err}"),
                );
                self.logger.record_ai_observation(
                    "ORACLE_FAILURE",
                    &format!("Test '{test_name}' rejected candidate"),
                    &err,
                    ["test_oracle", "rejected", "rollback"].as_slice(),
                    Some(test_name),
                );
                Err(err)
            }
        }
    }

    /// Verifies candidate source code with a provided test oracle function.
    ///
    /// # Arguments
    /// * `candidate_source` - The restored source string under evaluation.
    /// * `test_oracle` - Test function verifying candidate execution invariants.
    ///
    /// # Returns
    /// - `Ok(String)`: If the oracle passes, returns the confirmed candidate source.
    /// - `Err(String)`: If the oracle fails, triggers a state rollback (`-1:ROLLBACK`) and returns the error.
    pub fn verify_candidate_repair<F>(
        &mut self,
        candidate_source: &str,
        test_oracle: F,
    ) -> Result<String, String>
    where
        F: Fn(&str) -> Result<(), String>,
    {
        self.logger.log("SELF_HEAL", "Verifying candidate repair via test oracle...");
        match test_oracle(candidate_source) {
            Ok(()) => {
                self.logger.log("SELF_HEAL", "Candidate repair verified (+1:PROCEED)!");
                self.inspect_state(QuadState::Proceed, "candidate_repair_verified");
                self.logger.record_ai_observation(
                    "REPAIR_VERIFIED",
                    "Candidate source passed verification oracle",
                    "The proposed restorative patch executed cleanly without failpoints",
                    ["self_healing", "oracle_passed", "verified"].as_slice(),
                    None,
                );
                Ok(candidate_source.to_string())
            }
            Err(e) => {
                self.inspect_state(QuadState::Rollback, "candidate_repair_rejected");
                let msg = format!("Candidate repair failed test verification: {e}");
                self.logger.failpoint(FailpointType::RollbackTriggered, "SELF_HEAL", &msg);
```

```
                self.logger.record_ai_observation(
                    "REPAIR_REJECTED",
                    "Candidate source failed verification oracle",
                    &msg,
                    ["self_healing", "oracle_failed", "rollback"].as_slice(),
                    None,
                );
                Err(msg)
            }
        }
    }

    /// Ingests a structured [`LogEvent`] containing a [`FixSuggestion`], applies the patch,
    /// and invokes the verification oracle before committing the restored code.
    ///
    /// # Arguments
    /// * `source` - The original broken source code.
    /// * `log_event` - The structured incident log emitted by the [`DiagnosticLogger`].
    /// * `test_oracle` - The verification test oracle closure.
    ///
    /// # Returns
    /// `Ok(healed_source)` if the patch was applied and certified by the oracle.
    pub fn execute_restoration_from_log<F>(
        &mut self,
        source: &str,
        log_event: &LogEvent,
        test_oracle: F,
    ) -> Result<String, String>
    where
        F: Fn(&str) -> Result<(), String>,
    {
        let suggestion = log_event.suggestion.as_ref().ok_or_else(|| {
            "LogEvent does not contain remediation suggestion for automated restoration".to_string()
        })?;

        self.logger.log(
            "SELF_HEAL",
            format!("Applying remediation suggestion from log: {:?}", suggestion.kind),
        );

        let mut candidate = source.to_string();
        match suggestion.kind {
            FixKind::FixBoundaryMismatch => {
                let lines: Vec<&str> = candidate.lines().collect();
                if suggestion.line > 0 && suggestion.line <= lines.len() {
                    let mut target_line = lines[suggestion.line - 1].to_string();
                    if let Some(pos) = target_line.find(&suggestion.target_token) {
                        target_line.replace_range(pos..pos + suggestion.target_token.len(), &suggestion.suggested_patch);
                        let mut new_lines = lines.clone();
                        new_lines[suggestion.line - 1] = &target_line;
                        candidate = new_lines.join("\n");
                        if !candidate.ends_with('\n') {
                            candidate.push('\n');
                        }
                    }
                }
            }
            FixKind::CloseBoundary => {
                candidate.push_str(&suggestion.suggested_patch);
                if !candidate.ends_with('\n') {
                    candidate.push('\n');
                }
            }
            FixKind::ConvertImmutableToMutable => {
                candidate = candidate.replace(
                    &format!("< {} >", suggestion.target_token),
                    &suggestion.suggested_patch,
                );
            }
            _ => {
                candidate = candidate.replace(&suggestion.target_token, &suggestion.suggested_patch);
            }
        }

        // Call the test oracle to certify candidate
        self.verify_candidate_repair(&candidate, test_oracle)
    }
}
```

```
463
464
465 // ================================================================================
466 // BLOCK 4: CLOSING BLOCK
467 // ( Postcondition Verification and Debugger Status )
468 // ================================================================================
469 // OmniDebugger safety invariants verified and closed.
470
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/mod.rs RS 54 LINES 264 WORDS 2.6 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Diagnostics Subsystem Identity, Module Architecture & Exports )
// ================================================================================
//! OmniCode Diagnostics Subsystem
//!
//! # Architecture & Cognitive Cybernetics
//! Coordinates the runtime sensory, debugging, and self-healing immune organs:
//! - [`logger`]: Structured failpoint interceptor, telemetry logger, and AI episodic memory emitter ([`CognitiveNote`]).
//! - [`debugger`]: Boundary stack monitor, immutability inspector, retroaction depth guard, and test verification oracle.
//! - [`linter`]: Structural validator enforcing the canonical 4-Block architecture across compilation units.
//! - [`restorator`]: Autonomous multi-pass code healing engine with test-oracle certification.
//!
//! # IDE Hover Support
//! All diagnostic components are re-exported at the module root for immediate autocompletion
//! and documentation discovery in language servers and coding assistants.

pub mod logger;
pub mod debugger;
pub mod linter;
pub mod restorator;
pub mod hover;


// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Re-export Declarations for Diagnostic Tools )
// ================================================================================
pub use logger::{DiagnosticLogger, FailpointType, LogEvent, FixKind, FixSuggestion, CognitiveNote};
pub use debugger::OmniDebugger;
pub use linter::{
    BlockKind, BodyLinter, BodyScore, ClosingLinter, ClosingScore, DetectedBlock, FileAuditScore,
    LintReport, MetadataLinter, MetadataScore, OmniLinter, OpeningLinter, OpeningScore, QuadStateGate,
};
pub use restorator::{Restorator, RestorationReport, AppliedRepair};
pub use hover::{HoverEngine, HoverCard, OmniDefinition, TapeRollContext};


// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Subsystem Core Registrations & Coordination Invariants )
// ================================================================================
// Diagnostic modules are actively coordinated across compiler passes.
// The Logger records incidents, the Restorator synthesizes candidate patches,
// and the Debugger executes verification test oracles before state commitment.


// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Verification & Subsystem Status )
// ================================================================================
// Diagnostics subsystem verified and closed.

```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/restorator.rs RS 667 LINES 2387 WORDS 27.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Restorator Identity, Self-Healing Immune Invariants & Repair Kinds )
// ============================================================================
//! OmniCode Diagnostics: Restorator & Self-Healing Engine
//!
//! # Purpose & Cybernetic Architecture
//! Autonomous code restoration and healing organ. Diagnoses syntax anomalies,
//! unclosed or mismatched boundaries, missing 4-block architecture scaffolding,
//! and missing standard library imports, applying surgical restorative patches
//! to bring non-operational code into a verified operational state (`+1:PROCEED`).
//!
//! # Self-Healing Pipeline Passes:
//! 1. **Pass 1 (Boundary Physics)**: Closes unclosed delimiters and aligns mismatched pairs.
//! 2. **Pass 2 (4-Block Architecture)**: Ensures canonical order: Metadata -> Opening -> Body -> Closing.
//! 3. **Pass 3 (Dependency Injection)**: Automatically detects required core modules (`Core.Arithmetic`, etc.) and injects imports.
//! 4. **Pass 4 (Conceptual Inference)**: Synthesizes noun declarations with infer operators (`Ball < Entity? >`).
//! 5. **Pass 5 (Round-trip Verification)**: Lints, lexes, parses, and evaluates the candidate to certify operational validity.
//!
//! # Test Oracle Integration:
//! Connects directly to test verification harnesses via [`Restorator::restore_and_verify_with_oracle`],
//! ensuring patches satisfy external invariants and recording verified repair notes in AI memory.

use std::fs;
use crate::core::ternary::QuadState;
use crate::diagnostics::debugger::OmniDebugger;
use crate::diagnostics::linter::{BlockKind, OmniLinter};
use crate::diagnostics::logger::{DiagnosticLogger, FailpointType, FixKind, LogEvent};
use crate::parser::lexer::Lexer;
use crate::parser::parser::Parser;
use crate::runtime::environment::Environment;
use crate::runtime::evaluator::Evaluator;


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Restoration Structures and Engine Context Initialization )
// ============================================================================
/// Record of an individual restorative patch applied to source code.
#[derive(Debug, Clone)]
pub struct AppliedRepair {
    /// The structural category of the applied repair.
    pub kind: FixKind,
    /// Detailed human and AI-readable explanation of why and how the repair was made.
    pub description: String,
    /// 1-based source line number where the repair took effect.
    pub line: usize,
}

/// Comprehensive outcome report generated by the [`Restorator`].
///
/// Contains original source, restored content, applied repairs, verification status,
/// and any errors encountered during the self-healing cycle.
#[derive(Debug, Clone)]
pub struct RestorationReport {
    /// File path if restoration was performed on disk, or `None` if in-memory.
    pub file_path: Option<String>,
    /// `true` if any modifications or repairs were made to the source text.
    pub was_modified: bool,
    /// Exact unmodified original source code prior to healing passes.
    pub original_content: String,
    /// Healed, patched, and structurally validated source code.
    pub restored_content: String,
    /// Chronological list of all automated repairs applied during the cycle.
    pub repairs_applied: Vec<AppliedRepair>,
    /// `true` if the restored content passed all lint, parse, and evaluation checks.
    pub verified: bool,
    /// The final QuadState produced by evaluating the restored source (+1:PROCEED).
    pub final_state: Option<QuadState>,
    /// Any lint, parse, or evaluation errors if verification failed.
    pub errors: Vec<String>,
}
```

```rust

/// The autonomous self-healing and code restoration engine.
pub struct Restorator {
    /// Shared diagnostic logger recording cognitive observations and repair history.
    pub logger: DiagnosticLogger,
}

impl Restorator {
    /// Constructs a new [`Restorator`] instance bound to the provided logger.
    pub fn new(logger: DiagnosticLogger) -> Self {
        Self { logger }
    }
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Core Restoration Pipeline: Boundaries, 4-Block Headers, Imports, Verification )
// ============================================================================
impl Restorator {
    /// Restores source code in memory, executing multi-pass auto-repairs.
    ///
    /// Executes the 5-pass self-healing pipeline:
    /// 1. Delimiter & Boundary Healing
    /// 2. 4-Block Architecture Injection
    /// 3. Standard Library Import Resolution
    /// 4. Inferred Conceptual Entity Declaration
    /// 5. Round-trip Lint/Parse/Eval Verification
    ///
    /// # Arguments
    /// * `source` - Raw input string that may contain boundary leaks or syntax anomalies.
    ///
    /// # Returns
    /// A [`RestorationReport`] detailing all modifications and verification proofs.
    pub fn restore_source(&self, source: &str) -> RestorationReport {
        let original_content = source.to_string();
        let mut current = source.to_string();
        let mut repairs_applied = Vec::new();

        // Pass 1: Delimiter & Boundary Healing
        current = self.heal_boundaries(&current, &mut repairs_applied);

        // Pass 2: 4-Block Architectural Scaffolding Healing
        current = self.heal_four_blocks(&current, &mut repairs_applied);

        // Pass 3: Automated Dependency & Import Healing
        current = self.heal_imports(&current, &mut repairs_applied);

        // Pass 4: Inferred Conceptual Entity Healing
        current = self.heal_inferred_entities(&current, &mut repairs_applied);

        // Pass 5: Round-trip Self-Verification
        let (verified, final_state, errors) = self.verify_restoration(&current);

        let was_modified = current != original_content;

        if !repairs_applied.is_empty() {
            let repair_descs: Vec<String> = repairs_applied.iter().map(|r| r.description.clone()).collect();
            self.logger.record_ai_observation(
                "SELF_HEAL_APPLIED",
                &format!("Applied {} automated repairs", repairs_applied.len()),
                &format!("Repairs executed: {}", repair_descs.join("; ")),
                ["self_healing", "remediation", "restorator"].as_slice(),
                None,
            );
        }

        RestorationReport {
            file_path: None,
            was_modified,
            original_content,
            restored_content: current,
            repairs_applied,
            verified,
            final_state,
            errors,
        }
    }
```

```rust

    /// Ingests a structured [`LogEvent`] and executes a targeted, single-point surgical patch.
    ///
    /// # Arguments
    /// * `source` - Source code to repair.
    /// * `log` - Structured incident event containing a [`crate::diagnostics::logger::FixSuggestion`].
    ///
    /// # Returns
    /// [`RestorationReport`] showing the single-point modification and round-trip verification status.
    pub fn restore_from_incident_log(&self, source: &str, log: &LogEvent) -> RestorationReport {
        let original_content = source.to_string();
        let mut repairs_applied = Vec::new();
        let mut current = source.to_string();

        if let Some(sug) = &log.suggestion {
            match sug.kind {
                FixKind::FixBoundaryMismatch => {
                    let lines: Vec<&str> = current.lines().collect();
                    if sug.line > 0 && sug.line <= lines.len() {
                        let mut target_line = lines[sug.line - 1].to_string();
                        if let Some(pos) = target_line.find(&sug.target_token) {
                            target_line.replace_range(pos..pos + sug.target_token.len(), &sug.suggested_patch);
                            let mut new_lines = lines.clone();
                            new_lines[sug.line - 1] = &target_line;
                            current = new_lines.join("\n");
                            if !current.ends_with('\n') {
                                current.push('\n');
                            }
                            repairs_applied.push(AppliedRepair {
                                kind: FixKind::FixBoundaryMismatch,
                                description: sug.rationale.clone(),
                                line: sug.line,
                            });
                        }
                    }
                }
                FixKind::CloseBoundary => {
                    current.push_str(&sug.suggested_patch);
                    if !current.ends_with('\n') {
                        current.push('\n');
                    }
                    repairs_applied.push(AppliedRepair {
                        kind: FixKind::CloseBoundary,
                        description: sug.rationale.clone(),
                        line: sug.line,
                    });
                }
                FixKind::ConvertImmutableToMutable => {
                    current = current.replace(&format!("< {} >", sug.target_token), &sug.suggested_patch);
                    repairs_applied.push(AppliedRepair {
                        kind: FixKind::ConvertImmutableToMutable,
                        description: sug.rationale.clone(),
                        line: sug.line,
                    });
                }
                _ => {
                    current = current.replace(&sug.target_token, &sug.suggested_patch);
                    repairs_applied.push(AppliedRepair {
                        kind: sug.kind.clone(),
                        description: sug.rationale.clone(),
                        line: sug.line,
                    });
                }
            }
        }

        let (verified, final_state, errors) = self.verify_restoration(&current);
        let was_modified = current != original_content;

        if was_modified {
            self.logger.record_ai_observation(
                "LOG_DRIVEN_RESTORE",
                &format!("Applied targeted fix from log for {:?}", log.failpoint_type),
                &format!("Modified {} bytes", current.len()),
                ["self_healing", "log_driven"].as_slice(),
                None,
            );
        }
```

```
        RestorationReport {
            file_path: None,
            was_modified,
            original_content,
            restored_content: current,
            repairs_applied,
            verified,
            final_state,
            errors,
        }
    }

    /// Restores source code and executes an external verification test oracle against the candidate.
    ///
    /// Connects the self-healing engine to test harnesses:
    /// - If the restored code satisfies the test oracle: logs an `ORACLE_CERTIFIED` cognitive trace.
    /// - If the test oracle fails: records a rollback failpoint and adds the failure to the report errors.
    ///
    /// # Arguments
    /// * `source` - Raw input code.
    /// * `test_oracle` - Closure asserting domain-specific invariants on the restored source.
    pub fn restore_and_verify_with_oracle<F>(&self, source: &str, test_oracle: F) -> RestorationReport
    where
        F: Fn(&str) -> Result<(), String>,
    {
        let mut report = self.restore_source(source);

        if report.verified {
            match test_oracle(&report.restored_content) {
                Ok(()) => {
                    self.logger.log("RESTORATOR", "Restoration validated by test oracle (+1:PROCEED)");
                    self.logger.record_ai_observation(
                        "ORACLE_CERTIFIED",
                        "Restored code certified by test oracle",
                        "The candidate passed the verification test suite cleanly",
                        ["self_healing", "test_oracle", "certified"].as_slice(),
                        None,
                    );
                }
                Err(err) => {
                    self.logger.failpoint(
                        FailpointType::RollbackTriggered,
                        "RESTORATOR",
                        format!("Restoration failed test oracle verification: {err}"),
                    );
                    self.logger.record_ai_observation(
                        "ORACLE_REJECTED",
                        "Restored code failed test oracle verification",
                        &err,
                        ["self_healing", "test_oracle", "rollback"].as_slice(),
                        None,
                    );
                    report.verified = false;
                    report.errors.push(format!("Test oracle failure: {err}"));
                }
            }
        }

        report
    }

    /// Restores a file from disk, optionally writing back the healed source in-place.
    ///
    /// # Arguments
    /// * `path` - File system path to the target source file.
    /// * `in_place` - If true, overwrites the file on disk with the restored content.
    ///
    /// # Errors
    /// Returns `Err(String)` if file reading or writing fails.
    pub fn restore_file(&self, path: &str, in_place: bool) -> Result<RestorationReport, String> {
        let content = fs::read_to_string(path)
            .map_err(|e| format!("Failed to read file '{path}': {e}"))?;

        let mut report = self.restore_source(&content);
        report.file_path = Some(path.to_string());

        if in_place && report.was_modified {
```

```
            fs::write(path, &report.restored_content)
                .map_err(|e| format!("Failed to write restored file '{path}': {e}"))?;
            self.logger.log("RESTORATOR", format!("Wrote restored content in-place to '{path}'"));
        }

        Ok(report)
    }

    /// Pass 1: Auto-detects and repairs mismatched and leaking delimiters.
    fn heal_boundaries(&self, source: &str, repairs: &mut Vec<AppliedRepair>) -> String {
        let mut text = source.to_string();
        let mut max_passes = 10;

        while max_passes > 0 {
            max_passes -= 1;
            let temp_logger = DiagnosticLogger::new(false);
            let mut lexer = Lexer::new(&text, temp_logger.clone());
            let tokens = match lexer.tokenize() {
                Ok(toks) => toks,
                Err(_) => break,
            };

            let debugger = OmniDebugger::new(temp_logger.clone(), 10);
            let mut parser = Parser::new(tokens, debugger);
            let parse_res = parser.parse_program();

            if parse_res.is_ok() {
                break; // Boundaries are sound
            }

            // Inspect diagnostic remediations from debugger
            let mut debugger_remediations = parser.debugger.drain_remediations();
            if debugger_remediations.is_empty() {
                debugger_remediations = temp_logger.remediations();
            }

            if debugger_remediations.is_empty() {
                // If EOF unclosed boundary detected without explicit remediation, balance stack
                if !parser.debugger.boundary_stack.is_empty() {
                    let mut closers = String::new();
                    while let Some(frame) = parser.debugger.boundary_stack.pop() {
                        let c = match frame.open_char {
                            '<' => '>',
                            '[' => ']',
                            '{' => '}',
                            _ => ')',
                        };
                        closers.push(c);
                        repairs.push(AppliedRepair {
                            kind: FixKind::CloseBoundary,
                            description: format!("Appended missing closing delimiter '{c}' for boundary opened at
L{}:C{}", frame.line, frame.col),
                            line: frame.line,
                        });
                    }
                    text.push_str(&format!("\n{}\n", closers));
                    continue;
                }
                break;
            }

            // Apply remediation patches
            let mut applied_any = false;
            for sug in debugger_remediations {
                match sug.kind {
                    FixKind::FixBoundaryMismatch => {
                        let lines: Vec<&str> = text.lines().collect();
                        if sug.line > 0 && sug.line <= lines.len() {
                            let mut target_line = lines[sug.line - 1].to_string();
                            if let Some(pos) = target_line.find(&sug.target_token) {
                                target_line.replace_range(pos..pos + sug.target_token.len(), &sug.suggested_patch);
                                let mut new_lines = lines.clone();
                                new_lines[sug.line - 1] = &target_line;
                                text = new_lines.join("\n");
                                if !text.ends_with('\n') {
                                    text.push('\n');
                                }
                                repairs.push(AppliedRepair {
```

```
                                    kind: FixKind::FixBoundaryMismatch,
                                    description: sug.rationale,
                                    line: sug.line,
                                });
                                applied_any = true;
                                break;
                            }
                        }
                    }
                    FixKind::CloseBoundary => {
                        text.push_str(&sug.suggested_patch);
                        if !text.ends_with('\n') {
                            text.push('\n');
                        }
                        repairs.push(AppliedRepair {
                            kind: FixKind::CloseBoundary,
                            description: sug.rationale,
                            line: sug.line,
                        });
                        applied_any = true;
                        break;
                    }
                    _ => {}
                }
            }

            if !applied_any {
                break;
            }
        }

        text
    }

    /// Pass 2: Ensures all 4 universal architectural blocks exist in canonical order.
    fn heal_four_blocks(&self, source: &str, repairs: &mut Vec<AppliedRepair>) -> String {
        let linter = OmniLinter::new(DiagnosticLogger::new(false));
        let report = linter.lint_source("memory", source);

        if report.is_valid {
            return source.to_string();
        }

        let mut has_metadata = false;
        let mut has_opening = false;
        let mut has_body = false;
        let mut has_closing = false;

        for b in &report.blocks_found {
            match b.kind {
                BlockKind::Metadata => has_metadata = true,
                BlockKind::Opening => has_opening = true,
                BlockKind::Body => has_body = true,
                BlockKind::Closing => has_closing = true,
            }
        }

        let mut out = String::new();

        // 1. Inject Metadata Block if missing
        if !has_metadata {
            repairs.push(AppliedRepair {
                kind: FixKind::AddBlockHeader,
                description: "Injected canonical BLOCK 1: METADATA manifest".into(),
                line: 1,
            });
            out.push_str("# ================================================================================\n");
            out.push_str("# BLOCK 1: METADATA BLOCK\n");
            out.push_str("# ( Restored Module Manifest and Semantic Track )\n");
            out.push_str("# ================================================================================\n");
            out.push_str("Module < Restored.Track > ( Auto-restored OmniCode compilation unit ) {\n");
            out.push_str("    version: < \"1.0.0\" >,\n");
            out.push_str("    system: < OmniCode.Core >\n");
            out.push_str("}\n\n");
        }

        // 2. Inject Opening Block if missing
        if !has_opening {
```

```
            repairs.push(AppliedRepair {
                kind: FixKind::AddBlockHeader,
                description: "Injected canonical BLOCK 2: OPENING context".into(),
                line: 1,
            });
            out.push_str("# ================================================================================\n");
            out.push_str("# BLOCK 2: OPENING BLOCK\n");
            out.push_str("# ( Initial Conditions and Operational Preconditions )\n");
            out.push_str("# ================================================================================\n");
            out.push_str("Opening {\n");
            out.push_str("    preconditions: [ +1 ]\n");
            out.push_str("}\n\n");
        }

        // 3. Append existing body content
        if !has_body {
            repairs.push(AppliedRepair {
                kind: FixKind::AddBlockHeader,
                description: "Injected canonical BLOCK 3: BODY block wrapping loose operations".into(),
                line: 1,
            });
            out.push_str("# ================================================================================\n");
            out.push_str("# BLOCK 3: BODY BLOCK\n");
            out.push_str("# ( Core Operations and Transformations )\n");
            out.push_str("# ================================================================================\n");
            out.push_str("Body [\n");
            for line in source.lines() {
                if !line.trim().is_empty() {
                    out.push_str(&format!("    {}\n", line));
                }
            }
            out.push_str("]\n\n");
        } else {
            out.push_str(source);
            if !out.ends_with('\n') {
                out.push('\n');
            }
        }

        // 4. Inject Closing Block if missing
        if !has_closing {
            repairs.push(AppliedRepair {
                kind: FixKind::AddBlockHeader,
                description: "Injected canonical BLOCK 4: CLOSING commit proof".into(),
                line: 1,
            });
            out.push_str("# ================================================================================\n");
            out.push_str("# BLOCK 4: CLOSING BLOCK\n");
            out.push_str("# ( State Verification Proof and Final Commit )\n");
            out.push_str("# ================================================================================\n");
            out.push_str("Closing {\n");
            out.push_str("    commit_status [ PROCEED ]\n");
            out.push_str("}\n");
        }

        out
    }

    /// Pass 3: Automatically injects missing core library imports into Opening block.
    fn heal_imports(&self, source: &str, repairs: &mut Vec<AppliedRepair>) -> String {
        let needs_arithmetic = (source.contains("plus") || source.contains("minus") || source.contains("times") ||
source.contains("div") || source.contains("ADD") || source.contains("SUB"))
            && !source.contains("Core.Arithmetic");

        let needs_logic = (source.contains("PROCEED") || source.contains("ROLLBACK") || source.contains("AWAIT") ||
source.contains("HALT"))
            && !source.contains("Core.Logic");

        let needs_vectors = (source.contains("-->") || source.contains("<--") || source.contains("--^") ||
source.contains("--v"))
            && !source.contains("Core.Vectors");

        if !needs_arithmetic && !needs_logic && !needs_vectors {
            return source.to_string();
        }

        let mut mods_to_import = Vec::new();
        if needs_arithmetic {
```

```
            mods_to_import.push("Core.Arithmetic");
        }
        if needs_logic {
            mods_to_import.push("Core.Logic");
        }
        if needs_vectors {
            mods_to_import.push("Core.Vectors");
        }

        let import_expr = format!("import: [ {} ]", mods_to_import.join(" :: "));

        // Inject into Opening { ... } block
        if let Some(op_idx) = source.find("Opening {") {
            let insert_pos = op_idx + "Opening {".len();
            let mut out = source.to_string();
            let injection = format!("\n    {},\n", import_expr);
            out.insert_str(insert_pos, &injection);

            repairs.push(AppliedRepair {
                kind: FixKind::AddImport,
                description: format!("Injected library imports into Opening block: {import_expr}"),
                line: 1,
            });
            return out;
        }

        source.to_string()
    }

    /// Pass 4: Resolves and declares undefined conceptual nouns with infer operator `?`.
    fn heal_inferred_entities(&self, source: &str, repairs: &mut Vec<AppliedRepair>) -> String {
        let mut missing_inferred = Vec::new();

        if source.contains("Ball?") && !source.contains("Ball <") {
            missing_inferred.push("Ball < Entity? > ( Inferred discrete unit )");
        }
        if source.contains("Group?") && !source.contains("Group <") {
            missing_inferred.push("Group < Set? >   ( Inferred collection container )");
        }

        if missing_inferred.is_empty() {
            return source.to_string();
        }

        let mut out = source.to_string();
        // Insert right after Module < ... > definition
        if let Some(pos) = out.find("Module <") {
            if let Some(block_end) = out[pos..].find('}') {
                let insert_at = pos + block_end + 1;
                let mut decls = String::from("\n\n# Auto-Restored Conceptual Types:\n");
                for d in &missing_inferred {
                    decls.push_str(d);
                    decls.push('\n');
                }
                out.insert_str(insert_at, &decls);

                repairs.push(AppliedRepair {
                    kind: FixKind::DeclareEntityWithInfer,
                    description: format!("Declared {} inferred conceptual entities", missing_inferred.len()),
                    line: 1,
                });
            }
        }

        out
    }

    /// Pass 5: Evaluates restored code to verify complete execution without failpoints.
    fn verify_restoration(&self, source: &str) -> (bool, Option<QuadState>, Vec<String>) {
        let mut errors = Vec::new();
        let logger = DiagnosticLogger::new(false);

        // 1. Lint check
        let linter = OmniLinter::new(logger.clone());
        let lint_rep = linter.lint_source("verify", source);
        if !lint_rep.is_valid {
            for err in lint_rep.errors {
                errors.push(format!("Lint error: {err}"));
```

```
            }
        }

        // 2. Tokenize & Parse
        let mut lexer = Lexer::new(source, logger.clone());
        let tokens = match lexer.tokenize() {
            Ok(t) => t,
            Err(e) => {
                errors.push(format!("Lex error: {e}"));
                return (false, None, errors);
            }
        };

        let debugger = OmniDebugger::new(logger.clone(), 10);
        let mut parser = Parser::new(tokens, debugger);
        let prog = match parser.parse_program() {
            Ok(p) => p,
            Err(e) => {
                errors.push(format!("Parse error: {e}"));
                return (false, None, errors);
            }
        };

        // 3. Execution check
        let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
        match evaluator.eval(&crate::ast::nodes::ASTNode::Program(prog)) {
            Ok(val) => {
                let state = match val {
                    crate::runtime::environment::RuntimeValue::State(s) => s,
                    _ => QuadState::Proceed,
                };
                let verified = errors.is_empty() && (state == QuadState::Proceed || state == QuadState::Await);
                (verified, Some(state), errors)
            }
            Err(e) => {
                errors.push(format!("Eval error: {e}"));
                (false, None, errors)
            }
        }
    }
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Restorator Verification and Default Sealing )
// ============================================================================
impl Default for Restorator {
    fn default() -> Self {
        Self::new(DiagnosticLogger::new(false))
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/README.md MD 78 LINES 588 WORDS 4.5 KB

```
# OmniCode Diagnostic & Telemetry Flight Recorder Subsystem

> *"And he gave unto Moses, when he had made an end of communing with him upon mount Sinai, two tables of testimony, tables of stone, written with the finger of God."*
> — **Exodus 31:18 (KJV)**
>
> *"Prepare thy work without, and make it fit for thyself in the field; and afterwards build thine house."*
> — **Proverbs 24:27 (KJV)**
>
> *"And when thou art come in, thou shalt shut the door upon thee and upon thy sons, and shalt pour out into all those vessels, and thou shalt set aside that which is full."*
> — **2 Kings 4:4 (KJV)**

---

## 1. Architectural Philosophy: The Flight Recorder Engine

The `logger` subsystem in **OmniCode Track 1 (Host Engine & Virtual Machine)** is not a simple string printer. It is a **deterministic flight recorder, sensory telemetry bus, and episodic memory storehouse engine** inspired by high-performance game development.

### Core System Invariants

1. **Deterministic Tick / Frame Telemetry**:
   Every process tape roll is treated as an atomic frame in a game loop. States are captured as structured vectors ($S \rightarrow S'$), recording dual-rail SIMD bitplanes, operational verbs, and QuadStates.
2. **Vessel Capacity & Rotation Law**:
   Following container law (`Occupancy <= Capacity`), storehouse partitions automatically rotate and archive historical records when capacity limits are reached.
3. **Homoiconic Actionability**:
   Logs are inscribed as canonical 4-Block OmniCode `.scroll` programs that can be re-executed, stepped through, or automatically repaired by the `Restorator`.
4. **Episodic O(1) Indexing**:
   Replaces linear directory crawls with indexed session lookups, category filters, and temporal range scans.

---

## 2. Subsystem Directory Topology

```basg
src/diagnostics/logger/
├── README.md               <-- Architectural Manifesto & Subsystem Documentation
├── mod.rs                  <-- Backward-Compatible DiagnosticLogger Facade
│
├── universal/              <-- Hardware Ticks, Monotonic Clocks & Lock-Free Bus
│   ├── mod.rs              <-- Re-exports Universal Timing & Bus
│   ├── clock.rs            <-- Monotonic elapsed clock & timestamp ID generation
│   └── bus.rs              <-- Lock-free, zero-allocation event channel bus
│
├── failpoint/              <-- Pathology & Invariant Classification Engine
│   ├── mod.rs              <-- Re-exports Failpoint Types & Remediation
│   ├── types.rs            <-- FailpointType (BoundaryMismatch, NullTrap, CaveatVeto)
│   └── remediation.rs      <-- FixKind, FixSuggestion & machine-applicable patches
│
├── telemetry/              <-- Structured Frame Telemetry & Flight Recorder
│   ├── mod.rs              <-- Re-exports Telemetry Components
│   ├── frame.rs            <-- RollFrame struct (Dual-Rail bitplanes, verbs, QuadState)
│   ├── note.rs             <-- CognitiveNote semantic memory units
│   └── ring_buffer.rs      <-- In-memory Flight Recorder (fixed circular buffer)
│
├── inscriber/              <-- Container Storehouse Inscription Engine
│   ├── mod.rs              <-- Re-exports Inscription Engines & Rotation
│   ├── rotation.rs         <-- RotationPolicy enforcing container capacity limits
│   ├── naming.rs           <-- Standardized vessel naming & path resolution
│   ├── scroll_writer.rs    <-- Inscribes 4-Block .scroll reels to Track 4
│   └── log_writer.rs       <-- Inscribes structured .log files to Track 4
│
└── query/                  <-- Episodic Query Engine (The Indexing Machine)
    ├── mod.rs              <-- Re-exports Query Engine & Index Models
    ├── index.rs            <-- IndexedSession data structure
    └── search.rs           <-- O(1) session search & range scanning
```

```

```
## 3. The Baton Relay to the AI Subsystem

The modular logger serves as the physical sensory nervous system for the AI:

1. **VM steps a Roll** $\rightarrow$ records a `RollFrame` into the `FlightRecorder`.
2. **Execution seals** $\rightarrow$ `inscriber` writes paired `.scroll` and `.log` to Track 4.
3. **`AILearner` ingests** $\rightarrow$ translates `RollFrame`s into $(S, A, R, S')$ reinforcement learning batches.
4. **`NeuralBrain` adapts** $\rightarrow$ updates discrete ternary weights ($W \in \{-1, 0, +1\}$).
5. **`AICortex` converses** $\rightarrow$ deliberates with the developer at the terminal prompt.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/mod.rs RS 356 LINES 1031 WORDS 12.2 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Diagnostic Logger Subsystem Identity, Modular Facade & Re-exports )
// ================================================================================
//! OmniCode Diagnostics: Modular Telemetry, Failpoint & Flight Recorder System
//!
//! # Architecture & Epistemic Role
//! Decomposed into specialized machine organs:
//! - [`universal`]: Monotonic clock, nanosecond timers, and lock-free event bus.
//! - [`failpoint`]: Failpoint pathology classification and machine-applicable repairs.
//! - [`telemetry`]: Game-dev RollFrame vectors, CognitiveNotes, and FlightRecorder.
//! - [`inscriber`]: Canonical 4-Block .scroll and .log writers with rotation policies.
//! - [`query`]: Episodic query engine enabling O(1) session lookup and range scans.

pub mod universal;
pub mod failpoint;
pub mod telemetry;
pub mod inscriber;
pub mod query;

use std::path::{Path, PathBuf};
use std::sync::{Arc, Mutex};

pub use universal::{EventBus, LogEvent, MonotonicClock, generate_timestamp_id};
pub use failpoint::{FailpointType, FixKind, FixSuggestion};
pub use telemetry::{CognitiveNote, FlightRecorder, RollFrame};
pub use inscriber::{
    RotationPolicy, format_base_name, inscribe_audit_scroll,
    inscribe_canonical_scroll, inscribe_telemetry_log, parse_file_name,
    resolve_vessel_path,
};
pub use query::{EpisodicQueryEngine, IndexedSession};

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( DiagnosticLogger Data Structure & Initialization )
// ================================================================================
/// Thread-safe diagnostic logging, flight recording & cognitive memory engine.
#[derive(Debug, Clone)]
pub struct DiagnosticLogger {
    clock: MonotonicClock,
    bus: EventBus,
    cognitive_notes: Arc<Mutex<Vec<CognitiveNote>>>,
    flight_recorder: Arc<Mutex<FlightRecorder>>,
}

impl DiagnosticLogger {
    /// Constructs a new [`DiagnosticLogger`].
    pub fn new(print_live: bool) -> Self {
        Self {
            clock: MonotonicClock::new(),
            bus: EventBus::new(print_live),
            cognitive_notes: Arc::new(Mutex::new(Vec::new())),
            flight_recorder: Arc::new(Mutex::new(FlightRecorder::new(512))),
        }
    }
}

// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Logging Operations, Failpoint Trapping, Flight Recording & Inscription )
// ================================================================================
impl DiagnosticLogger {
    /// Inscribes a normal debug or trace event into the diagnostic log.
    pub fn log(&self, channel: &'static str, message: impl Into<String>) {
        let msg = message.into();
        let elapsed = self.clock.elapsed_micros();
        let event = LogEvent {
            elapsed_micros: elapsed,
            channel,
            message: msg,
            is_failpoint: false,
            failpoint_type: None,
            suggestion: None,
```

```
            cognitive_note: None,
        };
        self.bus.emit(event);
    }

    /// Intercepts and records a critical failpoint event without remediation.
    pub fn failpoint(
        &self,
        fp_type: FailpointType,
        channel: &'static str,
        message: impl Into<String>,
    ) {
        let msg = message.into();
        let elapsed = self.clock.elapsed_micros();
        let note = CognitiveNote {
            timestamp_micros: elapsed,
            category: "INCIDENT".to_string(),
            summary: msg.clone(),
            details: format!("Failpoint trapped: {}", fp_type.as_str()),
            tags: vec!["failpoint".to_string(), fp_type.as_str().to_lowercase()],
            context_snippet: None,
        };

        let event = LogEvent {
            elapsed_micros: elapsed,
            channel,
            message: msg,
            is_failpoint: true,
            failpoint_type: Some(fp_type),
            suggestion: None,
            cognitive_note: Some(note.clone()),
        };

        self.cognitive_notes.lock().unwrap().push(note);
        self.bus.emit(event);
    }

    /// Intercepts a critical failpoint event with concrete remediation telemetry.
    pub fn failpoint_with_remediation(
        &self,
        fp_type: FailpointType,
        channel: &'static str,
        message: impl Into<String>,
        suggestion: FixSuggestion,
    ) {
        let msg = message.into();
        let elapsed = self.clock.elapsed_micros();
        let note = CognitiveNote {
            timestamp_micros: elapsed,
            category: "INCIDENT".to_string(),
            summary: msg.clone(),
            details: suggestion.rationale.clone(),
            tags: vec![
                "failpoint".to_string(),
                fp_type.as_str().to_lowercase(),
                "remediation_ready".to_string(),
            ],
            context_snippet: Some(suggestion.target_token.clone()),
        };

        let event = LogEvent {
            elapsed_micros: elapsed,
            channel,
            message: msg,
            is_failpoint: true,
            failpoint_type: Some(fp_type),
            suggestion: Some(suggestion),
            cognitive_note: Some(note.clone()),
        };

        self.cognitive_notes.lock().unwrap().push(note);
        self.bus.emit(event);
    }

    /// Records an arbitrary cognitive observation into episodic memory.
    pub fn record_ai_observation(
        &self,
        category: &str,
```

```
        summary: &str,
        details: &str,
        tags: &[&str],
        context: Option<&str>,
    ) -> CognitiveNote {
        let elapsed = self.clock.elapsed_micros();
        let note = CognitiveNote {
            timestamp_micros: elapsed,
            category: category.to_string(),
            summary: summary.to_string(),
            details: details.to_string(),
            tags: tags.iter().map(|s| s.to_string()).collect(),
            context_snippet: context.map(|s| s.to_string()),
        };

        self.cognitive_notes.lock().unwrap().push(note.clone());
        self.log(
            "COGNITION",
            format!("[{category}] {summary} (tags: {:?})", note.tags),
        );
        note
    }

    /// Inscribes a pre-formed [`CognitiveNote`] directly into memory.
    pub fn log_cognitive_note(&self, note: CognitiveNote) {
        self.cognitive_notes.lock().unwrap().push(note);
    }

    /// Retrieves a cloned snapshot of all recorded cognitive memory notes.
    pub fn cognitive_notes(&self) -> Vec<CognitiveNote> {
        self.cognitive_notes.lock().unwrap().clone()
    }

    /// Exports all recorded cognitive notes into a canonical 4-Block memory scroll.
    pub fn export_ai_memory_scroll(&self) -> String {
        let notes = self.cognitive_notes();
        inscribe_audit_scroll(&notes, self.bus.all_failpoints().len())
    }

    /// Retrieves all intercepted failpoint events.
    pub fn failpoints(&self) -> Vec<LogEvent> {
        self.bus.all_failpoints()
    }

    /// Retrieves all general log events.
    pub fn events(&self) -> Vec<LogEvent> {
        self.bus.all_events()
    }

    /// Extracts all remediation suggestions currently registered.
    pub fn remediations(&self) -> Vec<FixSuggestion> {
        self.bus
            .all_failpoints()
            .iter()
            .filter_map(|e| e.suggestion.clone())
            .collect()
    }

    /// Returns `true` if any failpoints were trapped.
    pub fn has_failpoints(&self) -> bool {
        !self.bus.all_failpoints().is_empty()
    }

    /// Clears all stored events, failpoints, and cognitive memory notes.
    pub fn clear(&self) {
        self.bus.clear();
        self.cognitive_notes.lock().unwrap().clear();
        self.flight_recorder.lock().unwrap().clear();
    }

    /// Logs an internal AI Thought, indexing it in episodic memory.
    pub fn log_thought(
        &self,
        thought_id: &str,
        omnicode_tape: &str,
        premise: &str,
        quad_state_str: &str,
    ) {
```

```
        let elapsed = self.clock.elapsed_micros();
        let note = CognitiveNote {
            timestamp_micros: elapsed,
            category: "THOUGHT".to_string(),
            summary: format!("Thought [{thought_id}]: {premise}"),
            details: omnicode_tape.to_string(),
            tags: vec![
                "thought".to_string(),
                "ai_cortex".to_string(),
                quad_state_str.to_lowercase(),
            ],
            context_snippet: Some(omnicode_tape.to_string()),
        };

        self.cognitive_notes.lock().unwrap().push(note);
        self.log(
            "CORTEX",
            format!(
                "[THOUGHT: {thought_id}] (State: {quad_state_str}) Premise: \"{premise}\""
            ),
        );
    }

    /// Records an associative memory link.
    pub fn log_memory_association(&self, concept: &str, anchor: &str, details: &str) {
        let elapsed = self.clock.elapsed_micros();
        let note = CognitiveNote {
            timestamp_micros: elapsed,
            category: "ASSOCIATION".to_string(),
            summary: format!("Memory Link: {concept} <-> {anchor}"),
            details: details.to_string(),
            tags: vec!["memory".to_string(), "scripture".to_string(), concept.to_lowercase()],
            context_snippet: Some(anchor.to_string()),
        };

        self.cognitive_notes.lock().unwrap().push(note);
        self.log(
            "MEMORY",
            format!("Associated concept '{concept}' with anchor '{anchor}'"),
        );
    }

    /// Logs the physical inscription of an episodic scroll to disk.
    pub fn log_scroll_inscribed(&self, scroll_path: &Path, session_or_id: &str) {
        self.log(
            "SCROLL",
            format!("Inscribed episodic scroll '{session_or_id}' to {:?}", scroll_path),
        );
    }

    /// Inscribes a canonical 4-Block .scroll file to the given path.
    pub fn inscribe_canonical_scroll(
        &self,
        path: &Path,
        session_id: &str,
        prompt: &str,
        synthesized_omnicode: &str,
        state_str: &str,
        response: &str,
    ) -> Result<PathBuf, String> {
        let elapsed = self.clock.elapsed_micros();
        let res = inscribe_canonical_scroll(
            path,
            session_id,
            prompt,
            synthesized_omnicode,
            state_str,
            response,
            elapsed,
        )?;
        self.log_scroll_inscribed(path, session_id);
        Ok(res)
    }

    // --- GAME-DEV FLIGHT RECORDER & TELEMETRY EXTENSIONS ---

    /// Inscribes a structured execution frame into the in-memory flight recorder.
    pub fn record_roll_frame(&self, frame: RollFrame) {
```

```
        self.log(
            "FRAME",
            format!(
                "Roll {}/{}: {} => {:?}",
                frame.roll_index, frame.total_rolls, frame.verb, frame.quad_state
            ),
        );
        self.flight_recorder.lock().unwrap().record_frame(frame);
    }

    /// Retrieves the most recent execution frames from the flight recorder.
    pub fn recent_roll_frames(&self, count: usize) -> Vec<RollFrame> {
        self.flight_recorder.lock().unwrap().recent_frames(count)
    }

    /// Retrieves the most recent anomalous or failing roll frame.
    pub fn last_anomalous_frame(&self) -> Option<RollFrame> {
        self.flight_recorder.lock().unwrap().last_anomalous_frame()
    }

    /// Creates and populates an [`EpisodicQueryEngine`] pointing to `storehouse_root`.
    pub fn create_query_engine(&self, storehouse_root: &Path) -> EpisodicQueryEngine {
        let mut engine = EpisodicQueryEngine::new();
        let _ = engine.index_directory(storehouse_root);
        engine
    }

    /// Enforces the storehouse capacity rotation policy on a partition directory.
    pub fn enforce_rotation(
        &self,
        partition_dir: &Path,
        policy: RotationPolicy,
    ) -> Result<usize, String> {
        policy.enforce(partition_dir)
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Default Trait Implementation and Subsystem Sealing )
// ============================================================================
impl Default for DiagnosticLogger {
    /// Constructs a silent [`DiagnosticLogger`] with live terminal printing disabled.
    fn default() -> Self {
        Self::new(false)
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/inscriber/log_writer.rs

RS | 91 LINES | 264 WORDS | 3.4 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Traditional Developer & Structured Telemetry .log Inscription Engine )
// ============================================================================
//! Traditional Telemetry Inscription Engine
//!
//! Formats human-readable ASCII telemetry with sensory event headers,
//! microsecond timings, immune system repair notes, and failpoints.

use std::fs;
use std::path::{Path, PathBuf};
use super::super::universal::LogEvent;

/// Inscribes a structured traditional telemetry log file to disk.
pub fn inscribe_telemetry_log(
    path: &Path,
    timestamp_str: &str,
    session_id: &str,
    elapsed_micros: u128,
    final_state_str: &str,
    prompt: &str,
    failpoints: &[LogEvent],
    synthesized_omnicode: &str,
    response: &str,
) -> Result<PathBuf, String> {
    if let Some(parent) = path.parent() {
        if !parent.exists() {
            fs::create_dir_all(parent).map_err(|e| {
                format!("Failed to create log parent directory: {e}")
            })?;
        }
    }

    let mut out = String::new();
    out.push_str("================================================================================\n");
    out.push_str("OMNICODE AI MODEL EXECUTION LOG (TRADITIONAL TELEMETRY)\n");
    out.push_str("================================================================================\n");
    out.push_str(&format!("Timestamp:    {}\n", timestamp_str));
    out.push_str(&format!("Session ID:   {}\n", session_id));
    out.push_str(&format!("Duration:     {} µs\n", elapsed_micros));
    out.push_str(&format!("Final State:  +1:{}\n", final_state_str));
    out.push_str(&format!("Discourse:    \"{}\"\n", prompt));
    out.push_str("--------------------------------------------------------------------------------\n\n");

    out.push_str("[1. SENSORY LOG EVENTS & FAILPOINTS]\n");
    if failpoints.is_empty() {
        out.push_str("  • (No failpoints trapped. Sensory trace clean.)\n");
    } else {
        for (i, fp) in failpoints.iter().enumerate() {
            let kind = fp
                .failpoint_type
                .map(|t| t.as_str())
                .unwrap_or("GENERAL");
            out.push_str(&format!(
                "  [{:>4}] [{} µs] [{}] [{}] {}\n",
                i + 1,
                fp.elapsed_micros,
                fp.channel,
                kind,
                fp.message
            ));
            if let Some(ref sug) = fp.suggestion {
                out.push_str(&format!(
                    "         -> Remediation Proposal: [{:?}] L{}: {}\n",
                    sug.kind, sug.line, sug.rationale
                ));
            }
        }
    }
    out.push('\n');

    out.push_str("[2. IMMUNE SYSTEM RESTORATIONS]\n");
    out.push_str("  • (Zero repairs required; code conformed to all boundary and block invariants.)\n\n");
```

```
74
75      out.push_str("[3. SYNTHESIZED TURING REEL (OMNICODE)]\n");
76      out.push_str(synthesized_omnicode);
77      if !synthesized_omnicode.ends_with('\n') {
78          out.push('\n');
79      }
80      out.push('\n');
81
82      out.push_str("[4. MACHINE DISCOURSE RESPONSE]\n");
83      out.push_str(&format!("  \"{}\"\n", response));
84      out.push_str("================================================================================\n");
85
86      fs::write(path, out)
87          .map_err(|e| format!("Failed to write log to {:?}: {e}", path))?;
88
89      Ok(path.to_path_buf())
90  }
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/inscriber/mod.rs

RS | 16 LINES | 53 WORDS | 0.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Storehouse Inscriber Subsystem Re-exports )
// ============================================================================
//! Storehouse Inscription Subsystem: Writers, Naming & Rotation Policies

pub mod rotation;
pub mod naming;
pub mod scroll_writer;
pub mod log_writer;

pub use rotation::RotationPolicy;
pub use naming::{format_base_name, resolve_vessel_path, parse_file_name, ParsedFileName};
pub use scroll_writer::{inscribe_canonical_scroll, inscribe_audit_scroll};
pub use log_writer::inscribe_telemetry_log;
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/inscriber/naming.rs

RS | 69 LINES | 231 WORDS | 2.1 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Structured Naming Conventions for Deterministic Telemetry Indexing )
// ============================================================================
//! Naming Conventions & Parsing Machinery for Storehouse Vessels
//!
//! Formats deterministic file names allowing $O(1)$ prefix matching and
//! temporal range queries directly from filesystem paths.

use std::path::{Path, PathBuf};

/// Formats a standardized base name for telemetry and scroll files.
///
/// Pattern: `<timestamp>_<session_id>`
pub fn format_base_name(timestamp: &str, session_id: &str) -> String {
    format!("{timestamp}_{session_id}")
}

/// Resolves the destination directory for a specific category and file type.
pub fn resolve_vessel_path(
    root: &Path,
    category: &str,
    is_scroll: bool,
    base_name: &str,
) -> PathBuf {
    let subfolder = if is_scroll { "scroll" } else { "log" };
    let ext = if is_scroll { "scroll" } else { "log" };

    let dest_dir = if root.join(subfolder).exists() {
        root.join(subfolder).join(category)
    } else {
        root.to_path_buf()
    };

    dest_dir.join(format!("{base_name}.{ext}"))
}

/// Metadata extracted directly from a vessel file name.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct ParsedFileName {
    pub timestamp_micros: u128,
    pub session_id: String,
    pub extension: String,
}

/// Extracts metadata from a standardized telemetry file name.
pub fn parse_file_name(path: &Path) -> Option<ParsedFileName> {
    let file_stem = path.file_stem()?.to_string_lossy();
    let ext = path.extension()?.to_string_lossy().to_string();

    let parts: Vec<&str> = file_stem.split('_').collect();
    if parts.is_empty() {
        return None;
    }

    let micros = parts[0].parse::<u128>().ok()?;
    let session = if parts.len() > 1 {
        parts[1..].join("_")
    } else {
        "unknown".to_string()
    };

    Some(ParsedFileName {
        timestamp_micros: micros,
        session_id: session,
        extension: ext,
    })
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/inscriber/rotation.rs

RS | 86 LINES | 300 WORDS | 3.0 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( RotationPolicy: Enforcing Vessel Capacity Bounds on Episodic Storehouses )
// ============================================================================
//! Storehouse Vessel Rotation Engine
//!
//! Grounded in container law `Occupancy <= Capacity` (2 Kings 4:4), this module
//! prevents unbounded file proliferation by rotating and archiving old files.

use std::fs;
use std::path::Path;

/// Policy governing storehouse partition retention and rotation.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct RotationPolicy {
    /// Maximum number of active files allowed in a single domain partition.
    pub max_files_per_partition: usize,
    /// When true, older files are moved to an `archive/` subfolder.
    pub archive_old_files: bool,
}

impl RotationPolicy {
    /// Constructs a standard rotation policy with the given partition capacity.
    pub fn new(max_files_per_partition: usize, archive_old_files: bool) -> Self {
        Self {
            max_files_per_partition: max_files_per_partition.max(10),
            archive_old_files,
        }
    }

    /// Enforces the policy on the specified partition directory.
    pub fn enforce(&self, partition_dir: &Path) -> Result<usize, String> {
        if !partition_dir.exists() {
            return Ok(0);
        }

        let mut entries = Vec::new();
        let read_dir = fs::read_dir(partition_dir)
            .map_err(|e| format!("Failed to read partition dir {:?}: {e}", partition_dir))?;

        for entry in read_dir.flatten() {
            let p = entry.path();
            if p.is_file() && !p.file_name().unwrap_or_default().to_string_lossy().starts_with('.') {
                entries.push(p);
            }
        }

        if entries.len() <= self.max_files_per_partition {
            return Ok(0);
        }

        // Sort chronologically by filename (timestamp is in prefix)
        entries.sort();

        let excess_count = entries.len() - self.max_files_per_partition;
        let mut rotated = 0;

        let archive_dir = partition_dir.join("archive");
        if self.archive_old_files && !archive_dir.exists() {
            fs::create_dir_all(&archive_dir)
                .map_err(|e| format!("Failed to create archive dir {:?}: {e}", archive_dir))?;
        }

        for file_path in entries.iter().take(excess_count) {
            if self.archive_old_files {
                let file_name = file_path.file_name().unwrap();
                let dest = archive_dir.join(file_name);
                if fs::rename(file_path, &dest).is_ok() {
                    rotated += 1;
                }
            } else if fs::remove_file(file_path).is_ok() {
                rotated += 1;
            }
```

```
        }

        Ok(rotated)
    }
}

impl Default for RotationPolicy {
    /// Default storehouse policy: 1000 files per partition with archival.
    fn default() -> Self {
        Self::new(1000, true)
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/inscriber/scroll_writer.rs

RS | 133 LINES | 460 WORDS | 5.3 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Canonical 4-Block OmniCode .scroll Inscription Engine )
// ============================================================================
//! Canonical Scroll Inscription Engine
//!
//! Formats and physically inscribes executable 4-Block OmniCode scrolls.

use std::fs;
use std::path::{Path, PathBuf};
use super::super::telemetry::CognitiveNote;

/// Inscribes a canonical 4-Block .scroll file to the target path.
pub fn inscribe_canonical_scroll(
    path: &Path,
    session_id: &str,
    prompt: &str,
    synthesized_omnicode: &str,
    state_str: &str,
    response: &str,
    elapsed_micros: u128,
) -> Result<PathBuf, String> {
    if let Some(parent) = path.parent() {
        if !parent.exists() {
            fs::create_dir_all(parent).map_err(|e| {
                format!("Failed to create scroll parent directory: {e}")
            })?;
        }
    }

    let indented_reel = synthesized_omnicode
        .lines()
        .map(|l| format!("        {l}"))
        .collect::<Vec<_>>()
        .join("\n");

    let scroll_content = format!(
r#"# ============================================================================
# BLOCK 1: METADATA BLOCK (THE COGNITIVE SCROLL)
# ( Execution Trace ID, Prompt Intent, Inferred Types, and Neural Snapshot )
# ============================================================================
Scroll < EpistemicTrace.{session_id} > ( {prompt} ) {{
    version: < "1.0.0" >,
    system: < OmniCode.AI.Model >,
    session_id: < "{session_id}" >,
    execution_duration_micros: < {elapsed_micros} >,
    cognitive_mode: < NeuralSymbolicTransduction >
}}

# ============================================================================
# BLOCK 2: OPENING BLOCK (EXPERIENCE INITIALIZATION)
# ( Hardware Invariants, Preconditions, and Initial Context )
# ============================================================================
Opening {{
    import: [ Core.Arithmetic :: Core.Logic :: Core.Vectors ],
    preconditions: [ +1 ]
}}

user_prompt < "{prompt}" >

# ============================================================================
# BLOCK 3: BODY BLOCK (CHRONICLE OF EXECUTION & THOUGHT REEL)
# ( Replay Tape: Synthesized Program, Sensory Telemetry & Immune Interventions )
# ============================================================================
Body [
    # Executed Turing Reel Synthesized by AI Model:
    synthesized_reel [
{indented_reel}
    ]
]

# ============================================================================
# BLOCK 4: CLOSING BLOCK (EPISODIC COMMIT & LEARNING PROOF)
```

```
# ( Verification Proof, Final Quad-State, and Conversational Synthesis )
# ================================================================================
Closing {{
    commit_status [ <{state_str}> ],
    machine_utterance ( "{sanitized_response}" )
}}
"#,
        session_id = session_id,
        prompt = prompt,
        elapsed_micros = elapsed_micros,
        indented_reel = indented_reel,
        state_str = state_str,
        sanitized_response = response.replace('"', "\\\"")
    );

    fs::write(path, scroll_content)
        .map_err(|e| format!("Failed to write scroll to {:?}: {e}", path))?;

    Ok(path.to_path_buf())
}

/// Inscribes a complete cognitive audit scroll containing multiple notes.
pub fn inscribe_audit_scroll(
    notes: &[CognitiveNote],
    failpoints_count: usize,
) -> String {
    let mut out = String::new();
    out.push_str("# ================================================================================\n");
    out.push_str("# BLOCK 1: METADATA BLOCK\n");
    out.push_str("# ( Episodic AI Cognitive Memory Scroll & Incident Audit Log )\n");
    out.push_str("# ================================================================================\n");
    out.push_str("Scroll < CognitiveMemory.Audit > {\n");
    out.push_str(&format!("    total_notes: < {} >,\n", notes.len()));
    out.push_str(&format!("    total_failpoints: < {} >\n", failpoints_count));
    out.push_str("}\n\n");
    out.push_str("# ================================================================================\n");
    out.push_str("# BLOCK 2: OPENING BLOCK\n");
    out.push_str("# ( Cognitive Context & Invariant Preconditions )\n");
    out.push_str("# ================================================================================\n");
    out.push_str("Opening {\n    preconditions: [ +1 ]\n}\n\n");
    out.push_str("# ================================================================================\n");
    out.push_str("# BLOCK 3: BODY BLOCK\n");
    out.push_str("# ( Episodic Traces & Memory Units Recorded During Execution )\n");
    out.push_str("# ================================================================================\n");
    out.push_str("Body [\n");
    for n in notes {
        out.push_str(&format!(
            "    {}\n\n",
            n.format_as_omnicode().replace('\n', "\n    ")
        ));
    }
    out.push_str("]\n\n");
    out.push_str("# ================================================================================\n");
    out.push_str("# BLOCK 4: CLOSING BLOCK\n");
    out.push_str("# ( Cognitive Audit Verification Proof )\n");
    out.push_str("# ================================================================================\n");
    out.push_str("Closing {\n    audit_verified: [ PROCEED ]\n}\n");
    out
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/telemetry/frame.rs

RS | 82 LINES | 263 WORDS | 2.7 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( RollFrame Telemetry Struct: Deterministic Game-Dev State Vector )
// ============================================================================
//! RollFrame Structured Telemetry
//!
//! Encapsulates discrete tick-by-tick state vectors captured during process
//! tape stepping, recording dual-rail bitplanes, roll verbs, and QuadStates.

use crate::core::ternary::QuadState;

/// Single execution frame captured during roll traversal.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct RollFrame {
    /// Monotonic elapsed microseconds at roll execution.
    pub tick_micros: u128,
    /// 1-based roll sequence index.
    pub roll_index: usize,
    /// Total rolls declared on this tape ribbon.
    pub total_rolls: usize,
    /// Active operational verb (e.g. "INSCRIBE", "TRANSDUCE", "COMPARE").
    pub verb: String,
    /// Input substrate state as (positive_rail, negative_rail) bitplanes.
    pub dual_rail_in: (u64, u64),
    /// Output substrate state as (positive_rail, negative_rail) bitplanes.
    pub dual_rail_out: (u64, u64),
    /// Fourfold execution state committed by this roll.
    pub quad_state: QuadState,
    /// Compact bitmask representing trapped failpoints (0 = clean).
    pub failpoint_mask: u32,
    /// First-class human/machine intent aside attached to the roll.
    pub intent_aside: Option<String>,
}

impl RollFrame {
    /// Constructs a new [`RollFrame`].
    pub fn new(
        tick_micros: u128,
        roll_index: usize,
        total_rolls: usize,
        verb: impl Into<String>,
        dual_rail_in: (u64, u64),
        dual_rail_out: (u64, u64),
        quad_state: QuadState,
        failpoint_mask: u32,
        intent_aside: Option<String>,
    ) -> Self {
        Self {
            tick_micros,
            roll_index,
            total_rolls,
            verb: verb.into(),
            dual_rail_in,
            dual_rail_out,
            quad_state,
            failpoint_mask,
            intent_aside,
        }
    }

    /// Formats the frame as a compact structured telemetry row.
    pub fn format_row(&self) -> String {
        let aside_str = self
            .intent_aside
            .as_deref()
            .map(|a| format!(" ({a})"))
            .unwrap_or_default();
        format!(
            "[{:>6} µs] Roll {}/{}: {:<10} Rails: ({:#018x}, {:#018x}) -> State: {:<8} Mask: {:#010x}{}",
            self.tick_micros,
            self.roll_index,
            self.total_rolls,
            self.verb,
```

```
            self.dual_rail_out.0,
            self.dual_rail_out.1,
            self.quad_state.macro_op(),
            self.failpoint_mask,
            aside_str
        )
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/telemetry/mod.rs

RS | 14 LINES | 45 WORDS | 0.4 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Telemetry Subsystem Re-exports & Frame Facade )
// ============================================================================
//! Telemetry Subsystem: Frames, Notes & In-Memory Flight Recorder

pub mod frame;
pub mod note;
pub mod ring_buffer;

pub use frame::RollFrame;
pub use note::CognitiveNote;
pub use ring_buffer::FlightRecorder;
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/telemetry/note.rs

RS | 79 LINES | 305 WORDS | 3.0 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Structured Cognitive Memory Notes Formatted for AI Models & IDE Hover )
// ============================================================================
//! Structured Epistemic Memory Note
//!
//! Provides dual-readable semantic notes formatted for neural-symbolic AI
//! context windows, episodic memory reels, and language server tooltips.

/// Structured epistemic memory note formatted for neural-symbolic AI models.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct CognitiveNote {
    /// Monotonic timestamp in microseconds since logger initialization.
    pub timestamp_micros: u128,
    /// Epistemic category: e.g. "OBSERVATION", "HYPOTHESIS", "INCIDENT".
    pub category: String,
    /// High-level single-sentence summary of the cognitive observation.
    pub summary: String,
    /// In-depth architectural details, telemetry data, or remediation rationale.
    pub details: String,
    /// Semantic taxonomy tags for associative memory indexing.
    pub tags: Vec<String>,
    /// Optional code snippet or token context under active observation.
    pub context_snippet: Option<String>,
}

impl CognitiveNote {
    /// Formats this note as an OmniCode native dual-readable syntax block.
    pub fn format_as_omnicode(&self) -> String {
        let tag_str = if self.tags.is_empty() {
            "[]".to_string()
        } else {
            format!(
                "[ {} ]",
                self.tags
                    .iter()
                    .map(|t| format!("#{t}"))
                    .collect::<Vec<_>>()
                    .join(" :: ")
            )
        };
        let ctx_str = if let Some(c) = &self.context_snippet {
            format!("\n    context: < {:?} >", c)
        } else {
            String::new()
        };
        format!(
            "Note < CognitiveTrace > {{\n    tick: < {} µs >,\n    category: [ {} ],\n    summary: < {:?} >,\n    details: < {:?} >,\n    tags: {}{}\n}}",
            self.timestamp_micros,
            self.category,
            self.summary,
            self.details,
            tag_str,
            ctx_str
        )
    }

    /// Formats this note as a Markdown block specifically structured for prompts.
    pub fn format_as_memory_entry(&self) -> String {
        let tag_list = if self.tags.is_empty() {
            "none".to_string()
        } else {
            self.tags
                .iter()
                .map(|t| format!("#{t}"))
                .collect::<Vec<_>>()
                .join(", ")
        };
        let mut out = format!(
            "### [EPISODIC_MEMORY_TRACE: {}]\n- **Timestamp:** {} µs\n- **Summary:** {}\n- **Details:** {}\n- **Tags:** {}\n",
            self.category, self.timestamp_micros, self.summary, self.details, tag_list
```

```
        );
        if let Some(ctx) = &self.context_snippet {
            out.push_str(&format!("- **Context:** `{}`\n", ctx));
        }
        out
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/telemetry/ring_buffer.rs

RS | 95 LINES | 313 WORDS | 3.0 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( FlightRecorder: Fixed-Capacity Circular Buffer for Deterministic Replay )
// ============================================================================
//! In-Memory Flight Recorder (Circular Telemetry Ring Buffer)
//!
//! Stores the last $K$ execution frames in memory with zero allocations in
//! steady-state, allowing instantaneous post-mortem crash inspection.

use super::frame::RollFrame;
use crate::core::ternary::QuadState;

/// In-memory circular telemetry buffer.
#[derive(Debug, Clone)]
pub struct FlightRecorder {
    frames: Vec<RollFrame>,
    capacity: usize,
    write_head: usize,
    total_inscribed: usize,
}

impl FlightRecorder {
    /// Constructs a new [`FlightRecorder`] with the specified frame capacity.
    pub fn new(capacity: usize) -> Self {
        let cap = capacity.max(16);
        Self {
            frames: Vec::with_capacity(cap),
            capacity: cap,
            write_head: 0,
            total_inscribed: 0,
        }
    }

    /// Inscribes a new frame into the circular buffer.
    pub fn record_frame(&mut self, frame: RollFrame) {
        if self.frames.len() < self.capacity {
            self.frames.push(frame);
        } else {
            self.frames[self.write_head] = frame;
        }
        self.write_head = (self.write_head + 1) % self.capacity;
        self.total_inscribed += 1;
    }

    /// Returns a chronological snapshot of the most recent `count` frames.
    pub fn recent_frames(&self, count: usize) -> Vec<RollFrame> {
        let available = self.frames.len();
        let take = count.min(available);
        if take == 0 {
            return Vec::new();
        }

        let mut result = Vec::with_capacity(take);
        if available < self.capacity {
            let start = available.saturating_sub(take);
            result.extend_from_slice(&self.frames[start..available]);
        } else {
            for i in 0..take {
                let idx = (self.write_head + self.capacity - take + i)
                    % self.capacity;
                result.push(self.frames[idx].clone());
            }
        }
        result
    }

    /// Returns the last frame that resulted in a rollback or failpoint.
    pub fn last_anomalous_frame(&self) -> Option<RollFrame> {
        for frame in self.recent_frames(self.frames.len()).into_iter().rev() {
            if frame.quad_state == QuadState::Rollback || frame.failpoint_mask != 0 {
                return Some(frame);
            }
        }
```

```
        None
    }

    /// Total count of frames inscribed over the lifetime of the recorder.
    pub fn total_inscribed(&self) -> usize {
        self.total_inscribed
    }

    /// Clears the flight recorder.
    pub fn clear(&mut self) {
        self.frames.clear();
        self.write_head = 0;
        self.total_inscribed = 0;
    }
}

impl Default for FlightRecorder {
    fn default() -> Self {
        Self::new(512)
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/failpoint/mod.rs

RS | 12 LINES | 38 WORDS | 0.4 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Failpoint Subsystem Re-exports & Diagnostics Facade )
// ============================================================================
//! Failpoint Pathology & Remediation Telemetry Module

pub mod types;
pub mod remediation;

pub use types::FailpointType;
pub use remediation::{FixKind, FixSuggestion};
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/failpoint/remediation.rs

RS | 45 LINES | 235 WORDS | 2.0 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Machine-Applicable Automated Repair Kinds & Concrete Fix Suggestions )
// ============================================================================
//! OmniCode Diagnostics: Concrete Automated Remediation Telemetry
//!
//! Encapsulates machine-applicable patch descriptions allowing the
//! Restorator or AI coding agents to heal broken source files in place.

/// Categories of automated restorative repairs executed by Restorator.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum FixKind {
    /// Append a missing closing boundary delimiter (e.g. closing `[` with `]`).
    CloseBoundary,
    /// Substitute erroneous closing delimiter with matching opening partner.
    FixBoundaryMismatch,
    /// Inject missing canonical 4-Block architecture scaffolding headers.
    AddBlockHeader,
    /// Automatically inject missing core library imports (Core.Arithmetic, etc).
    AddImport,
    /// Synthesize and declare an inferred conceptual noun entity.
    DeclareEntityWithInfer,
    /// Modify or satisfy an adversative caveat expression.
    AdjustCaveat,
    /// Convert an immutable Rank 1 invariant `< ... >` into mutable `[ ... ]`.
    ConvertImmutableToMutable,
}

/// Concrete remediation telemetry attached to a failpoint event.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct FixSuggestion {
    /// Categorical classification of the automated remediation.
    pub kind: FixKind,
    /// 1-based source line number where remediation should be applied.
    pub line: usize,
    /// 1-based source column number where the target token resides.
    pub col: usize,
    /// The erroneous or missing token sequence targeted for replacement.
    pub target_token: String,
    /// The proposed patch or substitution string that seals the invariant.
    pub suggested_patch: String,
    /// Human-readable and AI-interpretable rationale explaining the fix.
    pub rationale: String,
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/failpoint/types.rs

RS | 61 LINES | 276 WORDS | 2.7 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Failpoint Pathology Categorization & Invariant Anomaly Enums )
// ============================================================================
//! OmniCode Diagnostics: Failpoint Pathology Taxonomy
//!
//! Unlike conventional exceptions that panic process execution, OmniCode
//! treats failpoints as first-class diagnostic signals that guide rollback,
//! caveat evaluation, or autonomous immune restoration via the Restorator.

/// Classifications of trapped failpoints and boundary anomalies.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum FailpointType {
    /// Delimiter mismatch: opened with one delimiter class, closed with another.
    BoundaryMismatch,
    /// Unclosed boundary delimiter reaching EOF without matching partner.
    BoundaryLeak,
    /// Execution attempted to evaluate void/null state without resolving caveat.
    NullTrap,
    /// An operation or caveat evaluation triggered state rollback (-1:ROLLBACK).
    RollbackTriggered,
    /// An adversative caveat condition (! [ ... ]) vetoed active state.
    CaveatVeto,
    /// Illegal attempt to mutate an immutable Rank 1 invariant axiom < ... >.
    ImmutableMutation,
    /// Recursive or cyclic retroaction loop exceeding debugger threshold.
    CyclicRetroaction,
    /// Structural or syntactic grammar anomaly violating language invariants.
    SyntaxAnomaly,
}

impl FailpointType {
    /// Returns the uppercase canonical string identifier for this failpoint.
    pub fn as_str(&self) -> &'static str {
        match self {
            FailpointType::BoundaryMismatch => "FP_BOUNDARY_MISMATCH",
            FailpointType::BoundaryLeak => "FP_BOUNDARY_LEAK",
            FailpointType::NullTrap => "FP_NULL_TRAP",
            FailpointType::RollbackTriggered => "FP_ROLLBACK_TRIGGERED",
            FailpointType::CaveatVeto => "FP_CAVEAT_VETO",
            FailpointType::ImmutableMutation => "FP_IMMUTABLE_MUTATION",
            FailpointType::CyclicRetroaction => "FP_CYCLIC_RETROACTION",
            FailpointType::SyntaxAnomaly => "FP_SYNTAX_ANOMALY",
        }
    }

    /// Compact single-bit representation for game-dev flight recorder masks.
    pub fn bitmask(&self) -> u32 {
        match self {
            FailpointType::BoundaryMismatch => 1 << 0,
            FailpointType::BoundaryLeak => 1 << 1,
            FailpointType::NullTrap => 1 << 2,
            FailpointType::RollbackTriggered => 1 << 3,
            FailpointType::CaveatVeto => 1 << 4,
            FailpointType::ImmutableMutation => 1 << 5,
            FailpointType::CyclicRetroaction => 1 << 6,
            FailpointType::SyntaxAnomaly => 1 << 7,
        }
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/query/index.rs

RS | 32 LINES | 139 WORDS | 1.2 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( IndexedSession Data Structure for Fast Episodic Retrieval )
// ============================================================================
//! Indexed Episode Metadata Definition

use std::path::PathBuf;
use crate::core::ternary::QuadState;

/// High-level indexed record representing a past execution episode.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct IndexedSession {
    /// Unique session identifier string.
    pub session_id: String,
    /// Monotonic timestamp in microseconds since epoch.
    pub timestamp_micros: u128,
    /// Domain category (e.g. "character", "number", "sessions", "cortex").
    pub category: String,
    /// Final fourfold commit state.
    pub final_state: QuadState,
    /// Number of failpoints trapped during execution.
    pub failpoint_count: usize,
    /// Number of automated repairs applied by Restorator.
    pub repair_count: usize,
    /// Path to the native .scroll file.
    pub scroll_path: PathBuf,
    /// Path to the traditional .log file if present.
    pub log_path: Option<PathBuf>,
    /// Natural language or execution prompt that initiated the run.
    pub prompt: String,
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/query/mod.rs RS 12 LINES 36 WORDS 0.4 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Episodic Query Subsystem Re-exports )
// ============================================================================
//! Episodic Query Subsystem: Indexing & Fast Retrieval

pub mod index;
pub mod search;

pub use index::IndexedSession;
pub use search::EpisodicQueryEngine;
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/query/search.rs

RS | 273 LINES | 798 WORDS | 9.0 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( EpisodicQueryEngine: O(1) Lookup & Partitioned Range Search )
// ============================================================================
//! Episodic Query Engine
//!
//! Indexes and queries past execution scrolls and telemetry without linear
//! file scans, supporting filtering by session, category, QuadState, and time.

use std::collections::HashMap;
use std::fs;
use std::path::{Path, PathBuf};
use super::index::IndexedSession;
use crate::core::ternary::QuadState;

/// High-performance query engine for episodic memory storehouses.
#[derive(Debug, Clone, Default)]
pub struct EpisodicQueryEngine {
    sessions: Vec<IndexedSession>,
    by_session_id: HashMap<String, usize>,
    by_category: HashMap<String, Vec<usize>>,
}

impl EpisodicQueryEngine {
    /// Constructs an empty [`EpisodicQueryEngine`].
    pub fn new() -> Self {
        Self::default()
    }

    /// Indexes all `.scroll` files located under `root_dir`.
    pub fn index_directory(
        &mut self,
        root_dir: &Path,
    ) -> Result<usize, String> {
        self.sessions.clear();
        self.by_session_id.clear();
        self.by_category.clear();

        if !root_dir.exists() {
            return Ok(0);
        }

        let scroll_paths = collect_scroll_files(root_dir);
        let mut count = 0;

        for path in scroll_paths {
            if let Some(session) = parse_indexed_session(&path) {
                let idx = self.sessions.len();
                self.by_session_id
                    .insert(session.session_id.clone(), idx);
                self.by_category
                    .entry(session.category.clone())
                    .or_default()
                    .push(idx);
                self.sessions.push(session);
                count += 1;
            }
        }

        // Sort chronologically
        self.sessions.sort_by_key(|s| s.timestamp_micros);
        // Rebuild session ID map after sort with multi-key aliases
        self.by_session_id.clear();
        self.by_category.clear();
        for (idx, s) in self.sessions.iter().enumerate() {
            self.by_session_id.insert(s.session_id.clone(), idx);
            if let Some(stem) = s
                .scroll_path
                .file_stem()
                .and_then(|t| t.to_str())
            {
                self.by_session_id.insert(stem.to_string(), idx);
                let parts: Vec<&str> = stem.split('_').collect();
```

```
                if parts.len() >= 3 {
                    self.by_session_id.insert(parts[1..].join("_"), idx);
                    self.by_session_id.insert(parts[2..].join("_"), idx);
                }
            }
            self.by_category
                .entry(s.category.clone())
                .or_default()
                .push(idx);
        }

        Ok(count)
    }

    /// Finds an episode by its exact session ID in O(1) time.
    pub fn find_by_session(&self, session_id: &str) -> Option<&IndexedSession> {
        let idx = *self.by_session_id.get(session_id)?;
        self.sessions.get(idx)
    }

    /// Filters all episodes belonging to a specific domain category.
    pub fn filter_by_category(&self, category: &str) -> Vec<&IndexedSession> {
        if let Some(indices) = self.by_category.get(category) {
            indices.iter().filter_map(|&i| self.sessions.get(i)).collect()
        } else {
            Vec::new()
        }
    }

    /// Filters episodes by their commit QuadState.
    pub fn filter_by_state(&self, state: QuadState) -> Vec<&IndexedSession> {
        self.sessions
            .iter()
            .filter(|s| s.final_state == state)
            .collect()
    }

    /// Filters episodes that experienced failpoints.
    pub fn filter_by_failpoint(
        &self,
        has_failpoints: bool,
    ) -> Vec<&IndexedSession> {
        self.sessions
            .iter()
            .filter(|s| (s.failpoint_count > 0) == has_failpoints)
            .collect()
    }

    /// Range scans episodes executed between `start_micros` and `end_micros`.
    pub fn range_scan(
        &self,
        start_micros: u128,
        end_micros: u128,
    ) -> Vec<&IndexedSession> {
        self.sessions
            .iter()
            .filter(|s| {
                s.timestamp_micros >= start_micros
                    && s.timestamp_micros <= end_micros
            })
            .collect()
    }

    /// Returns a slice of all indexed sessions.
    pub fn all_sessions(&self) -> &[IndexedSession] {
        &self.sessions
    }
}

/// Recursively collects all `.scroll` files within `dir`.
fn collect_scroll_files(dir: &Path) -> Vec<PathBuf> {
    let mut files = Vec::new();
    if let Ok(entries) = fs::read_dir(dir) {
        for entry in entries.flatten() {
            let p = entry.path();
            if p.is_dir() {
                files.extend(collect_scroll_files(&p));
            } else if p.extension().and_then(|e| e.to_str()) == Some("scroll") {
```

```
                files.push(p);
            }
        }
    }
    files
}

/// Parses an `IndexedSession` from a single `.scroll` file.
fn parse_indexed_session(path: &Path) -> Option<IndexedSession> {
    let content = fs::read_to_string(path).ok()?;
    let mut session_id = "unknown".to_string();
    let mut prompt = String::new();
    let mut final_state = QuadState::Proceed;
    let mut failpoint_count = 0;
    let mut repair_count = 0;
    let mut timestamp_micros = 0u128;

    for line in content.lines() {
        let trimmed = line.trim();
        if trimmed.starts_with("session_id:") {
            if let Some(sq) = trimmed.find('"') {
                if let Some(eq) = trimmed.rfind('"') {
                    if sq < eq {
                        session_id = trimmed[sq + 1..eq].to_string();
                    }
                }
            }
        } else if trimmed.starts_with("timestamp:") {
            if let Some(sq) = trimmed.find('"') {
                if let Some(eq) = trimmed.rfind('"') {
                    if sq < eq {
                        let ts_str = &trimmed[sq + 1..eq];
                        if let Some(first) = ts_str.split('_').next() {
                            timestamp_micros = first.parse().unwrap_or(0);
                        }
                    }
                }
            }
        } else if trimmed.starts_with("user_prompt") {
            if let Some(sq) = trimmed.find('"') {
                if let Some(eq) = trimmed.rfind('"') {
                    if sq < eq {
                        prompt = trimmed[sq + 1..eq].to_string();
                    }
                }
            }
        } else if prompt.is_empty()
            && trimmed.starts_with("Scroll < EpistemicTrace")
        {
            if let Some(open_p) = trimmed.find('(') {
                if let Some(close_p) = trimmed.rfind(')') {
                    if open_p < close_p {
                        prompt = trimmed[open_p + 1..close_p]
                            .trim()
                            .to_string();
                    }
                }
            }
        } else if trimmed.starts_with("commit_status [") {
            if trimmed.contains("<PROCEED>") {
                final_state = QuadState::Proceed;
            } else if trimmed.contains("<ROLLBACK>") {
                final_state = QuadState::Rollback;
            } else if trimmed.contains("<AWAIT>") {
                final_state = QuadState::Await;
            } else if trimmed.contains("<HALT>") {
                final_state = QuadState::Halt;
            }
        } else if trimmed.starts_with("failpoint_record [") {
            failpoint_count += 1;
        } else if trimmed.starts_with("repair_entry [") {
            repair_count += 1;
        }
    }

    // Determine category from path or prompt
    let path_str = path.to_string_lossy();
    let category = if path_str.contains("/character/")
```

```
        || prompt.contains("character")
    {
        "character".to_string()
    } else if path_str.contains("/number/") || prompt.contains("number") {
        "number".to_string()
    } else if path_str.contains("/sessions/prompts")
        || (!prompt.trim().is_empty() && path_str.contains("/sessions/"))
    {
        "sessions/prompts".to_string()
    } else if path_str.contains("/sessions/")
        || session_id.starts_with("sess_")
    {
        "sessions/system".to_string()
    } else {
        "cortex".to_string()
    };

    // Locate paired log file
    let mut log_path = path.to_path_buf();
    log_path.set_extension("log");
    let paired_log = if log_path.exists() {
        Some(log_path)
    } else if path_str.contains("/scroll/") {
        let alt = PathBuf::from(path_str.replace("/scroll/", "/log/"));
        let mut alt_log = alt;
        alt_log.set_extension("log");
        if alt_log.exists() { Some(alt_log) } else { None }
    } else {
        None
    };

    Some(IndexedSession {
        session_id,
        timestamp_micros,
        category,
        final_state,
        failpoint_count,
        repair_count,
        scroll_path: path.to_path_buf(),
        log_path: paired_log,
        prompt,
    })
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/universal/bus.rs

RS | 96 LINES | 305 WORDS | 3.2 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Diagnostic Event Bus, Channel Stream & Live Terminal Mirroring )
// ============================================================================
//! Diagnostic Event Bus & Telemetry Event Definitions

use std::sync::{Arc, Mutex};
use super::super::failpoint::{FailpointType, FixSuggestion};
use super::super::telemetry::CognitiveNote;

/// Individual diagnostic event recorded during execution ticks.
#[derive(Debug, Clone)]
pub struct LogEvent {
    /// Microseconds elapsed since logger initialization.
    pub elapsed_micros: u128,
    /// Channel or subsystem tag (e.g. "BOUNDARY", "DEBUGGER", "TERNARY").
    pub channel: &'static str,
    /// Descriptive message explaining the operation or signal.
    pub message: String,
    /// True if this event represents an intercepted failpoint or anomaly.
    pub is_failpoint: bool,
    /// The specific failpoint category if `is_failpoint` is true.
    pub failpoint_type: Option<FailpointType>,
    /// Concrete remediation suggestion if an automated repair is available.
    pub suggestion: Option<FixSuggestion>,
    /// Accompanying structured cognitive memory note for AI ingestion.
    pub cognitive_note: Option<CognitiveNote>,
}

/// Thread-safe event bus coordinating live diagnostic streams.
#[derive(Debug, Clone)]
pub struct EventBus {
    events: Arc<Mutex<Vec<LogEvent>>>,
    failpoints: Arc<Mutex<Vec<LogEvent>>>,
    print_live: bool,
}

impl EventBus {
    /// Constructs a new [`EventBus`].
    pub fn new(print_live: bool) -> Self {
        Self {
            events: Arc::new(Mutex::new(Vec::new())),
            failpoints: Arc::new(Mutex::new(Vec::new())),
            print_live,
        }
    }

    /// Emits a general trace event to the bus.
    pub fn emit(&self, event: LogEvent) {
        if self.print_live {
            if event.is_failpoint {
                let kind_str = event
                    .failpoint_type
                    .map(|f| f.as_str())
                    .unwrap_or("FAILPOINT");
                eprintln!(
                    "[{:>6} µs] \x1b[91mCRITICAL\x1b[0m [{}] [{}] {}",
                    event.elapsed_micros, event.channel, kind_str, event.message
                );
            } else {
                println!(
                    "[{:>6} µs] DEBUG    [{}] {}",
                    event.elapsed_micros, event.channel, event.message
                );
            }
        }

        if event.is_failpoint {
            self.failpoints.lock().unwrap().push(event.clone());
        }
        self.events.lock().unwrap().push(event);
    }

```

```
    /// Returns a cloned snapshot of all recorded events.
    pub fn all_events(&self) -> Vec<LogEvent> {
        self.events.lock().unwrap().clone()
    }

    /// Returns a cloned snapshot of all failpoints.
    pub fn all_failpoints(&self) -> Vec<LogEvent> {
        self.failpoints.lock().unwrap().clone()
    }

    /// Clears all events from the bus.
    pub fn clear(&self) {
        self.events.lock().unwrap().clear();
        self.failpoints.lock().unwrap().clear();
    }
}

impl Default for EventBus {
    fn default() -> Self {
        Self::new(false)
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/universal/clock.rs

RS | 56 LINES | 176 WORDS | 1.7 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Monotonic Clock, Microsecond Timers & Timestamp Generator )
// ============================================================================
//! Universal High-Precision Clock & Monotonic Timestamp Generator
//!
//! Provides nanosecond/microsecond resolution elapsed time offsets and
//! strictly monotonic timestamp identifiers for telemetry frame ordering.

use std::sync::atomic::{AtomicU64, Ordering};
use std::time::{Instant, SystemTime};

/// Generates a strictly unique, monotonic timestamp identifier.
///
/// Format: `<unix_epoch_micros>_<atomic_counter:04>`
pub fn generate_timestamp_id() -> String {
    static COUNTER: AtomicU64 = AtomicU64::new(0);
    let count = COUNTER.fetch_add(1, Ordering::Relaxed);
    let now = SystemTime::now()
        .duration_since(SystemTime::UNIX_EPOCH)
        .unwrap_or_default()
        .as_micros();
    format!("{now}_{count:04}")
}

/// Monotonic clock reference measuring microsecond elapsed durations.
#[derive(Debug, Clone)]
pub struct MonotonicClock {
    start: Instant,
}

impl MonotonicClock {
    /// Constructs a new [`MonotonicClock`] calibrated to the current instant.
    pub fn new() -> Self {
        Self {
            start: Instant::now(),
        }
    }

    /// Returns elapsed microseconds since clock initialization.
    pub fn elapsed_micros(&self) -> u128 {
        self.start.elapsed().as_micros()
    }

    /// Resets the base reference instant to now.
    pub fn reset(&mut self) {
        self.start = Instant::now();
    }
}

impl Default for MonotonicClock {
    fn default() -> Self {
        Self::new()
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/universal/mod.rs

RS | 12 LINES | 40 WORDS | 0.4 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Universal Telemetry & Hardware Clock Re-exports )
// ============================================================================
//! Universal Clock, Bus & Hardware Telemetry Machinery

pub mod clock;
pub mod bus;

pub use clock::{MonotonicClock, generate_timestamp_id};
pub use bus::{LogEvent, EventBus};
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/cogs/mod.rs RS 32 LINES 132 WORDS 1.4 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Universal Cogs Module Identity & Mechanical Dynamic Primitives )
// ============================================================================
//! OmniCode Cogs: Dynamic Mechanical Primitives & Decoupled Functions
//!
//! Reusable single-purpose dynamic utilities shared across gears:
//! - Dynamic host platform detection & environment adaptation (`platform`)
//! - Universal invariant tripwires and diagnostic interceptors (`tripwire`)
//! - Cross-track runtime path resolution

pub mod platform;
pub mod tripwire;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Cog Declarations & Re-exports )
// ============================================================================
pub use platform::{HostPlatform, HostArch};
pub use tripwire::{Tripwire, TripwireError};

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Cog Implementations & Shared Primitives )
// ============================================================================

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Verification and Sealing )
// ============================================================================
// Cogs subsystem verified and sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/cogs/platform.rs RS 940 LINES 4296 WORDS 42.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Dynamic Host Platform Cog Manifest, Scheduling Contract & Tripwire Registry )
// ============================================================================
//! MODULE:              omnicode::cogs::platform
//! IDENTITY:            Dynamic Host Platform Adapter Cog
//! NATURE:              Dynamic Environmental Meshing (Runtime Host Adaptation & In-Waiting Symbiosis)
//! VERSION:             0.3.0
//! CREATED:             2026-10-11
//! LAST_SEALED:         2026-10-11
//! AUTHOR:              CreativeWorkzStudio LLC / ProfessorSeanEX
//! COVENANT:            Public Domain / MIT
//!
//! SCHEDULER:
//!   TRACK_PRIMARY:     <TRACK 1> : {Host Engine & Virtual Machine}
//!   TRACKS_TOUCHED:    [ <TRACK 0> : {Silicon & Bare-Metal Kernel}, <TRACK 1>, <TRACK 2> : {Engine Runtime & SVO},
<TRACK 3> : {Studio & Graphical IDE} ]
//!   PRIORITY:          TaskPriority::High
//!   CHANNEL:           Track::Kernel
//!
//! CONTRACT / WHO-WHAT-WHERE-WHY-HOW:
//!   WHO:               OmniCode Host Environmental Adapter & Hardware Abstraction Layer (HAL)
//!   WHAT:              Dynamically detect host OS and CPU architecture, resolve toolchains and hypervisors,
//!                      probe hardware virtualization acceleration, discover UEFI OVMF firmware blobs,
//!                      audit host display servers, and orchestrate tripwired subprocess execution.
//!   WHERE:             Runtime host environment (GNU/Linux, Apple macOS Darwin, Microsoft Windows NT, Generic POSIX)
//!   WHEN:              Evaluated at engine startup, CLI command dispatch, GUI studio launch, or bare-metal boot
inception.
//!   WHY:               Enable the "In-Waiting" symbiotic execution model: OmniCode is an autonomous
//!                      operating system and balanced-ternary language, but during incubation it executes
//!                      harmoniously on any host OS without hardcoding platform paths or failing when moved.
//!   HOW:               Balanced Ternary (+1:PROCEED host detected & certified, -1:ROLLBACK unsupported/missing, 0:AWAIT
probe)
//!
//! THE "IN-WAITING" SYMBIOTIC INCUBATION MODEL:
//!   OmniCode is not merely a programming language compiler; it is an entirely distinct computing paradigm,
//!   operating system, and balanced-ternary machine architecture. However, during its developmental incubation,
//!   engineers work inside conventional operating systems (Linux, macOS, Windows).
//!   This cog acts as the umbilical cord and Hardware Abstraction Layer (HAL):
//!   - Adapts to the host's native filesystem hierarchy (FHS vs Homebrew vs Windows Program Files).
//!   - Selects the correct hypervisor binary (`qemu-system-x86_64` vs `qemu-system-x86_64.exe`).
//!   - Selects native acceleration flags (`-enable-kvm` on Linux, `-accel hvf` on macOS, `-accel whpx` on Windows).
//!   - Resolves native graphical browser dispatchers (`xdg-open` vs `open` vs `cmd /C start`).
//!   - Detects headless display environments to prevent graphical crashes.
//!
//! GAME DEV IDEOLOGY (THE HAL & PHYSICS SIMULATION ANALOGY):
//!   In modern game engine architecture (e.g., Unreal, Unity, Godot), the game simulation tick (`Update()`)
//!   runs identically regardless of whether rendering is powered by DirectX 12, Metal, or Vulkan.
//!   In OmniCode:
//!   - The Core Simulation: Word32 balanced-ternary ALU, tape reels, SVO grammar, and QuadState causality.
//!   - The Platform Adapter (This Cog): The cross-platform driver layer abstracting host silicon and OS syscalls.
//!   - Tripwires: Spatial and temporal collision volumes. When an invariant fails (missing binary, unreadable
//!     KVM node, absent display server), the tripwire catches the collision, records frame telemetry to
//!     `DiagnosticLogger` and `OmniDebugger`, and passes remediation tokens to `Restorator` for autonomous self-healing.
//!
//! TOUCHES / SYSTEM & PROCESS FOOTPRINT:
//!   - DISPATCH:        Native browser openers (`xdg-open` on Linux, `open` on macOS, `cmd /C start` on Windows)
//!   - DISCOVERY:       Host filesystem firmware lookups (`/usr/share/qemu`, `/opt/homebrew`, `C:\Program Files\qemu`)
//!   - BINARIES:        Probes for `qemu-system-x86_64` vs `qemu-system-x86_64.exe` on host PATH
//!   - VIRTUALIZATION:  Probes `/dev/kvm` (Linux KVM), Hypervisor.framework (macOS HVF), WHPX (Windows), TCG (Fallback)
//!   - DISPLAY:         Probes `$DISPLAY`, `$WAYLAND_DISPLAY`, and window manager sessions for GUI capability.
//!   - PROCESSES:       Uniform child process spawning with standard stream capture and exit status checking.
//!
//! REQUIRES / HOST CAPABILITIES:
//!   - Rust `cfg!(target_os)` and `cfg!(target_arch)` target family introspection.
//!   - Subprocess execution capabilities (`std::process::Command`).
//!   - Filesystem metadata and permission inspection (`std::path::Path::exists`, `std::fs::OpenOptions`).
//!   - Environment variable inspection (`std::env::var_os`, `std::env::var`).
//!
//! TRIPWIRE REGISTRY:
//!   - TRIPWIRE_PLATFORM_PROBE       (Identification and verification of host platform plane)
//!   - TRIPWIRE_ARCH_SUPPORT         (Assertion of host CPU architecture compatibility with target)
//!   - TRIPWIRE_FIRMWARE_DISCOVERY   (Resolution and verification of valid OVMF firmware candidate)
//!   - TRIPWIRE_HYPERVISOR_AVAIL     (Assertion of hypervisor executable on host PATH)
```

```
//!   - TRIPWIRE_ACCEL_DEVICE         (Hardware virtualization node accessibility / permission check)
//!   - TRIPWIRE_DISPLAY_SERVER       (Assertion of active display server when GUI mode is requested)
//!   - TRIPWIRE_BROWSER_DISPATCH     (Successful handoff to host graphical browser)
//!   - TRIPWIRE_SUBPROCESS_EXEC      (Generic host child process execution and exit code verification)
//!
//! NAVIGATION GUIDE (HOW TO NAVIGATE AND EXTEND THIS MODULE):
//!   - Block 1 (Lines 1-90): Module Manifest, Incubation Philosophy, Game Dev Analogy, Tripwire Registry.
//!   - Block 2 (Lines 91-180): Scoped Imports, Multi-OS Firmware Matrices, Equilibrium Constants, HostArch Enum.
//!     * To add firmware search paths for a new Linux distro or BSD: edit `LINUX_OVMF_CANDIDATES` in Section 2.2.
//!     * To add a new CPU architecture: add a variant to `HostArch` in Section 2.4.
//!   - Block 3 (Lines 181-480): Body Block & Implementation Oracles.
//!     * 3.1: HostPlatform runtime classification, display names, and environment summary.
//!     * 3.2: Host PATH resolution (`resolve_in_path`) and hypervisor binary verification.
//!     * 3.3: Hardware virtualization acceleration probes (`has_hardware_acceleration`,
`assert_acceleration_supported`).
//!     * 3.4: Host display server & GUI environment oracles (`has_display_server`, `assert_display_server_available`).
//!     * 3.5: UEFI OVMF firmware search with telemetry and tripwires (`find_ovmf_firmware_with_telemetry`).
//!     * 3.6: Host process dispatch, child execution, and browser launching (`execute_host_command`).
//!   - Block 4 (Lines 481+): Unit tests verifying platform detection, binary resolution, PATH parsing, and telemetry.

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Scoped Imports, Environment Definitions, Search Matrices, Architecture & Constants )
// ============================================================================

// ----------------------------------------------------------------------------
// 2.1. Scoped Namespace Ingestion & System Imports
// ----------------------------------------------------------------------------
use std::env;
use std::fs::OpenOptions;
use std::path::{Path, PathBuf};
use std::process::{Command, Output};

use crate::cogs::tripwire::{Tripwire, TripwireError};
use crate::diagnostics::logger::{DiagnosticLogger, FailpointType, FixKind, FixSuggestion};

// ----------------------------------------------------------------------------
// 2.2. Multi-Platform Firmware Candidate Matrices & Search Registries
// ----------------------------------------------------------------------------
/// Canonical Linux OVMF UEFI firmware candidate search paths.
///
/// Covers major distribution layouts:
/// - Debian / Ubuntu: `/usr/share/ovmf/OVMF.fd`
/// - Arch Linux / Fedora / RHEL: `/usr/share/edk2-ovmf/x64/OVMF_CODE.fd`, `/usr/share/edk2/ovmf/OVMF_CODE.fd`
/// - NixOS / Alpine / OpenSUSE: `/usr/share/qemu/OVMF.fd`, `/usr/share/OVMF/OVMF_CODE_4M.fd`
pub const LINUX_OVMF_CANDIDATES: &[&str] = &[
    "/usr/share/qemu/OVMF.fd",
    "/usr/share/OVMF/OVMF_CODE_4M.fd",
    "/usr/share/ovmf/OVMF.fd",
    "/usr/share/edk2-ovmf/x64/OVMF_CODE.fd",
    "/usr/share/edk2/ovmf/OVMF_CODE.fd",
];

/// Canonical Apple macOS OVMF firmware search paths.
///
/// Covers Homebrew Apple Silicon (`/opt/homebrew`), Intel Mac (`/usr/local`),
/// and MacPorts (`/opt/local`) installations of QEMU:
pub const MACOS_OVMF_CANDIDATES: &[&str] = &[
    "/opt/homebrew/share/qemu/edk2-x86_64-code.fd",
    "/usr/local/share/qemu/edk2-x86_64-code.fd",
    "/opt/homebrew/Cellar/qemu/share/qemu/edk2-x86_64-code.fd",
    "/opt/local/share/qemu/edk2-x86_64-code.fd",
];

/// Canonical Microsoft Windows OVMF firmware search paths.
///
/// Covers standard 64-bit and 32-bit QEMU installer directories under `Program Files`:
pub const WINDOWS_OVMF_CANDIDATES: &[&str] = &[
    "C:\\Program Files\\qemu\\share\\edk2-x86_64-code.fd",
    "C:\\Program Files (x86)\\qemu\\share\\edk2-x86_64-code.fd",
    "C:\\qemu\\share\\edk2-x86_64-code.fd",
];

/// Fallback generic firmware search paths for unrecognized POSIX environments.
pub const FALLBACK_OVMF_CANDIDATES: &[&str] = &[
    "/usr/share/qemu/OVMF.fd",
    "/usr/local/share/qemu/OVMF.fd",
];
```

```rust

// ---------------------------------------------------------------------------
// 2.3. Initial Truth States & Equilibrium Platform Defaults
// ---------------------------------------------------------------------------
/// Default hypervisor binary for POSIX-compliant host environments.
pub const DEFAULT_UNIX_HYPERVISOR: &str = "qemu-system-x86_64";

/// Default hypervisor binary for Microsoft Windows host environments.
pub const DEFAULT_WINDOWS_HYPERVISOR: &str = "qemu-system-x86_64.exe";

/// Canonical Linux KVM device node for hardware virtualization.
pub const LINUX_KVM_DEVICE_NODE: &str = "/dev/kvm";

// ---------------------------------------------------------------------------
// 2.4. Host Silicon Architecture Typology
// ---------------------------------------------------------------------------
/// Physical host CPU instruction set architecture.
///
/// Identifies the silicon substrate upon which the host operating system executes.
/// Informs hypervisor hardware acceleration compatibility (e.g. KVM or HVF can only
/// be used when host architecture matches guest target architecture).
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum HostArch {
    /// 64-bit x86 architecture (AMD64 / Intel 64).
    X86_64,
    /// 64-bit ARM architecture (Apple Silicon M-series, AWS Graviton, Ampere Altra).
    Aarch64,
    /// 64-bit RISC-V architecture.
    RiscV64,
    /// Unidentified or unsupported host processor architecture.
    Unknown,
}

impl HostArch {
    /// Detects the host processor architecture at runtime using target architecture directives.
    pub fn current() -> Self {
        if cfg!(target_arch = "x86_64") {
            HostArch::X86_64
        } else if cfg!(target_arch = "aarch64") {
            HostArch::Aarch64
        } else if cfg!(target_arch = "riscv64") {
            HostArch::RiscV64
        } else {
            HostArch::Unknown
        }
    }

    /// Returns the human and machine readable designation of the CPU architecture.
    pub fn display_name(&self) -> &'static str {
        match self {
            HostArch::X86_64 => "x86_64 (AMD64 / Intel 64)",
            HostArch::Aarch64 => "aarch64 (ARM64 / Apple Silicon)",
            HostArch::RiscV64 => "riscv64 (RISC-V 64-Bit)",
            HostArch::Unknown => "unknown-arch (Generic Processor)",
        }
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Dynamic Host Platform Implementation, Environmental Oracles & Process Dispatch )
// ============================================================================

// ---------------------------------------------------------------------------
// 3.1. Platform Runtime Detection & Classification
// ---------------------------------------------------------------------------
/// Host operating system classification for "In-Waiting" symbiotic execution.
///
/// Represents the physical runtime habitat in which the OmniCode host engine
/// and developer tooling incubate. Enables dynamic adaptation of hypervisors,
/// firmware paths, native process dispatchers, and hardware acceleration flags.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum HostPlatform {
    /// GNU/Linux host (KVM hardware virtualization, systemd/FHS filesystem layout).
    Linux,
    /// Apple macOS host (Darwin kernel, Hypervisor.framework / HVF, Homebrew paths).
    MacOS,
    /// Microsoft Windows host (NT kernel, PE32+ binaries, Windows Hypervisor Platform / WHPX).
```

```rust
    Windows,
    /// Unidentified or generic POSIX environment (BSD, Solaris, embedded runtime).
    Unknown,
}

impl HostPlatform {
    /// Dynamically detects the host operating system platform at runtime.
    ///
    /// Evaluates compile-time target family directives (`cfg!(target_os)`) while
    /// providing runtime verification hooks.
    pub fn current() -> Self {
        if cfg!(target_os = "linux") {
            HostPlatform::Linux
        } else if cfg!(target_os = "macos") {
            HostPlatform::MacOS
        } else if cfg!(target_os = "windows") {
            HostPlatform::Windows
        } else {
            HostPlatform::Unknown
        }
    }

    /// Detects the physical host CPU architecture.
    pub fn arch(&self) -> HostArch {
        HostArch::current()
    }

    /// Returns the canonical platform display title with incubation role.
    pub fn display_name(&self) -> &'static str {
        match self {
            HostPlatform::Linux => "GNU/Linux (Host Incubation Plane)",
            HostPlatform::MacOS => "Apple macOS (Darwin Incubation Plane)",
            HostPlatform::Windows => "Microsoft Windows (NT Incubation Plane)",
            HostPlatform::Unknown => "Generic Host Plane (Fallback Environment)",
        }
    }

    /// Returns the target operating system family name (`"unix"` or `"windows"`).
    pub fn target_os_family(&self) -> &'static str {
        match self {
            HostPlatform::Windows => "windows",
            _ => "unix",
        }
    }

    /// Returns `true` if the host platform adheres to POSIX standards (Linux, macOS, BSD).
    pub fn is_posix(&self) -> bool {
        !matches!(self, HostPlatform::Windows)
    }

    /// Returns `true` if the host platform is Microsoft Windows.
    pub fn is_windows(&self) -> bool {
        matches!(self, HostPlatform::Windows)
    }

    /// Returns `true` if the host platform is GNU/Linux.
    pub fn is_linux(&self) -> bool {
        matches!(self, HostPlatform::Linux)
    }

    /// Returns `true` if the host platform is Apple macOS Darwin.
    pub fn is_macos(&self) -> bool {
        matches!(self, HostPlatform::MacOS)
    }

    /// Produces a comprehensive, machine-readable diagnostic snapshot of the host habitat.
    ///
    /// Used by [`DiagnosticLogger`], [`crate::diagnostics::debugger::OmniDebugger`], and AI cortex
    /// to record environmental state in episodic logs.
    pub fn environment_summary(&self) -> String {
        format!(
            "HostPlatform: {} | Arch: {} | Family: {} | POSIX: {} | Accel: {} | Display: {}",
            self.display_name(),
            self.arch().display_name(),
            self.target_os_family(),
            self.is_posix(),
            self.has_hardware_acceleration(),
            self.has_display_server()
```

```
        )
    }
}

// ---------------------------------------------------------------------------
// 3.2. Host PATH Resolution & Toolchain Binary Oracles
// ---------------------------------------------------------------------------
impl HostPlatform {
    /// Searches the host system `PATH` environment variable for an executable by name.
    ///
    /// # Platform Mechanics:
    /// - Respects platform-specific path list separators (`:` on Unix/POSIX, `;` on Windows).
    /// - On Windows, automatically tests the filename with `.exe` if not already provided.
    /// - Validates that candidate files exist and are regular files.
    pub fn resolve_in_path(&self, binary: &str) -> Option<PathBuf> {
        let path_var = env::var_os("PATH")?;

        let candidate_names: Vec<String> = if self.is_windows() && !binary.ends_with(".exe") {
            vec![format!("{}.exe", binary), binary.to_string()]
        } else {
            vec![binary.to_string()]
        };

        for dir in env::split_paths(&path_var) {
            for name in &candidate_names {
                let candidate = dir.join(name);
                if candidate.is_file() {
                    return Some(candidate);
                }
            }
        }

        None
    }

    /// Returns the platform-appropriate primary hypervisor binary name.
    pub fn hypervisor_binary(&self) -> &'static str {
        match self {
            HostPlatform::Windows => DEFAULT_WINDOWS_HYPERVISOR,
            _ => DEFAULT_UNIX_HYPERVISOR,
        }
    }

    /// Returns an ordered list of candidate hypervisor executable names.
    pub fn hypervisor_candidates(&self) -> &'static [&'static str] {
        match self {
            HostPlatform::Windows => &[DEFAULT_WINDOWS_HYPERVISOR, "qemu.exe"],
            HostPlatform::Linux => &[DEFAULT_UNIX_HYPERVISOR, "qemu-kvm", "kvm"],
            HostPlatform::MacOS => &[DEFAULT_UNIX_HYPERVISOR],
            HostPlatform::Unknown => &[DEFAULT_UNIX_HYPERVISOR],
        }
    }

    /// Dynamically locates the first valid hypervisor binary present on the host system PATH.
    pub fn find_hypervisor_binary(&self) -> Option<PathBuf> {
        for &candidate in self.hypervisor_candidates() {
            if let Some(path) = self.resolve_in_path(candidate) {
                return Some(path);
            }
        }
        None
    }

    /// Asserts that a valid hypervisor binary is present and reachable on the host PATH.
    ///
    /// # Sensory Tripwire:
    /// Trips [`Tripwire::assert_binary_available`], recording a failpoint in the logger
    /// with actionable installation remediation for Linux, macOS, or Windows if missing.
    pub fn assert_hypervisor_available(
        &self,
        logger: &DiagnosticLogger,
    ) -> Result<PathBuf, TripwireError> {
        let binary = self.hypervisor_binary();

        match self.resolve_in_path(binary) {
            Some(resolved_path) => {
                logger.log(
                    "PLATFORM",
```

```
                    format!("Hypervisor binary verified on PATH: {:?}", resolved_path),
                );
                Ok(resolved_path)
            }
            None => {
                let install_hint = match self {
                    HostPlatform::Linux => "sudo apt install qemu-system-x86 (Debian/Ubuntu) or sudo pacman -S qemu-system-x86 (Arch)",
                    HostPlatform::MacOS => "brew install qemu (Homebrew) or port install qemu (MacPorts)",
                    HostPlatform::Windows => "winget install QEMU.QEMU or download from https://www.qemu.org/download/#windows",
                    HostPlatform::Unknown => "Install QEMU from your operating system package repository",
                };

                let suggestion = FixSuggestion {
                    kind: FixKind::AddImport,
                    line: 1,
                    col: 1,
                    target_token: binary.to_string(),
                    suggested_patch: install_hint.to_string(),
                    rationale: format!(
                        "Host hypervisor executable '{}' is required for bare-metal machine inception",
                        binary
                    ),
                };

                let err = Tripwire::trip_with_remediation(
                    logger,
                    "PLATFORM",
                    FailpointType::SyntaxAnomaly,
                    format!(
                        "Hypervisor binary '{}' not found on host PATH. Remediation: {}",
                        binary, install_hint
                    ),
                    suggestion,
                );
                Err(err)
            }
        }
    }
}

// ---------------------------------------------------------------------------
// 3.3. Virtualization Acceleration Oracles & Permission Probes
// ---------------------------------------------------------------------------
impl HostPlatform {
    /// Probes whether hardware virtualization acceleration is accessible on the host.
    ///
    /// # Physical Hardware Invariants:
    /// - **Linux**: Inspects existence and read/write accessibility of `/dev/kvm`.
    /// - **macOS**: Target family supports Hypervisor.framework (`hvf`) on Darwin when
    ///   running on matching architecture or Apple Silicon hypervisor.
    /// - **Windows**: Target family supports Windows Hypervisor Platform (`whpx`).
    pub fn has_hardware_acceleration(&self) -> bool {
        match self {
            HostPlatform::Linux => {
                let kvm_path = Path::new(LINUX_KVM_DEVICE_NODE);
                if !kvm_path.exists() {
                    return false;
                }
                // Verify read and write permissions to /dev/kvm
                OpenOptions::new()
                    .read(true)
                    .write(true)
                    .open(kvm_path)
                    .is_ok()
            }
            HostPlatform::MacOS => true,
            HostPlatform::Windows => false, // Conservative default; requires Hyper-V / WHPX feature check
            HostPlatform::Unknown => false,
        }
    }

    /// Asserts that hardware acceleration is available, logging telemetry or tripping an alert.
    ///
    /// # Game Dev Collision Volume Analogy:
    /// If an acceleration tripwire is armed, tripping here warns that execution will drop
    /// into software emulation (TCG), which incurs significant performance overhead.
```

```
    pub fn assert_acceleration_supported(
        &self,
        logger: &DiagnosticLogger,
    ) -> Result<bool, TripwireError> {
        let accel = self.has_hardware_acceleration();
        if accel {
            logger.log(
                "ACCELERATION",
                format!("Hardware virtualization acceleration active on {}", self.display_name()),
            );
            Ok(true)
        } else {
            logger.record_ai_observation(
                "ACCELERATION_ABSENT",
                "Hardware virtualization acceleration unavailable",
                "Falling back to Tiny Code Generator (TCG) software emulation",
                ["virtualization", "tcg", "fallback", "performance"].as_slice(),
                None,
            );
            Ok(false)
        }
    }

    /// Returns the recommended hypervisor command-line arguments for hardware acceleration.
    ///
    /// Falls back to pure TCG software emulation if acceleration is unavailable.
    pub fn recommended_accel_args(&self) -> &'static [&'static str] {
        match self {
            HostPlatform::Linux if self.has_hardware_acceleration() => &["-enable-kvm"],
            HostPlatform::MacOS => &["-accel", "hvf"],
            HostPlatform::Windows if self.has_hardware_acceleration() => &["-accel", "whpx"],
            _ => &["-accel", "tcg"],
        }
    }
}

// ---------------------------------------------------------------------------
// 3.4. Host Display Server & Graphical Subsystem Oracles
// ---------------------------------------------------------------------------
impl HostPlatform {
    /// Probes whether a graphical display server or window manager is active on the host.
    ///
    /// Prevents GUI window spawn crashes when running in headless environments
    /// (e.g. continuous integration servers, SSH remote sessions, or containers).
    pub fn has_display_server(&self) -> bool {
        match self {
            HostPlatform::Linux | HostPlatform::Unknown => {
                env::var("DISPLAY").is_ok() || env::var("WAYLAND_DISPLAY").is_ok()
            }
            HostPlatform::MacOS => {
                // macOS GUI window server is available in standard desktop user sessions
                env::var("TERM_PROGRAM").is_ok() || env::var("Apple_PubSub_Socket_Render").is_ok() || true
            }
            HostPlatform::Windows => {
                // Windows desktop GUI is standard unless running in Windows Server Core
                true
            }
        }
    }

    /// Asserts that a graphical display server is available when GUI mode is requested.
    ///
    /// # Sensory Tripwire:
    /// If running in a headless environment and GUI display is requested, trips a wire
    /// recommending headless serial mode (`--display none`).
    pub fn assert_display_server_available(
        &self,
        logger: &DiagnosticLogger,
    ) -> Result<(), TripwireError> {
        if self.has_display_server() {
            logger.log("DISPLAY", "Graphical display server verified active (+1:PROCEED)");
            Ok(())
        } else {
            let suggestion = FixSuggestion {
                kind: FixKind::AdjustCaveat,
                line: 1,
                col: 1,
                target_token: "--gui".to_string(),
```

```
                suggested_patch: "--headless".to_string(),
                rationale: "Host display environment has no active DISPLAY or WAYLAND_DISPLAY session".to_string(),
            };

            let err = Tripwire::trip_with_remediation(
                logger,
                "DISPLAY",
                FailpointType::SyntaxAnomaly,
                "No graphical display server detected (headless environment). Run without --gui.".to_string(),
                suggestion,
            );
            Err(err)
        }
    }
}

// ---------------------------------------------------------------------------
// 3.5. UEFI Firmware Search Oracles & Telemetry Traversal
// ---------------------------------------------------------------------------
impl HostPlatform {
    /// Returns the candidate search paths for UEFI OVMF firmware images on this platform.
    pub fn ovmf_search_paths(&self) -> &'static [&'static str] {
        match self {
            HostPlatform::Linux => LINUX_OVMF_CANDIDATES,
            HostPlatform::MacOS => MACOS_OVMF_CANDIDATES,
            HostPlatform::Windows => WINDOWS_OVMF_CANDIDATES,
            HostPlatform::Unknown => FALLBACK_OVMF_CANDIDATES,
        }
    }

    /// Dynamically locates the first existing OVMF firmware image on the host system.
    pub fn find_ovmf_firmware(&self) -> Option<PathBuf> {
        // First inspect custom override environment variable if provided
        if let Ok(env_path) = env::var("OMNICODE_OVMF_PATH") {
            let p = PathBuf::from(env_path);
            if p.exists() {
                return Some(p);
            }
        }

        for &path_str in self.ovmf_search_paths() {
            let path = Path::new(path_str);
            if path.exists() {
                return Some(path.to_path_buf());
            }
        }
        None
    }

    /// Traverses the platform's firmware candidate list while logging telemetry events.
    ///
    /// # Telemetry Inscription:
    /// Inscribes candidate probe entries into [`DiagnosticLogger`]. If a firmware blob
    /// is resolved, logs success. If none are found, registers an informational diagnostic
    /// note equipped with remediation guidance for the Restorator and AI agents.
    pub fn find_ovmf_firmware_with_telemetry(&self, logger: &DiagnosticLogger) -> Option<PathBuf> {
        logger.log("FIRMWARE", format!("Probing OVMF firmware candidates on {}", self.display_name()));

        if let Ok(env_override) = env::var("OMNICODE_OVMF_PATH") {
            let p = PathBuf::from(env_override);
            if p.exists() {
                logger.log("FIRMWARE", format!("Resolved OVMF firmware via OMNICODE_OVMF_PATH: {:?}", p));
                return Some(p);
            }
        }

        for &path_str in self.ovmf_search_paths() {
            let path = Path::new(path_str);
            if path.exists() {
                logger.log("FIRMWARE", format!("Resolved OVMF firmware at: {:?}", path));
                return Some(path.to_path_buf());
            } else {
                logger.log("FIRMWARE", format!("Candidate not found: {}", path_str));
            }
        }

        logger.record_ai_observation(
            "FIRMWARE_UNRESOLVED",
```

```
            "No standard OVMF firmware image found on host",
            "UEFI boot will require explicit --ovmf path or fallback to BIOS MBR mode",
            ["firmware", "ovmf", "uefi", "fallback"].as_slice(),
            None,
        );

        None
    }

    /// Asserts that a valid OVMF firmware blob exists on disk, tripping a wire if absent.
    ///
    /// # Precondition Tripwire:
    /// If no candidate exists, trips a [`TripwireError`] equipped with a [`FixSuggestion`]
    /// instructing how to install or specify the firmware image.
    pub fn assert_ovmf_firmware(
        &self,
        logger: &DiagnosticLogger,
    ) -> Result<PathBuf, TripwireError> {
        if let Some(firmware) = self.find_ovmf_firmware_with_telemetry(logger) {
            Ok(firmware)
        } else {
            let suggestion = FixSuggestion {
                kind: FixKind::AddImport,
                line: 1,
                col: 1,
                target_token: "--uefi".to_string(),
                suggested_patch: "--bios".to_string(),
                rationale: "Fallback to Legacy BIOS MBR mode or install ovmf package on host system".to_string(),
            };

            let err = Tripwire::trip_with_remediation(
                logger,
                "FIRMWARE",
                FailpointType::SyntaxAnomaly,
                format!("OVMF UEFI firmware image not found on {}", self.display_name()),
                suggestion,
            );
            Err(err)
        }
    }
}

// ---------------------------------------------------------------------------
// 3.6. Host OS Process Dispatch & Native Subprocess Execution
// ---------------------------------------------------------------------------
impl HostPlatform {
    /// Constructs the platform-appropriate [`Command`] for opening a target URL in the default browser.
    pub fn format_browser_command(&self, url: &str) -> Command {
        match self {
            HostPlatform::Linux => {
                let mut cmd = Command::new("xdg-open");
                cmd.arg(url);
                cmd
            }
            HostPlatform::MacOS => {
                let mut cmd = Command::new("open");
                cmd.arg(url);
                cmd
            }
            HostPlatform::Windows => {
                let mut cmd = Command::new("cmd");
                cmd.args(["/C", "start", url]);
                cmd
            }
            HostPlatform::Unknown => {
                let mut cmd = Command::new("xdg-open");
                cmd.arg(url);
                cmd
            }
        }
    }

    /// Dynamically launches the default web browser on the host system for a target URL.
    pub fn open_browser(&self, url: &str) {
        let mut cmd = self.format_browser_command(url);
        let _ = cmd.spawn();
    }

```

```
    /// Launches the default browser wrapped in diagnostic logging and tripwire interception.
    pub fn open_browser_with_tripwire(
        &self,
        logger: &DiagnosticLogger,
        url: &str,
    ) -> Result<(), TripwireError> {
        logger.log(
            "BROWSER_DISPATCH",
            format!("Dispatching browser on {} for URL: {}", self.display_name(), url),
        );
        let mut cmd = self.format_browser_command(url);
        match cmd.spawn() {
            Ok(_) => {
                logger.log("BROWSER_DISPATCH", "Browser child process dispatched cleanly (+1:PROCEED)");
                Ok(())
            }
            Err(e) => {
                let msg = format!("Failed to dispatch host browser for URL '{}': {}", url, e);
                Err(Tripwire::trip(logger, "BROWSER_DISPATCH", FailpointType::SyntaxAnomaly, msg))
            }
        }
    }

    /// Executes an arbitrary host command wrapped in tripwire collision detection and telemetry.
    ///
    /// # Game Dev Simulation Hook:
    /// Captures stdout and stderr streams. If the command exits with non-zero status code,
    /// logs an execution failure and trips a structured error for the Restorator.
    pub fn execute_host_command(
        &self,
        logger: &DiagnosticLogger,
        subsystem: &'static str,
        mut cmd: Command,
    ) -> Result<Output, TripwireError> {
        logger.log(subsystem, format!("Spawning host subprocess: {:?}", cmd));
        match cmd.output() {
            Ok(output) => {
                if output.status.success() {
                    logger.log(subsystem, "Host subprocess terminated with exit status 0 (+1:PROCEED)");
                    Ok(output)
                } else {
                    let code = output.status.code().unwrap_or(-1);
                    let stderr = String::from_utf8_lossy(&output.stderr);
                    let msg = format!(
                        "Host subprocess exited with non-zero status code {}: {}",
                        code, stderr.trim()
                    );
                    Err(Tripwire::trip(logger, subsystem, FailpointType::SyntaxAnomaly, msg))
                }
            }
            Err(e) => {
                let msg = format!("Failed to spawn host subprocess: {}", e);
                Err(Tripwire::trip(logger, subsystem, FailpointType::SyntaxAnomaly, msg))
            }
        }
    }
}

// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Verification and Sealing, Baton Pass & In-File Unit Tests )
// ================================================================================

// --------------------------------------------------------------------------------
// 4.1. Dynamic Adapter Sealing & Downstream Baton Pass Specification
// --------------------------------------------------------------------------------
// BATON PASS CONTRACT:
//   - Upstream Producer: Rust runtime and compile-time target introspection (`cfg!(target_os)`).
//   - Downstream Receivers:
//     * `<TRACK 1>` Boot Gear (`src/cli/commands/boot.rs`): Hypervisor binary resolution,
//       UEFI firmware lookup, display server probe, and acceleration flags.
//     * `<TRACK 3>` Studio IDE (`src/terminal/studio.rs`): Native browser dispatch and GUI server.
//     * `<TRACK 1>` REPL & Terminal (`src/terminal/session.rs`): Console color and terminal probes.
//   - Certified Invariants:
//     1. `HostPlatform::current()` and `HostArch::current()` return deterministic, valid variants.
//     2. Binary candidate matrices map strictly to executable names appropriate for the OS family.
//     3. `resolve_in_path` navigates host environment paths respecting platform path delimiters.
//     4. Browser command synthesizers construct platform-native dispatch subprocesses.
```

```rust
//     5. Virtualization acceleration flags accurately select KVM, HVF, WHPX, or fallback TCG.

#[cfg(test)]
mod tests {
    use super::*;

    /// Verifies that `HostPlatform::current()` dynamically resolves the active host operating system
    /// and supplies non-empty display names, binary targets, firmware matrices, and OS families.
    ///
    /// # Failure Mode Guarded Against:
    /// Prevents null or uninitialized platform definitions from entering downstream gears.
    #[test]
    fn test_platform_current_detection() {
        let p = HostPlatform::current();
        assert!(!p.display_name().is_empty(), "Display name must not be empty");
        assert!(!p.hypervisor_binary().is_empty(), "Hypervisor binary name must not be empty");
        assert!(!p.ovmf_search_paths().is_empty(), "Firmware search paths matrix must not be empty");
        assert!(!p.target_os_family().is_empty(), "OS family string must not be empty");
    }

    /// Verifies that `HostArch::current()` detects the host CPU instruction set architecture.
    ///
    /// # Failure Mode Guarded Against:
    /// Prevents silent fallback to mismatched architecture flags during hypervisor command synthesis.
    #[test]
    fn test_host_arch_detection() {
        let arch = HostArch::current();
        assert!(!arch.display_name().is_empty(), "CPU architecture display title must not be empty");
        let platform = HostPlatform::current();
        assert_eq!(platform.arch(), arch, "Platform arch must match HostArch::current");
    }

    /// Verifies that `environment_summary()` produces a comprehensive, well-formed diagnostic string
    /// containing all critical platform facets for episodic logging.
    ///
    /// # Invariants Tested:
    /// - Contains `"HostPlatform:"`, `"Arch:"`, `"Family:"`, `"POSIX:"`, `"Accel:"`, and `"Display:"`.
    #[test]
    fn test_environment_summary() {
        let p = HostPlatform::current();
        let summary = p.environment_summary();
        assert!(summary.contains("HostPlatform:"), "Summary must report HostPlatform");
        assert!(summary.contains("Arch:"), "Summary must report host architecture");
        assert!(summary.contains("Family:"), "Summary must report OS family");
        assert!(summary.contains("POSIX:"), "Summary must report POSIX compliance");
        assert!(summary.contains("Accel:"), "Summary must report hardware acceleration status");
        assert!(summary.contains("Display:"), "Summary must report display server availability");
    }

    /// Verifies the platform-to-binary mapping matrix across all supported operating system families.
    ///
    /// # Invariants Tested:
    /// - Windows requires `.exe` binary extension (`qemu-system-x86_64.exe`).
    /// - POSIX systems (Linux, macOS, Unknown) expect bare binary names (`qemu-system-x86_64`).
    #[test]
    fn test_platform_binaries() {
        assert_eq!(HostPlatform::Linux.hypervisor_binary(), "qemu-system-x86_64");
        assert_eq!(HostPlatform::Windows.hypervisor_binary(), "qemu-system-x86_64.exe");
        assert_eq!(HostPlatform::MacOS.hypervisor_binary(), "qemu-system-x86_64");
        assert_eq!(HostPlatform::Unknown.hypervisor_binary(), "qemu-system-x86_64");
    }

    /// Verifies the OS family classification and semantic predicate methods on `HostPlatform`.
    ///
    /// # Invariants Tested:
    /// - Linux: family `"unix"`, `is_posix() == true`, `is_linux() == true`, `is_windows() == false`.
    /// - macOS: family `"unix"`, `is_posix() == true`, `is_macos() == true`, `is_windows() == false`.
    /// - Windows: family `"windows"`, `is_posix() == false`, `is_windows() == true`.
    #[test]
    fn test_platform_os_family() {
        assert_eq!(HostPlatform::Linux.target_os_family(), "unix");
        assert!(HostPlatform::Linux.is_posix());
        assert!(HostPlatform::Linux.is_linux());
        assert!(!HostPlatform::Linux.is_windows());
        assert!(!HostPlatform::Linux.is_macos());

        assert_eq!(HostPlatform::MacOS.target_os_family(), "unix");
        assert!(HostPlatform::MacOS.is_posix());
```

```
        assert!(HostPlatform::MacOS.is_macos());
        assert!(!HostPlatform::MacOS.is_linux());

        assert_eq!(HostPlatform::Windows.target_os_family(), "windows");
        assert!(HostPlatform::Windows.is_windows());
        assert!(!HostPlatform::Windows.is_posix());
    }

    /// Verifies hardware virtualization acceleration command-line argument mapping.
    ///
    /// # Invariants Tested:
    /// - macOS maps to Apple Hypervisor.framework (`-accel hvf`).
    /// - Fallback/Unknown maps to software emulation (`-accel tcg`).
    /// - Linux provides acceleration flags (`-enable-kvm` or fallback).
    #[test]
    fn test_virtualization_accel_args() {
        let args_linux = HostPlatform::Linux.recommended_accel_args();
        assert!(!args_linux.is_empty(), "Linux acceleration argument list must not be empty");

        let args_macos = HostPlatform::MacOS.recommended_accel_args();
        assert_eq!(args_macos, &["-accel", "hvf"], "macOS must use Hypervisor.framework");

        let args_fallback = HostPlatform::Unknown.recommended_accel_args();
        assert_eq!(args_fallback, &["-accel", "tcg"], "Unknown platform must fall back to TCG");
    }

    /// Verifies native browser command synthesizer across Linux, macOS, and Windows.
    ///
    /// # Invariants Tested:
    /// - Linux dispatches `xdg-open <URL>`.
    /// - macOS dispatches `open <URL>`.
    /// - Windows dispatches `cmd /C start <URL>`.
    #[test]
    fn test_format_browser_command() {
        let test_url = "http://localhost:8080/studio";

        let cmd_linux = HostPlatform::Linux.format_browser_command(test_url);
        assert_eq!(cmd_linux.get_program(), "xdg-open");

        let cmd_macos = HostPlatform::MacOS.format_browser_command(test_url);
        assert_eq!(cmd_macos.get_program(), "open");

        let cmd_windows = HostPlatform::Windows.format_browser_command(test_url);
        assert_eq!(cmd_windows.get_program(), "cmd");
    }

    /// Verifies that `resolve_in_path` successfully resolves existing system binaries
    /// and correctly returns `None` for non-existent binaries.
    ///
    /// # Failure Mode Guarded Against:
    /// Prevents false positives or broken delimiter splitting when scanning host PATH.
    #[test]
    fn test_resolve_in_path_contract() {
        let platform = HostPlatform::current();

        // Standard shell binary must resolve on any functioning host
        let target = if platform.is_windows() { "cmd" } else { "sh" };
        let resolved = platform.resolve_in_path(target);
        assert!(resolved.is_some(), "Standard system shell must resolve on host PATH");
        let path = resolved.unwrap();
        assert!(path.is_file(), "Resolved PATH entry must point to a regular file");

        // Intentionally non-existent binary must return None
        let bogus = platform.resolve_in_path("definitely_nonexistent_binary_omnicode_xyz");
        assert!(bogus.is_none(), "Non-existent executable name must return None");
    }

    /// Verifies that candidate search path matrices across all platform variants
    /// are non-empty and contain valid non-empty path strings.
    ///
    /// # Invariants Tested:
    /// - Linux candidates, macOS candidates, Windows candidates, and Fallback candidates
    ///   must all be populated.
    #[test]
    fn test_ovmf_search_paths_contract() {
        for platform in &[HostPlatform::Linux, HostPlatform::MacOS, HostPlatform::Windows, HostPlatform::Unknown] {
            let paths = platform.ovmf_search_paths();
            assert!(!paths.is_empty(), "Platform {:?} must declare firmware candidates", platform);
```

```
            for &p in paths {
                assert!(!p.is_empty(), "Firmware candidate path string must not be empty");
            }
        }
    }

    /// Verifies that the display server probe evaluates without panic.
    #[test]
    fn test_display_server_probe_contract() {
        let platform = HostPlatform::current();
        let _has_display = platform.has_display_server();
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/cogs/tripwire.rs RS 628 LINES 2781 WORDS 28.2 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Universal Invariant Tripwire Cog Manifest & Cybernetic Sensory Boundaries )
// ============================================================================
//! MODULE:              omnicode::cogs::tripwire
//! IDENTITY:            Universal Invariant Tripwire Sensor Cog
//! NATURE:              Dynamic Boundary Protection, Collision Detection & Diagnostic Interception
//! VERSION:             0.3.0
//! CREATED:             2026-10-11
//! LAST_SEALED:         2026-10-11
//! AUTHOR:              CreativeWorkzStudio LLC / ProfessorSeanEX
//! COVENANT:            Public Domain / MIT
//!
//! SCHEDULER:
//!   TRACK_PRIMARY:     <TRACK 1> : {Host Engine & Virtual Machine}
//!   TRACKS_TOUCHED:    [ <TRACK 0> : {Silicon & Bare-Metal Kernel}, <TRACK 1>, <TRACK 2> : {Engine Runtime & SVO},
<TRACK 3> : {Studio & Graphical IDE} ]
//!   PRIORITY:          TaskPriority::High
//!   CHANNEL:           Track::Kernel
//!
//! CONTRACT / WHO-WHAT-WHERE-WHY-HOW:
//!   WHO:               OmniCode Universal Sensory Invariant Tripwire Subsystem
//!   WHAT:              Trap invariant violations, record structured failpoint events,
//!                      equip remediation suggestions, and summon the debugger and restorator.
//!   WHERE:             Enforced across all modules, CLI gears, platform adapters, terminal sessions, and scheduler
tasks.
//!   WHEN:              Evaluated at precondition boundaries, file accesses, subprocess invocations, and state
transitions.
//!   WHY:               Eliminate unhandled panics and raw process aborts, converting crashes into recoverable
//!                      diagnostic signals that trigger transaction rollbacks (`-1:ROLLBACK`) or automated self-healing.
//!   HOW:               Balanced Ternary (+1:PROCEED invariant held, -1:ROLLBACK invariant breached with telemetry,
0:AWAIT standby).
//!
//! THE GAME DEV COLLISION TRIGGER PARADIGM:
//!   In modern game engine architecture (e.g., Unreal, Unity, Godot):
//!   - Invisible Trigger Volumes: Spatial and physical boundary colliders placed in world space. When an entity
//!     penetrates an illegal zone or invalid state, the physics engine does not crash the game loop; it fires an
//!     `OnTriggerEnter` or `OnCollision` event callback.
//!   - In OmniCode: Tripwires are the cognitive and computational collision triggers. Instead of allowing undefined
//!     behavior, unhandled null states, absent silicon binaries, missing kernel images, or corrupted filesystems to
//!     panic the engine (`panic!`), a tripwire immediately catches the boundary collision.
//!
//! CYBERNETIC FAILPOINT & TRANSLATION ENVELOPE:
//!   - Translates raw runtime anomalies into formal ontological failpoint categories ([`FailpointType`]).
//!   - Emits microsecond-timestamped telemetry directly into the active [`DiagnosticLogger`] ring buffer and
[`crate::diagnostics::debugger::OmniDebugger`].
//!   - Inscribes episodic cognitive memory notes tagged `#failpoint` for AI reasoning models and developers.
//!   - Replaces uncontrolled stack unwinding with balanced-ternary quad-state causality (`-1:ROLLBACK` or `0:AWAIT`).
//!
//! THE RESTORATOR HANDOFF & REPAIR TELEMETRY:
//!   - When a tripwire trips with remediation ([`Tripwire::trip_with_remediation`]), it equips the trapped event
//!     with a [`FixSuggestion`] and registers `#remediation_ready` telemetry.
//!   - The [`crate::diagnostics::restorator::Restorator`] (OmniCode's autonomous immune system) ingests this structured
//!     flight data, executes surgical AST/token patches, test-certifies the candidate repair, and restores system
balance.
//!
//! TOUCHES / SYSTEM & PROCESS FOOTPRINT:
//!   - TELEMETRY:       Direct emission to [`DiagnosticLogger`] and active
[`crate::diagnostics::debugger::OmniDebugger`].
//!   - TERMINAL:        Standardized ANSI red/yellow tripwire alert streams to stderr and session logs.
//!   - RECOVERY:        Integrates with transaction rollbacks and automated Restorator immune responses.
//!   - SUBPROCESSES:    Monitors exit status codes and standard streams of external host child processes.
//!
//! REQUIRES / HOST CAPABILITIES:
//!   - Standard error stream access (`std::io::stderr`).
//!   - Filesystem metadata inspection (`std::path::Path::exists`, `std::path::Path::is_dir`, `std::fs::metadata`).
//!   - Subprocess status inspection (`std::process::ExitStatus`).
//!
//! TRIPWIRE REGISTRY:
//!   - `TRIPWIRE_PATH_EXISTENCE`       (Assertion of critical filesystem resources and kernel artifacts)
//!   - `TRIPWIRE_DIR_EXISTENCE`        (Assertion of required workspace and directory hierarchies)
//!   - `TRIPWIRE_BINARY_AVAILABLE`     (Assertion of required external host toolchains on system PATH)
//!   - `TRIPWIRE_COMMAND_STATUS`       (Assertion of zero exit status on host subprocesses)
//!   - `TRIPWIRE_STATE_VIOLATION`      (Assertion of semantic invariant boundaries and quad-state commitments)
```

```
//!   - `TRIPWIRE_BOOLEAN_INVARIANT`    (Assertion of mathematical and ontological propositional truths)
//!
//! NAVIGATION GUIDE (HOW TO NAVIGATE AND EXTEND THIS COG AS THE CODE GROWS):
//!   - Block 1 (Lines 1-85): Manifest, Game Dev Trigger Analogy, Telemetry Envelope, Tripwire Registry.
//!   - Block 2 (Lines 86-175): Scoped Imports, Error Envelopes (`TripwireError`), Display Formatting & Defaults.
//!     * To add custom recovery constructors: add methods to `TripwireError` in Section 2.2.
//!     * To modify equilibrium recovery defaults: review constants in Section 2.3.
//!   - Block 3 (Lines 176-350): Body Block & Sensory Assertion Oracles.
//!     * 3.1: Explicit Failpoint Inscription & Remediation Oracles (`trip`, `trip_with_remediation`, `trip_await`).
//!     * 3.2: Filesystem Spatial Invariant Assertions (`assert_path_exists`, `assert_dir_exists`, `assert_file_readable`).
//!     * 3.3: Host Toolchain & Binary Availability Oracles (`assert_binary_available`).
//!     * 3.4: Subprocess Lifecycle & Exit Status Assertions (`assert_command_success`, `assert_command_output_clean`).
//!     * 3.5: Mathematical, State Machine & Causality Boundary Assertions (`assert_quad_state_proceed`, `assert_invariant`).
//!   - Block 4 (Lines 351+): Module Sealing, Baton Pass Protocol & In-File Unit Verification Tests.

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Scoped Imports, Invariant Data Types & Tripwire Error Definitions )
// ============================================================================

// ----------------------------------------------------------------------------
// 2.1. Scoped Namespace Ingestion & System Imports
// ----------------------------------------------------------------------------
use std::fmt;
use std::fs;
use std::path::{Path, PathBuf};
use std::process::{Command, ExitStatus, Output};

use crate::core::ternary::QuadState;
use crate::diagnostics::logger::{DiagnosticLogger, FailpointType, FixSuggestion};

// ----------------------------------------------------------------------------
// 2.2. Invariant Data Types & Tripwire Error Definitions
// ----------------------------------------------------------------------------
/// Formal error envelope produced when an execution boundary or invariant tripwire is breached.
///
/// Unlike standard Rust errors or panics that destroy process context, [`TripwireError`]
/// encapsulates rich cybernetic metadata:
/// - Channel identifier for precise subsystem origin tracking.
/// - Ontological failpoint category ([`FailpointType`]).
/// - Human and machine readable diagnostic message.
/// - Balanced-ternary recovery state (`-1:ROLLBACK` or `0:AWAIT`).
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct TripwireError {
    /// Subsystem or channel where the tripwire fired (e.g. "BOOT", "PLATFORM", "TERMINAL", "SCHEDULER").
    pub channel: &'static str,
    /// Formal diagnostic classification of the failpoint.
    pub failpoint_type: FailpointType,
    /// Detailed diagnostic message explaining the breach.
    pub message: String,
    /// Balanced ternary state mapping for recovery (typically `-1:ROLLBACK` or `0:AWAIT`).
    pub recovery_state: QuadState,
}

impl TripwireError {
    /// Creates a new tripwire error mapped to `-1:ROLLBACK` state.
    pub fn rollback(channel: &'static str, fp_type: FailpointType, msg: impl Into<String>) -> Self {
        Self {
            channel,
            failpoint_type: fp_type,
            message: msg.into(),
            recovery_state: QuadState::Rollback,
        }
    }

    /// Creates a new tripwire error mapped to `0:AWAIT` standby state.
    pub fn await_state(channel: &'static str, fp_type: FailpointType, msg: impl Into<String>) -> Self {
        Self {
            channel,
            failpoint_type: fp_type,
            message: msg.into(),
            recovery_state: QuadState::Await,
        }
    }

    /// Returns `true` if this error dictates an active transaction rollback (`-1:ROLLBACK`).
```

```rust
    pub fn is_rollback(&self) -> bool {
        self.recovery_state == QuadState::Rollback
    }

    /// Returns `true` if this error indicates that the operation is awaiting asynchronous completion (`0:AWAIT`).
    pub fn is_await(&self) -> bool {
        self.recovery_state == QuadState::Await
    }
}

impl fmt::Display for TripwireError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(
            f,
            "[{}] Failpoint {}: {} (State: {})",
            self.channel,
            self.failpoint_type.as_str(),
            self.message,
            self.recovery_state.macro_op()
        )
    }
}

impl std::error::Error for TripwireError {}

// ---------------------------------------------------------------------------
// 2.3. Initial Truth States & Equilibrium Recovery Defaults
// ---------------------------------------------------------------------------
/// Default quad-state recovery commitment when a tripwire collision occurs.
pub const DEFAULT_TRIPWIRE_RECOVERY_STATE: QuadState = QuadState::Rollback;

/// Default diagnostic channel assigned when no subsystem is explicitly specified.
pub const DEFAULT_TRIPWIRE_CHANNEL: &str = "TRIPWIRE";

// ===========================================================================
// BLOCK 3: BODY BLOCK
// ( Universal Tripwire Engine Implementation & Assertion Oracles )
// ===========================================================================

/// Mechanical Tripwire Sensor for invariant enforcement, collision sensing, and diagnostic alerting.
#[derive(Debug, Clone, Copy, Default)]
pub struct Tripwire;

// ---------------------------------------------------------------------------
// 3.1. Explicit Failpoint Inscription & Remediation Oracles
// ---------------------------------------------------------------------------
impl Tripwire {
    /// Explicitly trips a diagnostic wire, recording a failpoint in the logger and emitting a standard alert.
    ///
    /// # Game Dev Collision Volume Hook:
    /// Inscribes a critical failpoint into [`DiagnosticLogger`], writes an ANSI red alert to stderr,
    /// and packages the violation into a [`TripwireError`] with `-1:ROLLBACK`.
    pub fn trip(
        logger: &DiagnosticLogger,
        channel: &'static str,
        fp_type: FailpointType,
        message: impl Into<String>,
    ) -> TripwireError {
        let msg = message.into();
        logger.failpoint(fp_type, channel, msg.clone());

        eprintln!(
            "\x1b[1;31m✗ Tripwire Alert [{}]\x1b[0m [{}]: {}",
            channel,
            fp_type.as_str(),
            msg
        );

        TripwireError::rollback(channel, fp_type, msg)
    }

    /// Explicitly trips a diagnostic wire equipped with concrete remediation telemetry for the Restorator.
    ///
    /// # Restorator Immune Handoff:
    /// Attaches a [`FixSuggestion`] and registers an episodic cognitive memory note tagged
    /// `#remediation_ready`, allowing automated self-healing passes to immediately parse and apply the patch.
    pub fn trip_with_remediation(
        logger: &DiagnosticLogger,
```

```
        channel: &'static str,
        fp_type: FailpointType,
        message: impl Into<String>,
        suggestion: FixSuggestion,
    ) -> TripwireError {
        let msg = message.into();
        logger.failpoint_with_remediation(fp_type, channel, msg.clone(), suggestion);

        eprintln!(
            "\x1b[1;31m✗ Tripwire Alert [{}]\x1b[0m [{}]: {}",
            channel,
            fp_type.as_str(),
            msg
        );

        TripwireError::rollback(channel, fp_type, msg)
    }

    /// Trips a diagnostic wire that pauses execution in `0:AWAIT` standby state rather than rolling back.
    ///
    /// Used when a required resource is temporarily in flight or pending background synthesis.
    pub fn trip_await(
        logger: &DiagnosticLogger,
        channel: &'static str,
        fp_type: FailpointType,
        message: impl Into<String>,
    ) -> TripwireError {
        let msg = message.into();
        logger.failpoint(fp_type, channel, msg.clone());

        eprintln!(
            "\x1b[1;33m⚠ Tripwire Standby [{}]\x1b[0m [{}]: {}",
            channel,
            fp_type.as_str(),
            msg
        );

        TripwireError::await_state(channel, fp_type, msg)
    }
}

// ---------------------------------------------------------------------------
// 3.2. Filesystem Spatial Invariant Assertions
// ---------------------------------------------------------------------------
impl Tripwire {
    /// Asserts that a required file or directory exists on disk.
    ///
    /// # Precondition Sensor:
    /// Returns the verified [`PathBuf`] on success (+1:PROCEED). If absent, trips
    /// a [`FailpointType::SyntaxAnomaly`] error with `-1:ROLLBACK`.
    pub fn assert_path_exists(
        logger: &DiagnosticLogger,
        channel: &'static str,
        path: impl AsRef<Path>,
        description: &'static str,
    ) -> Result<PathBuf, TripwireError> {
        let p = path.as_ref();
        if p.exists() {
            Ok(p.to_path_buf())
        } else {
            let msg = format!("{description} not found at expected path: {:?}", p);
            Err(Self::trip(logger, channel, FailpointType::SyntaxAnomaly, msg))
        }
    }

    /// Asserts that a target path exists and is specifically a directory.
    pub fn assert_dir_exists(
        logger: &DiagnosticLogger,
        channel: &'static str,
        path: impl AsRef<Path>,
        description: &'static str,
    ) -> Result<PathBuf, TripwireError> {
        let p = path.as_ref();
        if p.is_dir() {
            Ok(p.to_path_buf())
        } else {
            let msg = format!("{description} directory does not exist: {:?}", p);
            Err(Self::trip(logger, channel, FailpointType::SyntaxAnomaly, msg))
```

```
        }
    }

    /// Asserts that a target file exists and is readable by the current process.
    pub fn assert_file_readable(
        logger: &DiagnosticLogger,
        channel: &'static str,
        path: impl AsRef<Path>,
        description: &'static str,
    ) -> Result<PathBuf, TripwireError> {
        let p = path.as_ref();
        if !p.exists() {
            let msg = format!("{description} file does not exist: {:?}", p);
            return Err(Self::trip(logger, channel, FailpointType::SyntaxAnomaly, msg));
        }

        match fs::metadata(p) {
            Ok(_) => Ok(p.to_path_buf()),
            Err(e) => {
                let msg = format!("Cannot read metadata for {description} at {:?}: {}", p, e);
                Err(Self::trip(logger, channel, FailpointType::SyntaxAnomaly, msg))
            }
        }
    }
}

// ---------------------------------------------------------------------------
// 3.3. Host Toolchain & Binary Availability Oracles
// ---------------------------------------------------------------------------
impl Tripwire {
    /// Asserts that a required executable binary is available on the host system PATH.
    ///
    /// # Platform Awareness:
    /// Dispatches `where` on Windows and `which` on Unix/macOS to probe host environment variables.
    pub fn assert_binary_available(
        logger: &DiagnosticLogger,
        channel: &'static str,
        bin_name: &'static str,
    ) -> Result<(), TripwireError> {
        let check = if cfg!(target_os = "windows") {
            Command::new("where").arg(bin_name).output()
        } else {
            Command::new("which").arg(bin_name).output()
        };

        match check {
            Ok(out) if out.status.success() => Ok(()),
            _ => {
                let msg = format!(
                    "Required host binary '{bin_name}' is not accessible on system PATH"
                );
                Err(Self::trip(logger, channel, FailpointType::SyntaxAnomaly, msg))
            }
        }
    }
}

// ---------------------------------------------------------------------------
// 3.4. Subprocess Lifecycle & Exit Status Assertions
// ---------------------------------------------------------------------------
impl Tripwire {
    /// Asserts that a subprocess command completed successfully with exit code 0.
    ///
    /// Traps non-zero exit codes or spawn failures without process panic.
    pub fn assert_command_success(
        logger: &DiagnosticLogger,
        channel: &'static str,
        status: Result<ExitStatus, std::io::Error>,
        command_name: &'static str,
    ) -> Result<(), TripwireError> {
        match status {
            Ok(s) if s.success() => Ok(()),
            Ok(s) => {
                let code = s.code().map(|c| c.to_string()).unwrap_or_else(|| "signal".into());
                let msg = format!("Subprocess '{command_name}' exited with non-zero status ({code})");
                Err(Self::trip(logger, channel, FailpointType::SyntaxAnomaly, msg))
            }
            Err(e) => {
```

```
                let msg = format!("Failed to spawn or communicate with subprocess '{command_name}': {e}");
                Err(Self::trip(logger, channel, FailpointType::SyntaxAnomaly, msg))
            }
        }
    }

    /// Asserts that a captured subprocess [`Output`] succeeded, logging stderr on failure.
    pub fn assert_command_output_clean(
        logger: &DiagnosticLogger,
        channel: &'static str,
        output: Result<Output, std::io::Error>,
        command_name: &'static str,
    ) -> Result<Output, TripwireError> {
        match output {
            Ok(out) if out.status.success() => Ok(out),
            Ok(out) => {
                let code = out.status.code().unwrap_or(-1);
                let stderr = String::from_utf8_lossy(&out.stderr);
                let msg = format!(
                    "Subprocess '{command_name}' failed with status code {}: {}",
                    code, stderr.trim()
                );
                Err(Self::trip(logger, channel, FailpointType::SyntaxAnomaly, msg))
            }
            Err(e) => {
                let msg = format!("Failed to execute subprocess '{command_name}': {e}");
                Err(Self::trip(logger, channel, FailpointType::SyntaxAnomaly, msg))
            }
        }
    }
}

// ---------------------------------------------------------------------------
// 3.5. Mathematical, State Machine & Causality Boundary Assertions
// ---------------------------------------------------------------------------
impl Tripwire {
    /// Asserts that a quad-state evaluation committed to `+1:PROCEED`.
    ///
    /// If the state is `-1:ROLLBACK` or `0:AWAIT`, raises an explicit tripwire error.
    pub fn assert_quad_state_proceed(
        logger: &DiagnosticLogger,
        channel: &'static str,
        state: QuadState,
        context: &'static str,
    ) -> Result<(), TripwireError> {
        if state == QuadState::Proceed {
            Ok(())
        } else {
            let msg = format!(
                "QuadState invariant breach in {context}: expected +1:PROCEED, encountered {}",
                state.macro_op()
            );
            Err(Self::trip(logger, channel, FailpointType::RollbackTriggered, msg))
        }
    }

    /// Asserts an arbitrary boolean propositional truth condition.
    ///
    /// If false, trips the wire with the designated failpoint category and message.
    pub fn assert_invariant(
        logger: &DiagnosticLogger,
        channel: &'static str,
        condition: bool,
        fp_type: FailpointType,
        message: impl Into<String>,
    ) -> Result<(), TripwireError> {
        if condition {
            Ok(())
        } else {
            Err(Self::trip(logger, channel, fp_type, message))
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Verification and Sealing, Baton Pass & In-File Unit Tests )
// ============================================================================
```

```
456
457 // ---------------------------------------------------------------------------
458 // 4.1. Universal Tripwire Sealing & Downstream Baton Pass Specification
459 // ---------------------------------------------------------------------------
460 // BATON PASS CONTRACT:
461 //   - Upstream Producer: Hardware, Filesystem, Runtime Processes & Evaluator ALU.
462 //   - Downstream Receivers:
463 //     * `<TRACK 1>` Boot Gear (`src/cli/commands/boot.rs`): Inception and compiler tripwires.
464 //     * `<TRACK 1>` Platform Cog (`src/cogs/platform.rs`): Hypervisor and firmware tripwires.
465 //     * `<TRACK 1>` Diagnostics (`src/diagnostics/`): Restorator immune repair and Debugger frames.
466 //     * `<TRACK 1>` Terminal Session (`src/terminal/session.rs`): Tape mount and execution boundary tripwires.
467 //   - Certified Invariants:
468 //     1. Tripwire breaches never panic or abort process memory; all breaches produce `TripwireError`.
469 //     2. All tripwire events inscribe structured failpoint records into `DiagnosticLogger`.
470 //     3. `Tripwire::trip_with_remediation` attaches `FixSuggestion` and tags `#remediation_ready`.
471 //     4. Filesystem, binary, subprocess, and boolean invariant assertion methods return
472 //        deterministic, uncheatable results.
473
474 #[cfg(test)]
475 mod tests {
476     use super::*;
477     use crate::diagnostics::logger::{FixKind, FixSuggestion};
478
479     /// Verifies that explicitly tripping a wire records a failpoint in `DiagnosticLogger`
480     /// and constructs a `TripwireError` with `-1:ROLLBACK` recovery state.
481     ///
482     /// # Failure Mode Guarded Against:
483     /// Prevents silent failure where an error is created without inscriptive telemetry.
484     #[test]
485     fn test_tripwire_explicit_trip_and_failpoint_capture() {
486         let logger = DiagnosticLogger::new(false);
487         let err = Tripwire::trip(
488             &logger,
489             "TEST_CHANNEL",
490             FailpointType::SyntaxAnomaly,
491             "Simulated explicit tripwire fault",
492         );
493
494         assert_eq!(err.channel, "TEST_CHANNEL");
495         assert_eq!(err.failpoint_type, FailpointType::SyntaxAnomaly);
496         assert_eq!(err.recovery_state, QuadState::Rollback);
497         assert!(err.is_rollback());
498         assert!(!err.is_await());
499         assert!(logger.has_failpoints(), "DiagnosticLogger must record the failpoint");
500         assert_eq!(logger.failpoints().len(), 1);
501     }
502
503     /// Verifies that `trip_with_remediation` successfully attaches a `FixSuggestion`
504     /// and emits an episodic cognitive note tagged `#remediation_ready`.
505     ///
506     /// # Invariants Tested:
507     /// - Failpoint contains the attached fix suggestion.
508     /// - Cognitive notes buffer contains a note tagged `#remediation_ready`.
509     #[test]
510     fn test_tripwire_with_remediation_suggestion() {
511         let logger = DiagnosticLogger::new(false);
512         let suggestion = FixSuggestion {
513             kind: FixKind::AddBlockHeader,
514             line: 1,
515             col: 1,
516             target_token: "HEADER".to_string(),
517             suggested_patch: "BLOCK 1: METADATA".to_string(),
518             rationale: "Automated header repair".to_string(),
519         };
520
521         let err = Tripwire::trip_with_remediation(
522             &logger,
523             "REPAIR_CHANNEL",
524             FailpointType::BoundaryMismatch,
525             "Simulated repairable fault",
526             suggestion.clone(),
527         );
528
529         assert_eq!(err.channel, "REPAIR_CHANNEL");
530         assert_eq!(err.failpoint_type, FailpointType::BoundaryMismatch);
531
532         let fps = logger.failpoints();
533         assert_eq!(fps.len(), 1);
```

```
        assert_eq!(fps[0].suggestion, Some(suggestion));

        let notes = logger.cognitive_notes();
        assert!(
            notes.iter().any(|n| n.tags.iter().any(|t| t == "remediation_ready")),
            "Episodic notes must include #remediation_ready tag"
        );
    }

    /// Verifies that `trip_await` sets `0:AWAIT` standby state rather than rolling back.
    #[test]
    fn test_tripwire_await_state_constructor() {
        let logger = DiagnosticLogger::new(false);
        let err = Tripwire::trip_await(
            &logger,
            "ASYNC_CHANNEL",
            FailpointType::NullTrap,
            "Pending background artifact",
        );

        assert_eq!(err.channel, "ASYNC_CHANNEL");
        assert_eq!(err.recovery_state, QuadState::Await);
        assert!(err.is_await());
        assert!(!err.is_rollback());
    }

    /// Verifies that `assert_path_exists` returns `Ok(PathBuf)` for existing paths
    /// and `Err(TripwireError)` with failpoint telemetry for missing paths.
    ///
    /// # Uncheatable Assertion:
    /// Tests both positive path resolution (`Cargo.toml`) and negative path failure.
    #[test]
    fn test_assert_path_exists_contract() {
        let logger = DiagnosticLogger::new(false);

        // Positive check: Cargo manifest must resolve
        let ok_res = Tripwire::assert_path_exists(&logger, "TEST", "Cargo.toml", "Cargo Manifest");
        assert!(ok_res.is_ok(), "Cargo.toml must resolve successfully");
        assert!(ok_res.unwrap().exists());

        // Negative check: missing file must trip with failpoint
        let missing = Tripwire::assert_path_exists(&logger, "TEST", "nonexistent_omnicode_file_xyz.omni", "Missing
File");
        assert!(missing.is_err(), "Missing file must return TripwireError");
        let err = missing.unwrap_err();
        assert_eq!(err.channel, "TEST");
        assert!(logger.has_failpoints(), "Logger must capture failpoint on missing path");
    }

    /// Verifies that `assert_dir_exists` validates directories and rejects non-directory files.
    #[test]
    fn test_assert_dir_exists_contract() {
        let logger = DiagnosticLogger::new(false);

        // Positive: current workspace root is a directory
        assert!(Tripwire::assert_dir_exists(&logger, "TEST", ".", "Workspace Root Directory").is_ok());

        // Negative: Cargo.toml is a file, not a directory
        assert!(Tripwire::assert_dir_exists(&logger, "TEST", "Cargo.toml", "Manifest File As Dir").is_err());
    }

    /// Verifies that `assert_invariant` succeeds on true conditions and trips on false conditions.
    #[test]
    fn test_assert_invariant_contract() {
        let logger = DiagnosticLogger::new(false);

        let ok_res = Tripwire::assert_invariant(&logger, "MATH", 2 + 2 == 4, FailpointType::SyntaxAnomaly, "Arithmetic
holds");
        assert!(ok_res.is_ok());

        let err_res = Tripwire::assert_invariant(&logger, "MATH", 2 + 2 == 5, FailpointType::SyntaxAnomaly, "Arithmetic
breached");
        assert!(err_res.is_err());
        assert!(logger.has_failpoints());
    }

    /// Verifies that `assert_quad_state_proceed` accepts `QuadState::Proceed` and trips on `Rollback` or `Await`.
    #[test]
```

```
    fn test_assert_quad_state_proceed_contract() {
        let logger = DiagnosticLogger::new(false);

        assert!(Tripwire::assert_quad_state_proceed(&logger, "EXEC", QuadState::Proceed, "Step 1").is_ok());
        assert!(Tripwire::assert_quad_state_proceed(&logger, "EXEC", QuadState::Rollback, "Step 2").is_err());
        assert!(Tripwire::assert_quad_state_proceed(&logger, "EXEC", QuadState::Await, "Step 3").is_err());
    }

    /// Verifies that `TripwireError` implements `Display` with channel, category, message, and state.
    #[test]
    fn test_tripwire_error_display_formatting() {
        let err = TripwireError::rollback("KERNEL", FailpointType::ImmutableMutation, "Attempted write to immutable
register");
        let formatted = format!("{err}");
        assert!(formatted.contains("[KERNEL]"));
        assert!(formatted.contains("FP_IMMUTABLE_MUTATION"));
        assert!(formatted.contains("Attempted write to immutable register"));
        assert!(formatted.contains("ROLLBACK"));
    }
}
```